

Project Report for FlickPond: A Reusable Cloud Media Processing Platform

Practice Module for Graduate Certificate in Architecting Scalable Systems

Team 17

Members:

Guruprasath Gopal

Ibrahim Mammadov

Nguyễn Kim Long

SWE5001 Architecting Scalable Systems · SE34FT · September–November 2026

Repository: github.com/Collaboration95/dc2 · Report version 1.1 · Submitted for 16 November 2026

CONTENTS

1. INTRODUCTION 4
   1.1 Background 4
   1.2 Business Needs 5
   1.3 Stakeholders 5
   1.4 Project Scope 6
2. PROJECT CONDUCT 8
   2.1 Project Plan 8
   2.2 Project Status 13
   2.3 Project Metrics 14
3. SOLUTION OVERVIEW 16
   3.1 Logical Architecture & Design 16
   3.2 Physical Architecture & Design 30
   3.3 Other Architectural Decisions 54
   3.4 Architectural Limitation 55
4. QUALITY ATTRIBUTES 56
   4.1 Performance 57
   4.2 Availability 58
   4.3 Security 58
   4.4 Extensibility and Maintainability 60
5. DEVOPS AND DEVELOPMENT LIFECYCLE 62
   5.1 Source Control Strategy 62
   5.2 Continuous Integration 63
   5.3 Continuous Delivery 65
6. OTHER THINGS TO BE HIGHLIGHTED 68
   6.1 Presentation Feedback and Response 68
   6.2 Use Case Model 69
   6.3 Analysis-to-Design Transition Strategies 71
   6.4 Analysis and Design Models 73
   6.5 Design Problems and Patterns 126

6.6 Member Contributions and Lessons Learned 140
7. APPENDICES 143
7.1 Glossary 143
7.2 Use-Case Traceability Matrix 143
7.3 Supporting Assets 144

1. Introduction

1.1 Background

FlickPond began as a Project 1 video application. Users upload their own videos, FFmpeg workers turn each upload into an MP4 and adaptive HLS streams (360p, 480p and 720p), and the owner plays the result through an owner-checked link. Project 1 ran FastAPI, PostgreSQL, Redis with RQ, MinIO, nginx and a browser frontend on one virtual machine as **one application**: a single API handled identity, uploads, jobs and playback, and the jobs table had a foreign key to the users table. Another product could not reuse the video workflow without importing FlickPond's code or reading its database.

Team 17 was formed for this practice module with the special permission of Prof Boon Kui Heng (22 September 2026), as a three-member team different from the Project 1 teams. The team chose the frozen FlickPond code as its baseline because it already contained a real, CPU-heavy asynchronous workload. The proposal names repository `Flickpond/Media_player`, commit `890c5ec`; the working plan used the team's local checkout of the same frozen code, commit `8854537` (decision D-01, 9 October). Inherited capabilities are credited to FlickPond and are not claimed as Project 2 work.

The proposal (4 October 2026) set the goal: turn FlickPond into a **reusable media processing platform**. Applications use one documented API for upload, processing, status and playback, and processing runs as an independently owned and scaled service. The lecturer accepted the proposal and asked the team to confirm the microservice choice through detailed domain analysis (feedback F-01); section 3.1.4 records that analysis.

Current status. This is report version 1.1, submitted for **16 November 2026**. Team 17 presented on 26 October, and v1.1 incorporates the presentation feedback. The implementation is in release freeze, and the persistent AWS environment `dc2-live` remains available until after submission.

- **Media Management** and **Processing** own separate databases, images, IAM roles, CI lanes and release units. Their business commands and results use versioned asynchronous SQS messages. API Gateway uses Media's authorizer to establish trusted caller identity.
- The **web app** and the independent **Publisher CLI** use the published API. All seven scoped use cases are implemented, tested and backed by dated experiments (sections 4 and 6).
- The standard lane launches finite Fargate encoders within a three-slot cap; a separate Lambda lane handles eligible short clips. Each service has its own database cluster.

Area	Inherited from FlickPond (Project 1)	Added by Team 17 (Project 2)
Service structure	One FastAPI application; workers and recovery task share its job code	Two bounded contexts, Media Management and Processing, found through DDD; business commands and results use asynchronous, versioned contracts
Data ownership	One database; jobs reference users	`media_db` and `processing_db`, each on its own Aurora cluster, with separate migration histories and least-privilege roles; no cross-database keys
Reuse	One browser application	A second, independent client (Publisher CLI) built against the published API only

Area	Inherited from FlickPond (Project 1)	Added by Team 17 (Project 2)
Runtime	One VM with nginx, Redis/RQ and MinIO	Serverless AWS: CloudFront, API Gateway, Lambda container images, SQS, Aurora Serverless v2, S3, finite ECS Fargate encoders that scale 0 → 3 → 0, and a Lambda lane for short clips
Security	Login, owner checks, TLS, signed output links, CI scans	Media-owned Gateway authorizer, trusted actor context, per-handler IAM roles, queue resource policies, bounded direct-to-S3 upload grants, pinned source versions, cloud negative tests
Delivery	Tests and scans in CI; manual deployment	Path-filtered per-service CI lanes, image scans and DAST; GitHub Actions CD over OIDC with per-owner release, smoke test and automatic restore
Evidence	Load tests that mostly read one completed video	Scaling, latency, recovery, release, security and reuse experiments with retained receipts

1.2 Business Needs

Small content teams, such as training providers, course teams, small studios and community publishers, keep needing the same video capabilities:

- secure upload of videos they own;
- conversion into MP4 and best-effort HLS for supported browser playback;
- visibility of processing progress and failures;
- playback restricted to the owner.

Today each application rebuilds these capabilities or copies another team's code, which duplicates effort and produces inconsistent security and operations. Transcoding is also CPU-heavy and bursty: when it shares a deployment unit with the user-facing API, a backlog of uploads competes with logins and status checks.

The business drivers follow from this:

1. **Reuse.** A new application gets upload, processing, status and owner-checked playback by calling one published API, with no copied code and no database access.
2. **Independent capacity.** Encoder capacity follows the backlog without scaling or redeploying the user-facing API.
3. **Repeatable, secure releases.** Each service is tested, scanned and released on its own from an immutable image, then verified by an automated upload-to-playback smoke test.
4. **Low idle cost.** The platform scales to zero, with a design target of about USD 3–5 a month at idle (AD-27).

In platform terms (section 3.1.4), the **seed** is an owned video asset and its processing job. **Producers** are content owners submitting videos through the web app or the Publisher CLI. **Consumers** are owners playing their processed output. **Integrators** build new clients against the OpenAPI contract. **Operators** release and scale the platform.

1.3 Stakeholders

The project has no external sponsor; it is team-proposed. Four planning personas, defined in week 1 (`project-roadmap/business-scope.json`), represent the business stakeholders. The persona cast illustration and its provenance are in the asset register.

Stakeholder	Type	Interest and need	Use cases
Content owners, e.g. course coordinator “Maya” (P-01)	Business: primary producer	Submit a folder of recordings once, see each result, retry failures and check playback	UC-1 to UC-5
Independent trainers, e.g. “Arun” (P-02)	Business: secondary producer and viewer	Upload one video in the browser, follow its status, watch the result; nobody else can see it	UC-1 to UC-4
Application developers, e.g. “Jules” (P-03)	Business/IT: integrator	Use the published API without copying service code or holding database credentials	UC-1, UC-3 to UC-5
Platform operators, e.g. “Sam” (P-04)	IT: operations	Change processing capacity and release Processing without touching Media; see failures and recover	UC-6, UC-7
Course lecturer and assessors (Prof Boon Kui Heng, NUS-ISS)	Academic sponsor and reviewer	Approved the team and the proposal; assesses the architecture, evidence and delivery	All
Team 17 members	Delivery team	Guruprasath Gopal (cloud and delivery), Ibrahim Mammadov (backend services), Nguyễn Kim Long (domain, contracts and verification)	All
FlickPond Project 1 team	Upstream code owner	Authors of the inherited baseline; credited for inherited capabilities	Baseline
Amazon Web Services	IT: cloud provider	Hosts the managed services in `ap-southeast-1`	Runtime
GitHub	IT: tooling provider	Source control, pull requests, Actions CI/CD, Pages for the study guide	Delivery

1.4 Project Scope

The scope is the proposal’s seven use cases (UC-1 to UC-7), delivered as two services, two clients and one AWS environment, at three levels:

- **Architected.** The two-context platform, its contracts, the AWS topology and the operating model.
- **Implemented.** The seven use cases, with three capabilities built behind flags (AD-22): an unattended encoder launcher, a Lambda short-encode lane (#93) and one database cluster per service (#94, AD-29). All three are enabled in the V2-live deployment.
- **Experimentally verified.** The results in section 4, each labelled with its environment: local, AWS V1 per-run stack, or the persistent V2-live environment.

The project ran in three phases.

Phase	Dates	Delivered
0. Baseline and design	22 Sep – 9 Oct	Team approval, proposal (4 Oct), lecturer feedback, frozen baseline, personas and use cases, DDD model, contracts
1. V1: two services on AWS, per-run stacks	10 – 18 Oct	Processing extracted, Media and Publisher adapted, AWS template and pipeline, cloud proof runs; tag `project2-v1` (18 Oct)
2. V2-live: one persistent environment	18 Oct – 16 Nov	`dc2-live`, automatic in-place CD, unattended 0 → 3 → 0 encoders, security and rollback proofs, freeze (20 Oct), presentation (26 Oct), final report (16 Nov)

The detailed scope sources are the proposal (§3), the use-case catalogue (`docs/architecture/use-case-design.md`) and the proposal-to-as-built deviation table (`docs/architecture/decisions.md` §4). Section 3 explains each deviation.

1.4.1 Functionality in scope

ID	Use case	Clients	Status
UC-1	Register, log in and upload an owned video	Web app, CLI	Implemented. At most 100 MiB; a 15-minute owner-scoped direct-to-S3 grant, byte verification and pinned source version
UC-2	Process the video into playable outputs	Platform for the owner	Implemented. MP4 plus best-effort HLS (360p/480p/720p, without upscaling); standard finite Fargate encoders and a Lambda short-clip lane
UC-3	Track status and play the result	Web app, CLI	Implemented. Owner-filtered Media projection and expiring MP4/HLS playback access
UC-4	Retry from the stored source	Web app, CLI	Implemented. A new fenced attempt uses the same pinned source without re-upload
UC-5	Bulk-publish a folder and report results	CLI	Implemented. One public-API workflow and retained outcome per file
UC-6	Scale processing capacity independently	Operator / platform	Implemented. V1 operator batches of 1–3; V2-live automatic finite Fargate launching within three reserved slots; short-lane admission is separate
UC-7	Release Processing on its own and demonstrate rollback	Operator / pipeline	Implemented. Per-owner image/alias promotion, smoke and restore of the previous runtime; schema migrations are not reversed

Section 6.2, Figure 29, shows the actors, the seven scoped goals and the grouped exclusions. Three supporting capabilities reuse the same aggregates and appear as variants in the use-case model: owner **edit** operations (clip, crop, scale, convert) that create a derived asset; **delete**, recorded as a tombstone in both contexts and followed by source cleanup; and operator-only **admin** routes.

1.4.2 Functionality out of scope

These requirements are excluded, as recorded in the proposal (§3.5):

- The rest of the Project 1 feature list: profiles, password reset, titles and tags, sharing modes and external videos.
- Viewer identities other than the owner: public sharing, student or LMS access, shared team ownership, tenants and billing. The contract accepts a `viewer` role that nothing issues.
- Search, public discovery, moderation workflows and product analytics.
- Crop or clip editor UI, thumbnails, 4K output, and direct/resumable uploads above 100 MiB.
- Kubernetes, general-purpose autoscaling, and multi-AZ database or multi-region failover.
- Historical data migration, malware scanning and full key rotation.
- A third client, a developer portal, live streaming, and a separate staging environment.

One planning exclusion changed. The proposal excluded autoscaling; V2-live starts finite encoders automatically from the SQL backlog, bounded by a hard cap of three (AD-27). This is bounded task launching, not a general autoscaling service.

1.4.3 User journey evidence

The presentation recordings illustrate the owner workflow. Figure 1 shows a completed video in the library player. Other frames show the library with playback, upload screen, accepted upload awaiting processing, Publisher batch results, foreign-owner and anonymous access

denials, and bounded encoder activity. The frames were captured on `dc2-live` on 23 October; sources and timecodes are in the asset register.

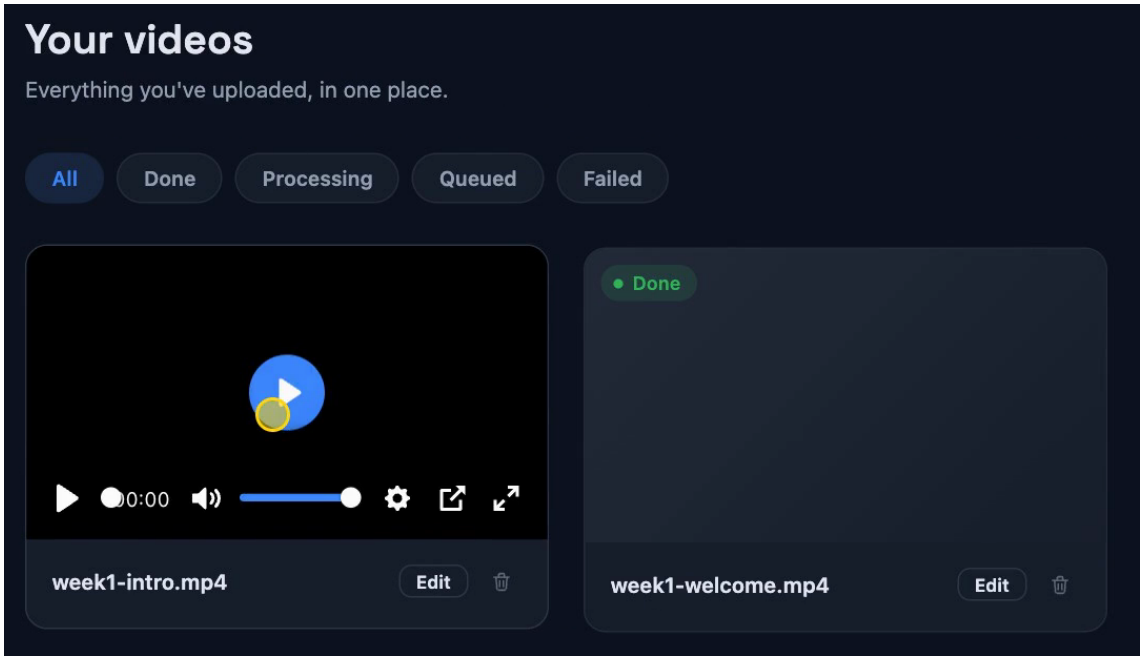


Figure 1. Owner library with the processed video loaded in the player and the other items marked Done; D1 recording, 00:14.5, cropped.

2. Project Conduct

2.1 Project Plan

The briefing expects about 10 person-days (pd) per participant. The proposal (§4.1) estimated **31 pd** for three members across ten work packages (WBS), each with a lead member. Project Metrics gives the effort per member against these estimates.

#	Work package	Lead	Estimate (pd)
1	Freeze baseline, reproducible local setup, fix the inherited scan finding	Guruprasath	2
2	Domain analysis, service contracts and OpenAPI documents	Long (team review)	3
3	Extract Processing: service, own database and migrations, workers and recovery task	Ibrahim	6
4	Adapt Media Management: asset records, Processing client, web app routes and retry	Ibrahim	3
5	Boundary security: identity, owner checks, database roles, presigned URLs, negative tests	Guruprasath	3
6	Publisher CLI and contract tests	Long	2
7	AWS environment, per-service deployment, pipeline updates, smoke test	Guruprasath	4
8	Experiments: scaling, load, recovery and playback; raw results	Long	3
9	Integration fixes and repeat checks	All	2
10	Presentation and final report	All	3

#	Work package	Lead	Estimate (pd)
	Total		**31**

2.1.1 Responsibility for analysis and design

Each use case's **analysis and design**, and each design problem, has one responsible member, following the team's three specialisms.

Item	Analysis and design responsibility
UC-1 Register, log in and upload an owned video	Ibrahim Mammadov
UC-2 Process the video into playable outputs	Ibrahim Mammadov
UC-3 Track status and play the result	Nguyễn Kim Long
UC-4 Retry from the stored source	Ibrahim Mammadov
UC-5 Bulk-publish a folder and report results	Nguyễn Kim Long
UC-6 Scale processing capacity independently	Guruprasath Gopal
UC-7 Release Processing on its own and demonstrate rollback	Guruprasath Gopal
DP-1 Bounded cloud capacity and safe per-owner release	Guruprasath Gopal
DP-2 Durable asynchronous Processing handoff	Ibrahim Mammadov
DP-3 Trusted owner identity and reusable public contract	Nguyễn Kim Long

2.1.2 Product backlog

GitHub Project 14 holds the delivery backlog. Each issue states an outcome, acceptance criteria and verification; the local contracts are also kept in `docs/execution/backlog.json` and the versioned AWS backlog directories. On 10 November the board held **118 items: 111 Done, 6 Todo and 1 In Progress**.

Field	Values and purpose
Workflow	Backlog, Ready, In progress, Review, Blocked, Done; authoritative delivery state
Wave	Wave 0–4; dependency order for local delivery, followed by AWS bundles and close-out
Priority	P0, P1, P2; implementation ordering
Persona	Maya, Arun, Jules, Sam, or combinations; intended beneficiary
Risk / Area / Sequence	Delivery risk, affected component and execution order
Status	GitHub's Todo / In Progress / Done summary

The Wave field orders the work, and the iterations below give it a calendar. The epic index is:

Delivery group	Epics and acceptance focus
Local baseline and integration	E0 #1 reviewed plan; E1 #2 reproducible workspace; E2 #3 Processing; E3 #4 Media; E4 #5 web/Publisher; E5 #6 integrated evidence
Required AWS V1	#41 domain APIs and S3 intake; #42 managed runtime and budget; #43 Redis-free delivery and releases; #44 acceptance and immutable baseline
Close-out	#178 visible pipeline; #179 cloud proof; #180 as-built documentation; #181 presentation/report; #195 microservice hardening
Persistent demonstration environment	#234 always-on in-place delivery and scale-to-zero, with #235–#238 for supporting delivery and verification
Measured improvements	E6 #33 performance experiments; #159 selective CI and full verification

Delivery group	Epics and acceptance focus
Optional experiments	#80 optional migrations (closed as not planned); #68 launcher task, adopted into V2-live

2.1.3 User stories

The stories come from the backlog and the planning personas in `project-roadmap/business-scope.json`. Section 6.2 describes the use cases and section 6.4 gives the detailed models.

Use case	Persona story	Backlog link
UC-1	As a content owner (Arun), I want to register, log in and upload my video, so that I can prepare my own content for playback.	Media #15, epic #4
UC-2	As a content owner (Arun), I want the platform to convert my video to playable outputs, so that I can use it without running a converter myself.	Processing #13, epic #3
UC-3	As a course coordinator (Maya), I want to follow status and play my completed recordings, so that I can check each result while keeping access restricted to my account.	Media #16, epic #4
UC-4	As a content owner (Arun), I want to retry a failed encode from its stored source, so that I can recover without uploading the video again.	Media #16, browser retry #19
UC-5	As a course coordinator (Maya), I want to bulk-publish a folder and receive an outcome for every file, so that I can prepare a course's recordings with one command; Jules integrates this workflow through the published API.	Publisher #18, epic #5
UC-6	As a platform operator (Sam), I want to change Processing capacity independently, so that a backlog can be handled without scaling or redeploying Media.	Scaling #22, launcher #68
UC-7	As a platform operator (Sam), I want to release Processing separately and restore a failed candidate, so that I can change it while preserving Media's release.	Release #23, pipeline epic #178

2.1.4 Iterations and sprint plans

The team delivered in five iterations that follow the proposal schedule (§4.2) and the delivery waves. The sprint plans, reviews and retrospectives below are compiled from the dated worklog (`docs/project2-worklog.txt`), review records, release manifest and issue register. Figure 2 shows the plan.

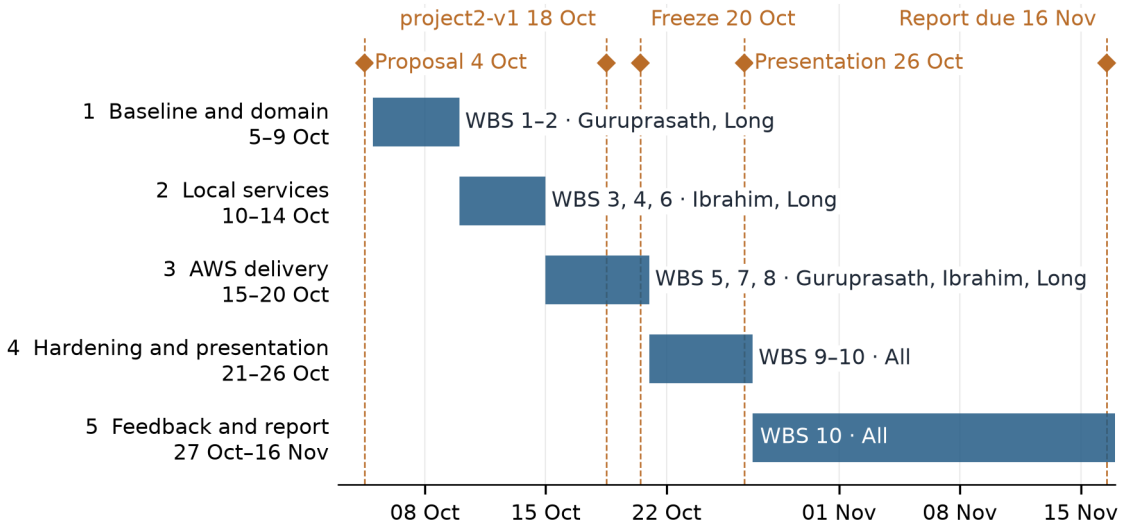


Figure 2. Iteration plan: windows, proposal work packages and leads, with the main milestones.

Iteration	Goal	Committed issues and epics	Outcome
1 Baseline and domain, 5–9 Oct	Freeze the baseline, define ownership boundaries and agree the seven use cases	WBS 1–2; decisions F-01 and D-01–D-07; later tracked by epics #1 and #7	Baseline and domain/design pack completed 7–9 Oct; personas, contracts and build preparation ready
2 Local services, 10–14 Oct	Deliver the independently owned services, the browser flow and an API-only Publisher, verified locally	E0–E5 (#1–#6); foundation #9–#11, Processing #12–#14, Media #15–#17, clients #18–#19, verification #20–#24	Processing 12 Oct; Media and Publisher 13 Oct; integrated local workflow, fault, release and experiment evidence to 14 Oct
3 AWS delivery, 15–20 Oct	Deploy both services on AWS, prove security and independent operations, freeze a demonstration release	V1 #41–#44; cloud proof #63–#67; close-out #178–#180; hardening #195; audit #258; manifest #255	Cloud proofs `r1017b`, `r1018b`, `r1019a`; `project2-v1` and `dc2-live` 18 Oct; CI/CD; security, scaling and rollback evidence; freeze 20 Oct
4 Hardening and presentation, 21–26 Oct	Resolve release blockers, record the workflows and present	Presentation #192, recordings #254; repairs #285/#287 via PRs #286/#288; short lane #93 and per-service databases #94	Verified repair `c7c489c` 21 Oct; short lane and database split 21–22 Oct; D1–D5 recorded; presented 26 Oct
5 Feedback and report, 27 Oct–16 Nov	Incorporate the presentation feedback and complete the final report	Report #193, epic #181, effort #191, recordings #254, course checks #257	W1–W6 feedback packets and report v1.1, submitted for 16 Nov

Iteration	Review: increment and evidence	Went well	Improvement and action
1	Decision record of 9 Oct: frozen baseline accepted; lecturer asked for detailed domain analysis	Domain and contract decisions gave implementation a shared starting point	Carry acceptance and environment prerequisites into executable issue contracts; Wave 0 planning review (`docs/execution/reviews/planning-rereview.md`)
2	PRs #28–#30; mixed batch of 30 valid files plus one refusal and a stored-source retry; experiment review of 14 Oct	Separate service branches and public-contract tests made integration straightforward	Preserve request identity and fence delayed responses when users change assets or sessions; client regressions kept (`clients-fixes.md`)
3	PR #194 V1 acceptance baseline; audit #258; PR #278 freeze and effort ledger	Cloud receipts tied each operational claim to its environment and source	Test duplicate messages, idle database behaviour and deployment scripts as operating behaviours; #270/#272 drain re-test, #273 idle fix, #265/#267 pipeline regressions
4	PRs #286, #288 and CD run 39545554380; D1–D5 clips; feedback of 26 Oct	A frozen baseline and recorded workflows made the presentation repeatable	Show object responsibilities, lane selection and security tools in diagrams; carried into the W1–W6 packets
5	Feedback mapped in `feedback-20261026/REPORT-CHANGES.md`; response in section 6.1	Retained source, receipts and feedback made the revision traceable	Keep issue bookkeeping in step with delivery; close tracking issues at submission

2.1.5 Burndown, burnup and issue reduction

Figures 3 to 5 are derived from GitHub issue timestamps captured on **10 November 2026 at 23:15 SGT**. The register holds 156 issues: 137 tasks and 19 parent epics. Counts use each issue's creation time and latest closure time.

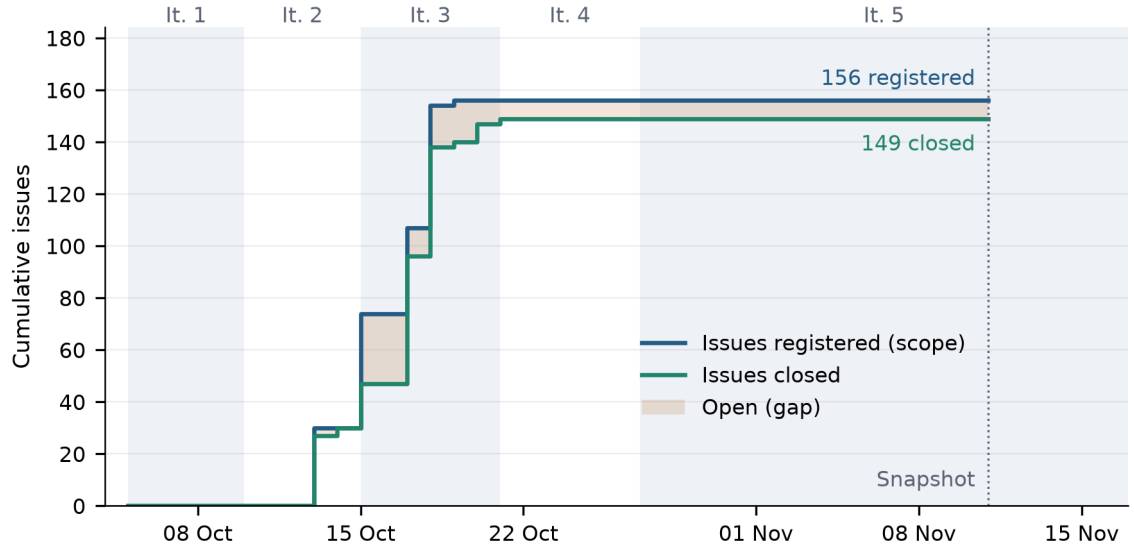


Figure 3. Overall burnup: issues registered (scope) against issues closed, with iteration windows shaded. The gap between the lines is the open work.

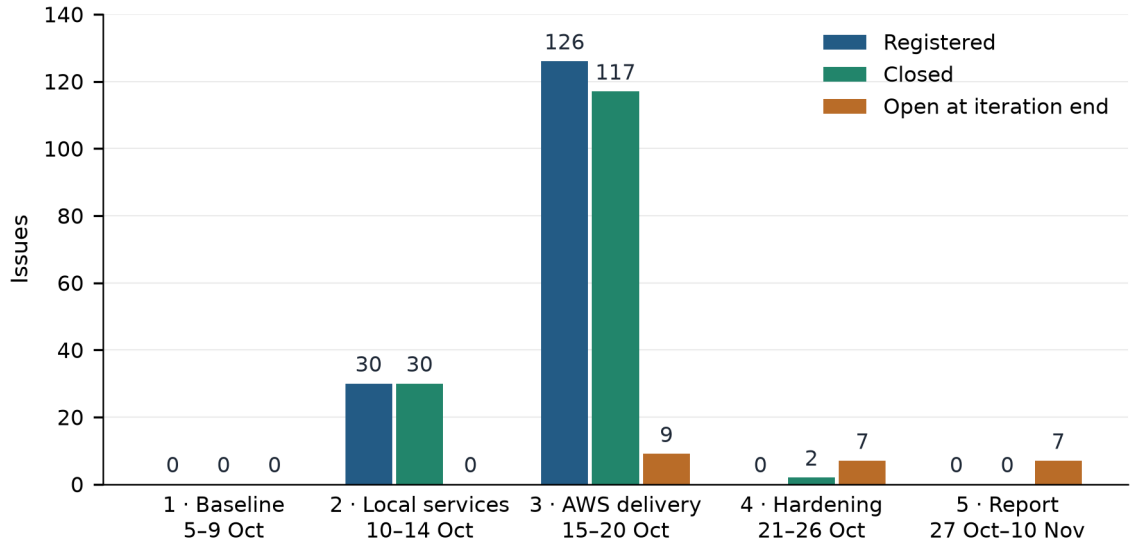


Figure 4. Per-iteration burndown: issues registered and closed in each iteration, and issues open at its end. Iteration 5 is counted to 10 November.

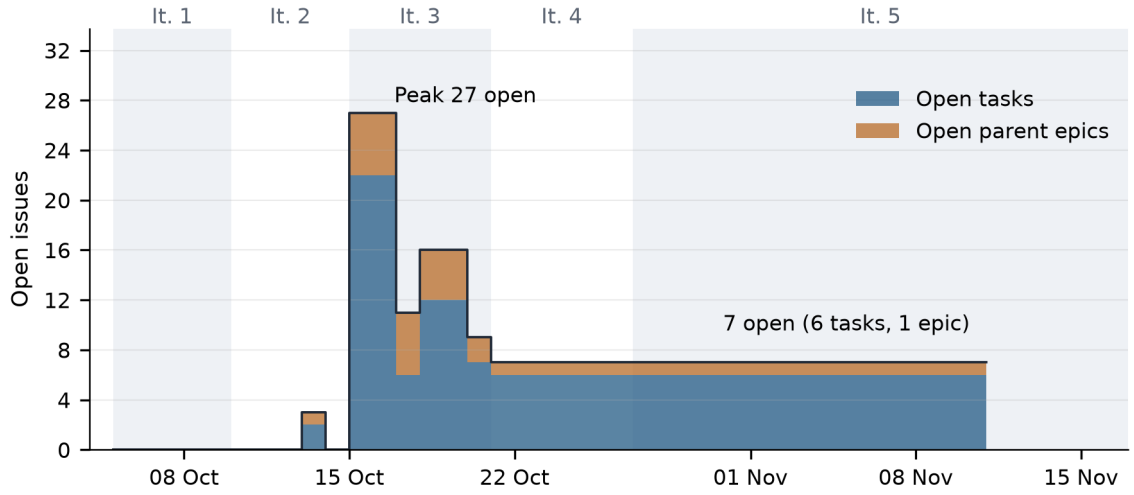


Figure 5. Issue reduction: open tasks and open parent epics over time. Seven remain open on 10 November.

Issues were registered from 13 October, once the wave plan existed, so Iteration 1 work appears under its later planning epics. Scope peaked during the AWS iteration, when 126 issues were registered and 117 closed. The seven open records are tracking issues for the presentation and report (#181, #191, #192, #193), recordings (#254), course checks (#257) and the audit index (#258). The data is in conduct-issues.csv and conduct-daily-issues.csv; `python tools/conduct_burndown.py` regenerates the four figures, and `--refresh` captures a new snapshot through `gh`.

2.2 Project Status

Report v1.1 is submitted for **16 November 2026**. All seven use cases are implemented and the release is frozen; status data was captured on 10 November.

Area	Status
Implementation	UC-1 to UC-7 implemented; tests and dated local and AWS experiments indexed in `docs/evidence-index.md`.
Release freeze	Demo freeze accepted 20 October at `daee44b`; `project2-v1` is immutable. Later engineering deliveries have their own verification addenda in `docs/release/demo-release-manifest.md`.
Live demonstration	`dc2-live` runs with finite standard encoders and separate short-lane admission. On 10 November the frontend, Media health and Processing health returned 200; these are liveness checks, and the health endpoints neither query nor wake Aurora, so they do not establish database readiness.
Presentation and report	Presented 26 October. v1.1 adds the requested DDD, deployment, component, schema, analysis/design and pipeline views; section 6.1 maps each remark to its response.
CI	The scheduled run on Main at `76282d3` passed on 1 November (41294802851). The scheduled run of 8 November (42468024855) stopped in its tool-installation steps; Full verification passed again on 11 November for PR #326 into Main (42949129268).
Outstanding	Close the seven tracking issues at submission; tear down `dc2-live` once after 16 November. Open measurement items are in section 3.4 (AISS-01, AISS-05).

Status is tracked in the effort ledger (`project-documentation/effort-ledger.md`, issue #191) and GitHub Project 14. The measured quality results are in section 4.

2.3 Project Metrics

2.3.1 Milestones planned against actual

Planned (proposal §4.2)	Milestone	Actual	Evidence
4 Oct	Proposal	Submitted 4 Oct	`Team17-Proposal.docx`
5–7 Oct	Baseline, domain/contracts and lecturer review	Completed 7–9 Oct	`project-roadmap/records/project-decisions.txt`
8–14 Oct	Local Processing, Media, web and Publisher	Processing 12 Oct; Media/Publisher 13 Oct; integrated evidence 14 Oct	PRs #28–#32; worklog
15–20 Oct	Security, AWS, pipeline and experiments	AWS tooling 15 Oct; cloud proofs 17–18 Oct; `project2-v1` and `dc2-live` 18 Oct; CD 18–19 Oct; freeze 20 Oct	Evidence index; PR #278; release manifest
21–25 Oct	Hardening and presentation preparation	Verified repair 21 Oct; short lane and database split 21–22 Oct; clips recorded 23 Oct	Engineering receipts; release addendum
26 Oct	Team 17 presentation	Presented 26 Oct; feedback recorded the same day	`presentation/feedback-20261026/FEEDBACK.md`
27 Oct–16 Nov	Feedback revision and final report	Report v1.1 submitted for 16 Nov	Section 6.1; report sources; #193

Baseline and design finished on 9 October, two days after the proposal's window. Every later milestone landed within its planned window.

2.3.2 Effort per member

Team 17 was formed with the professor's special permission as a team different from the Project 1 teams, so these figures cover Project 2 only. Estimates come from the proposal WBS (§4.1), with each package assigned to its lead and shared packages split evenly. Actual hours follow the work allocation in the team's effort ledger (#191). One person-day is 8 hours.

Member	#	Work package	Estimated hours	Actual (rough, team-agreed) hours
Guruprasath Gopal	1	Baseline, setup and inherited scan fix	16	14
	2	Domain, contracts and OpenAPI (review)	0	2
	5	Boundary security	24	5
	7	AWS environment, CI/CD pipeline and smoke test	32	46
	8	Experiments (cloud runs)	0	3
	9	Integration fixes and repeat checks	5	12
	10	Presentation and final report	8	14
	11	Release freeze and evidence index	0	1
Ibrahim Mammadov	1	Baseline, setup and inherited scan fix	0	2
	2	Domain, contracts and OpenAPI (review)	0	2
	3	Processing extraction	48	36
	4	Media adaptation	24	22
	5	Boundary security	0	13
	7	AWS environment (service images)	0	2
	9	Integration fixes and repeat checks	5	4
	10	Presentation and final report	8	14
Nguyễn Kim Long	2	Domain, contracts and OpenAPI	24	22
	3	Processing extraction (contract review)	0	2
	4	Media adaptation (contract review)	0	2
	6	Publisher CLI and contract tests	16	16
	8	Scaling, recovery and playback experiments	24	22
	9	Integration fixes and repeat checks	6	8
	10	Presentation and final report	8	17
	11	Release freeze and evidence index	0	7
Guruprasath Gopal total			**85**	**97**
Ibrahim Mammadov total			**85**	**95**
Nguyễn Kim Long total			**78**	**96**
Team total			**248 (31 pd)**	**288 (36 pd)**

Member	#	Work package	Estimated hours	Actual (rough, team-agreed) hours
Team average per member			**82.7**	**96.0**

The team spent about 16% more than the estimate. Work package 7 grew because the CD workflow (#178) was added on 18–19 October, and package 10 grew with the feedback revision of the report. Boundary security moved from Guruprasath to Ibrahim, next to the service code it protects, and package 11 (release freeze and evidence index) was added after the proposal.

2.3.3 Delivery and verification counts

Repository and Project counts were refreshed through `gh` on **10 November 2026**; the snapshot, board items and PR records are under `data/conduct-*`. Overlapping test suites are not summed.

Metric	Value and evidence
Issues	**156 total: 149 closed, 7 open**; 137 tasks and 19 parent epics
Pull requests	**173 total: 164 merged, 5 closed unmerged, 4 open**
Project 14	**118 items: 111 Done, 6 Todo, 1 In Progress**
Python fast suite	**2,986 passed, 3 skipped, 278 subtests** in the scheduled 1 Nov CI run at `76282d3`; real integrations passed 44 inherited, 47 Processing and 39 Media tests
V1 operational checks	**1,520 passed** in the 1 Nov offline control-plane job
Frontend	**177 passed** in the same 1 Nov CI run
Complete local regression	**3,697 Python tests plus 177 frontend tests passed**, 20 Oct validation at `514914a`
Coverage	Media 93.91%, Processing 93.90% combined statement/branch coverage; frontend statements 96.24% (`docs/results/engineering-20261020/local-validation-514914a.json`)

3. Solution Overview

This section describes the solution **as built** at the delivered engineering source. The proposal used synchronous `SubmitJob`, Redis/RQ and a service token. The major changes are recorded in `docs/architecture/decisions.md` and summarised here. AD-01 to AD-29 keep the register's numbering; AD-30 and AD-31 continue it for the report's load-distribution and database analyses (section 3.2).

Evidence labels used in this report:

- **[AWS]**: recorded in a live cloud receipt.
- **[Local]**: tests on real PostgreSQL 16 and FFmpeg, with Moto standing in for S3 and SQS.
- **[Code]**: read from source or the rendered CloudFormation template.
- **[NOT_RUN]**: implemented, never run against AWS.

3.1 Logical Architecture & Design

Architecture views. Each view answers a different question while keeping the same names, identities and ownership boundaries.

View	Question answered	Report home / course connection
Context and scenarios	Who uses the platform, for what?	Introduction and use-case models; ARCHSS / SAD
Logical and source structure	Which responsibilities and interfaces collaborate?	This section, tiers and components; ARCHSS / 4+1 logical and development concerns
Domain and consistency	Why these contexts and aggregate boundaries?	Platform Design; Platform Engineering
Runtime and trust	Who authorizes, commits and publishes?	Identity and intake flows here; Detailed Design; SAD / SDP
Deployment and capacity	Where do artifacts execute, and how is load bounded?	Physical Architecture; Cloud Native Solution Design
Information	What persists, with which keys and rules?	Persistence Design; Platform Engineering / SAD
Delivery and verification	How are changes admitted, released and checked?	DevOps; DevSecOps

The views organize context, logical, development, runtime, deployment, information and delivery concerns.

The architecturally significant elements, their types and their providers are listed below. “Team 17” marks custom code written in this project; “inherited” marks code reshaped from the FlickPond baseline. The logical view (Figure 6) groups them into tiers. Each domain service has its own API, domain and persistence layers. The contract and technical libraries are copied into each image at build time; domain models and databases stay separate.

Element	Type	Provider	Responsibility
FlickPond web app (`frontend/`)	Custom browser SPA (plain JavaScript, vendored hls.js and Plyr)	Team 17, adapted from inherited UI	Register, log in, direct upload, status polling, MP4/HLS playback, retry, edit, delete
Publisher CLI (`publisher/`)	Custom Python client (stdlib plus `httpx`)	Team 17 (new)	Bulk publish, status, output and retry through the public HTTPS API only
Media Management (`media/`)	Custom FastAPI service and Lambda handlers	Team 17	Accounts, ownership, upload grants and verification, the public job view, playback signing, deletion
Media authorizer (`media/authorizer.py`)	Custom Lambda REQUEST authorizer	Team 17 (part of Media)	Verifies the session cookie against the current user row; returns the trusted actor
Processing (`processing/`)	Custom FastAPI service and Lambda handlers	Team 17	Jobs, attempts, leases, retry and cancel, output descriptors, recovery, encoder launch
Finite encoder (`processing/worker/`, `media_engine/`)	Custom container task	Team 17; FFmpeg command builders inherited and reshaped	Claims one attempt, verifies the pinned source, encodes the primary output (MP4, or MKV/MP3 for a conversion edit) and the optional HLS ladder, writes outputs
Shared contract and technical libraries (`contracts/`, `delivery/`, `runtime_support/`)	Custom libraries copied into both images	Team 17	Versioned envelope and actor contracts, outbox/inbox and SQS adapters, Lambda DB sessions and settings
Operator and delivery tooling (`scripts/aws/v1/`, `scripts/ci/`)	Custom Python CLIs and shell	Team 17	Template rendering, deploy, session, release and restore, evidence harnesses, CD admission
FFmpeg and ffprobe	COTS open source	FFmpeg project (Debian package)	Probing and transcoding

Element	Type	Provider	Responsibility
FastAPI, Pydantic, SQLAlchemy, Alembic, asyncpg, boto3, PyJWT, bcrypt	Open-source libraries	Python community	Web framework, validation, ORM, migrations, PostgreSQL driver, AWS SDK, JWT, password hashing
Relational database (PostgreSQL 16)	Managed service	AWS (Aurora Serverless v2)	`media_db` and `processing_db`, one cluster each (AD-29)
Object storage	Managed service	AWS (S3)	Source, output and frontend buckets
Messaging	Managed service	AWS (SQS)	Four queues and four dead-letter queues with short encodes enabled
Edge, compute and operations	Managed services	AWS (CloudFront, API Gateway, Lambda, ECS Fargate, EventBridge, SSM, Secrets Manager, ECR, CloudWatch, Budgets, IAM)	Section 3.2
Source control, CI/CD and scanners	SaaS and open-source tools	GitHub (Actions, OIDC); Aqua Trivy, gitleaks, OWASP ZAP, Ruff	Section 5

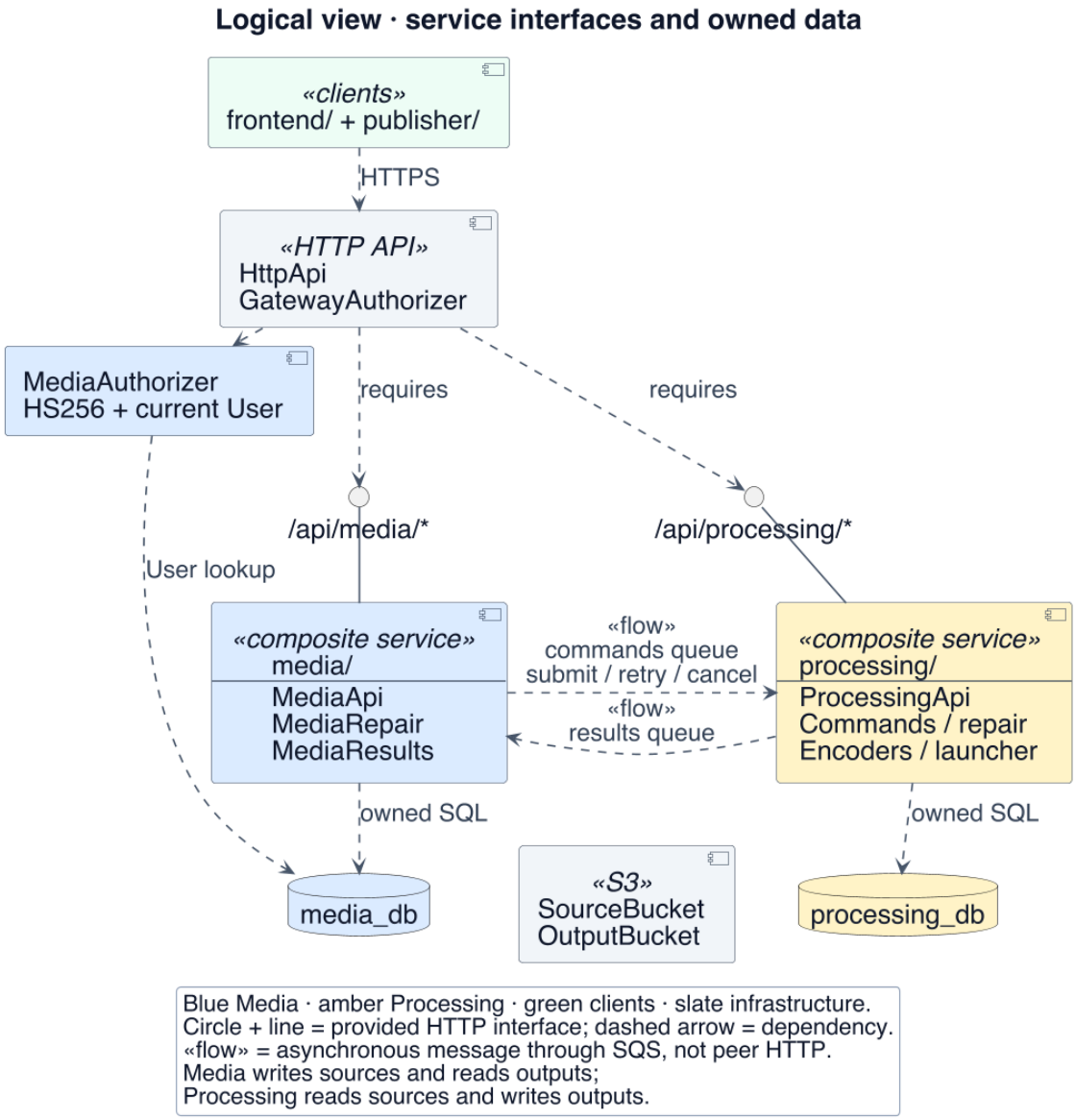


Figure 6. Logical responsibilities, provided HTTP interfaces, asynchronous business messages and owned data. The Media authorizer is a synchronous Gateway dependency for both protected APIs.

Media and Processing exchange business commands and state snapshots through SQS. Neither business handler calls the other's API or database. A protected Processing HTTP request still depends on Media's authorizer and current User lookup. Encoders commit state and result intent in Processing SQL; a Processing relay publishes the results, and no encoder sends queue messages directly.

Public request and identity boundary. The normal path is CloudFront `/api/*` → API Gateway HTTP API → the selected API's `live` alias. For protected routes, Gateway first invokes the Media-owned Lambda REQUEST authorizer (`GatewayAuthorizer` in `scripts/aws/v1/definitions.py`), with Cookie as identity source and a result-cache TTL of zero. The authorizer verifies the JWT and looks up the current Media `User`, denying on missing identity, unsupported role or lookup failure (`media/authorizer.py`). `contracts/gateway.py` then strips supplied owner/role headers and injects the authorizer's `Actor`; Media revalidates the cookie and current User (`media/api/deps.py`), while Processing trusts only the injected Actor (`processing/auth.py`). Health and Media register/login/logout bypass the hook. DP-3 develops this design.

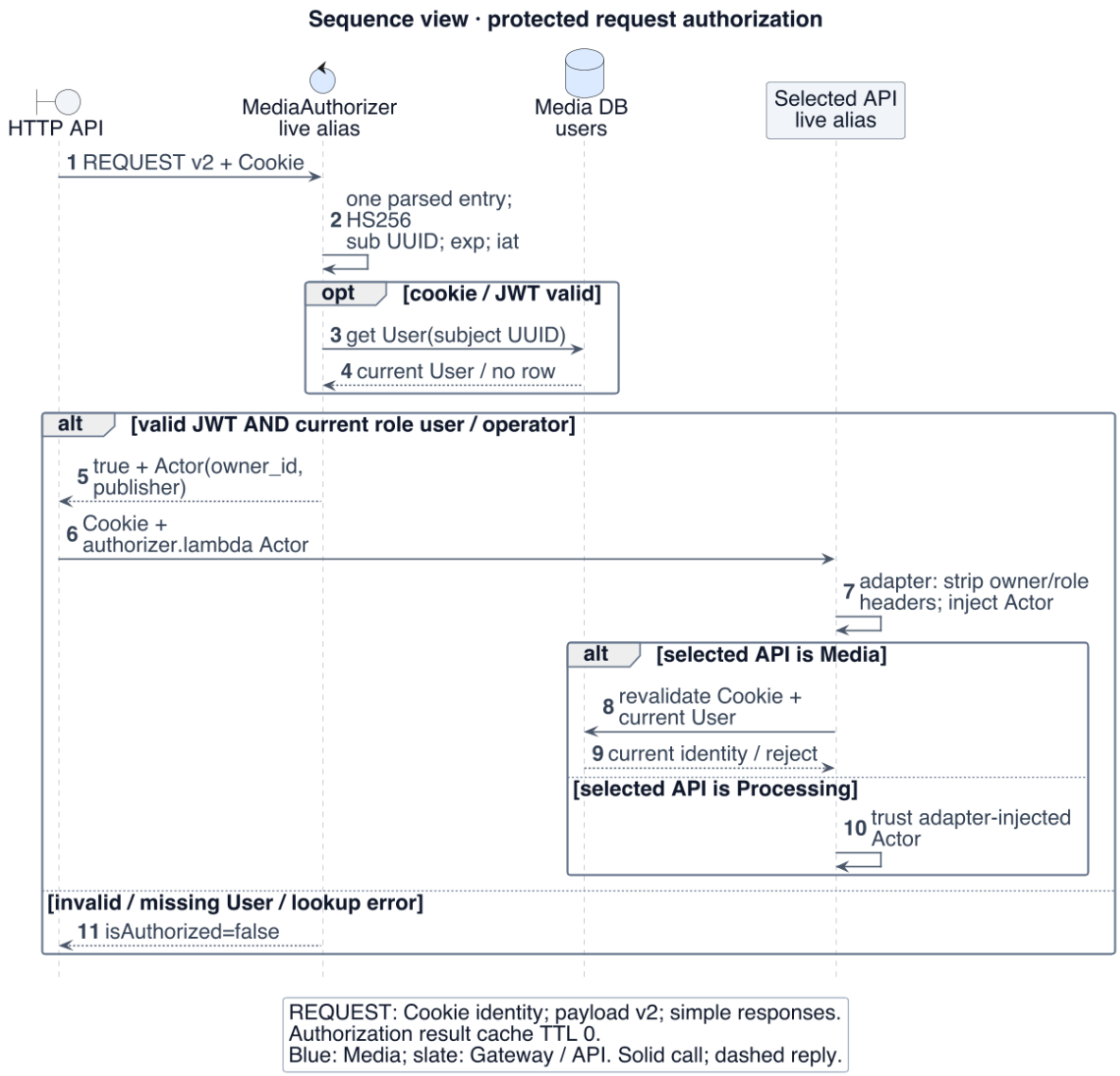


Figure 7. Gateway invokes the Media-owned REQUEST authorizer before its API integration. Media revalidates current identity; Processing uses the trusted Actor.

Durable upload and edit acceptance. Both paths commit an adopted source and its `submit` outbox intent in one Media SQL transaction before anything is published (Figure 8). Direct upload uses reserve → client POST to S3 → finalize (`media/services/direct_intake.py`). Edit copies an owned completed parent's primary output and adopts it as a new Asset; the original stays intact (`media/services/v1_lifecycle.py`). `MediaRepair` relays the command to SQS after the commit, and Processing commits its inbox, guard, Job/Attempt and outboxes before acknowledging it. Uncertain sends may repeat, so inbox identity checks and lifecycle fences are part of the design. Section 3.2.4 and DP-2 give the operation-level protocol.

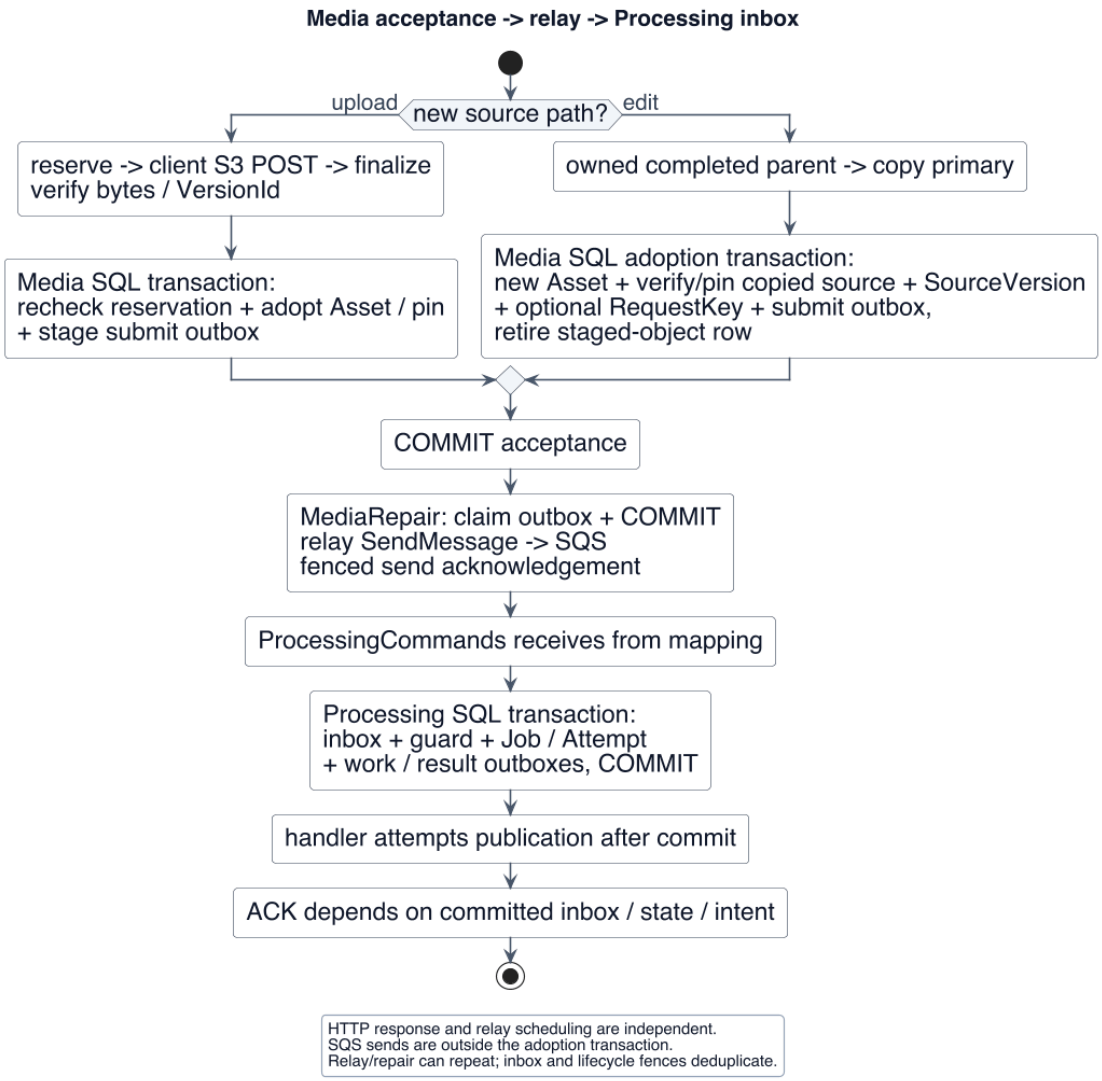


Figure 8. Upload and edit converge on SQL adoption plus outbox intent, followed by relay, SQS and the Processing inbox. SQS publication is outside the adoption transaction.

3.1.1 Key Architectural Decisions

The significant decisions reflected in the logical view are summarised below. AD-01–29 keep their register identities; the descriptions give the implemented mechanisms and the relevant alternatives.

Identifier	Decision	Rationale and trade-off	Status
AD-01	Extract **Processing** as the only new business service; Identity and Delivery stay inside Media	Processing has its own rules (claim, lease, attempt), lifecycle and CPU-bound scaling profile. Every playback or retry decision needs owner and asset together, so splitting Identity or Delivery out would turn single-database joins into cross-service calls. Four services were rejected as more contracts and operations than the need justifies.	Kept; boundary rationale below
AD-02	**Database per service**: `media_db` and `processing_db`, separate migration histories and roles (`*_runtime` DML only, `*_migrator` DDL, `*_owner` no login)	A real ownership boundary at low cost. V1 put both databases on one Aurora cluster; since 21 October each service has its own cluster (AD-29).	Kept (hosting refined by AD-12 and AD-29)
AD-09	**API Gateway HTTP API with a Media-owned Lambda authorizer**; the Gateway adapter builds the actor only from the authorizer context	Processing trusts the owner identity without holding Media's JWT secret or user table. TTL 0 requires a fresh authorization lookup for each protected request; Media also revalidates. The cost is a synchronous Media lookup even for Processing HTTP requests.	New (supersedes AD-06 service token)
AD-11	**SQS with an SQL outbox and inbox** in both databases, replacing Redis/RQ and synchronous `SubmitJob`	No always-on broker, and failure isolation: Media accepts uploads while Processing is down. The SQL-to-SQS handoff is not atomic, so repair is part of the design. Delivery is at least once, so consumers deduplicate.	New (supersedes AD-03; changes AD-04)
AD-14	**Direct-to-S3 intake**: a presigned, owner-scoped reservation, then a finalize step that verifies size, SHA-256 and file signature at a pinned S3 VersionId	No 100 MiB body passes through Gateway or Lambda, and the bytes that were verified are exactly the bytes encoded. Trade-off: a two-step protocol; the client hashes the file first, and versioned buckets need cleanup.	New (changes AD-05)
AD-21	**Media-owned projection** of Processing state, fenced by `state_version` and attempt	Media answers status and signs playback without querying Processing. The view is eventually consistent. Older or identical snapshots are no-ops; conflicting equal-version content is rejected. Higher versions still pass source, identity, attempt and transition checks (`media/projections.py`).	New
AD-24	**`contracts/` as published language and reused contract values**	Producers emit version 1 and `accepted_versions()` returns `{1}`. Additive N/N-1 evolution is the release policy for a future version 2.	New
AD-26	Retired peer-HTTP and Redis/RQ code **quarantined in `legacy/`** and excluded from images by a build-time check	The submission shows no dead dependency between the services. Trade-off: the legacy package and its CI lane remain to maintain.	New

3.1.2 Tiers and Layers

Tier	Elements	Layers and modules
Client	Browser SPA, Publisher CLI	Presentation only. Both call the same public API. The browser uploads bytes straight to S3 with a presigned POST.
Edge	CloudFront, API Gateway, Media authorizer	TLS termination, routing (`/api/media/*`, `/api/processing/*`), identity, stage throttle
Application	Media Management, Processing	**API layer**: `media/api/*`, `processing/v1_api.py`; transport is FastAPI behind `contracts/gateway.py`. **Domain and service layer**: `media/services/direct_intake.py`, `v1_lifecycle.py`, `media/projections.py`; `processing/messaging.py`, `lifecycle.py`, `launcher.py`. **Messaging layer**: `media/messaging.py`, `processing/messaging.py` over `delivery/`. **Persistence layer**: models, repositories and Alembic migrations per service. **Shared libraries**: `contracts/`, `delivery/`, `runtime_support/`.
Execution	Lambda handlers, Fargate encoder	Queue consumers, one-minute repair ticks, ECS and S3 event handlers, the launcher, the encoder and the short-encode Lambda (`processing/worker/` with `media_engine/`)
Data	Aurora, S3, SQS	`media_db`, `processing_db`; source, output and frontend buckets; commands, standard/short work and results queues

Figures 9 and 10 expand the layers into actual modules and exports. Storage and subprocess adapters perform I/O; `media_engine` builds commands and parses probe data without I/O. Interface symbols name provided exports.

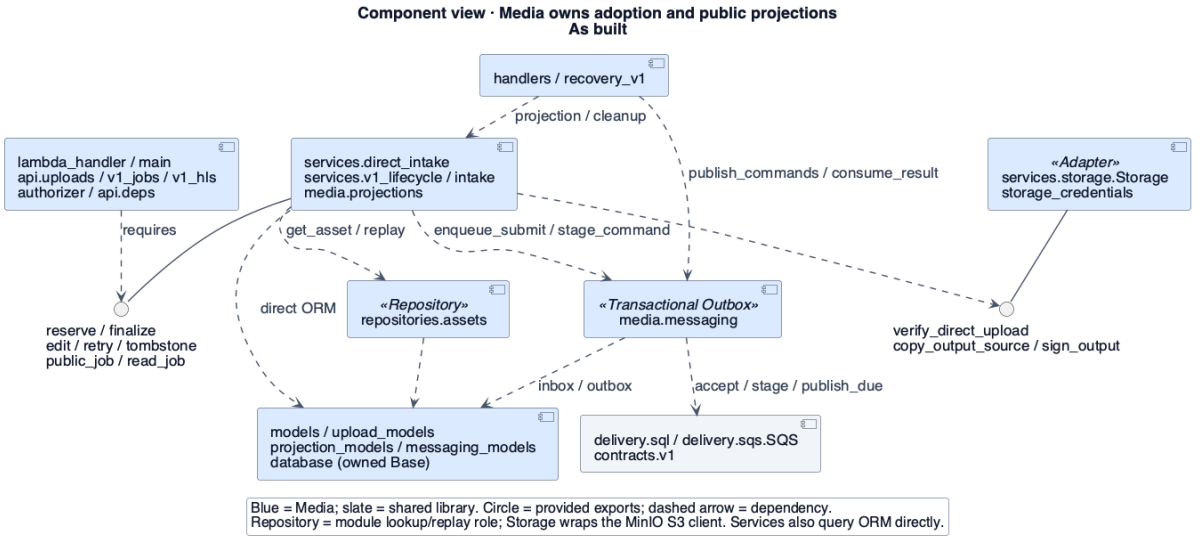


Figure 9. Media components: API entry points, adoption/projection policy, repository helpers, owned SQL and storage/messaging adapters.

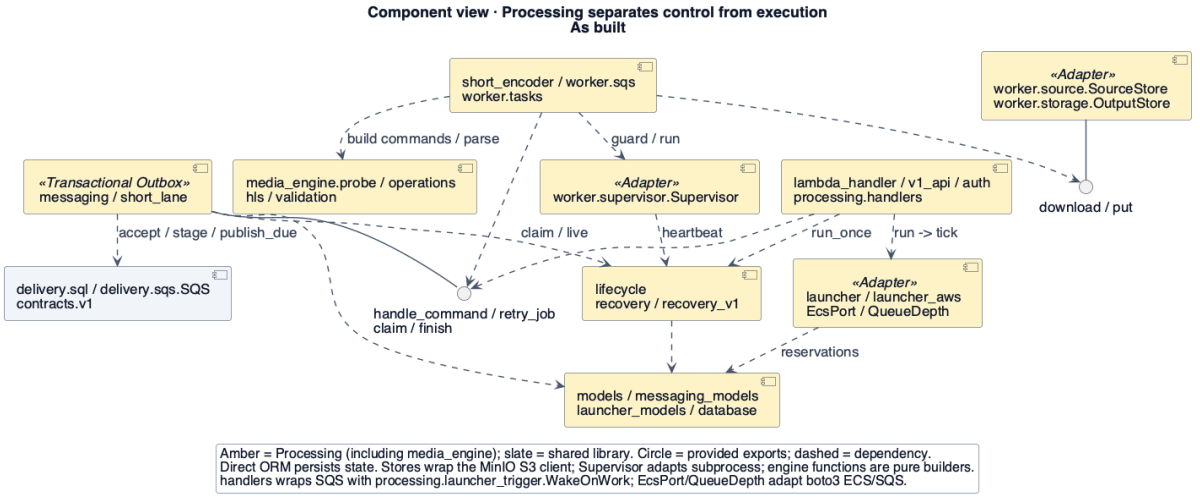


Figure 10. Processing components: command and lifecycle policy, shared encoder pipeline, storage/subprocess adapters and launcher capacity control.

The dependency rules are enforced, not just documented:

- Processing's API module imports no Media package.
- `deploy/v1/check_runtime.py` fails an image build if the opposite domain package, or any `legacy/` module, can be imported.
- The engine package is pure command construction with no I/O.

3.1.3 Nodes and Subsystems

Media and Processing are the two business subsystems. The API, repair, consumer and encoder processes below are execution roles of their owning context, not additional contexts. Section 3.2 maps them to images, aliases and authorities.

Subsystem	Responsibility	Nodes	Owner
Identity and intake	Accounts, JWT cookie, upload reservations, verified adoption of uploads	`MediaApi`, `MediaAuthorizer` Lambdas	Media
Playback	Owner-checked presigned MP4 URLs, HLS playlist and segment redirects	`MediaApi`	Media
Projection	Apply Processing results to Media's read model	`MediaResults` Lambda	Media
Media repair	Publish the outbox, expire grants, clean up deleted sources	`MediaRepair` Lambda (one-minute rule)	Media
Job orchestration	Job and attempt lifecycle, retry, cancel, leases, state versions	`ProcessingApi`, `ProcessingCommands` Lambdas	Processing
Recovery	Rebuild lost work, expire leases, react to stopped tasks and late outputs	`ProcessingRepair`, `ProcessingEcsEvents`, `ProcessingS3Events` Lambdas	Processing
Dispatch	Stage standard/short work using source size, feature flag and prior Lambda execution history	`messaging.emit_state` → `short_lane.work_lane` inside the state transaction	Processing
Standard capacity	Reconcile SQL backlog, visible standard work, tasks and reservations; reserve at most three slots	`ProcessingLauncher` Lambda	Processing
Short execution	Claim, download, probe admission, encode or fenced standard-lane redirect	`ProcessingShortEncoder`, Worker image, mapping cap 2	Processing
Standard execution	Long-poll, claim at most one Attempt, probe/encode/upload and fenced finish	Finite Fargate task, Worker image, 2 vCPU / 4 GiB, cap 3	Processing
Messaging	Envelope validation, outbox, inbox, SQS lanes, DLQ tooling	Library in both images	Shared libraries
Operations	Template, deploy, release and restore, evidence	CD runner or operator station	Platform

3.1.4 Platform Design

Domain-driven design explains the boundary requested in F-01 and developed after the 26 October feedback. Capabilities were grouped by vocabulary, invariants and required local consistency, then checked against code (`docs/architecture/ddd.md`).

Deriving the bounded contexts.

Capability	Vocabulary and rule	Required consistency	Context and boundary rationale
Account and ownership	User, current role, owner	Current User and owner-filtered Asset lookup	**Media Management** keeps account/access decisions near owned assets. Separate Identity would add a boundary without a separate implemented identity model.
Source adoption and edit	UploadReservation, verified pin, derived Asset	Reservation/new Asset/submit intent, or derived Asset/pin/request key/intent, commit locally	**Media Management** owns which bytes are accepted. Separate Upload would split the adoption transaction and require another durable handoff.
Status and playback	Public job, projection, playback link	Validate Processing state into an owned read model; authorize against Asset plus projection	**Media Management** keeps delivery as an access/view capability. A separate Delivery context would add remote coordination to every owned playback decision.
Execution and recovery	Job, Attempt, lease, heartbeat, outputs	Current Attempt/token/deadline and result/work intent agree at commit	**Processing** owns execution vocabulary and CPU-bound capacity independently of intake. A single service could implement both models, but would couple their execution and change responsibilities.
Pre-Job cancellation	SubmissionGuard, tombstone	Cancellation must survive a later or reordered submit	**Processing** owns durable execution refusal even before a Job exists. The guard is local policy support, not a third service.

Media owns users, assets, accepted sources and public projections in `media_db`. Processing owns jobs, attempts, leases, outputs and execution recovery in `processing_db`. Neither holds the other's database credentials. Processing stores owner/asset UUIDs without Media foreign keys; Media holds Processing identities in its projection. The split costs asynchronous integration and operational coordination, and in return each context keeps independent business state and execution recovery.

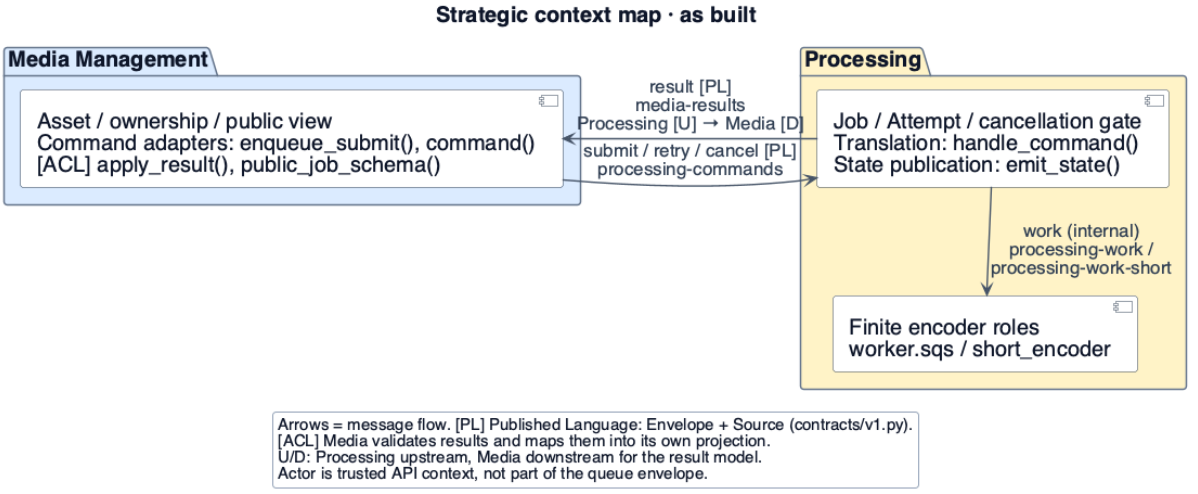


Figure 11. Context map: commands cross to Processing, results return through Media validation/mapping, and work remains inside Processing. PL denotes the published message language; U/D applies to the result model.

Context-map relationships. `contracts/v1.py` publishes Envelope and Source. Processing translates commands in `messaging.handle_command`; Media's `projections.apply_result` validates source, identity, attempt and output namespace, then maps the snapshot to its public model. This incoming validation and mapping is an anti-corruption layer. Processing is upstream and Media downstream for the **result vocabulary**. The services reuse contract and

technical libraries; the context map labels the implemented message and translation relationships. Actor is a separate trusted API-context value, not part of the queue envelope.

Ubiquitous language. Asset is an owned uploaded or edit-derived video; its UUID is Media's public `job_id`. Processing's ORM root is `Job`, with a distinct UUID and at most one Job per `asset_id`. An Attempt has its own UUID plus a positive per-Job number. `source_revision` is an integer, whereas S3 `version_id` identifies the exact object version. A source pin also includes bucket, key, SHA-256, byte size and media type. A lease is the token/expiry/deadline stored on Attempt and judged on the database clock. The projection is a validated Media snapshot, never a shared Job object.

Media domain model and invariants. Asset is the principal aggregate root. It owns accepted source identity and operations; an edit-derived pin is a `SourceVersion` value record keyed by Asset, and `contracts.v1.Source` is the frozen value object carrying the six pin fields. Direct uploads keep their pin on UploadReservation. User and pre-adoption UploadReservation have independent lifecycles, so neither is nested as an Asset child (Figures 12 and 13).

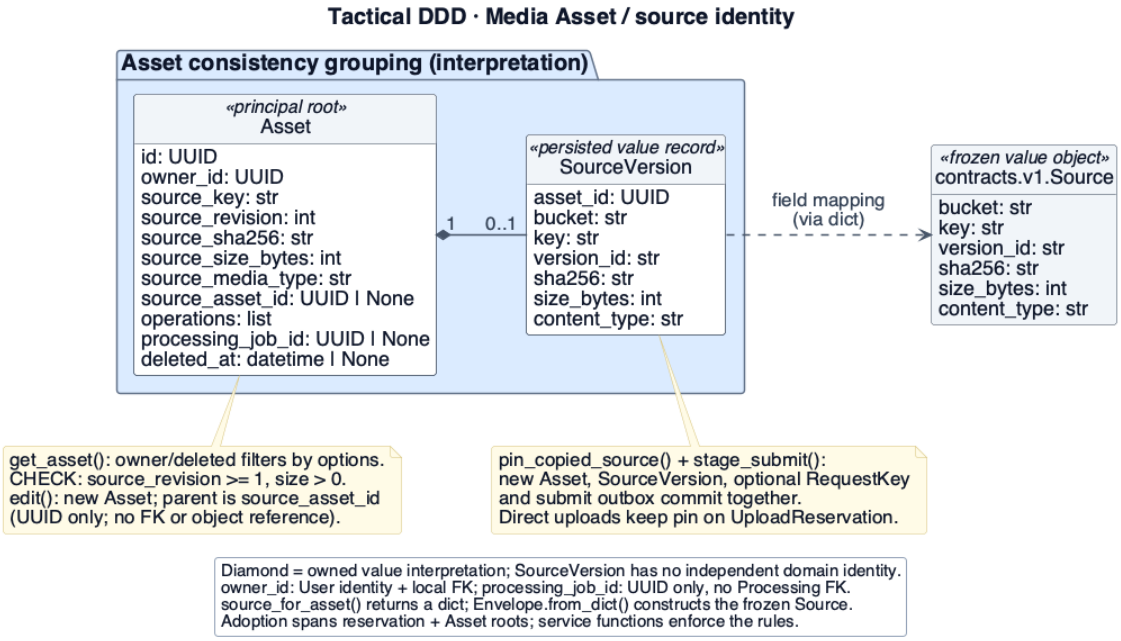


Figure 12. Media Asset root, typed source attributes and owned pin value; UUID references connect independent roots without sharing domain objects.

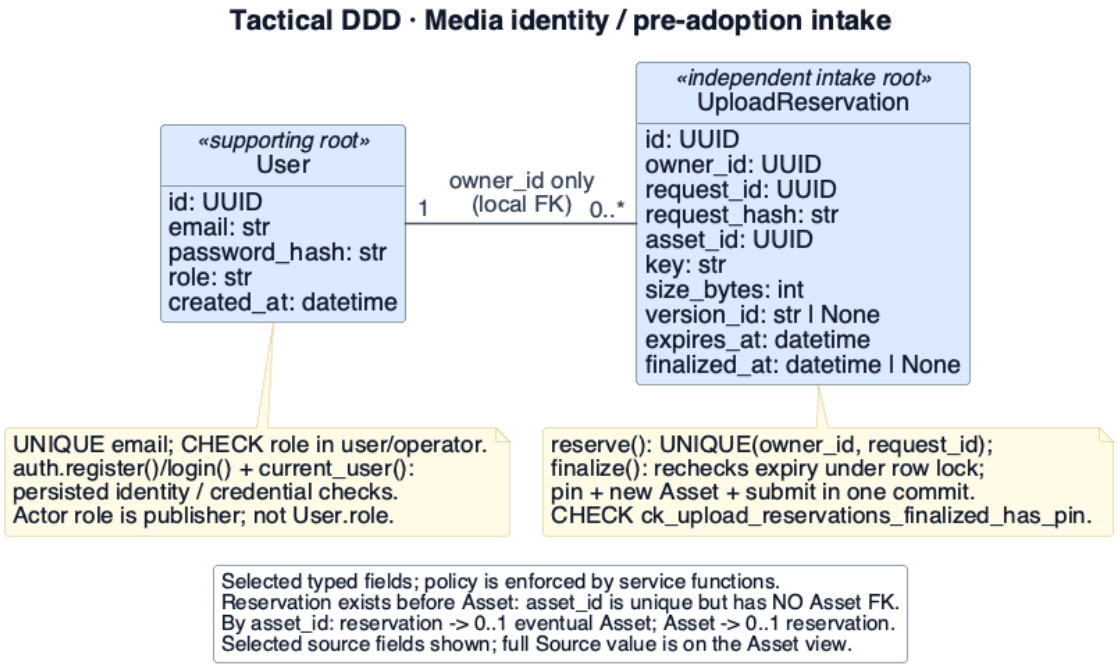


Figure 13. Independent Media identity and intake roots. A reservation exists before its eventual Asset; finalize spans both records in one local transaction.

The Media rules are enforced by `media/services/direct_intake.py`, `v1_lifecycle.py` and `media/repositories/assets.py`:

- Owner-filtered access returns 404 to another owner. User has unique email and a checked stored role (`user` or `operator`); the authorizer maps either to the API Actor role `publisher`.
- Reservation identity is unique per `(owner_id, request_id)`; replay must match its request hash. The 15-minute grant is adopted only after byte verification and an expiry recheck, and a finalized reservation holds a non-null S3 version pin.
- Editing creates a new Asset referring to its parent by UUID, with a copied, pinned source and normalized operations.
- Retry requires Media's failed projection and stages intent; Processing creates the new Attempt. Delete commits an Asset tombstone, cleanup intent and cancel outbox together.

Processing domain model and invariants. Job is the aggregate root for its numbered Attempt history and current execution authority. Attempt is a member entity with its own identity and lifecycle. Source data and output descriptors are embedded values; Output is a Pydantic value serialized to JSON, not another table (Figure 14).

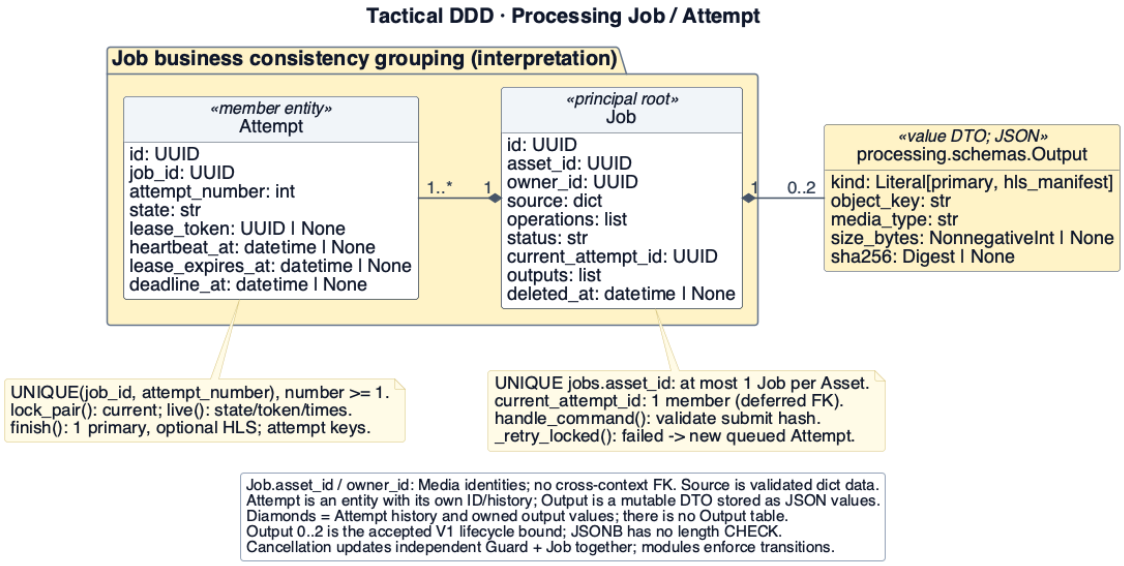


Figure 14. Processing Job root and Attempt entities, with typed lease attributes and output values. Current-attempt membership and lifecycle ordering are checked by service functions.

`processing/messaging.py` and `processing/lifecycle.py` enforce the following rules:

- Unique `Job.asset_id` permits at most one Job per Asset; a repeated submit must match the canonical source/operations identity.
- Attempt numbers are positive and unique per Job. `lock_pair()` locks Job, reads Attempt and verifies current membership. A claim acquires a token and DB-clock lease; heartbeat and finish require that token to be live and within the fixed deadline.
- A finite task binds to at most one Attempt. Retry and the fenced short-lane redirect (section 3.2) create a new numbered Attempt rather than reusing the claimed one.
- Successful finish requires exactly one verified primary output and optionally one HLS manifest in the current attempt's namespace. Old tokens cannot finish a successor.
- State version increases with emitted state, committed together with outgoing intent. A pre-Job cancel persists a SubmissionGuard tombstone, so a late submit cannot resurrect execution (supporting records: execution authority and cancellation model).

Supporting consistency boundaries. The ORM records declare fields and constraints; service functions enforce the business rules. Each transaction stays inside one context; none spans both databases.

Boundary / record	Identity and invariant	Enforcement
Media request/projection	RequestKey PK `(owner_id, operation, request_key)` maps a request hash to Asset; ProcessingProjection is keyed by Asset, with unique internal Job UUID	Replay helpers and `apply_result`: older/identical results are no-ops; conflicting equal versions fail; higher versions pass transition/source checks
Media source cleanup	CleanupIntent is keyed by Asset; StagedObject by source key and can exist before Asset	Tombstone/cancel confirmation and repeated cleanup preserve authority over uncertain or late writes
Processing cancellation/binding	SubmissionGuard is keyed by asset with nullable local Job FK; AttemptTask is keyed by Attempt with unique task name	Guard owner check and tombstone; atomic claim binding prevents two executor identities sharing an Attempt
Processing retry/state	RetryRequest has unique `(job_id, request_id)`; JobMessaging is keyed by Job with `state_version: int`	Retry and `emit_state` commit Attempt/version/outbox changes together

Boundary / record	Identity and invariant	Enforcement
Owned delivery ledgers	Each context has separate inbox/outbox rows, `message_id: UUID` PK and body hash	`delivery.sql.accept/stage/publish_due` ; same ID/body is a replay, changed body is a conflict (Figure 24; DP-2)
Launcher technical control	Singleton `LauncherControl.id = 1`; reservation token PK and unique `started_by`	`_reserve` serializes capacity before RunTask; ambiguous replies retain slots; reservations are separate from Attempt claims (Figure 23; UC-6)

Domain changes and messages. The platform records current state, not an event-sourced history. Accepted Processing changes emit a full `result` snapshot: submit, claim, success/failure, retry, cancel, interruption and redirect. Heartbeat emits no result. Five message kinds use four transport lanes when short execution is enabled:

Kind / intent	Logical queue	Producer → consumer	Contents / cause
`submit`	`processing-commands`	Media adoption outbox, relayed by MediaRepair → ProcessingCommands	Source pin and operations for a newly accepted Asset
`retry`	`processing-commands`	Media retry outbox, relayed by MediaRepair → ProcessingCommands	Failed projection authorizes intent over the accepted source
`cancel`	`processing-commands`	Media tombstone outbox, relayed by MediaRepair → ProcessingCommands	Empty payload; source absent in the current producer; Job ID if known
`work`	`processing-work`	Processing outbox/relay → finite Fargate encoder	Queued current Attempt, source and operations; internal to Processing
`work`	`processing-work-short`	Processing outbox/relay → ProcessingShortEncoder	Same work contract, selected by dispatch; internal to Processing
`result`	`media-results`	Processing state outbox/relay → MediaResults	Status, outputs, error and attempt number at a state version

Queue bodies carry identities and descriptors, never media bytes, credentials or signed URLs. Queue policies and per-handler IAM roles establish producer authority. Persistence Design (section 3.2.3) gives the Envelope and Source field types.

Platform producers and consumers.

Platform role	As built	Interaction
Seed	Owned Asset plus its Processing Job, linked by `asset_id`	Upload/finalize or edit adoption creates the source of processing value; Job has a distinct internal UUID
Producers	Web app, one file at a time; Publisher CLI, batches	Public HTTPS API for reserve/finalize/status/retry/edit/delete; bytes use the granted direct S3 POST
Consumers	Owners playing their own processed output	Signed MP4 with Range; HLS playlist through Media and segment redirects to signed S3 URLs
Integrators	Developers building additional clients	Two OpenAPI documents, published contracts and Publisher reference client; tests enforce no service imports or DB credentials in Publisher
Operators	Team 17	Delivery/session/release/restore tooling and evidence harnesses; unattended launch is a Processing execution role

3.2 Physical Architecture & Design

The deployment targets one AWS account in **ap-southeast-1 (Singapore)**, with CloudFront at the global edge. `scripts/aws/v1/definitions.py` generates CloudFormation; tests check that the checked-in default template is current. The figures show the persistent mode with

applications, unattended launch, short encodes and separate databases enabled, as configured in source on 10 November.

- **V1** used per-run October proof stacks (`r1017b`, `r1018b`, `r1019a`, tag `project2-v1`), closed outside booked sessions and deleted afterwards.
- **V2-live**, introduced 18 October, uses persistent `dc2-live`, updated by qualifying Main CI/CD and configured for idle compute to scale to zero. The 21 October receipts record short execution and the database split.

Figure 15 shows runtime placement and Figure 16 the network. Supporting views show the image artifacts, the queue lanes and consumers and the S3 byte paths.

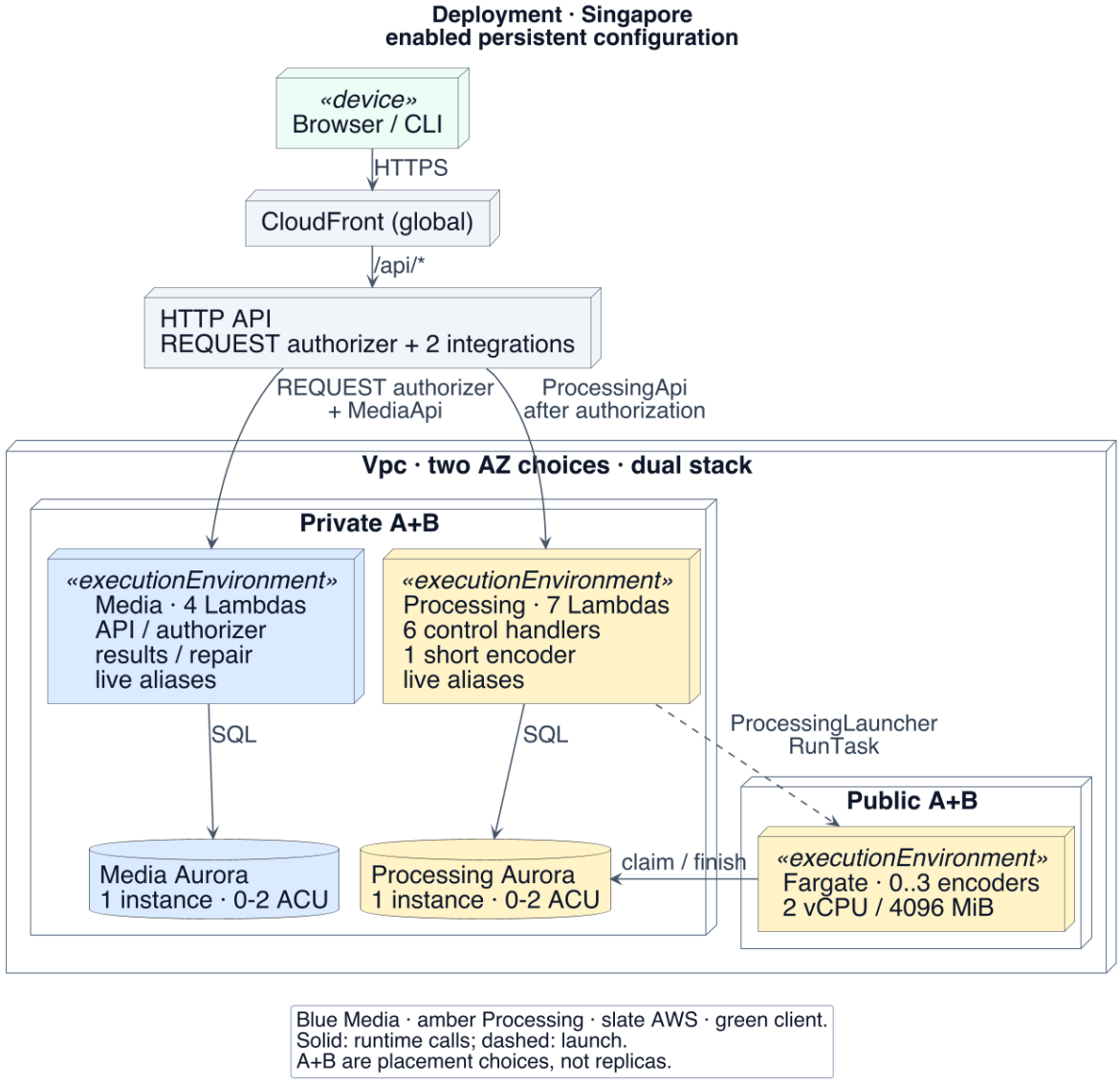


Figure 15. Configured persistent deployment in Singapore: Gateway authorizer and API integrations, private Lambda aliases, separate Aurora instances and launcher-created finite Fargate tasks. A+B denote subnet choices, not replica placement.

The table maps each deployed component to its artifact, trigger, placement and authority. Every Lambda has its own IAM role, published version and `live` alias; triggers invoke the alias.

Component	Image → entry point	Trigger	Placement and configured bound	Authority
Web app	Static files in FrontendBucket	CloudFront OAC GET	Global HTTPS edge; `/api/*` forwarded to the regional HTTP API	Read-only static assets
Public API	HTTP API, two integrations and the Media REQUEST authorizer	Client HTTPS via CloudFront	Stage throttle 20 requests/s, burst 50; 29 s integration timeout	Invokes `live` aliases only
MediaApi	Media → `media.lambda_handler.handler`	Gateway Media integration	Private A+B	Media SQL; owned S3 verify, copy and signing; no SQS send
MediaAuthorizer	Media → `media.lambda_handler.authorizer`	Gateway REQUEST authorizer	Private A+B	JWT secret; current User lookup
MediaResults	Media → `media.handlers.result_handler`	Results mapping, batch 10, cap 2	Private A+B	Media inbox, projection and cleanup
MediaRepair	Media → `media.handlers.repair_handler`	One-minute rule	Private A+B	Media outbox sends; source recovery
ProcessingApi	Processing → `processing.lambda_handler.handler`	Gateway Processing integration	Private A+B	Trusted Actor; Processing SQL; no S3 or SQS grant
ProcessingCommands	Processing → `processing.handlers.command_handler`	Commands mapping, batch 10, cap 2	Private A+B	Inbox, guard, Job/Attempt and work/result outboxes
ProcessingRepair, ProcessingEcsEvents, ProcessingS3Events	Processing → `repair_handler`, `ecs_handler`, `s3_handler`	One-minute rule; ECS STOPPED; output Object Created	Private A+B	Recovery, cleanup and outbox publication
ProcessingLauncher	Processing → `processing.handlers.launcher_handler`	One-minute rule, or async invocation after a standard send	Private A+B	Reservation rows, standard queue visibility, ECS RunTask
ProcessingShortEncoder	Worker → `short_encoder_handler` (`python -m awslambdaric`)	Short-work mapping, batch 1, cap 2	Private A+B; 3008 MB, 900 s, 2048 MB scratch	Claim, lease, finish; pinned S3 GET and output PUT
Standard encoder	Worker → `processing.v1_worker` (`worker.sqs.main`); finite Fargate task, no ECS service	Launcher RunTask; the task polls standard work	Public A+B; 2 vCPU / 4096 MiB; cap 3, zero when idle; no inbound listener	Same claim/finish authority; receive and delete work, no send
Operations tasks	Alembic from each domain image; `scripts.aws.v1.bootstrap` from Worker	One-off RunTask from deployment tooling	Public A+B; 256 CPU units / 512 MiB	Separate migrator roles; bootstrap only during provisioning
Databases	Aurora PostgreSQL 16 Serverless v2: `media_db` and `processing_db`, separate clusters	–	Private subnet group A+B; one instance per cluster, 0–2 ACU, 300 s auto-pause; encrypted, forced TLS	`*_runtime` DML and `*_migrator` DDL roles

Component	Image → entry point	Trigger	Placement and configured bound	Authority
Object storage	FrontendBucket, SourceBucket, OutputBucket	–	Regional; versioned, SSE-S3, public access blocked, TLS-only	Role-scoped bucket grants
Messaging	Commands, standard work, short work and results: 4 Standard SQS queues plus 4 DLQs	–	Regional, outside the VPC; DLQ after 5 receives; visibility 420 s (short 960 s)	Role-scoped queue policies
Scheduling and hints	Five EventBridge rules: three one-minute, ECS STOPPED, output Object Created	–	Regional	Hints lead back to SQL-authoritative recovery
Configuration and images	Five SecureString and four String SSM parameters; RDS-managed master secrets; three ECR repositories	–	Regional	Roles read only their parameters; image digests identify releases
Visibility	CloudWatch logs, ServicesDashboard, 15 alarms (11 function Errors + 4 DLQs), USD 20 budget	–	Seven-day logs; no paging action	–

Both Processing encoders share the Worker artifact and FFmpeg pipeline. Standard workers override receive visibility to 60 s and renew it while encoding. SQL remains authoritative: an uploaded object becomes a job only after Media finalize and adoption.

Network and identity. One dual-stack VPC (`10.62.0.0/16`) spans two AZ choices: Private A/B (`10.62.0.0/24`, `10.62.1.0/24`) and Public A/B (`10.62.2.0/24`, `10.62.3.0/24`), each with an IPv6 /64. All eleven Lambdas attach to Private A+B and reach dual-stack AWS APIs over IPv6 through an egress-only internet gateway; private subnets have no IPv4 default route or NAT gateway. Finite workers and operations tasks use public subnets with temporary public IPv4 for HTTPS egress. All four route tables use the S3 gateway endpoint. No compute security group admits inbound traffic. Media DB accepts the Lambda and migration groups; Processing DB also accepts the worker group on 5432. IAM parameter access and PostgreSQL roles separate the domains within the shared Lambda groups (Figure 16).

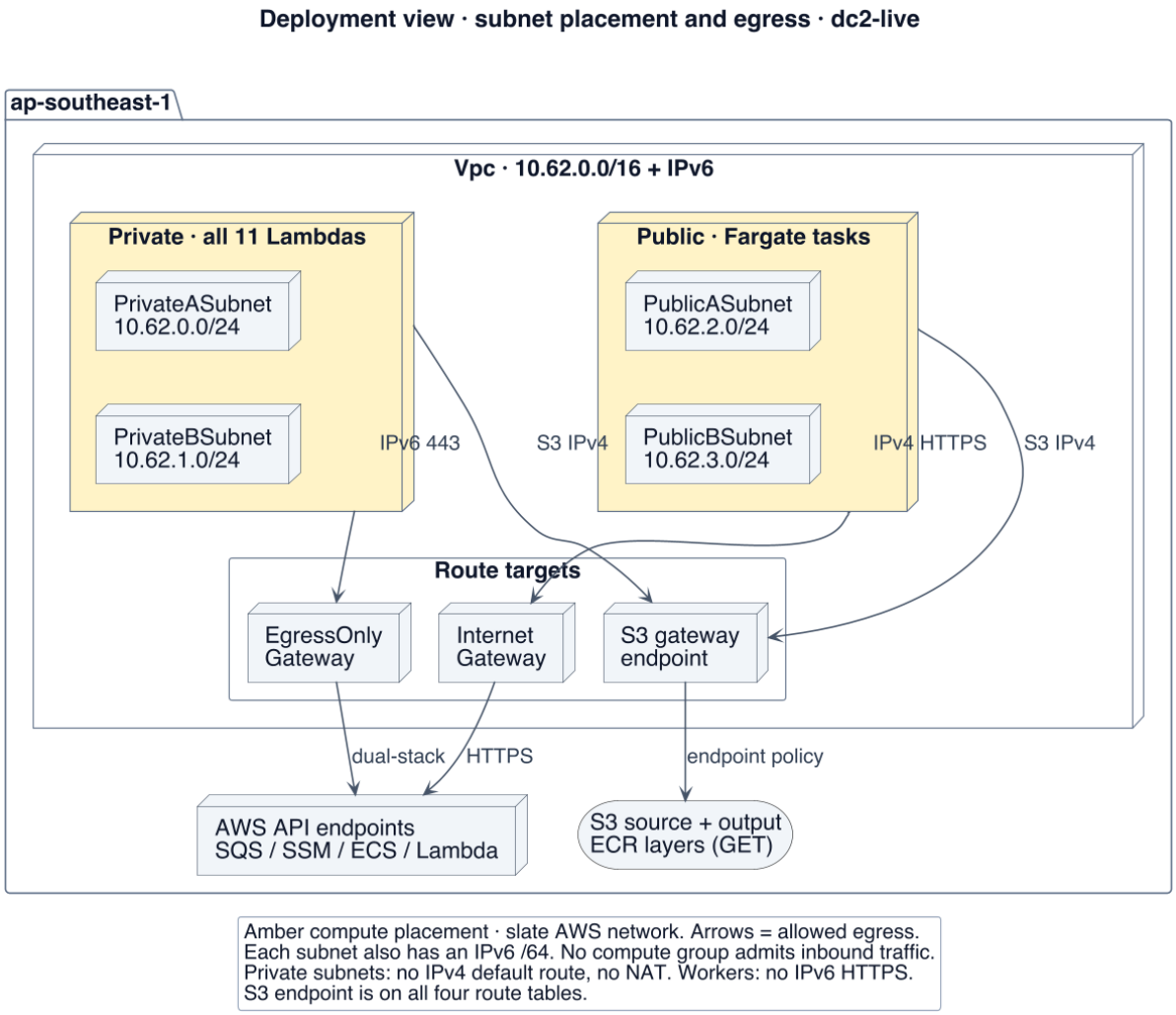


Figure 16. Network placement and allowed egress. Private Lambda IPv6 egress and public-task IPv4 HTTPS use different routes; all four route tables share the S3 gateway endpoint.

GitHub Actions uses temporary OIDC credentials through the `aws` environment's `dc2-github-deploy` role (`deploy/aws/v1/github-oidc.yaml`, AD-17). Runtime IAM roles, SSM parameter scopes and SQL DML/migrator roles are distinct authorities. Encoder source-read access covers the bucket; accepted source and owner/job/attempt checks bind execution before S3 access (AD-20).

3.2.1 Key Architectural Decisions

These decisions are reflected in the physical view. Section 3.1.1 holds the logical ones.

Identifier	Decision	Rationale and trade-off	Status
AD-10	**Lambda images for domain handlers; Worker artifact for both encoders**; handler command and `live` alias per function	Scale-to-zero and per-domain releases. Cold starts, one pooled SQL connection per warm Lambda environment and the 29 s Gateway timeout. Alternative: warm container API fleet (AD-30).	New, refines AD-07
AD-12	**Aurora Serverless v2**, one instance per cluster, default 0–2 ACU	Low idle compute and role separation; no reader failover. Paused-resume read: 4.7 s **[AWS]**. V1 shared a cluster; AD-29 split it.	New, refines AD-02
AD-29	**One cluster per service**, split in place	Separates failure and capacity boundaries. Processing received a copy-on-write clone; cleanup removed peer databases. A rehearsal and pre-split snapshot preceded the CD delivery on 21 October (`docs/results/engineering-20261021/database-split-rehearsal.json`). Additional storage and wake cost; Processing must be idle during the split.	New, refines AD-02/12
AD-13	**Finite standard encoders**, cap 3, zero when idle; optional short Lambda cap 2	One claimed Attempt per finite task. Always-on ECS workers and AWS Batch were rejected for idle cost and additional operations. Accepts startup latency and explicit launcher recovery.	New
AD-15	**CloudFront as normal frontend/API entry**	Managed HTTPS and one cookie origin. Generated hostname; execute-api is also reachable (AISS-02).	New
AD-16	**Owner image-digest/alias release and restore**	Processing can release independently. A failed owner-readiness smoke restores aliases; the later playback smoke is a separate boundary (section 5.3). Image-only restore requires additive schema changes.	New
AD-17	**GitHub Actions OIDC**, no stored cloud key	Repeatable delivery through the branch-restricted `aws` environment; promotion is approved at the Main pull request. The deploy role is broad (AdministratorAccess with a region lock; AISS-04).	New
AD-18	**Private Lambda IPv6 egress, S3 gateway, no NAT**	Avoids NAT and interface-endpoint idle charges. Dependencies require IPv6 or gateway reachability; finite tasks use public IPv4 for image pulls.	New
AD-20	**Per-handler IAM/SSM and queue policies**	Native credentials establish authority. MediaApi/ProcessingApi have no SQS grant; finite workers receive and delete but cannot send. Bucket-wide source read relies on code-level owner binding.	New, supersedes AD-06
AD-23	**One stack, infrastructure then digest-pinned applications**	Small operational surface; infrastructure changes together while code releases remain independent.	New
AD-25	**Per-service alarms/dashboard**	Owner-tagged function-error and DLQ alarms locate failures. No paging target is wired.	New
AD-27	**Persistent environment, update in place and zero idle compute**	Avoids per-check deploy and teardown. SQL plus standard-queue visibility, tasks and reservations drive the launcher. Throttle, cap and the USD 20 budget bound operation.	New, supersedes AD-19

AD-30 — distribute HTTP through Gateway/Lambda and encoding through queues plus finite capacity; retain no ALB. HTTP requests select the Media or Processing Lambda integration; Lambda supplies concurrent invocations. SQS buffers asynchronous commands, results and work. Mapped consumers have explicit concurrency bounds, while standard encoders are competing receivers under a launcher cap. The generated stack has no ALB,

HTTP target group or ECS service, and the encoder exposes no HTTP listener. This arrangement keeps idle compute at zero and accepts cold starts, finite-task startup latency and custom capacity and recovery logic.

The alternative, an **ALB plus ECS HTTP service**, offers health-checked warm HTTP targets, at the cost of standing infrastructure, service operations and a separate queue-capacity policy. AD-30 compares the alternatives against the deployed workload and idle-cost goals; it should be reassessed when sustained warm HTTP containers are needed.

Load distribution and execution admission.

Mechanism	Configured distribution / bound	Trade-off
HTTP	Gateway route integration, throttle 20/s and burst 50; Lambda concurrent invocations	Shared account quota; no reserved/provisioned concurrency; cold starts and 29 s integration
Commands/results	Separate SQS mappings, maximum 2 invocations each	At-least-once/unordered delivery requires inbox and version fences
Short execution	Short-work mapping, batch 1 and maximum 2	900 s invocation, 720 s pipeline budget with 180 s headroom; bounded scratch/CPU
Standard execution	Finite Fargate tasks, launcher cap 3	Queue/start delay; one Attempt per task, then exit
SQL	Separate clusters, one 0–2 ACU instance each	Cold resume, connections and lock/CPU contention; no reader failover

The configured encoder ceiling is **3 standard + 2 short = 5**. Lane choice and standard capacity are separate decisions, each made in one place:

Decision	Where it is made	Rule
Lane dispatch	`processing/short_lane.py:work_lane()`, called by `messaging.emit_state()` inside the state transaction	Short only when `PROCESSING_SHORT_LANE=true`, the source size is in (0, 64 MiB] and no earlier Attempt of the Job ran on a `lambda-` task; otherwise standard. Duration and height are not yet known (Figure 17).
Short admission	`short_lane.check_probe`, applied by `worker.sqs.execute_message` after claim and download	Finite positive duration ≤ 60 s and height ≤ 1080 px; a size-admitted source can still be rejected here.
Fenced redirect	`processing/messaging.py:redirect_to_long_lane()`	With a live DB-clock lease: abandon the short Attempt, record cleanup intent, create a new queued Attempt and stage standard work plus result. A stale token changes nothing; later retries stay standard (Figure 18).
Standard capacity	`processing/launcher.py:tick()` and `_reserve()`	Demand from SQL queued current Attempts and visible standard work, offset by idle or pending workers, bounded by cap 3. Reservations commit before RunTask; the worker claims an Attempt only after its own receive.
Short capacity	Short-work event source mapping	Maximum concurrency 2, independent of the launcher.

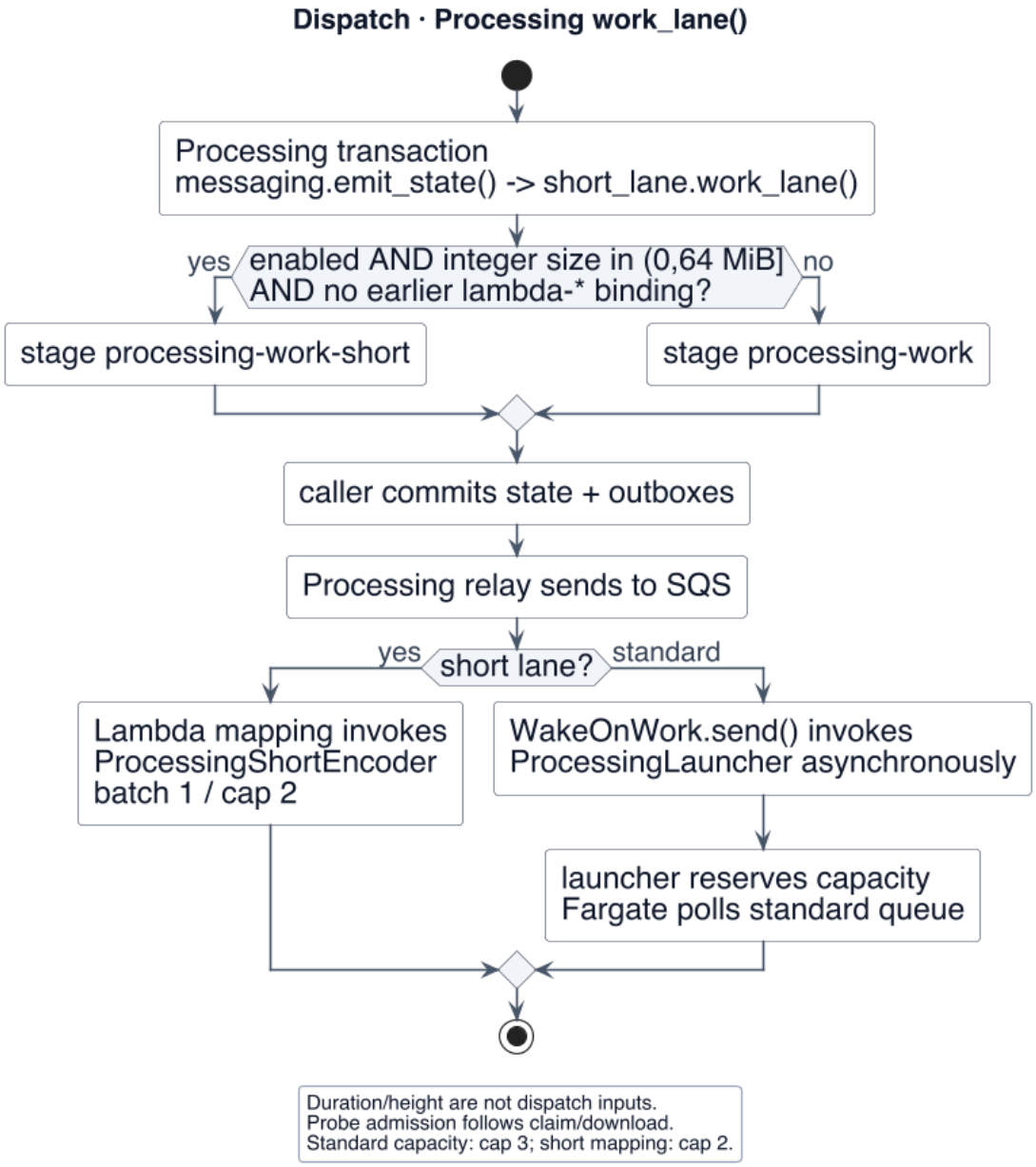


Figure 17. Dispatch selects a work lane inside the Processing transaction, then publication activates the short mapping or wakes the standard-capacity launcher.

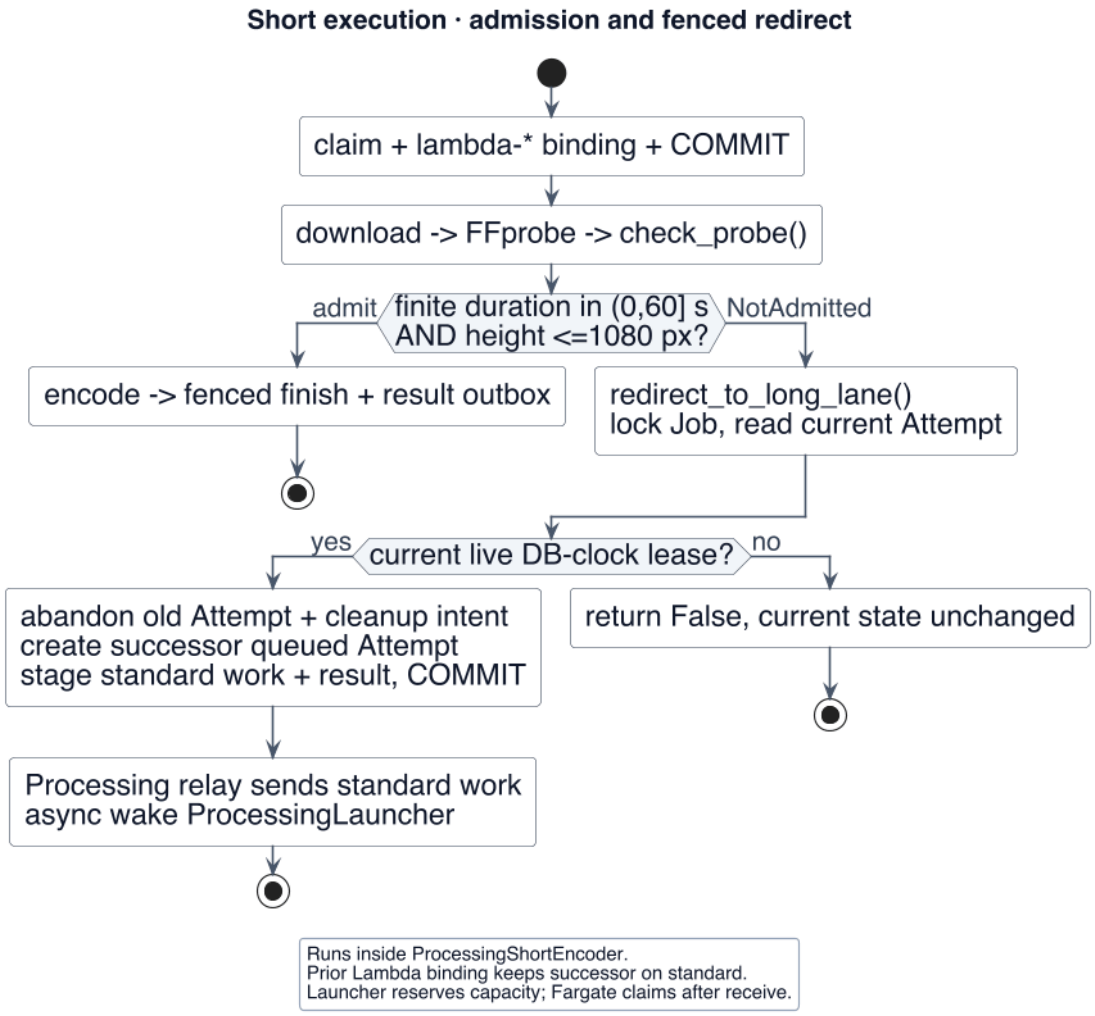


Figure 18. Probe admission follows download. A live-lease redirect creates a successor Attempt and standard work; stale execution cannot mutate current state.

Verification covers bounds and sticky fallback (`tests/processing/test_short_lane.py`), redirect and lease loss (`test_short_encoder.py`) and real-SQL handoff and capacity concurrency (`tests/v1/test_short_encoder_sql.py`, `test_launcher_sql.py`). UC-2 and UC-6 give the operation-level interactions.

3.2.2 Technology and Services

Technology or service	Used for	Rationale	Trade-off accepted
Python 3.12, FastAPI, Pydantic, SQLAlchemy 2 (async), Alembic	Services, worker and tooling	Continuity with the inherited code; typed validation at every boundary; one migration history per database	One lockfile; owner extras scope image dependencies
FFmpeg and ffprobe	Probing, MP4 encoding, HLS ladder (360/480/720, capped at the source height)	Industry standard and already proven in Project 1	CPU-bound; HLS is best effort; a large unfixed-CVE surface in the worker image (section 4.3)
Plain JavaScript SPA, hls.js, Plyr	Web client	No build step; browsers load `app.js` directly; Vitest tests it	No production bundling or minification

Technology or service	Used for	Rationale	Trade-off accepted
Python stdlib plus `httpx`	Publisher CLI	Proves reuse with the smallest dependency set: 495 physical Python lines across five files, including blank lines	Polling-based status; one cookie jar per process
AWS Lambda (container images)	APIs, consumers, repair, launcher	Pay per request, scale to zero, per-domain image release; also the short-encode lane	Cold starts; account concurrency limit of 10 observed
Amazon API Gateway HTTP API	Routing, authorizer hook, throttle	Cheapest managed HTTP front for Lambda; REQUEST authorizer	No WAF; one stage-level limit
Amazon CloudFront	HTTPS entry, static site, `/api/*` pass-through	Managed TLS, HTTP/2 and HTTP/3, no custom domain needed	Playback bytes come from S3 directly, not through CloudFront
Amazon SQS (Standard) with DLQs	Commands, work and results	Serverless, durable, cheap; redrive policy (5 receives)	Unordered and at least once, hence inbox dedupe and version fences
Amazon Aurora PostgreSQL Serverless v2	Two owned relational databases, one cluster each	Transactions, constraints and `SKIP LOCKED` for outbox claims; scales to 0 ACU	One instance per cluster; cold resume of several seconds
Amazon S3 (versioned)	Sources, outputs, frontend	Presigned POST/GET keep bytes off the API; versioning makes the pinned source immutable	Cleanup must delete every version
Amazon ECS on Fargate	Finite encoders, migrations, bootstrap task	No hosts to manage; per-task CPU caps make capacity predictable	Task start median 25.6 s **[AWS]**, which dominates queue wait
Amazon EventBridge	Repair ticks, ECS and S3 event hints	Rebuild lost work and react to stopped tasks without polling clients	One-minute granularity
AWS IAM, SSM Parameter Store, Secrets Manager	Per-handler roles, DSNs and JWT secret, Aurora master secret	Secrets stay out of code, images and the template	SSM read at cold start
Amazon ECR	Three image repositories	Immutable tags and digests are the release identity	Scan on push complements the CI and CD Trivy scans
Amazon CloudWatch, AWS Budgets	Logs, alarms, dashboard, cost alerts	Per-service observability at near-zero cost	7-day log retention; no paging
GitHub Actions, OIDC, Pages	CI, CD, study-guide publication	Integrated with pull requests; no long-lived cloud keys	Hosted runner limits shape the CI lanes

AD-31 — retain PostgreSQL authority with JSONB; consider NoSQL when access patterns justify it. Asset, source and output data include document-shaped values, and the schemas already store variable operations and descriptors in JSONB while S3 stores video bytes. PostgreSQL keeps owner-filtered reads, relational identities, unique request/message/task constraints, local state/inbox/outbox transactions and DB-clock lease fences in one authority. `delivery/sql.py` uses indexed due work and `FOR UPDATE SKIP LOCKED`; `processing/lifecycle.py` checks the current Attempt, token and deadline against database time (`media/models.py`, `processing/models.py`; Persistence Design).

Requirement	PostgreSQL retained	MongoDB alternative design	DynamoDB alternative design
Owned Asset and status	Owner-filtered local queries, projection and JSONB	Owner/Asset documents with appropriate indexes	Owner partition, Asset key and listing index
Domain + inbox + intent	One local SQL transaction	Bounded embedding or explicit multi-document transaction boundary	Conditional identity items and transaction/publication design
Replay/task uniqueness	Primary, unique and composite constraints	Unique indexes and transaction policy	Conditional identity reservations and transaction policy
Lease, due work, capacity	DB-time fences, row locks and due indexes	Compare-and-set/transaction plus clock and due-index policy	Conditional updates plus partition/due-query and clock policy

A NoSQL replacement needs explicit publication, concurrency and recovery policies, and an external SQS send still falls outside its transaction. Retaining SQL keeps flexible JSONB payloads and the current single authority. A future owner-facing read store would need backfill, tombstones, version/attempt fences and reconciliation, because result snapshots are not a complete event log.

Runtime and development tools are separate.

- Runtime images contain only the owning service's code and its runtime extras; `deploy/v1/check_runtime.py` verifies this.
- Development tools run only in CI and on developer machines: pytest, Ruff, Vitest, Playwright, Moto, cfn-lint, actionlint, CodeQL, Trivy, gitleaks and ZAP.

3.2.3 Persistence Design

Owned relational schemas. Media and Processing each own a PostgreSQL 16 database on a separate Aurora Serverless v2 cluster (AD-29). Ordered migrations give eleven application tables per database at heads `media_0008` and `processing_0007`, excluding `alembic_version`. The schemas follow this checkout's SQLAlchemy records and migrations (`media/models.py`, `media/upload_models.py`, `media/projection_models.py`, both `messaging_models.py` files, `processing/models.py`, `processing/launcher_models.py` and both `migrations/versions/` folders).

Figures 19–24 select typed columns and declared relationships. The dictionaries retain all 172 columns, 61 PK/FK/unique/CHECK constraints and twelve explicit indexes, including both physical copies of the delivery tables.

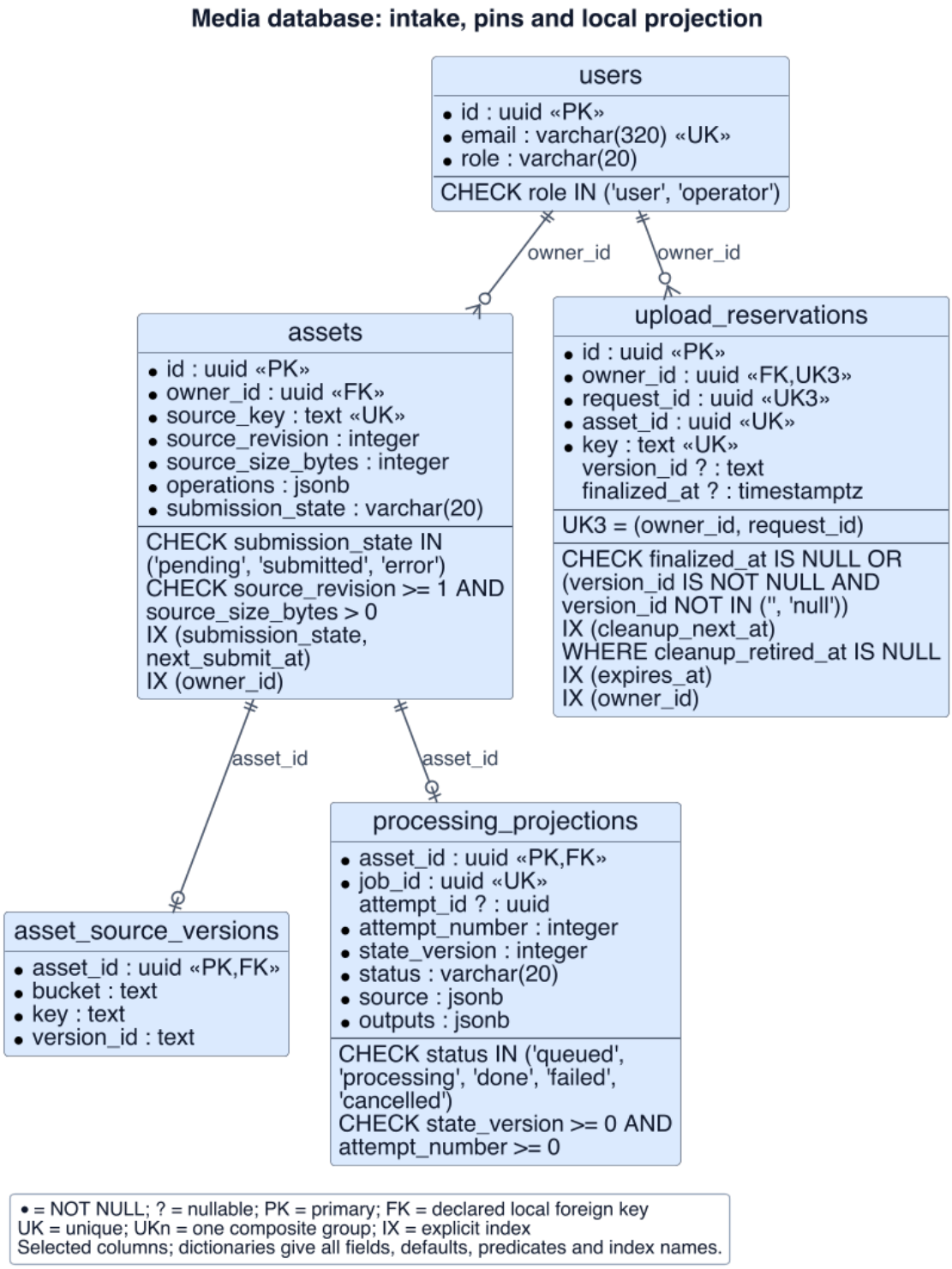


Figure 19. Media database: selected typed intake, source-pin and projection fields; only declared local foreign keys are connected.

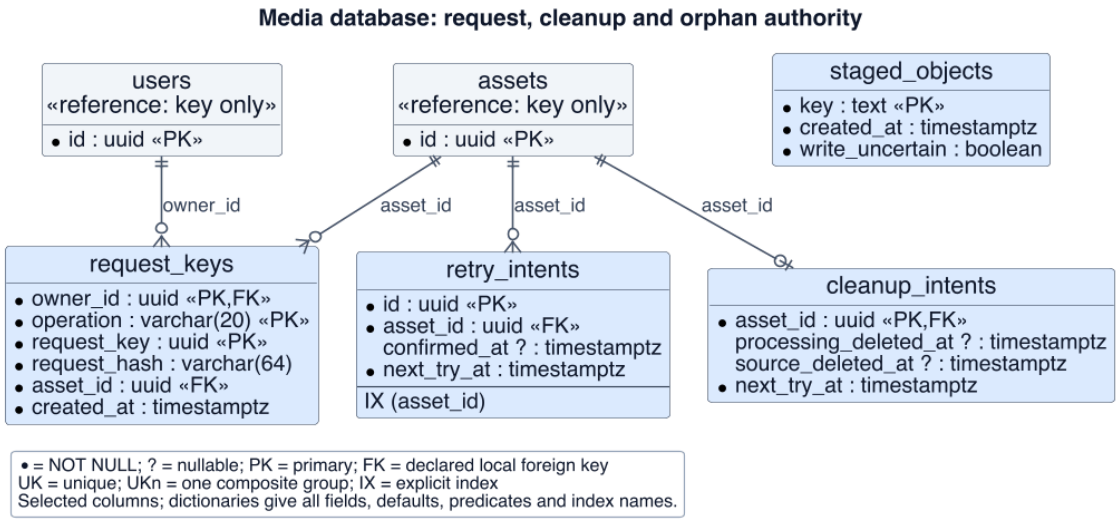


Figure 20. Media database: request, cleanup and orphan-write records; grey boxes repeat referenced keys.

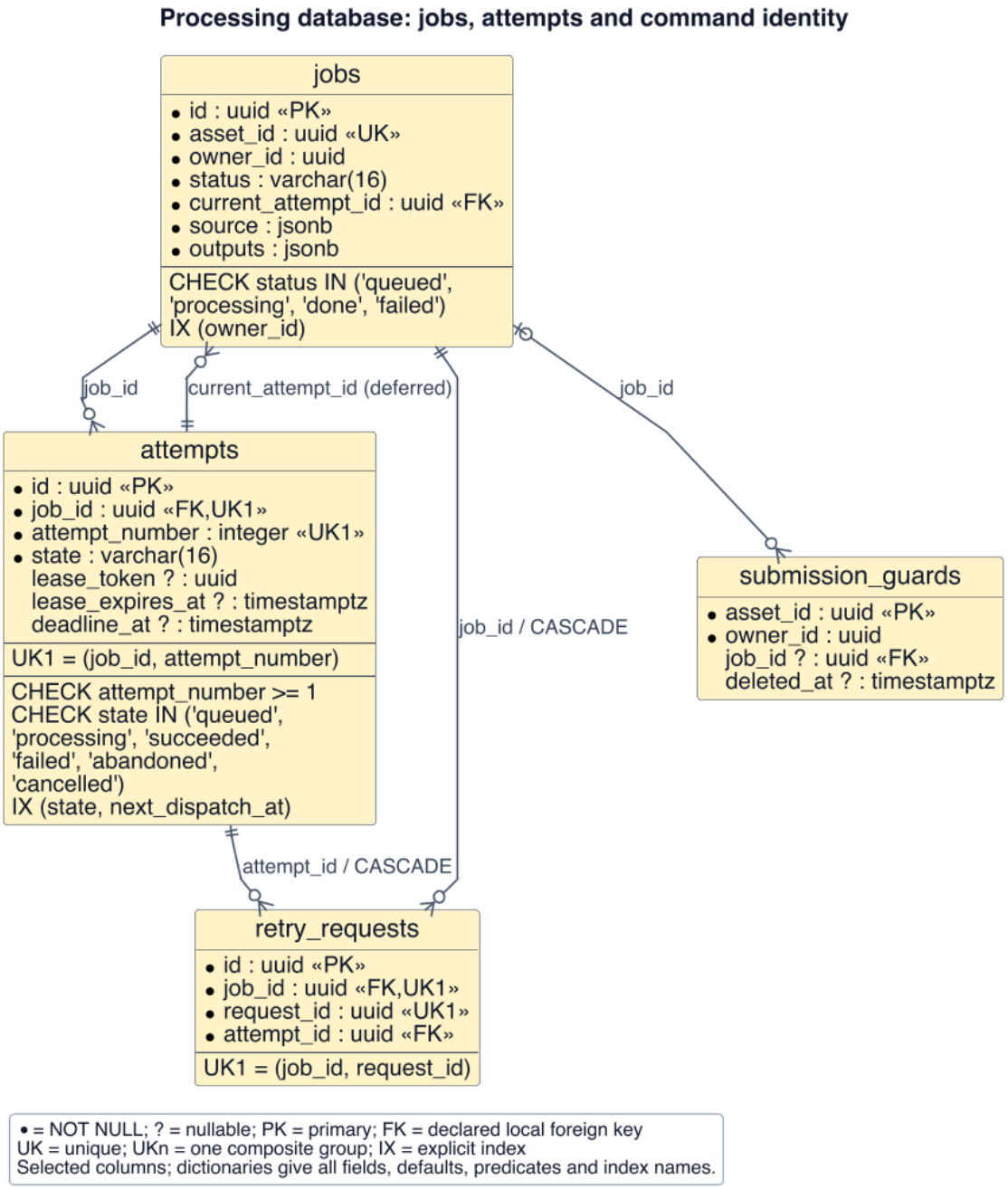


Figure 21. Processing database: selected typed Job/Attempt, retry and cancellation-guard fields. The current-attempt FK is deferred and non-unique.

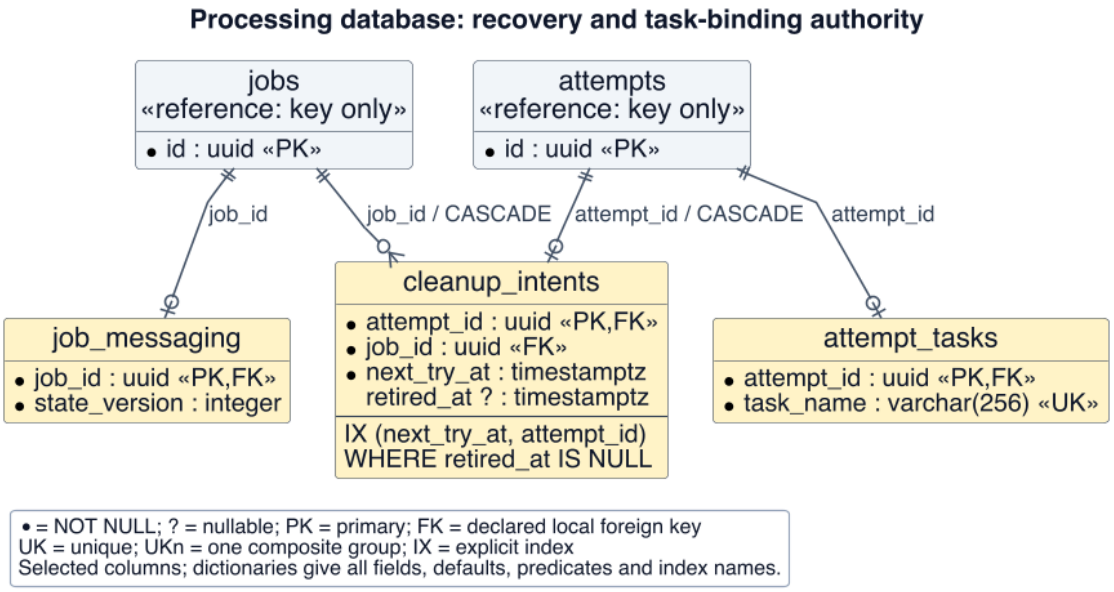


Figure 22. Processing database: cleanup, state-version and task-binding records.

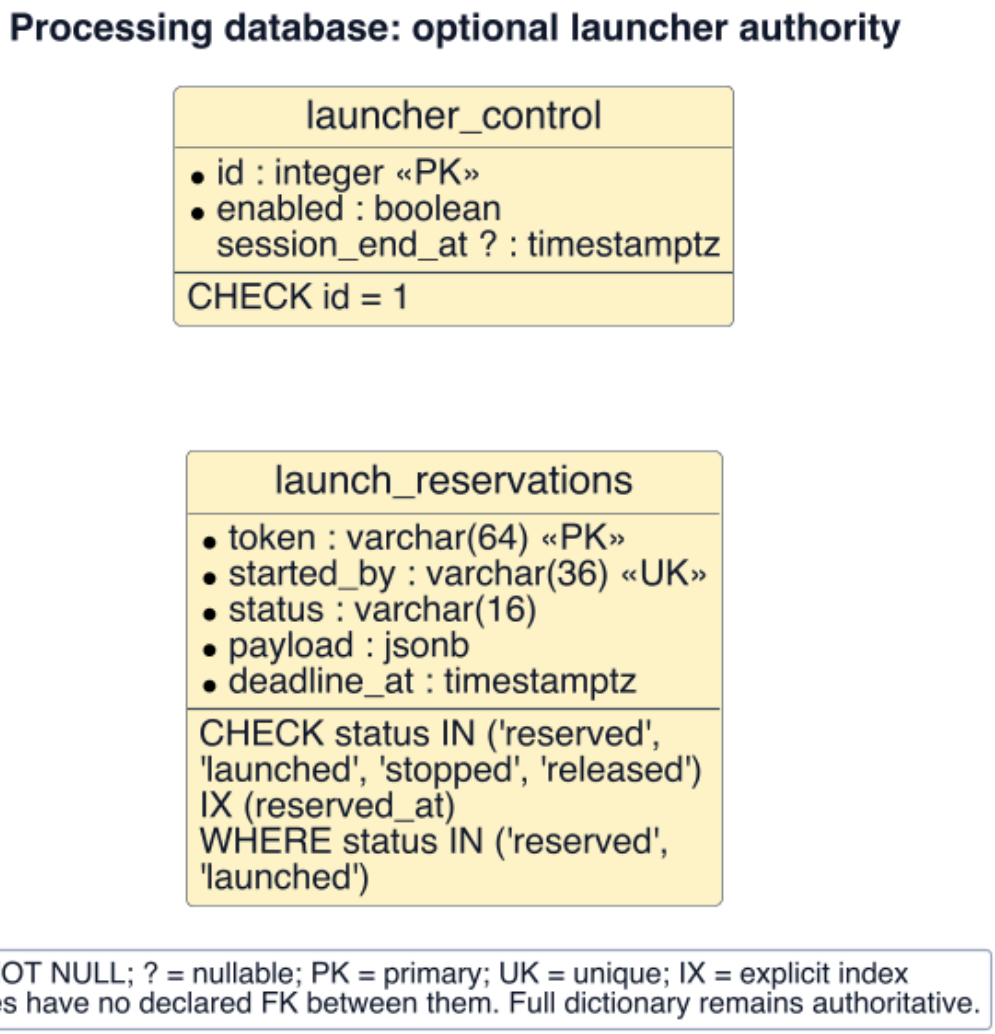


Figure 23. Processing database: optional launcher singleton and durable RunTask request, with unique token and startedBy identities.

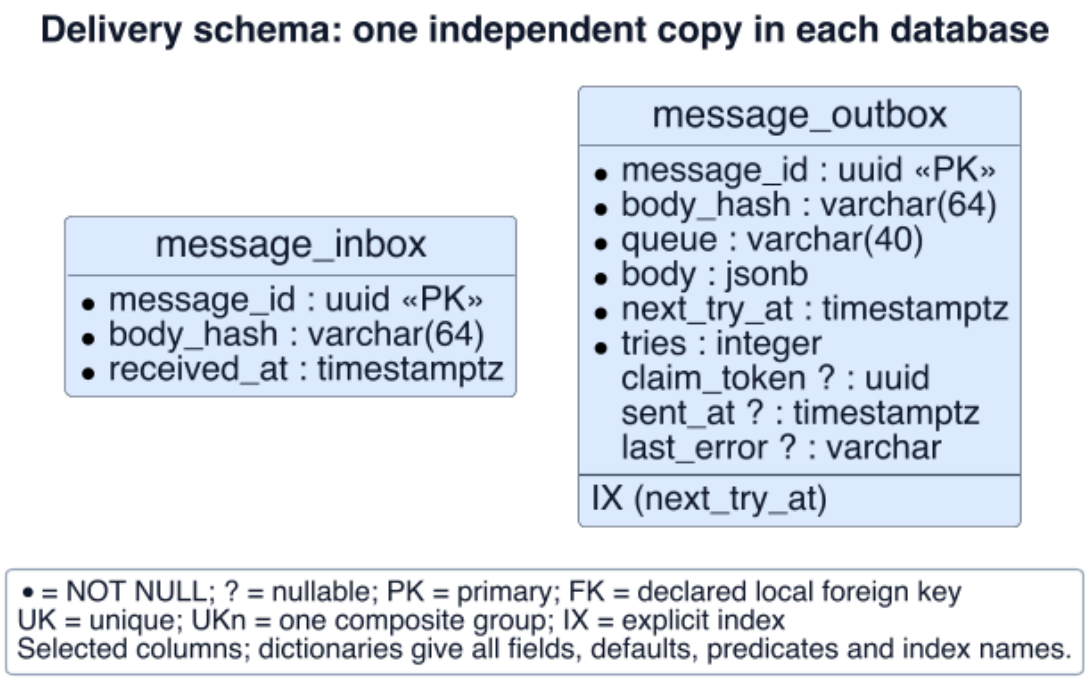


Figure 24. Identical inbox/outbox schemas exist independently in each database; no shared table or cross-database transaction is implied.

Cross-context owner/asset/job UUIDs carry identity without foreign keys. In Media, `upload_reservations.asset_id` is unique but has no Asset FK; a reservation precedes its Asset. `assets.source_asset_id` is also a plain UUID. Processing's nullable `submission_guards.job_id` is non-unique. `jobs.current_attempt_id` is non-unique and DEFERRABLE INITIALLY DEFERRED: SQL allows the Job/first Attempt insert cycle, while `processing/lifecycle.py:lock_pair` checks that the current Attempt belongs to that Job.

Database bootstrap creates `*_owner` (no login), `*_migrator` (inherits owner) and `*_runtime` (DML plus sequence use). `PUBLIC` has no access; login roles cannot connect to the other database; connections require verified TLS (`scripts/aws/v1/bootstrap.py`). Recorded AWS checks refused cross-database runtime login and runtime DDL **[AWS]**.

Compact physical data dictionaries. Each row names one table; comma-grouped column names share the following type. `?` marks nullable; all other columns are NOT NULL. `=value` is a SQL server default; omitted defaults are supplied by callers or Python, not SQL. `PK`, `UK`, `FK` and `IX` denote primary, unique, foreign and explicit index definitions. All FKs use ON UPDATE NO ACTION and ON DELETE NO ACTION unless CASCADE is shown. PK/UK backing indexes are implicit. `timestamptz` means timestamp with time zone; unbounded `varchar` remains distinct from `text`.

Media (`media_db.public`, head `media_0008`):

Table / purpose	Columns: name(s): SQL type	Keys, checks and explicit indexes
staged_objects — Crash-safe orphan ledger for uncertain source writes.	key: text; created_at: timestamptz; write_uncertain: boolean=true	PK (key)
users — Account identity, credentials and role.	id: uuid; email: varchar(320); password_hash: text; role: varchar(20); created_at: timestamptz	CHECK role IN ('user', 'operator'); PK (id); UK (email)
assets — Owned immutable asset intake and submission state.	id, owner_id, source_asset_id?, processing_job_id?: uuid; filename, source_media_type: varchar(100); source_key, last_submit_error?: text; source_revision, source_size_bytes, submit_attempts: integer; source_sha256: varchar(64); operations: jsonb; submission_state: varchar(20); submitted_at?, next_submit_at, created_at, deleted_at?: timestamptz	CHECK source_revision >= 1 AND source_size_bytes > 0; CHECK submission_state IN ('pending', 'submitted', 'error'); FK (owner_id) → users.id; PK (id); UK (source_key); IX assets_pending_due(submission_state,next_submit_at); IX ix_assets_owner_id(owner_id)
cleanup_intents — Deletion progress and retained rescan authority.	asset_id: uuid; processing_deleted_at?, source_deleted_at?, next_try_at: timestamptz; attempts: integer; last_error?: text	FK (asset_id) → assets.id; PK (asset_id)
request_keys — Owner/operation/request identity and immutable request hash.	owner_id, request_key, asset_id: uuid; operation: varchar(20); request_hash: varchar(64); created_at: timestamptz	FK (asset_id) → assets.id; FK (owner_id) → users.id; PK (owner_id, operation, request_key)
retry_intents — Retained legacy peer retry bookkeeping.	id, asset_id: uuid; confirmed_at?, rejected_at?, created_at, next_try_at: timestamptz; attempts: integer; last_error?: text	FK (asset_id) → assets.id; PK (id); IX ix_retry_intents_asset_id(asset_id)
asset_source_versions — Version pins for edit-derived copied sources.	asset_id: uuid; bucket, key, version_id: text; sha256: varchar(64); size_bytes: integer; content_type: varchar(100)	FK (asset_id) → assets.id; PK (asset_id)
upload_reservations — Direct-upload idempotency, immutable version pin and orphan rescan authority.	id, owner_id, request_id, asset_id: uuid; request_hash, sha256: varchar(64); filename, content_type: varchar(100); bucket, key, version_id?: text; size_bytes: integer; expires_at, finalized_at?, created_at, cleanup_checked_at, cleanup_retired_at?: timestamptz; cleanup_next_at: timestamptz=now(); cleanup_attempts: integer=0	CHECK finalized_at IS NULL OR (version_id IS NOT NULL AND version_id NOT IN ('', 'null')) [ck_upload_reservations_finalized_has_pin]; FK (owner_id) → users.id; PK (id); UK (asset_id); UK (key); UK (owner_id, request_id); IX ix_upload_reservations_cleanup_due(cleanup_next_at) WHERE cleanup_retired_at IS NULL; IX ix_upload_reservations_expires_at(expires_at); IX ix_upload_reservations_owner_id(owner_id)
processing_projections — Media-owned result snapshot.	asset_id, job_id, attempt_id?: uuid; attempt_number, state_version: integer; status: varchar(20); source, outputs: jsonb; error?: text; result_hash: varchar(64)	CHECK state_version >= 0 AND attempt_number >= 0; CHECK status IN ('queued', 'processing', 'done', 'failed', 'cancelled'); FK (asset_id) → assets.id; PK (asset_id); UK (job_id)

Processing (processing_db.public, head processing_0007):

Table / purpose	Columns: name(s): SQL type	Keys, checks and explicit indexes
jobs — Processing authority for one asset, source, operations, current attempt and output descriptors.	id, asset_id, owner_id, current_attempt_id: uuid; source, operations: jsonb; request_hash: varchar(64); source_url, error?: varchar; source_url_expires_at, created_at, updated_at, deleted_at?: timestamptz; status: varchar(16); outputs: jsonb='[]'::jsonb	CHECK status IN ('queued', 'processing', 'done', 'failed') [ck_jobs_status]; FK (current_attempt_id) → attempts.id; DEFERRABLE INITIALLY DEFERRED [fk_jobs_current_attempt]; PK (id); UK (asset_id); IX ix_jobs_owner_id(owner_id)
attempts — Numbered execution attempt, token lease, deadline and recovery schedule.	id, job_id, lease_token?: uuid; attempt_number: integer; state: varchar(16); queued_at, started_at?, finished_at?, heartbeat_at?, lease_expires_at?, deadline_at?, next_dispatch_at?: timestamptz; error?, dispatch_error?: varchar; dispatch_count: integer=0	CHECK attempt_number >= 1 [ck_attempts_number]; CHECK state IN ('queued', 'processing', 'succeeded', 'failed', 'abandoned', 'cancelled') [ck_attempts_state]; FK (job_id) → jobs.id; PK (id); UK (job_id, attempt_number) [uq_attempts_job_number]; IX ix_attempts_recovery(state,next_dispatch_at)
submission_guards — Durable owner/asset submission/cancellation boundary, including cancel-before-submit.	asset_id, owner_id, job_id?: uuid; deleted_at?: timestamptz	FK (job_id) → jobs.id; PK (asset_id)
retry_requests — Stable retry identity maps to the already-created attempt.	id, job_id, request_id, attempt_id: uuid	FK (attempt_id) → attempts.id; ON DELETE CASCADE; FK (job_id) → jobs.id; ON DELETE CASCADE; PK (id); UK (job_id, request_id)
cleanup_intents — Deletion progress and retained rescan authority.	attempt_id, job_id: uuid; next_try_at, retired_at?: timestamptz; tries: integer=0; last_error?: varchar; created_at: timestamptz=now()	FK (attempt_id) → attempts.id; ON DELETE CASCADE; FK (job_id) → jobs.id; ON DELETE CASCADE; PK (attempt_id); IX ix_cleanup_intents_due(next_try_at,attempt_id) WHERE retired_at IS NULL
job_messaging — Monotonic result/work state version per Job.	job_id: uuid; state_version: integer=0	FK (job_id) → jobs.id; PK (job_id)
attempt_tasks — One finite task-name binding per Attempt and unique task name.	attempt_id: uuid; task_name: varchar(256)	FK (attempt_id) → attempts.id; PK (attempt_id); UK (task_name)
launcher_control — Optional unattended launcher singleton enable/session gate.	id: integer=nextval('launcher_control_id_seq'::regclass); enabled: boolean=false; enabled_at?, session_end_at?: timestamptz; updated_at: timestamptz=now()	CHECK id = 1 [ck_launcher_control_singleton]; PK (id)
launch_reservations — Optional unattended launcher durable ECS request replay and capacity reservations.	token: varchar(64); started_by: varchar(36); status: varchar(16)=reserved; payload: jsonb; deadline_at, sent_at?, launched_at?, settled_at?: timestamptz; reserved_at: timestamptz=now(); task_arn?: varchar(512); claimed?: boolean; last_error?: varchar(120)	CHECK status IN ('reserved', 'launched', 'stopped', 'released') [ck_launch_reservations_status]; PK (token); UK (started_by); IX ix_launch_reservations_open(reserved_at) WHERE status IN ('reserved', 'launched')

The common delivery definition contributes two tables to **each** database:

Table / purpose	Columns: name(s): SQL type	Keys, checks and explicit indexes
message_inbox — Received message identity plus canonical body hash.	message_id: uuid; body_hash: varchar(64); received_at: timestamptz=now()	PK (message_id)
message_outbox — Canonical envelope and durable relay claim/replay schedule.	message_id, claim_token?: uuid; body_hash: varchar(64); queue: varchar(40); body: jsonb; next_try_at, sent_at?: timestamptz; tries: integer=0; last_error?: varchar	PK (message_id); IX ix_message_outbox_due(next_try_at)

The ORM omits the ix_attempts_recovery and ix_cleanup_intents_due declarations and uses a Python default of 1 for the launcher_control.id singleton; the dictionary follows the migration DDL for those indexes and the SERIAL default. retry_intents and jobs.source_url/source_url_expires_at are retained legacy structures; native intake writes an empty URL and database time, using the pinned JSONB source. The launcher tables are used only by the optional unattended launcher.

Transactions and concurrency. Every transaction is local to one database. SQL commits locally; durable outbox intent, inbox deduplication and repair bridge the SQS boundary. DP-2 (section 6.5) gives the full relay protocol.

Boundary	Locks and checks	Commits together	Failure or replay
Media adopt (direct_intake.finalize)	Verifies the explicit S3 VersionId outside row locks, then locks and rechecks UploadReservation	Pin, new Asset, submit MediaOutbox (delivery/sql.py:stage never commits for its caller)	Reservation (owner_id,request_id) plus request hash returns the adopted Asset
Media edit adoption (v1_lifecycle.edit)	Locks the staged row; verifies and pins the copied source inside the transaction	Derived Asset, SourceVersion, optional RequestKey, submit outbox; staged row removed	RequestKey(owner_id,operation, request_key) replays the winner; a loser discards its copy
Relay (delivery.sql.publish_due)	One due row FOR UPDATE SKIP LOCKED; UUID claim token and DB-time reservation	Claim; a second transaction records success or backoff only if the token still matches	SQS I/O runs outside SQL locks; an uncertain send repeats with the same message ID
Processing accept (messaging.handle_command)	Locks SubmissionGuard before an existing Job; delivery.sql.accept compares canonical body hashes	Inbox row, domain changes, work/result outbox	Same ID and body is a no-op; same ID with a different body raises MessageConflict; retry-before-submit rolls back for later redelivery; RetryRequest(job_id,request_id) fixes the successor
Claim, heartbeat, finish (processing/lifecycle.py)	lock_pair locks the Job and checks its current Attempt; token, lease and deadline use clock_timestamp()	Lease state; terminal result and result outbox	A stale token, expired lease or passed deadline cannot heartbeat or finish; AttemptTask allows one task binding per Attempt
Media result (media/messaging.py:consume_result)	Checks owner, source, Job and Attempt; accepts only a newer valid state_version	MediaInbox row and projection	Identical equal-version content is ignored; conflicting content is rejected and rolled back with the inbox (media/projections.py:validate_transition)

Encoding and downloads run between these transactions. Consumers acknowledge only after commit: command ACK follows committed acceptance, standard workers delete work after a durable outcome or SQL-proved staleness, the short Lambda returns partial batch failures, and result ACK follows Media's inbox/projection commit (processing/handlers.py, processing/worker/sqs.py, processing/short_encoder.py, media/handlers.py).

JSONB value schemas. These are application contracts, not SQL JSONB CHECK constraints.

Value / columns	Shape and validation boundary
Source — `jobs.source`, `processing_projections.source`	`{bucket:str,key:str,version_id:str,sha256:str,size_bytes:int,content_type:str}`. All six required; nonempty/non-`null` pin, lowercase 64-hex digest, strict size 1..100 MiB. `contracts/v1.py:Source` differs from legacy `processing.schemas.Source`. Workers check owner prefix and downloaded bytes.
Operations — `assets.operations`, `jobs.operations`	`[{operation,params}]`: clip `{start:number,end:number}` with `0≤start<end`; crop `{x:int≥0,y:int≥0,w:int>0,h:int>0}`; upscale/downscale `{height:int>0}`; convert `{format:"mkv" or "mp3"}`. No duplicate kinds or simultaneous upscale/downscale; probe-dependent bounds checked at execution (`media/schemas/edit.py`, `processing/schemas.py`, `processing/worker/tasks.py`).
Outputs — `jobs.outputs`, `processing_projections.outputs`	`[{kind:"primary" or "hls_manifest",object_key:str,media_type:str,size_bytes:int or null,sha256:str or null}]`. DTO permits null size/hash; successful `lifecycle.finish` requires positive size, digest, unique kinds, exactly one primary and current Attempt namespace.
Projection row	Job/Attempt IDs, attempt number, state version, status, Source, outputs/error and semantic result hash. Source/outputs are JSONB; other fields are typed columns. Media's local read model is independent of Processing's Job.
`message_outbox.body`	Canonical Envelope below; `delivery/envelope.py` normalises known fields before hashing. Inbox retains only identity/hash/time.
`launch_reservations.payload`	RunTask request: `{cluster,taskDefinition,launchType:"FARGATE",count:1,clientToken,startedBy,networkConfiguration:{awsvpcConfiguration:{subnets,securityGroups,assignPublicIp}},overrides:{containerOverrides:[{name,command}]}}`; replayed from SQL (`processing/launcher.py:run_payload`).

SQS wire model. `contracts/v1.py:Envelope` requires all ten keys below, even when values are nullable; UUIDs are JSON strings, version fields are strict integers, and schema version 1 is emitted and accepted.

```
Envelope = {schema_version:1, message_id:UUID,
kind:submit|retry|cancel|work|result,
            owner_id:UUID, asset_id:UUID, job_id:UUID|null, attempt_id:UUID|null,
            state_version:int>=0, source:Source|null, payload:object}
```

Kind	Producer shape / required binding
`submit`	Media: `{operations:[Operation]}`; Source required. Normal finalize emits null Job/Attempt and version 0; Processing assigns Job identity.
`retry`	Media: `{}`, stored Source; durable Job retains operations. Consumer uses optional `payload.request_id`, otherwise `message_id`, as retry identity. Envelope permits null Source; consumer compares it when supplied.
`cancel`	Media: `{}`, null Source; Job/Attempt may be null. `SubmissionGuard` persists cancel-before-submit.
`work`	Processing: `{operations:[Operation]}`; Job/Attempt/Source required. Stable UUID derives from Attempt and state version; worker compares stored acceptance before S3 GET. Both work lanes use this schema.
`result`	Processing: `{status,outputs:[Output],error:str or null,attempt_number:int≥0}` with Job/Attempt/Source. Status is queued/processing/done/failed/cancelled; cancellation derives from `Job.deleted_at`, not `jobs.status`.

Cancel-before-submit instead produces version 0 with null Job/Attempt/Source and exactly `{status:"cancelled",cancellation_confirmed:true,command_message_id:UUID}`, confirming a durable guard; Media matches the retained cancel command before authorising source

cleanup (`processing/messaging.py:_cancel_locked`, `media/projections.py:confirm_cancellation`). DLQs retain the same body.

Object storage. Source version pins and output Attempt namespaces serve different purposes:

Object	Exact key layout and authoritative identity
Direct source	`staging/{owner_id}/{reservation_id}/source`; adopted in place, with VersionId on finalized `UploadReservation`.
Edit source	`sources/{owner_id}/{asset_id}/source`; verified copy of parent's primary, with VersionId on `SourceVersion`.
Primary output	`jobs/{job_uuid}/attempts/{attempt_uuid}/primary.{mp4,mkv,mp3}`; extension follows operations, default MP4. Attempt UUID is not attempt number.
Optional HLS	Same Attempt prefix plus `hls/master.m3u8`, `hls/v{n}/index.m3u8`, `hls/v{n}/seg%05d.ts`; unedited inputs only, master uploaded last.
Frontend	`index.html`, `<stem>.<SHA256-prefix10>.<js/css>`, `vendor/...`; manifest records type, cache control, size, SHA-256 and MD5 (`scripts/aws/v1/frontend.py`).

Output descriptors have no output VersionId. Fenced SQL finish determines which Attempt's objects can be published. HLS names are validated before playback. Cleanup enumerates versions and delete markers and retains rescan authority for late writes (`processing/worker/tasks.py`, `media_engine/hls.py`, `processing/recovery_v1.py`, `media/recovery_v1.py`).

What persists, and for how long. These are the values configured in the template and code. The lifecycle of this deployment as a whole is set by AD-08 (fresh demonstration data only) and AD-27 (the persistent environment is torn down once, after 16 November 2026).

Data	Persists beyond	Period configured
Session (JWT cookie)	One request	1 hour (`jwt_ttl_seconds` 3600); not stored server-side
Upload grant	One request	15 minutes; an expired reservation is rescanned for late writes and retired after a clean scan beyond a one-day horizon
Signed playback URL	One request	At most 900 seconds (`signed_url_ttl_seconds`), shortened to remain within the signing credentials' lifetime
Accounts	Session	Until environment teardown. No account deletion feature or purge schedule exists
Assets, sources and outputs	Session	Until the owner deletes the asset. Deletion is a tombstone; every source and output version is then deleted asynchronously. No age-based expiry and no S3 lifecycle rule are configured
Jobs, attempts and tombstone rows	Session	Until environment teardown; no hard-purge job
Outbox and inbox rows	Transaction	Until environment teardown; sent rows are kept for replay; no pruning job
SQS messages	Transaction	7 days (DLQs 14 days); visibility: base 420 s, short queue 960 s, Fargate receive override normally 60 s; redrive after 5 receives
Lambda and task logs	Request	7 days
Aurora backups	n/a	1-day automated backup retention on each cluster; the pre-split snapshot of the original cluster is kept, and one final snapshot is kept at teardown
CI and CD artifacts	Workflow run	7, 14 or 60 days, depending on the artifact

Information architecture: data inventory and classification. The team uses four levels: **Restricted** (credentials, keys and secrets), **Confidential** (personal data and owners' content),

Internal (operational and processing metadata) and **Public** (published static assets). Shared controls (session cookie, encryption, IAM and network) are in section 4.3; the table keeps the controls specific to each group.

Data group (classification)	Format and store	Group-specific controls	Rationale and trade-off
Accounts and authentication (Confidential; hashes and secrets Restricted)	`users` rows (UUID, email, bcrypt hash at cost 10, role `user` or `operator`) in `media_db`, for unique constraints and joins with asset ownership; stateless HS256 JWT in the session cookie; signing secret as an SSM SecureString	Only Media roles read the JWT secret and Media DSN. With authorizer cache TTL 0, each protected request reads the current user row, and role checks use that identity (`media/authorizer.py`). Password bytes are length-limited before hashing.	Cognito would add MFA and recovery flows but a new dependency and token format; the inherited account model was kept, with verification at the edge (AD-09).
Owner media content (Confidential)	Accepted video sources of at most 100 MiB in the allowed types. Encoded outputs have no configured 100 MiB cap: the primary is MP4 by default, or MKV/MP3 for conversion; unedited inputs can also produce HLS playlists and segments. Versioned S3 buckets.	Public access blocked, TLS-only bucket policies, SSE-S3. Presigned POST fixes key, content type and length range. Downloads use owner-checked presigned GET links valid for at most 900 s, shortened when the signing credentials expire sooner; HLS segments use a 307 redirect to a fresh signature. CORS admits only the frontend origin. The worker reads only the source bucket, after checking the owner and source binding.	S3 stores video bytes; PostgreSQL stores transactional metadata and descriptors. Versioning gives each source an immutable VersionId. Proxying bytes through the API would exceed Gateway/Lambda payload limits; presigned URLs are bearer capabilities, so their lifetime is short.
Asset and upload metadata (Internal; original filenames Confidential)	`assets`, `upload_reservations`, `asset_source_versions`, `request_keys`, `cleanup_intents`, `staged_objects` and `processing_projections` in `media_db`; JSONB operations and outputs	Owner filter on every query, foreign IDs return 404. CHECK constraints enforce the pinned-source rule (`ck_upload_reservations_finalized_has_pin`) and status sets. Idempotency keys make finalize, edit and retry safe to repeat.	Media keeps a projection instead of querying Processing (AD-21): no runtime dependency, at the cost of eventual consistency.
Processing job data (Internal)	`jobs`, `attempts`, `submission_guards`, `retry_requests`, `attempt_tasks`, `job_messaging`, `cleanup_intents`, `launcher_control` and `launch_reservations` in `processing_db`; JSONB source and outputs	Only Processing roles hold its runtime DSN. Commands for an existing Job must match its owner and asset; supplied Job IDs and source descriptors must also match. A cancel can instead tombstone an owner-bound `SubmissionGuard` before a Job exists. Native workers require the durable source and current-attempt binding before claiming execution.	Leases, row locks and `SKIP LOCKED` need a transactional store with a trustworthy clock; the store comparison is AD-31 (section 3.2.2).
Messages in transit (Internal)	Envelope v1 (above) in SQS Standard queues and DLQs, plus `message_outbox` and `message_inbox`	Validation rejects credential/token/media-byte-like keys and HTTP(S) URL strings. Resource policies deny `SendMessage` to all but the listed producer roles, and receive/delete to all but the listed consumer roles.	FIFO queues would add ordering and deduplication at lower throughput and with message-group design; inbox and version fences already tolerate duplicates and reordering.

Data group (classification)	Format and store	Group-specific controls	Rationale and trade-off
Secrets and configuration (Restricted)	Four role DSNs and the JWT secret as SSM SecureStrings; the Aurora master password is RDS-managed in Secrets Manager. Four non-secret SSM epochs (`activity/{media,processing}/{touched,quiet}`) let repair skip idle passes (`runtime_support/activity.py`).	Each handler role reads only its own domain's parameters; only the bootstrap operator reaches the master secret. No AWS key is stored in GitHub.	SSM Parameter Store has no per-secret monthly fee but no automatic rotation; rotation is out of scope. Cold starts read the parameters once.
Telemetry and delivery evidence (Internal)	CloudWatch logs (bounded, redacted failure events), metrics and alarm state; Actions artifacts (7–60 days); sanitized receipts under `docs/results/` and `docs/execution/aws/v1/evidence/`; private versioned S3 bucket for CD state	Signed URLs, cookies, secrets and account IDs are stripped before commit. The state bucket is private because it holds the smoke account's credentials.	Seven-day logs keep cost and exposure low; committed receipts cover later reconstruction.
Static frontend (Public)	HTML, CSS and JavaScript with content-hashed names in a private S3 bucket behind CloudFront origin access control	Integrity comes from the deploy pipeline; the bucket is not directly public.	No material trade-off.

3.2.4 Detailed Design

Three representative collaborations are summarised here at class/object level. Section 6.4 contains the complete per-use-case analysis and design models; section 6.5 explains the design patterns.

Upload, process and play (UC-1–UC-3). Figure 8 locates the handoff. Bytes go straight to versioned S3 under an `UploadReservation`, and `direct_intake.finalize` adopts them as above. Processing accepts the command into a queued `Job` and `Attempt`, `short_lane.work_lane` chooses the lane, a worker claims under a DB-clock lease, and `lifecycle.finish` commits the terminal result; public status stays `accepted` until Media projects it. Figures 33, 35, 38, 39, 40 and 53 in section 6.4 give the typed records, operations and exceptions. `test_finalize_pins_the_exact_version_it_verified`, `test_finalize_intent_rolls_back_with_adoption_and_commits_once` and `test_success_output_namespace_validation_and_atomic_terminal_result` verify the pin, the atomic adoption and the terminal commit (`tests/v1/test_direct_intake_s3.py`, `test_media_messaging_sql.py`, `test_messaging_sql.py`).

Edit-derived intake. `media/api/v1_jobs.py:edit` calls `v1_lifecycle.edit`, which resolves request replay and checks an owned, nondeleted completed parent with a primary output. `intake.stage` commits a `StagedObject` before copying. A copy transaction locks the parent and staged row, copies the primary through `Storage.copy_output_source` and records writer completion. The adoption transaction then locks the staged row, creates and flushes the derived `Asset`, verifies and pins the copied source through `direct_intake.pin_copied_source`, writes `SourceVersion` and the optional `RequestKey`, stages submit and removes the staged row in one commit (Figure 25).

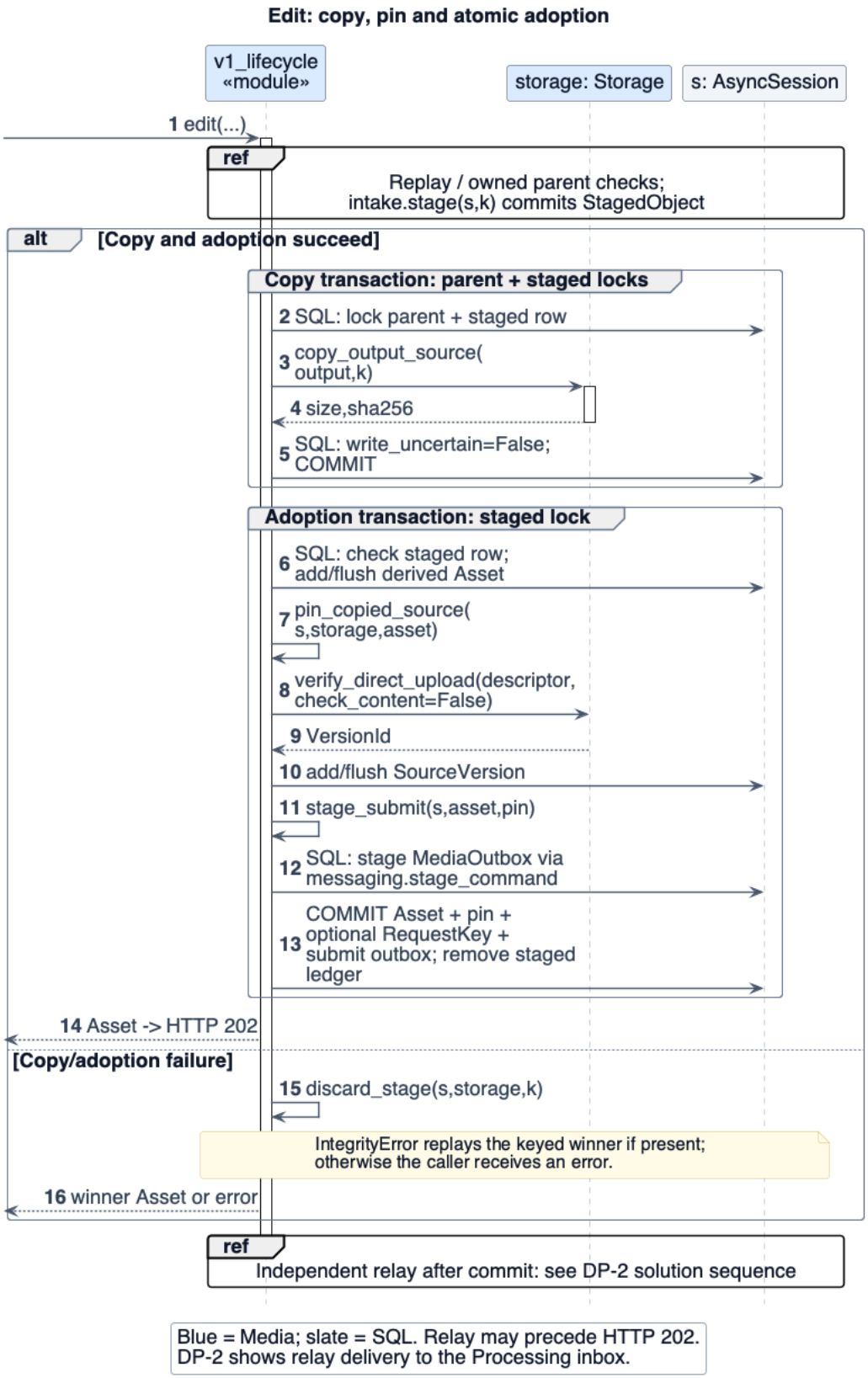


Figure 25. Edit copies the parent's primary, then verifies and pins it inside the adoption transaction that commits Asset, SourceVersion and submit intent.

Unlike direct finalize, pin verification happens **inside** edit adoption. The `MediaRepair` handler relays the submit command only after the adoption commit, possibly before HTTP 202 reaches

the caller; Figure 81 shows the relay and Processing inbox. A failed copy or adoption keeps its cleanup authority, and a losing keyed edit discards its copy and replays the winner (`test_edit_local_projection_adoption_and_submit` and `test_uncertain_copy_keeps_cleanup_and_never_submits` in `tests/media/test_v1_projection.py`).

Stored-source retry (UC-4). When execution authority expires, `processing/recovery.py:recover_expired` abandons the Attempt and fails the Job under the Job lock. `v1_lifecycle.retry` checks the owner, failed projection, stored pin and optional `RequestKey`, then commits the retry command. Processing's `_retry_locked` creates the next queued Attempt for a new `(job_id,request_id)`, keeping source and operations; a replay returns the current Attempt. Figures 44, 46 and 47 in section 6.4 show the records, the successor transaction and the fences that stop a superseded `Claim` from finishing (`tests/v1/test_messaging_sql.py:test_retry_idempotency_and_cancel_fence_old_and_current_attempts`).

Independent release (UC-7) is described in Continuous Delivery (section 5) and its class/object models in section 6.4; DP-1 in section 6.5 explains reservation and recovery-journal choices.

3.3 Other Architectural Decisions

The remaining register decisions, and the project decisions that shaped the architecture, are listed below. AD-29 is recorded with the physical decisions (section 3.2.1).

Identifier	Decision	Rationale and trade-off	Status
AD-03	Synchronous submit with asynchronous Redis/RQ processing (proposal)	Replaced so that no always-on broker runs and Media does not depend on Processing at request time; see AD-11	Superseded by AD-11
AD-04	Idempotent `SubmitJob` keyed on `asset_id`, plus reconciliation; outbox only if needed (proposal)	Idempotency (`SubmissionGuard`, unique `asset_id`) and reconciliation (`repair_work`, lease expiry) were kept. The outbox was adopted from the start because SQL plus SQS is not atomic.	Changed (AD-11)
AD-05	Presigned URLs for source reads and output delivery (proposal)	Uploads and playback stay presigned. The worker reads the source with an IAM grant pinned to a VersionId instead, which removes a live Media callback from the encode path. This is broader than a signed URL, and owner binding is checked in code (section 3.2.3).	Changed (AD-14, AD-20)
AD-06	Service token plus forwarded owner identity on a private network (proposal)	Peer business HTTP is unused. Gateway-trusted Actor and IAM establish the invocation boundary, with SQL roles enforcing data ownership	Superseded by AD-09, AD-20
AD-07	Containers on AWS in one small environment (proposal)	Same small-environment intent, managed execution: domain handlers use Lambda container images; finite encoders, migrations and bootstrap use Fargate tasks. WorkerImageUri also supplies the short Lambda.	Changed (AD-10, AD-12, AD-13, AD-15, AD-23)
AD-08	Fresh demonstration data; no Project 1 data migration	Keeps effort on the contribution and sets the data lifecycle (section 3.2.3). The deploy pipeline seeds one smoke account.	Kept
AD-19	Closed-by-default stack operated as booked sessions (V1)	Kept idle spend near zero, but made the platform unavailable outside sessions and cost a full deploy and teardown per check	Superseded by AD-27

Identifier	Decision	Rationale and trade-off	Status
AD-22	Opt-in V2 experiments in the same template, off by default: unattended launcher (#68), short encodes on Lambda (#93), one cluster per database (#94)	Keeps experiments revertible: with default flags the template is byte-identical to V1, which a test checks. #68 became the V2-live default (AD-27). #93 and #94 were enabled on `dc2-live` on 21 October (section 4.1, AD-29); per-run stacks keep the defaults.	New
AD-28	**Release admission scans both the candidate and the retained rollback image**, and handles a newly published advisory that affects only the retained image without bypassing the gate (section 5.3)	A restore target must not silently reintroduce a fixable HIGH/CRITICAL finding, but a newly disclosed advisory against the *old* image must not block the release that fixes it. The retained-image exception is Trivy-only; ECR completed-scan validation still applies. See `docs/architecture/decisions.md`.	New (21 October)

Project decisions with architectural effect.

ID	Decision	Effect
F-01	Lecturer accepted the proposal and asked for detailed DDD to confirm the service choice	The capability derivation, typed domain models and context map (section 3.1.4) explain AD-01
D-01	Baseline is the team's local frozen FlickPond checkout `8854537` (the proposal named `890c5ec`)	Inherited capability is identified separately from Team 17's work
Freeze, 20 Oct	Team-accepted demo release frozen at `daee44b` (`docs/release/demo-release-manifest.md`); afterwards, only blocker repairs with verification and a manifest update	Later engineering releases (`629119c` on 20 October, `c7c489c` on 21 October) are recorded as post-freeze addenda with their own receipts

3.4 Architectural Limitation

The table lists the material architectural limitations, their impact and a resolution horizon. Owners follow the agreed specialisms: cloud and delivery, Guruprasath Gopal; backend, Ibrahim Mammadov; domain, contracts and verification, Nguyễn Kim Long.

C means the course deadline, 16 November 2026. The post-course roadmap uses **R1** for month 1, **R2** for months 2–3 and **R3** for later work. The course environment is planned for teardown after 16 November; later work starts from the repository and template.

Identifier	Issue and impact	Description	Resolution	Owner	Status
AISS-01	Startup and finite-capacity latency	Finite Fargate task start dominates standard-lane queue wait. Startup latency and finite capacity constrain response time; measured results are in section 4.1.	R1: stage timestamps, repeated representative workloads and worker CPU/image profiling. R2: consider wider short-lane bounds after measurement.	Long, Guruprasath	Open
AISS-02	Public regional API, no WAF	The execute-api endpoint is reachable without CloudFront; the authorizer and stage throttle are the HTTP controls.	R1: evaluate CloudFront-only origin restriction and managed WAF rules with negative-access checks.	Guruprasath	Open
AISS-03	Image-only restore	Alias and image rollback does not restore schema. Consumers accept contract version 1; database changes must stay compatible with the retained image.	R2: expand-and-contract migrations and automated old-image-on-new-schema verification.	Ibrahim	Accepted; release policy

Identifier	Issue and impact	Description	Resolution	Owner	Status
AISS-04	Single-environment operations	`dc2-live` is the only environment. Main requires a pull request and Full verification, and delivery then runs automatically with no separate environment approval. The deploy role holds AdministratorAccess with a region lock. Alarms and a dashboard exist without paging or tracing; logs are kept seven days.	R1: least-privilege deployment role, paging target and alarm drill. R2: a staging environment from the same template.	Guruprasath	Open
AISS-05	Single-instance recovery	Separate clusters remove the shared capacity boundary, but each cluster has one instance and no reader failover. No restore drill or recovery-time measurement has been recorded.	R1: restore and recovery drills against unchanged bounds. Reassess replica capacity if availability needs justify it.	Long, Guruprasath	Open

At-least-once delivery, finite encoder caps, legacy quarantine and demonstration data retention are deliberate properties explained with their decisions. Scope exclusions are in section 1.4.2.

4. Quality Attributes

The results below keep the proposal's targets, workloads and timing boundaries; Figure 26 charts the speedup and release-time results. The measurement record is in `docs/results/actual-vs-target.md` and `docs/evidence-index.md`.

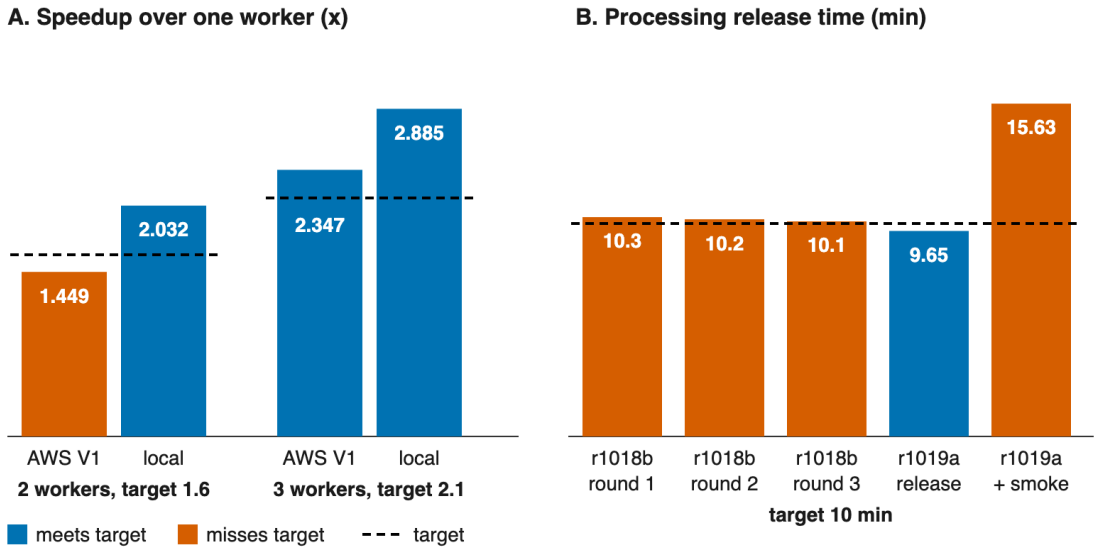


Figure 26. Speedup and Processing release time against the proposal targets (AWS V1 and local pre-V1 speedup; r1018b and r1019a release times).

Attribute	Scenario (proposal target)	Result	Verdict
Scalability	2 workers ≥ 1.6× and 3 workers ≥ 2.1× the jobs/min of 1 worker	AWS V1 (6 clips, one run per cap): 0.483 / 0.700 / 1.134 jobs/min → **1.449×** and **2.347×**	**FAIL** at 2; PASS at 3
Scalability	Median queue wait falls at each step; 0 failed jobs	274.3 s → 135.7 s → 9.6 s; 0 failed; 0 encoders left running	PASS
Scalability	(same, local pre-V1 design, 30 clips × 3 repeats)	2.032× and 2.885×	PASS (pre-V1 design)
Scalability	V2-live automatic scale 0 → 3 → 0	0 → 3 → 0 with 3 concurrent clips; 9-clip backlog at cap 3 in 239 s (2.26 jobs/min)	PASS
Performance	50 users polling status for 3 min: p90 ≤ 700 ms, < 1 % errors	p90 **58.2 ms**, 9,000 requests, 0 errors (in-region generator)	PASS
Performance	20 concurrent logins: p90 < 2 s	p90 **178 ms** (20 samples)	PASS
Performance	Warm-job queue wait ≤ 20 s (V2-live), standard Fargate lane	about **37 s**; Fargate start median 25.6 s	**FAIL**
Performance	Same wait on the Lambda short-encode lane (≤ 60 s clips, `sl1021`, 5 jobs per arm, one 24 s clip)	median **0.26 s** (max 2.6 s) against 32.6 s on Fargate; processing 5.3 s against 8.0 s; billed compute USD 0.0003 against 0.0021 per encode; 0 failures	PASS
Performance	Submit-to-done, same run (20 s aim; no proposal target)	Lambda cold 92.0 s (n = 1), warm median **45.2 s**; Fargate median 87.7 s	Above the 20 s aim
Availability	Worker killed mid-encode: re-queued or failed within 5 min	Stop detected in **5.8 s**; lease expiry marks the job `failed`; retry creates a new attempt	PASS
Availability	Lost queue entry re-enqueued within 2 min	23.5 s (local pre-V1); ≤ 120 s asserted on a simulated clock (local V1)	PASS locally; **NOT_RUN** on AWS
Availability	Processing temporarily unavailable while Media accepts uploads	Command consumer off for 48 s: Media answered 202 and held the command; job `done` 58.9 s after re-enable	PASS
Deployability	Release Processing alone, smoke-tested, in ≤ 10 min; Media unchanged	Operator promote 322.5 s; CD `Release` step **10.1–10.3 min** (`r1018b`) and 9 min 39 s (`r1019a`); release plus the separate upload smoke **15 min 38 s**; Media unchanged at every snapshot	**FAIL** for `r1018b` and release + smoke; PASS for the `r1019a` step
Deployability	Bad candidate restores automatically	CD run 39271754470 (`bad_candidate=true`) → `restored`	PASS (source `7928b7c`)
Security	Negative tests 100 % rejected; 0 fixable HIGH/CRITICAL at release	Selected negatives rejected on AWS V1 and on `dc2-live` (#229, 19 October); recorded releases met the fixable-image policy	PASS
Reusability	Publisher CLI: 0 service imports, 0 database credentials, ≤ 2 person-days	0 and 0, enforced by a test; 495 physical Python lines across five files; effort not timed	PASS; effort **NOT_MEASURED**

4.1 Performance

HTTP distribution, queue buffering and bounded encoding are separated, as recorded in **AD-30** (section 3.2). API Gateway routes requests to Media or Processing Lambda aliases with a stage throttle of 20 requests/s and a burst of 50. SQS buffers encode work; the Processing

launcher reserves capacity under a database lock and starts at most three finite Fargate tasks. Sources of at most 64 MiB, 60 s and 1080 pixels can use the short Lambda lane (batch size one, concurrency two, 3008 MB), which avoids a Fargate task start for small encodes; a refused probe redirects to the standard lane (`processing/short_lane.py`, `processing/short_encoder.py`; UC-2 in section 6.4).

Other tactics keep the API path short:

- Upload bytes go directly to S3 using presigned POST; MP4 range requests and HLS segments use signed object URLs. Lambda carries JSON control requests.
- CloudFront caches the content-hashed frontend. API caching is disabled and authenticated responses carry `Cache-Control: private, no-store`; playlists and signed URLs remain fresh.
- Media answers status from its own projection without a Processing business API call. Listing joins owned assets and projections in one query; a single-job read loads the owned asset and then its projection (`media/repositories/assets.py`, `media/projections.py`).
- Standard encoders use 2 vCPU / 4 GiB tasks. The HLS ladder stays within the source height.
- Warm Lambda environments reuse one pooled SQL connection; a 60 s server-side idle-session timeout releases idle connections so Aurora can auto-pause (`runtime_support/lambda_db.py`). The recorded first read after a pause took about 4.7 s; later reads took about 263 ms **[AWS]**.

4.2 Availability

Media commits accepted work and its outbox intent together, and Processing commits its inbox, job state and outboxes before acknowledging a command, so queue buffering separates accepted work from the moment an encoder can run (DP-2). Pending outbox rows are republished and missing queued work is rebuilt from SQL by a one-minute repair tick and by ECS `STOPPED` and S3 object events (`delivery/sql.py`, `processing/recovery_v1.py`). The command/result mappings each allow at most two concurrent Lambda invocations and report partial batch failures. With the standard cap of three and the short cap of two, a burst becomes backlog rather than an unbounded fleet. In the consumer-interruption experiment Media kept accepting uploads while Processing consumption was paused.

Database-clock leases, task bindings and deadlines fence stale workers. A failed attempt becomes visible, and owner retry creates a new attempt over the stored source. Workers cannot publish queue messages. DLQs hold poison messages after five receives; the recorded poison message reached its DLQ after 2,101 s **[AWS]**. Managed services and subnets across two availability zones provide the infrastructure boundary. Each service's Aurora cluster is a single instance in one region, with one-day backups (AD-29).

Owner-release smoke failure starts automatic restore (section 5.3). Stateful resources use `DeletionPolicy: Retain`, and deployment rejects replacement of Aurora, buckets or the distribution. The enabled configuration defines 15 alarms and a services dashboard (`scripts/aws/v1/definitions.py`).

4.3 Security

Every protected request passes the Media-owned Lambda REQUEST authorizer and then service-side checks of the current user and trusted Actor (section 3.1 and DP-3), so each service acts only for a verified current owner. Health and the Media register/login/logout routes are the only public exceptions.

Area	Implemented control
Owner scoping	Both services filter by the trusted owner; foreign job/asset IDs return 404. Administrative operations require the operator role.
Session and validation	Host-only `HttpOnly`, `Secure`, `SameSite=Strict` cookie; one-hour token; authenticated responses use `private, no-store`. bcrypt cost 10 hashes passwords. Upload validation enforces allowed types and 100 MiB; adoption verifies size, hash, signature and pinned S3 version.
Least-privilege IAM	Separate function, migration and worker roles; resource-scoped S3, queue and parameter permissions; queue policies restrict producers/consumers; separate database identities and no runtime DDL. The enabled configuration defines 19 IAM roles (17 at default flags).
Developer and delivery credentials	GitHub Actions assumes the AWS deploy role through OIDC and the branch-restricted `aws` environment, without long-lived AWS keys in GitHub. Runtime access uses AWS roles and scoped SSM SecureStrings.
Encryption	In transit: CloudFront HTTPS, TLS 1.2 to the API origin, TLS-only S3/SQS policies and Aurora `rds.force_ssl=1` with CA verification. At rest: encrypted Aurora storage, SSE-S3, SQS-managed encryption and SSM SecureStrings.
Network	Lambda and Aurora use private subnets. Media database ingress on TCP 5432 permits the Lambda and migration security groups; Processing additionally permits the worker security group. Worker tasks have no inbound application listener.
Source and supply chain	Ruff S and CodeQL provide SAST; gitleaks scans history; npm audit, pip-audit and Trivy check dependencies and images; cfn-lint and Checkov inspect generated infrastructure. Section 5.2 gives triggers and dispositions.
Dynamic checks	Authenticated OWASP ZAP runs against the disposable local V1 stack in CI and weekly (`dast.yml`); execution, coverage and cleanup are required, and findings are reported for triage.
Operations	Gateway throttles, service/DLQ alarms and a budget alarm support detection and containment; WAF/GuardDuty/IDS are tracked in AISS-02.

Infrastructure controls are generated in `scripts/aws/v1/definitions.py`; CI credential trust is specified in `deploy/aws/v1/github-oidc.yaml`. The 22 October CodeQL dismissals and the accepted-unfixed FFmpeg CVE-2025-52290 are dated triage records (`docs/execution/aws/v1/scan-accepted-unfixed-20261017.json`).

Verification. The following negative tests were recorded on AWS V1 stacks `r1017b`/`r1018b` and on `dc2-live` for issue #229 on 19 October **[AWS]**.

Control (proposal §3.3)	Result
Unauthenticated requests rejected	401 on all six probed job routes, through CloudFront and directly through Gateway
User A cannot read, retry, edit, delete or play User B's assets	404 for B on every Media and Processing route for A's job
Forged identity headers and token-less internal calls	Forged headers changed nothing; anonymous admin 401, non-operator 403
Direct queue send by an unlisted principal	403 on all three queues; 0 messages accepted
Each service's credentials refused on the other's database	`InsufficientPrivilegeError`; runtime DDL denied
Expired and tampered presigned URLs; no public listing	Tampered 400/403; both 900 s expiries waited out (206/204 before, 403 after); unsigned S3 403
Invalid and oversized uploads	100 MiB + 1 byte refused by S3 (400) and never adopted
External port scan; private datastores	Only 80 and 443 open; Aurora `PubliclyAccessible=false`; no `0.0.0.0/0` ingress
Secrets out of git and images	gitleaks green in CI

Control (proposal §3.3)	Result
Zero fixable HIGH/CRITICAL image findings at release	Met at the recorded releases; retained-image handling is described in section 5.3

The ZAP scope is seven GET operations on the local bridge: Media `auth/me`, `health`, `jobs`, `jobs/{job_id}` and Processing `health`, `jobs`, `jobs/{job_id}`, under `/api/<service>/`. The harness caps requests at 1,600, seeds two users and uploads, and invokes no workers. The 25 October run retained 16 findings and passed cleanup; ownership-negative fixtures are separate checks (`scripts/dev/v1_dast.py`).

4.4 Extensibility and Maintainability

Media separates routes (`media/api/`), application services (`media/services/`), repositories, owned models/migrations and projection, messaging and recovery modules. Processing separates API/authentication, lifecycle and messaging, launcher/adapters, recovery and worker execution (`processing/worker/`); `media_engine/` holds validation, probes and FFmpeg command builders. The services share only contract and technical libraries (`contracts/`, `delivery/`, `runtime_support/`).

A new operation extends `media_engine/` and the submit payload without changing queue transport. A new client uses the published OpenAPI contract and reserve/upload/finalize/status flow. The Publisher demonstrates this with 495 physical Python lines across five files, no service imports and no database credentials, checked by AST and public-contract tests. Image checks reject opposite-domain and legacy imports; generated-template freshness and IAM-shape tests protect infrastructure boundaries.

Contract changes are additive within a version, with consumer-owned fixtures. A breaking change first releases readers for N and N-1, then raises the emitted version after both readers are ready. `contracts/v1.py` accepts only positive supported versions, so the current N=1 accepts version 1; the rolling-upgrade policy is exercised with raised-version fixtures (`contracts/README.md`).

4.4.1 Unit tests for complex classes and modules

The modules below hold the most complex behavior. Counts are static `test_` definitions in each named file; the two parameterized files report collected cases. Shared files support several modules, so rows are not additive.

Complex module / class	What it does	Unit test file(s)	Number of tests
Media projections / `ProjectedJob`; `v1_lifecycle.edit`	Orders result snapshots, fences terminal state, adopts edits and handles replay	`tests/media/test_v1_projection.py`	12 definitions
Media `direct_intake`	Reserves and verifies uploads, adopts pinned versions, schedules late-write cleanup	`tests/media/test_direct_intake.py`	26 definitions
Media `Storage`	Signs, streams, verifies, copies and removes versioned objects	`tests/media/test_media_storage.py`; source-verification cases in `test_direct_intake.py` above	7 storage definitions
Media native recovery	Stops at deadlines, orders cleanup phases and caps batches	`tests/media/test_native_recovery.py`	13 cases
Processing launcher / `LauncherConfig`, `Observation`	Observes ECS/queue demand and controls bounded reservations	`tests/aws/v1/test_unattended_launch.py`	40 definitions

Complex module / class	What it does	Unit test file(s)	Number of tests
Processing launcher `tick()`	Avoids idle I/O, preserves unknown demand and stops refused/ambiguous sends	`tests/processing/test_launcher_tick.py`	12 cases
Processing messaging	Validates durable work identity and lane routing	`tests/processing/test_v1_delivery_rules.py`	10 definitions
Processing SQS worker	Bounds polling and fences claim, acknowledgement and visibility loss	`tests/processing/test_v1_worker.py`	32 definitions
Processing lifecycle / `Claim`	Claims, heartbeats and finishes under lease/attempt fences	`tests/processing/test_lifecycle.py`	10 definitions
Processing recovery helpers	Expiry and output-cleanup rules used by native recovery	`tests/processing/test_recovery.py`	10 definitions
Short-lane admission and encoder	Enforces size/probe bounds, redirects and handles partial batch failure	`tests/processing/test_short_lane.py`; `test_short_encoder.py`	16 + 16 definitions
Engine validation, commands, HLS / `SourceProbe`	Rejects invalid operations and builds safe FFmpeg/probe/HLS commands	`tests/media_engine/test_validation.py`; `test_commands.py`; `test_hls.py`	10 + 3 + 12 definitions
Publisher `PublicClient`, `Intents`, CLI	Persists cookies and upload/file identity; maps CLI outcomes	`tests/publisher/test_publisher.py`; `test_direct_upload.py`; `test_file_identity.py`	26 + 4 + 3 definitions
`Actor`, `Source`, `Envelope`, `GatewayAdapter`, delivery	Validates trusted boundaries, frozen messages, partial failures and poison handling	`tests/contracts/test_prepared_contracts.py`; `tests/v1/test_contract_fixtures.py`; `test_delivery.py`	7 + 17 + 5 definitions

All 25 native-recovery and launcher-tick cases passed, and eight deliberate mutations (deadlines, cleanup failure ordering, batch bounds, idle I/O, unknown demand and send termination) were each detected. Real PostgreSQL suites verify transactions, locking and concurrency (`tests/v1/test_media_cleanup_sql.py`, `test_launcher_sql.py`, `test_messaging_sql.py`, `test_delivery_recovery_sql.py`).

4.4.2 Test inventory and coverage gates

The 10 November audit recorded **1,316 offline cases passed** across two runs (820 service cases and 496 V1/launcher cases), with two skips; integration and e2e cases were deselected. Its static service inventory was **386 unit test definitions** across Media 150, Processing 167, engine 25, Publisher 34, contracts 7 and runtime support 3, before the 25 cases above were added.

Scope	Combined statement/branch minimum	Raw branch minimum	Enforcement
Every measured backend package	80%	Reported	Per-package gate; missing production files fail
Media and Processing packages	90%	85%	Additional gate with the complete five backend tiers
`media/recovery_v1.py`, `processing/recovery_v1.py`	80%	75%	Additional module gate with the complete five tiers
Frontend `app.js`	90% each for statements, branches, functions and lines	90%	Vitest coverage job

The exact gates are in `scripts/dev/coverage.py:19–39,97–202` and `frontend/package.json`. Backend evidence combines source-fingerprint-matched fast, legacy integration, Processing integration, Media integration and V1 integration fragments; CI retains XML, JSON and `gate.json`, plus JUnit and coverage summaries (`scripts/ci/quality_summary.py`).

5. DevOps and Development Lifecycle

5.1 Source Control Strategy

FlickPond uses the `Collaboration95/dc2` GitHub monorepo, with `Main` as the default branch. A contract change and its consumer tests can be reviewed together, while service images, CODEOWNERS and path-selected CI preserve ownership boundaries.

Path	Responsibility / release unit
`media/`	Media API, application services, persistence and migrations; Media image
`processing/`, `media_engine/`	Processing API, orchestration and encode pipeline; Processing and Worker images
`contracts/`, `delivery/`, `runtime_support/`	Shared contracts and technical libraries copied into service images
`frontend/`, `publisher/`	Browser client and independent Publisher CLI
`deploy/`, `scripts/aws/`	Image recipes, CloudFormation generation, deploy/release/operator tooling
`.github/workflows/`, `scripts/ci/`	CI selection, verification, security, delivery and publication
`tests/`	Unit, contract, integration, infrastructure, pipeline and browser checks
`docs/`, `project-documentation/`	Architecture decisions, runbooks, sanitized evidence and report sources
`app/`, `legacy/`, `migrations/`	Inherited/regression paths, excluded from V1 runtime images

Branches and pull requests. Feature/fix branches such as `fix/*`, `v2/*`, `ci/*` and `docs/*` merge by PR into `dev/closeout`, then an integration PR promotes the combined change to `Main`. The earlier V1 build used `dev/v1-integration` → `dev/v1` → `Main`. A PR records the problem, technical decisions and verification; CODEOWNERS identifies the relevant reviewer. Commit messages use short action-oriented descriptions, with `feat:` and `fix:` prefixes where appropriate. Source SHAs and image digests identify releases in receipts; `project2-v1` marks the V1 release.

A PR into `dev/closeout` normally uses changed-input selection and **Dev gate**; workflow, dependency or unknown inputs escalate it to full verification. PRs into other bases, including `Main`, run **Full verification**. `Main` is a protected branch: a merge requires a pull request and the required status check **Full verification**. Guruprasath Gopal, the cloud and delivery lead, approves promotion to `Main`. Main push CI then verifies the merged source again before automatic CD.

Developer authentication and protected secrets. Developers authenticate to GitHub with their GitHub accounts and use SSH keys or authenticated HTTPS credentials for Git transport. Repository credentials grant source access; AWS delivery uses a separate OIDC trust boundary. Secrets belong in protected GitHub Actions secrets or scoped AWS parameter/secret storage, and only sanitized evidence is committed. Push protection and full-history gitleaks provide secret checks; `.gitleaks.toml` holds the reviewed allowlist.

Submitted scripts. The repository is submitted with this report. The pipeline scripts are in `.github/workflows/` (CI, CD, DAST and advisory security scans), `scripts/ci/` (selection

policy, CD admission and delivery) and `scripts/aws/v1/` (template generation, deploy, release and restore). The test scripts are in `tests/` (unit, contract, integration, infrastructure and pipeline tests) and the frontend Vitest suites in `frontend/*.test.js`.

5.2 Continuous Integration

`.github/workflows/ci.yml` selects and verifies the candidate source. It calls `v1.yml` for V1 verification and `dast.yml` for authenticated local-stack DAST. Figure 27 places security tools inside the verification pipeline and distinguishes independent advisory workflows.

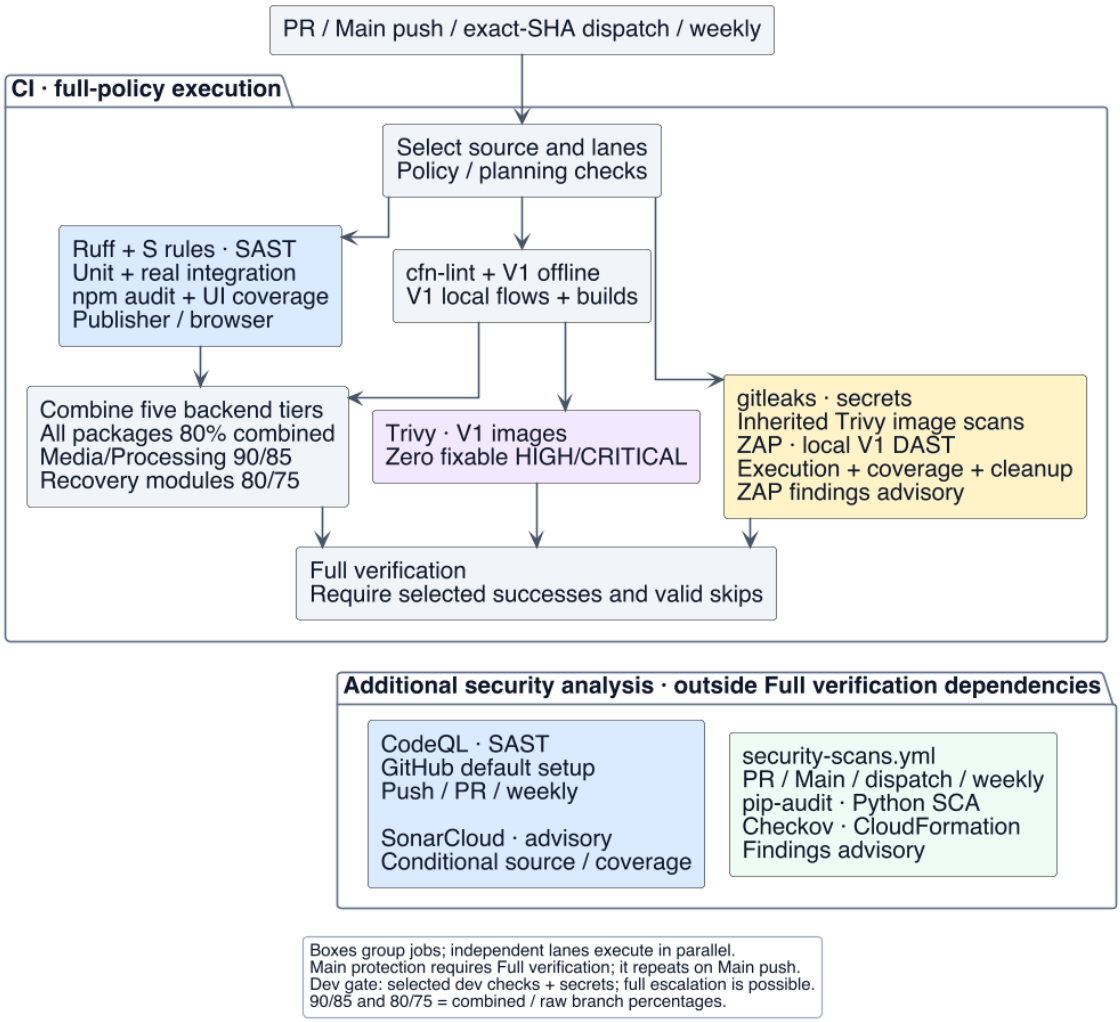


Figure 27. CI triggers, parallel verification lanes, security stages and aggregates. CodeQL and advisory scans execute independently of Full verification.

Trigger	Selection / purpose
PR into `dev/closeout`	Changed-input dev selection, with conservative full escalation
PR into any other base, including Main	Full verification of the PR merge source; required to merge into Main
Push to Main	Full verification of the exact merged source; eligible for automatic CD
Manual CI dispatch with `candidate_sha`	Full verification; exact SHA must match the dispatched ref
Sunday 03:00 UTC	Full scheduled regression on the default branch

`scripts/ci/policy.py` emits a source-bound `ci-selection` artifact. Both aggregates validate job results and intentional skips. Dev gate requires the selected dev checks and an

unconditional secret scan. Full verification additionally requires selected image security, DAST execution and combined coverage. Failed, cancelled or unexpectedly skipped required checks fail the aggregate. CodeQL, SonarCloud and `security-scans.yml` are outside its dependency list.

Verification and security stages. “Selected/full” means the lane is selected by policy and always included in full mode. Independent workflows have their own triggers.

Stage	Tool	What it checks	Trigger	Blocking or advisory
Selection and policy	Python policy/planning tests	Changed inputs, full/dev selection and aggregate rules	Every CI event	Blocking
Lint and source security (SAST)	Ruff, format check, explicit Ruff S rules	Maintained Python code and bootstrap; S rules are Bandit-derived	Lint selected/full	Blocking findings
Fast unit tests	pytest	Service/module behavior; fast coverage fragment	Lint/unit selected/full	Blocking
Frontend and dependency SCA	Vitest, npm audit	UI behavior/coverage and locked frontend test-tool dependencies	Frontend selected/full	Blocking; audit HIGH/CRITICAL
Real integration and public flows	pytest, PostgreSQL, storage, FFmpeg, Publisher HTTPS, browser trust	Owned persistence/recovery, migrations, TLS/cookies and browser/client acceptance; service image/runtime checks	Relevant lane selected/full	Blocking
V1 infrastructure validation	cfn-lint and offline tests	Generated shared/separate/split template schema, freshness, IAM and control-plane rules	V1 offline selected/full; manual V1	Blocking
V1 local flows	V1 harness, PostgreSQL, Moto, FFmpeg, browser	Current intake, processing, recovery and playback behavior	V1 live selected/full; manual V1	Blocking
Image security	Trivy; V1 scan validator	Inherited app/Media/Processing matrix and exact V1 Media/Processing/Worker images; source/config identity	Image lanes selected/full; manual V1 `images=true`	Blocking: zero fixable HIGH/CRITICAL; malformed/wrong-image reports fail
Secrets	gitleaks	Full Git history against the reviewed allowlist	Every CI event	Blocking; required by both aggregates
Dynamic security (DAST)	OWASP ZAP	Authenticated seven-route scan of the disposable local V1 stack	DAST selected/full; `dast.yml` exact-SHA dispatch; Monday 03:00 UTC	Execution, coverage and cleanup block; findings advisory
Combined coverage	coverage.py, Vitest	Five source-matched backend tiers; 80% package floor, Media/Processing 90/85 and recovery 80/75; frontend 90%	Backend coverage selected/full; frontend lane	Blocking; exact gates in section 4.4
Managed source security (SAST)	CodeQL default setup	Python, JavaScript/TypeScript and GitHub Actions	GitHub-managed push/PR analysis and weekly schedule	Independent; outside Full verification
Source/coverage analysis	SonarCloud	Conditional source-only or full-coverage analysis	Configured dev/full analysis jobs	Advisory; skipped in audited runs

Stage	Tool	What it checks	Trigger	Blocking or advisory
Python runtime dependency SCA	pip-audit 2.10.1	Frozen lockfile export: union of V1 Media/Processing/Worker runtime extras	`security-scans.yml` job "Python runtime dependency SCA (advisory)": PR, Main push, dispatch, Monday 03:30 UTC	Advisory findings; execution/report errors fail its job
Static infrastructure security	Checkov 3.4.3	Disposable shared/separate/split CloudFormation renders plus OIDC template	Same workflow, job "Static CloudFormation security (advisory)"	Advisory findings; execution/report errors fail its job
Aggregate	Dev gate / Full verification	Required selected job results and permitted skips	After dependencies complete	Blocking workflow verdict

`coverage-gate` combines the five backend fragments (fast, legacy, Processing, Media and V1 integration), and Full verification checks it with the other lanes (`ci.yml:581–643`, `scripts/dev/coverage.py`).

The two `security-scans.yml` jobs run without AWS credentials and retain JSON reports with source, lock-file and tool identity for 14 days (`scripts/ci/security_scans.py`). The recorded scan found **0 pip-audit findings in 40 applicable packages**. Checkov reported failed-check counts of **shared 26 / separate 31 / split 31 / OIDC 1**; these overlap across the templates and are advisory.

ZAP drives the actual API handlers and authorizer through a disposable HTTPS loopback bridge with PostgreSQL and Moto S3/SQS (routes in section 4.3). Sanitized ZAP findings are kept for 14 days and coverage and image receipts for 60 days; supported SARIF results are published to code scanning (`dast.yml`, `v1.yml`).

Dated verification evidence. The 21 October full Main run 39531399249 (`4ae6a0b`) passed 25 selected jobs with three intentional skips in 7 min 9 s.

Run / date	Source and event	Outcome
CI 39931072197, 23 October	`76282d3`, Main push	Full CI passed
CI 40174362164, 25 October	`76282d3`, scheduled regression on Main	Inherited app/Processing Trivy and Full verification failed; V1 images and local ZAP passed
CodeQL 40189407647, 25 October	`76282d3`, managed analysis	Python, JavaScript/TypeScript and Actions analyses completed
CI 41294802851, 1 November	`76282d3`, scheduled regression on Main	Full verification, coverage, secrets, images and CI-called local DAST passed
CI 42468024855, 8 November	`76282d3`, scheduled regression on Main	Failed after setup/download HTTP 504s; coverage skipped
CI 42949129268, 11 November	`3d75dd9`, PR #326 into Main	Full verification passed, including coverage, image scans, secrets and CI-called local DAST

The two October ZAP receipts recorded seven routes, 16 findings, 146 out-of-scope requests blocked and successful cleanup.

5.3 Continuous Delivery

`.github/workflows/cd.yml` invokes `scripts/ci/cd.sh`, which uses the existing deploy and per-owner release tools, including `scripts/aws/v1/release.py`. Figure 28 shows the automatic path and owner-release recovery boundary.

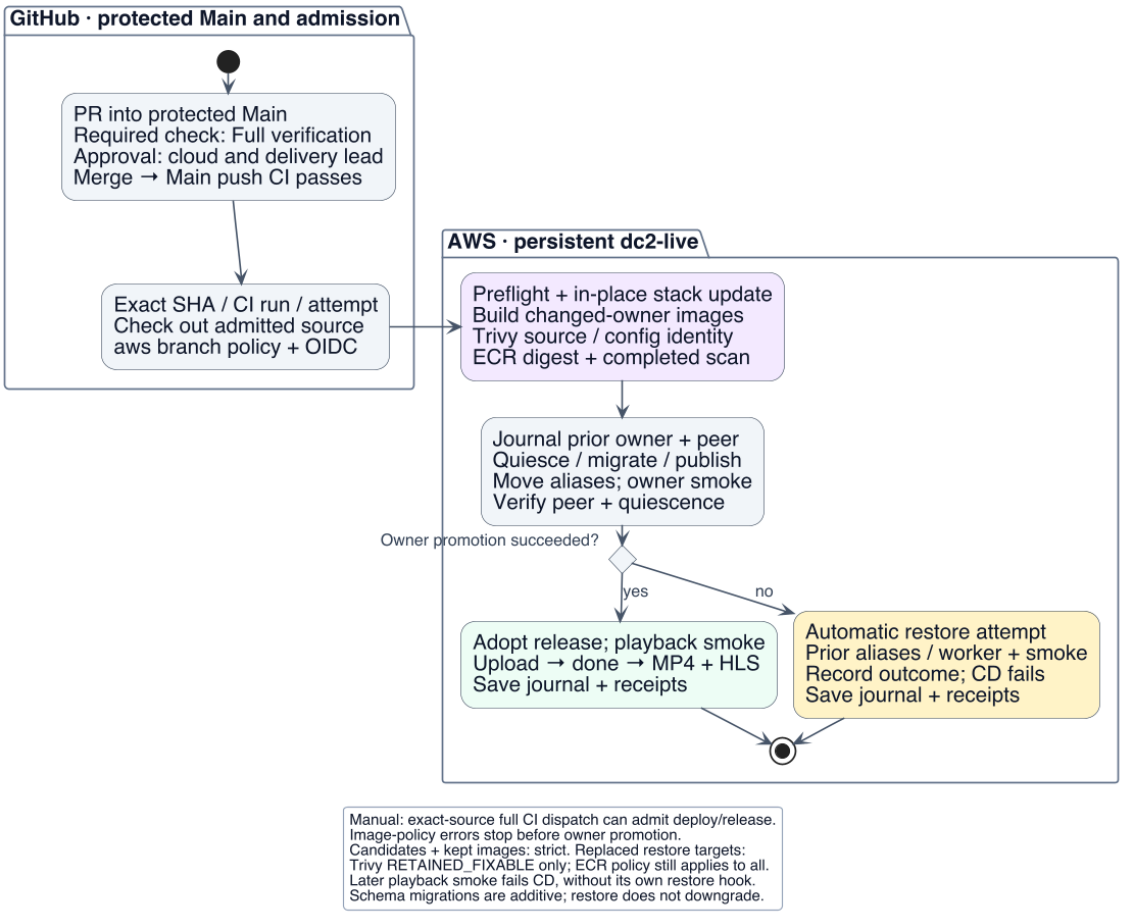


Figure 28. Delivery from the protected Main merge and exact-source CI admission through OIDC, image admission, journaled owner release, smoke/restore and playback verification.

Environment	Purpose and lifecycle
Local V1	Developer and CI disposable PostgreSQL/Moto/FFmpeg stack; fast and full public flows (`make setup-v1`, `make v1-up`, `make v1-demo`)
Per-run AWS V1 proof stacks	Disposable `r1017b`, `r1018b`, `r1019a` deployments for bounded cloud proofs, followed by teardown (AD-19)
Persistent `dc2-live`	Production-demo stack, updated in place after eligible Main CI (AD-27)

Promotion approval. Guruprasath Gopal, the cloud and delivery lead, approves the pull request that promotes a change to the protected `Main` branch; the merge also requires a green **Full verification**. CD then deploys automatically after a green Full verification on the Main push source, with no second human approval. The admission job binds the triggering repository, source SHA, run and attempt; PR and scheduled CI runs cannot admit this automatic path (`scripts/ci/cd_gate.py`). The `run` job checks out the admitted SHA and assumes the AWS role through GitHub OIDC in the branch-restricted **`aws` environment** (Main and `dev/closeout`). A concurrency group serializes deliveries. A manual deploy or release can instead use an eligible exact-source full CI dispatch.

Deployment verification.

1. **Admission and preflight.** Retain the exact-source admission record, read stack/journal state and inspect the change set; replacement of Aurora, buckets or the distribution is rejected.

2. **Image admission.** Release only owners whose image build inputs changed. Build and scan the promoted bytes, bind config/source identity to immutable registry digests, and require completed matching ECR scans. Candidate and kept-running image sets require zero fixable HIGH/CRITICAL Trivy findings. ECR separately rejects unaccepted HIGH/CRITICAL findings (`scripts/aws/v1/artifacts.py`, `scripts/aws/v1/proof/release_plan.py`).
3. **Journaled owner release.** Record prior owner and peer state, quiesce the owning work, run the migration task, publish Lambda versions, move live aliases (and, for Processing, the worker definition), journal each step and verify the peer is unchanged (`scripts/aws/v1/release.py:240–394`).
4. **Owner smoke and automatic restore.** The public owner smoke checks the candidate's contracts, including health/authentication and quiescence. On promotion failure, `cd.sh` restores the prior aliases/worker and smoke-tests the restored state. The journal records `restored`, `abandoned` when nothing changed, or `restore-failed`. Schema migrations are additive and are not downgraded (`scripts/ci/cd.sh:600–640`, `release.py:397–464`).
5. **Upload-to-playback smoke.** After deploy/release, upload a real clip through the public API, wait for `done`, then read MP4, HLS playlist and segment. Failure fails CD; this later smoke is outside the owner restore boundary (`cd.yml:230–241`).
6. **Receipts.** Retain sanitized admission, scan, release and playback results for 60 days. Save resumable private journal state to versioned S3, including after failed runs.

The Trivy-only `RETAINED_FIXABLE` exception (AD-28) applies to a replaced image held solely as a restore target: its findings are recorded without blocking remediation of that baseline. New candidates and kept-running companions stay strict, every image set still needs completed matching ECR validation, and wrong-image reports or scanner errors block. The manual `aws-prep.yml` workflow scans selected inherited-owner images under the same threshold and is separate from automatic Main CI/CD.

Recorded deliveries.

Run	Source	Outcome
39085599276 / 39087966647 / 39089615147 (`r1019a`)	`0b491e1`, `358d169`	V1 deploy with smoke PASS; Processing release 9 m 39 s with post-release smoke PASS; teardown `CLEAN`
39105817431	`d2070db`	First persistent `dc2-live` deploy; smoke PASS
39271754470	`7928b7c`	Deliberate bad Processing candidate: smoke failed (503) and the previous aliases were **restored** automatically
39286933644	`daee44b`	Deploy of the team-accepted frozen demo release
39416747925	`629119c`	Both owners promoted, deployed-image scans clean, upload-to-playback smoke passed, read-back of 3 images, 10 aliases and the worker definition
39532075705	`4ae6a0b`	**Failed**: the scan of the retained Media rollback image found 1 fixable HIGH/CRITICAL
39542434067	`6155951`	**Failed**: the stack-level public smoke met a paused Aurora cluster; login answered 503
39545554380	`c7c489c`	**Success**: both owners promoted; zero fixable HIGH/CRITICAL in the deployed images; upload-to-playback smoke PASS, job `done` in 101.2 s; 3 images, 10 aliases and the worker definition read back (`docs/results/engineering-20261021/delivery-c7c489c.json`)

The two 21 October failures led to two changes: AD-28 added the restore-target exception above, and the public smoke now retries its first Media login and Processing read on 502/503/504 or transport failure, pausing 5, 10 and 15 s while an Aurora cluster resumes (`scripts/aws/v1/public_smoke.py`). The next delivery, `c7c489c`, succeeded.

6. Other Things to Be Highlighted

This section responds to the presentation feedback and collects the supporting models, design decisions and team contributions:

- 6.1 Presentation Feedback and Response
- 6.2 Use Case Model
- 6.3 Analysis-to-Design Transition Strategies
- 6.4 Analysis and Design Models
- 6.5 Design Problems and Patterns
- 6.6 Member Contributions and Lessons Learned

6.1 Presentation Feedback and Response

The table answers each written comment and recorded spoken remark from the presentation on 26 October 2026 (`project-documentation/presentation/feedback-20261026/FEEDBACK.md`). The record does not name the spoken assessors. The last row is the team's own follow-up question about where the Lambda-or-Fargate decision is made.

Remark (assessor)	Response and evidence	Where in this report
Provide more architectural perspectives, including deployment (Boon Kui HENG, written).	The logical view gives responsibilities and interfaces (Figure 6). The deployment view maps image artifacts to Lambda aliases, finite Fargate tasks, owned stores and network placement (Figure 15).	Logical Architecture & Design; Physical Architecture & Design
Provide detailed DDD diagrams with entities (Boon Kui HENG, written).	The domain models show Asset and Job roots, Attempt entities, immutable source values and independent identity and intake lifecycles (Figures 11 and 14).	Platform Design
Provide class and sequence diagrams at class/object level for critical use cases (Boon Kui HENG, written).	Every scoped use case has analysis and design class and sequence models. The upload and processing designs name the actual records, functions, transactions and exception paths (Figures 35 and 40).	Analysis and Design Models: UC-1, UC-2
Provide data schemas/models for the data stores (Boon Kui HENG, written).	Typed relational models and physical dictionaries give keys, nullability, defaults, checks and indexes. JSONB values, SQS envelopes and S3 object namespaces have their own schemas (Figures 19 and 21).	Persistence Design
Expand the detail behind slide 20; make the report more detailed (spoken assessor remark).	The report follows an upload from identity resolution through reservation, direct S3 upload, verification and SQL adoption to later publication (Figures 8 and 35).	Logical Architecture & Design; Analysis-to-Design Transition Strategies; Analysis and Design Models: UC-1
Show the authorizer tied to API Gateway (spoken assessor remark).	Gateway invokes the Media-owned Lambda authorizer before the protected API. The adapter supplies trusted Actor context and Media revalidates the current user (Figures 7 and 85).	Logical Architecture & Design; DP-3
Editing does not go directly to SQS (spoken assessor remark).	An edit commits its new Asset, verified source pin and submit outbox row in one Media transaction. MediaRepair publishes after that commit and Processing accepts through its own inbox transaction (Figures 8 and 81).	Logical Architecture & Design; Detailed Design; DP-2

Remark (assessor)	Response and evidence	Where in this report
Explain grouping into bounded contexts and aggregates (spoken assessor remark).	Capabilities are grouped by vocabulary, invariants and local consistency. Account, intake and playback stay in Media; execution, leases and recovery belong to Processing (Figures 11 and 12).	Platform Design
Provide component-level detailed design (spoken assessor remark).	The component views show the real Media and Processing modules and adapters. The use-case and pattern sequences expand their collaborations, and unit tests cover the complex policy and orchestration modules (Figures 9 and 10).	Tiers and Layers; Detailed Design; Unit tests for complex classes and modules
Include database schemas and the data model (spoken assessor remark).	Each database has its own migration history, runtime and migrator roles, and model. Plain identifiers cross contexts; SQL constraints, version pins and lease fences protect local authority (Figures 20 and 22).	Persistence Design; Platform Design
The physical architecture lacks load balancing (spoken assessor remark).	AD-30 explains HTTP distribution through Gateway and Lambda, queue buffering with competing consumers, and SQL-reserved finite capacity. An ALB with an ECS HTTP service is the alternative it compares (Figure 15).	Physical Architecture & Design: Key Architectural Decisions; Performance
Consider NoSQL such as MongoDB (spoken assessor remark).	AD-31 compares PostgreSQL/JSONB with MongoDB and DynamoDB against owned reads, transactional inbox/outbox changes, uniqueness, leases and due-work queries. PostgreSQL keeps these rules; JSONB holds variable descriptors and S3 holds video bytes.	Physical Architecture & Design: Key Architectural Decisions, Technology and Services; Persistence Design
SAST and DAST were not visible in the pipeline (spoken assessor remark).	The scanner inventory lists the SAST (Ruff S rules, CodeQL), supply-chain (Trivy, pip-audit, Checkov) and DAST (authenticated local-stack OWASP ZAP) stages with scope, trigger and disposition (Figure 27).	Security; Continuous Integration
Where is the Lambda-or-Fargate decision made? (team follow-up to the presentation Q&A).	The Processing transaction chooses the lane by size, feature flag and history, and the short executor re-checks duration and height after download. A refused short encode becomes a new standard-lane Attempt, and the launcher decides standard capacity separately (Figures 17 and 18).	Physical Architecture & Design; Analysis and Design Models: UC-2, UC-6

6.2 Use Case Model

Figure 29 defines the FlickPond platform and its two supported clients. Owners submit videos and consume their own results; operators manage capacity and releases. UC-2 runs asynchronously on the owner's behalf, and workers, queues and the pipeline are internal participants. The personas Maya, Arun, Jules and Sam are described in the Introduction; they are planning personas, not application roles.

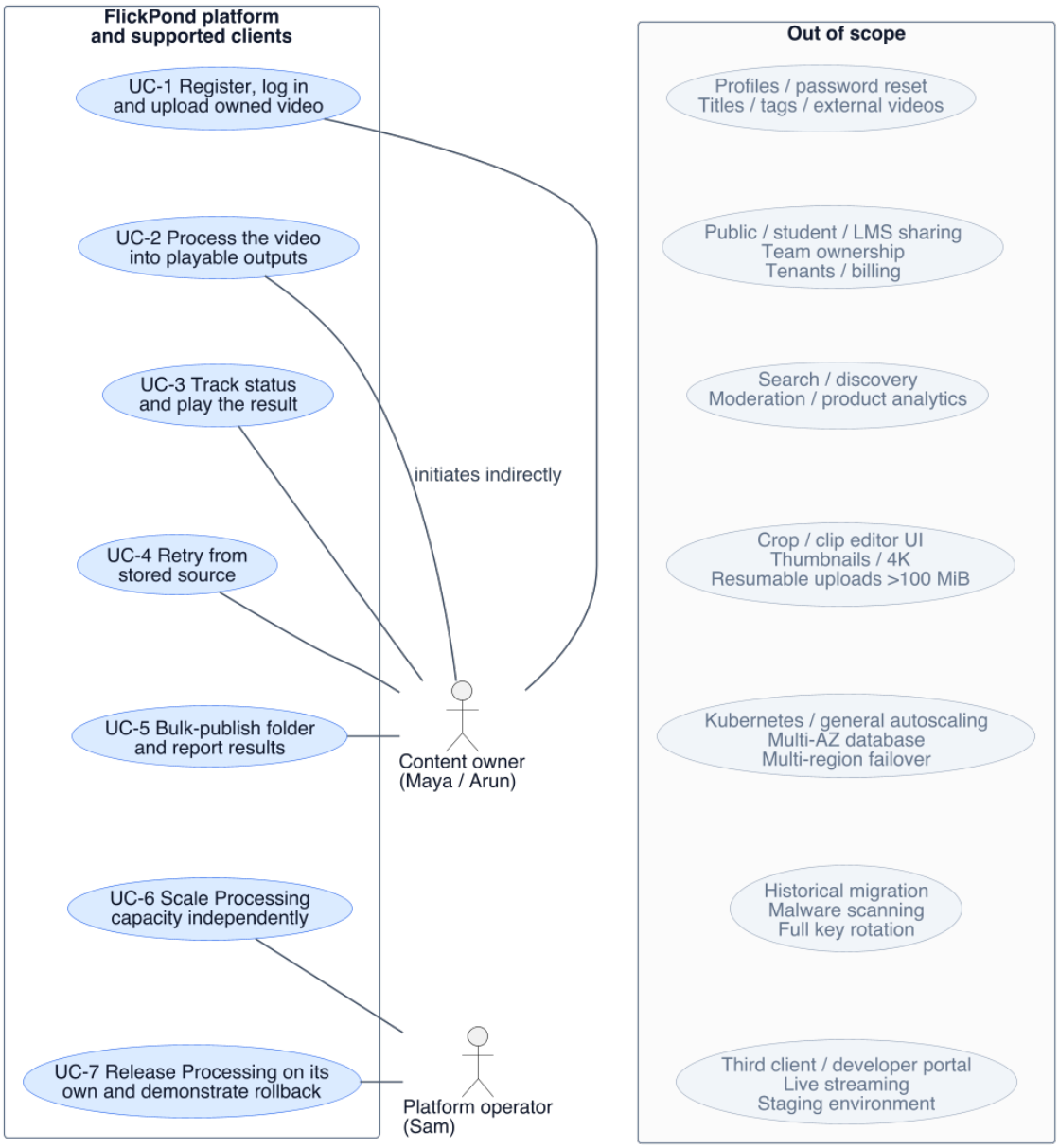


Figure 29. Use-case model: seven scoped goals and grouped exclusions. Grey cases sit outside the delivered system boundary.

The grey package groups the exclusions listed in the Introduction under “Functionality out of scope”. The editor exclusion concerns a crop/clip **UI**; API edit operations and deletion remain supporting capabilities. UC-6 uses bounded finite task launching, and general autoscaling is excluded. UC-3 authorises owners; student, LMS and public access are excluded.

No include/extend relation is asserted. Upload, processing and status reads depend on each other in time but remain independently observable goals. Retry is requested separately after failure. Publisher repeats the public upload protocol for each file.

The catalogue assigns responsibility for **analysis and design**. All seven proposal use cases have Must priority (BS-06); the backlog separately records implementation priority. Section 6.4 contains each owner’s descriptions, normal and exception flows, and analysis and design class and sequence models; the appendix traceability matrix links each case to its code modules and tests.

ID	Name	Primary actor	Planning persona	Analysis/design owner	Priority	Models
UC-1	Register, log in and upload an owned video	Content owner	Maya, Arun; Jules integrates	Ibrahim Mammadov	Must	§6.4, Ibrahim: UC-1
UC-2	Process the video into playable outputs	Content owner, through asynchronous execution	Maya, Arun	Ibrahim Mammadov	Must	§6.4, Ibrahim: UC-2
UC-3	Track status and play the result	Content owner as viewer	Maya, Arun	Nguyễn Kim Long	Must	§6.4, Long: UC-3
UC-4	Retry from the stored source	Content owner	Maya, Arun	Ibrahim Mammadov	Must	§6.4, Ibrahim: UC-4
UC-5	Bulk-publish a folder and report results	Content owner using Publisher	Maya; Jules integrates	Nguyễn Kim Long	Must	§6.4, Long: UC-5
UC-6	Scale processing capacity independently	Platform operator	Sam	Guruprasath Gopal	Must	§6.4, Guruprasath: UC-6
UC-7	Release Processing on its own and demonstrate rollback	Platform operator	Sam	Guruprasath Gopal	Must	§6.4, Guruprasath: UC-7

6.3 Analysis-to-Design Transition Strategies

These six strategies connect analysis objects to implemented classes and modules. Read their class and sequence changes alongside the analysis and design pairs in section 6.4. Conceptual associations express responsibilities; design associations identify persistence ownership, immutable references and execution authority. Function collections remain modules.

6.3.1 TR-1 Asset submission and status

Aspect	Transition
Analysis objects	Owner, UploadRequest, OwnedAsset, ProcessingJob, ProcessingAttempt, PlayableOutput.
Use cases	UC-1, UC-2, UC-3; UC-5 reuses intake and status.
Class-diagram changes	Map Owner to `User`, OwnedAsset to `Asset`, and UploadRequest to `UploadReservation`. Add Media's `ProcessingProjection`; keep `Job` and `Attempt` in Processing. One Job has multiple Attempts and a current-attempt reference. UUID references replace cross-context database associations.
Sequence-diagram changes	Replace conceptual submit → process → play with reserve → S3 upload → finalize, command/result delivery, then an owner-filtered local status read and signed playback access. Show queued status before a result arrives.
Reason	Separate ownership from execution so that Media accepts submissions and answers status without a synchronous Processing call. The projection records reported state and never authorises an encoder to finish.
Implemented in	`media/models.py`, `media/upload_models.py`, `media/projection_models.py`, `processing/models.py`, `media/projections.py:public_job`.

6.3.2 TR-2 Identity and authorisation

Aspect	Transition
Analysis objects	Owner, OwnedAsset, UploadRequest, IntegrationBatch.
Use cases	UC-1, UC-3, UC-4, UC-5.
Class-diagram changes	Separate persistent `User` from the `Actor` contract value. Add `GatewayAdapter` and the authorizer module; retain owner checks in Media and command handling in Processing.
Sequence-diagram changes	Add Gateway → authorizer → Gateway → API. Show current-user lookup, trusted context adaptation, Media revalidation, and refusal when identity or ownership is invalid.
Reason	A caller cannot choose an owner through a forwarded header, and Processing needs neither Media's JWT secret nor a peer authentication call.
Implemented in	`media/authorizer.py:authorize`, `contracts/gateway.py:GatewayAdapter.invoke`, `media/api/deps.py:current_user`. The authorizer issues only the `publisher` role.

6.3.3 TR-3 Upload and immutable source identity

Aspect	Transition
Analysis objects	UploadRequest, OwnedAsset, IntegrationBatch.
Use cases	UC-1, UC-5; derived-source edit variant.
Class-diagram changes	Add reservation request identity, expiry, checksum, size and S3 VersionId; use the `Source` contract descriptor. An adopted upload retains its pin in `UploadReservation`; `SourceVersion` records a derived edit source.
Sequence-diagram changes	Replace API byte transfer with reserve → scoped S3 POST → verification outside SQL locks → locked adoption transaction. Include replay, expiry and verification-failure alternatives. A finalized replay receives no new upload grant.
Reason	Avoid Lambda payload limits and bind processing to the exact verified bytes despite later writes to the same object key.
Implemented in	`media/services/direct_intake.py:reserve` and `finalize`; `media/services/v1_lifecycle.py:edit` (copy, then pin and stage the command in one adoption transaction); `publisher/client.py:PublicClient.upload`.

6.3.4 TR-4 Durable asynchronous handoff

Aspect	Transition
Analysis objects	OwnedAsset, ProcessingJob, ProcessingAttempt.
Use cases	UC-1, UC-2, UC-4; UC-5 through intake.
Class-diagram changes	Introduce `Envelope`, `MediaOutbox`, `MediaInbox`, `ProcessingOutbox` and `ProcessingInbox`, plus the Media projection. Keep message records local to each database.
Sequence-diagram changes	The sequence separates adoption commit, relay send and send acknowledgement; response loss leaves accepted work available for replay. It also shows consumer inbox/domain commit, duplicate delivery and scheduled repair.
Reason	SQL and SQS cannot commit atomically; durable intent and deduplication absorb at-least-once transport.
Implemented in	`delivery/sql.py:stage`, `publish_due` and `accept`, used by `media/messaging.py` and `processing/messaging.py`; relay I/O follows its reservation transaction.

6.3.5 TR-5 Retry, failure and output authority

Aspect	Transition
Analysis objects	ProcessingJob, ProcessingAttempt, PlayableOutput.
Use cases	UC-2, UC-4.
Class-diagram changes	Add `SubmissionGuard`, `RetryRequest`, `Claim`, `AttemptTask`, lease/deadline fields and `Output` descriptors under an attempt-specific object prefix.
Sequence-diagram changes	Separate owner retry/new Attempt from queued-work repair. Add claim → heartbeat → fenced finish, expired-lease failure, stale-worker/result rejection and short-lane redirect alternatives.
Reason	Only the current live attempt may publish a terminal outcome; duplicate messages and late workers must not overwrite a retry. Output validation in `processing/schemas.py` and `processing/lifecycle.py:finish` requires exactly one verified primary output in the current attempt namespace. Unedited uploads produce MP4; conversion edits can produce MKV or MP3. HLS remains optional.
Implemented in	`processing/lifecycle.py:claim`, `heartbeat` and `finish` (database-clock authority); `processing/recovery.py:recover_expired`; `processing/short_encoder.py:encode_record` and `processing/messaging.py:redirect_to_long_lane`. Both lanes keep the same fences.

6.3.6 TR-6 Bounded capacity and independent release

Aspect	Transition
Analysis objects	Operator, ProcessingCapacity, ReleaseCandidate.
Use cases	UC-6, UC-7 (Figures 63, 70).
Class-diagram changes	Map capacity to `LauncherControl` and `LaunchReservation`. Map release coordination to `release.promote` and `release.restore`, durable intent to `Journal` and `DeployJournal`, deployment adoption to `deploy.adopt_release`, and AWS operations to `OperatorTransport`.
Sequence-diagram changes	Add SQL-serialised slot reservation before RunTask, concurrent-wake and lost-reply branches (Figure 65). For release, show validation → pending intent → migration → publication marker → version → revision-checked alias change → owner smoke → adoption or restore, with unresolved intent kept for resumption (Figures 71, 72).
Reason	Bound outstanding standard encoder slots to three and preserve uncertain cloud outcomes while changing Processing independently of Media. The cap concerns standard finite Fargate capacity; the short Lambda lane has separate admission controls.
Implemented in	`processing/launcher.py:tick` and `_send` keep the RunTask token and reservation across ambiguous responses. `scripts/aws/v1/release.py:promote` and `restore` execute per-owner changes through `scripts/aws/v1/transport.py:OperatorTransport.call`; `scripts/aws/journal.py:Journal` retains intent and `scripts/aws/v1/deploy.py:adopt_release` records the new baseline. `CLITransport` is limited to planning and emulator execution. Restore does not reverse schema migrations.

6.4 Analysis and Design Models

Ibrahim Mammadov is responsible for the analysis and design of UC-1, UC-2 and UC-4. Each use case states its actors, conditions and flows, presents class and sequence models, and explains the analysis-to-design transition. Analysis objects use «`boundary`», «`control`» and «`entity`» stereotypes. Design models use implemented names, mark function collections as «`module`», and distinguish persistence records, adapters and contract values.

6.4.1 UC-1 Register, log in and upload an owned video

Description

Actors and conditions. Maya or Arun uses the web application or Publisher CLI; Media and versioned source storage support the interaction. The owner has a supported video of at most 100 MiB and registers or logs in before requesting an upload. Success leaves one owned `Asset`, a verified immutable source descriptor and durable `submit` intent. The public acceptance response uses `job_id=asset.id`; Processing assigns its internal Job identity asynchronously (`media/api/uploads.py`).

Normal flow. Media hashes the registration password or verifies existing credentials, then issues a Secure, HttpOnly, SameSite cookie. Protected requests resolve the current user. Media commits an owner/request reservation before returning its upload grant. The client uploads directly to S3, then finalizes. Media verifies the exact object version without holding a reservation row lock; a second transaction locks and rechecks the reservation, saves its pin, inserts the Asset and stages the command. The API returns HTTP 202 (`media/api/auth.py`, `media/api/deps.py`, `media/services/direct_intake.py`).

Exception flows. Duplicate registration returns 409; invalid login returns 401. Anonymous requests are refused. Reservation replay requires the same declaration and, once the request is finalized, returns no new upload grant; changed input conflicts and an expired pending reservation returns 410. A foreign reservation returns 404. Missing bytes return a recoverable 409; truncated, changed, unsupported or unversioned content cannot be adopted. Finalization replay returns the existing Asset without another command. Expiry is checked again after storage verification, covering slow uploads and concurrent finalization (`media/services/storage.py`).

Analysis class diagram

Figure 30 separates identity control from intake control. An upload request exists before an owned asset, so a reservation is not an accepted video.

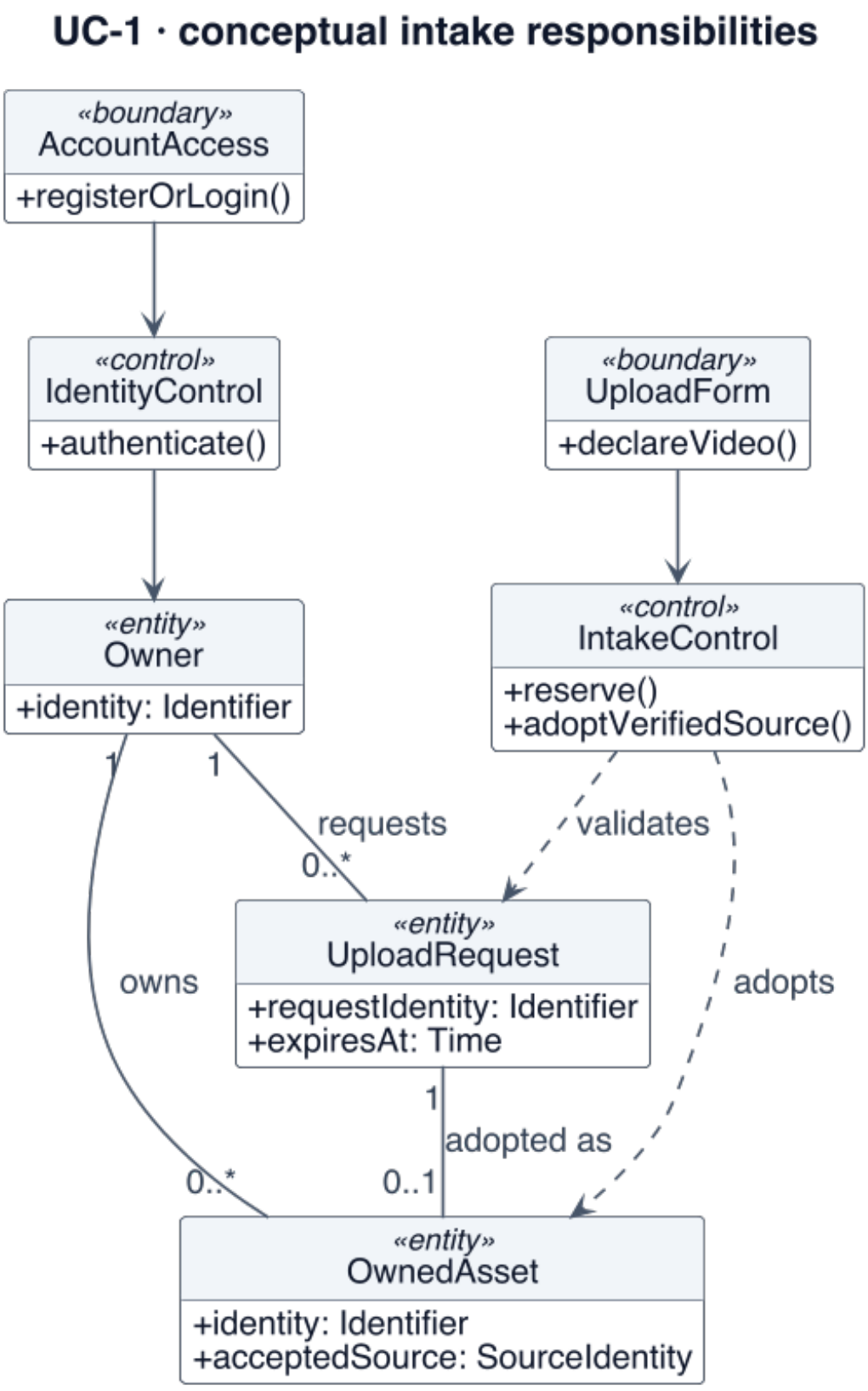


Figure 30. UC-1 conceptual account and intake responsibilities.

Analysis sequence diagrams

Authentication and source adoption have different failure boundaries. Figure 31 merges registration and login; Figure 32 adds replay and the main refused-adoption branch.

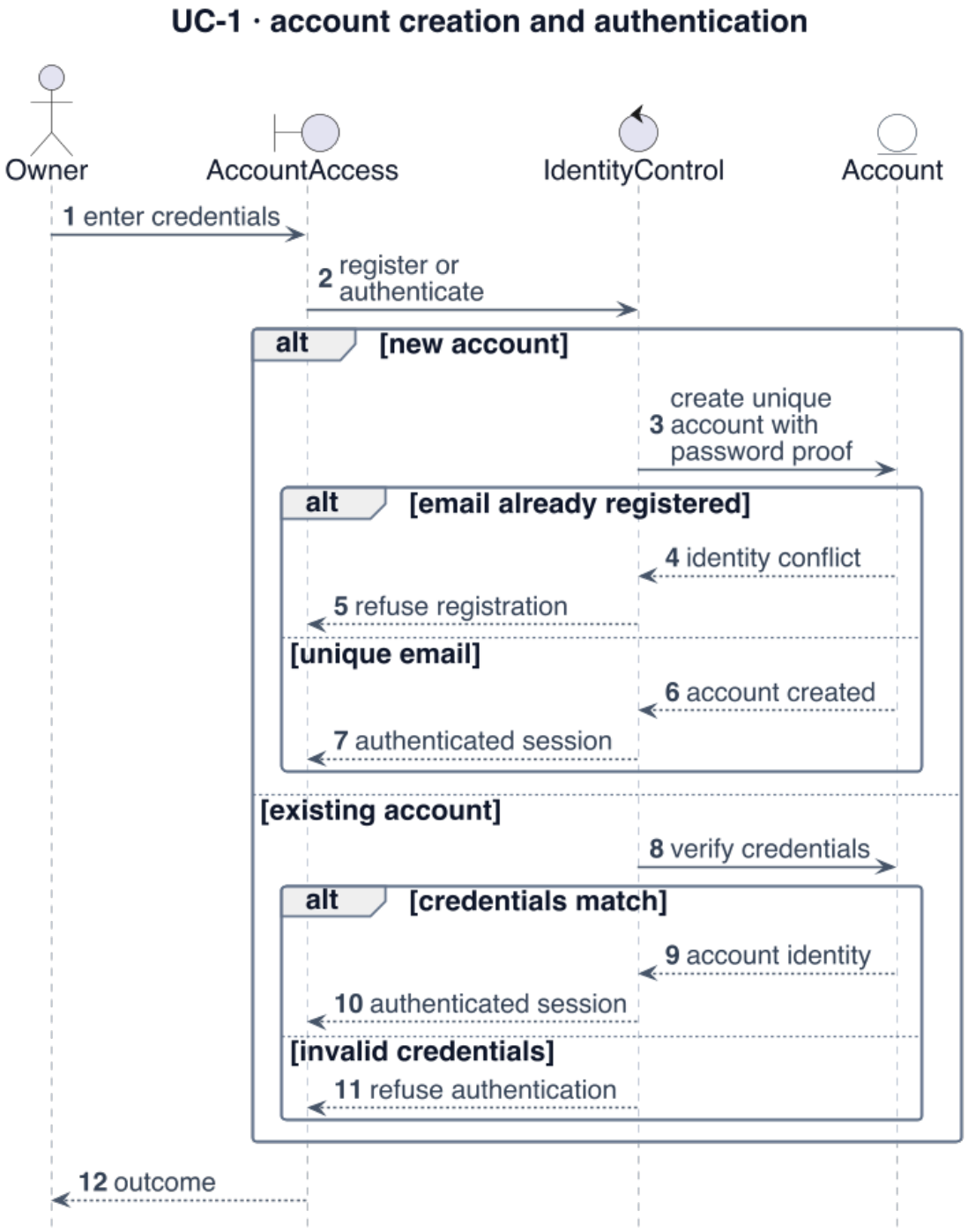


Figure 31. UC-1 account creation, login and credential refusal.

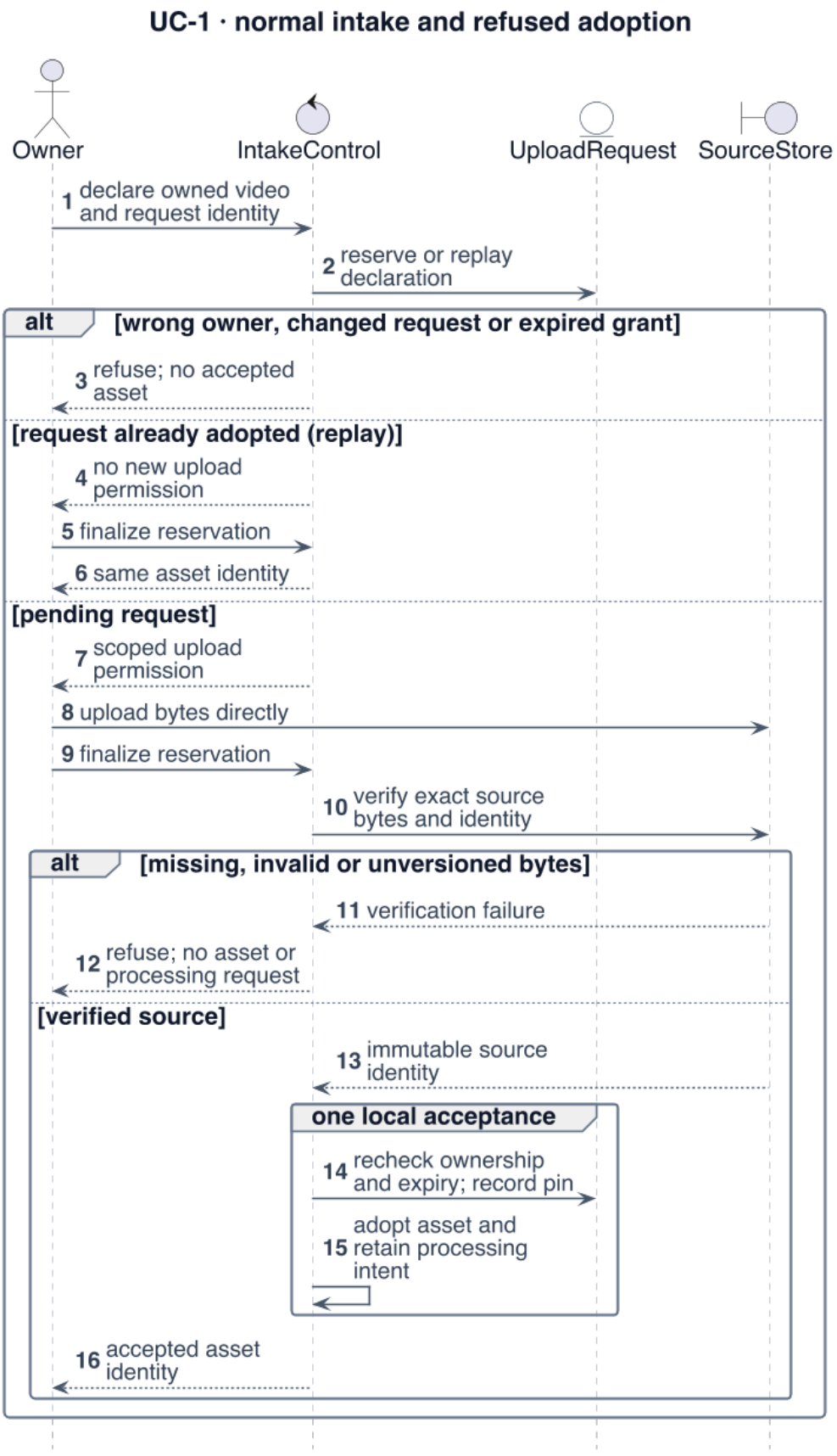


Figure 32. UC-1 direct upload, verification and refused adoption.

Design class diagram

Figure 33 uses the actual records and functions. The accepted upload's version pin belongs to `UploadReservation`; `SourceVersion` is used for derived edit sources. The reservation's preallocated `asset_id` is a logical reference without an Asset foreign key (`media/upload_models.py`). ORM annotations are shown without their `Mapped[...]` wrapper.

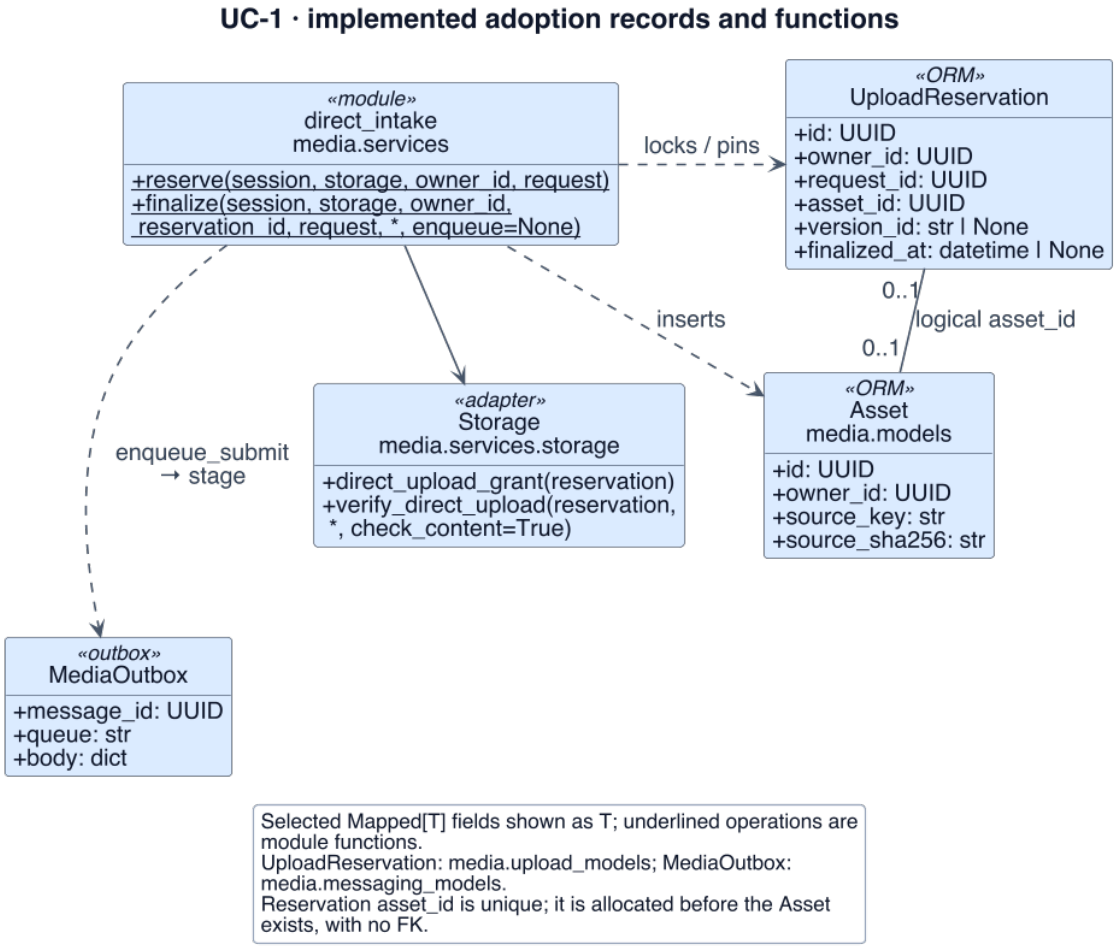


Figure 33. UC-1 implemented adoption records and application functions.

Design sequence diagrams

Figure 34 follows the account functions through their real caller. Figure 35 starts after `CurrentUser` resolution; behind Gateway, Media's authorizer validates the cookie and current user and the adapter builds trusted context (`media/authorizer.py`, `contracts/gateway.py`). Only a pending reservation receives a presigned POST, which permits 1..declared-size bytes; finalization enforces the exact size and hash. Adoption commits before relay publication.

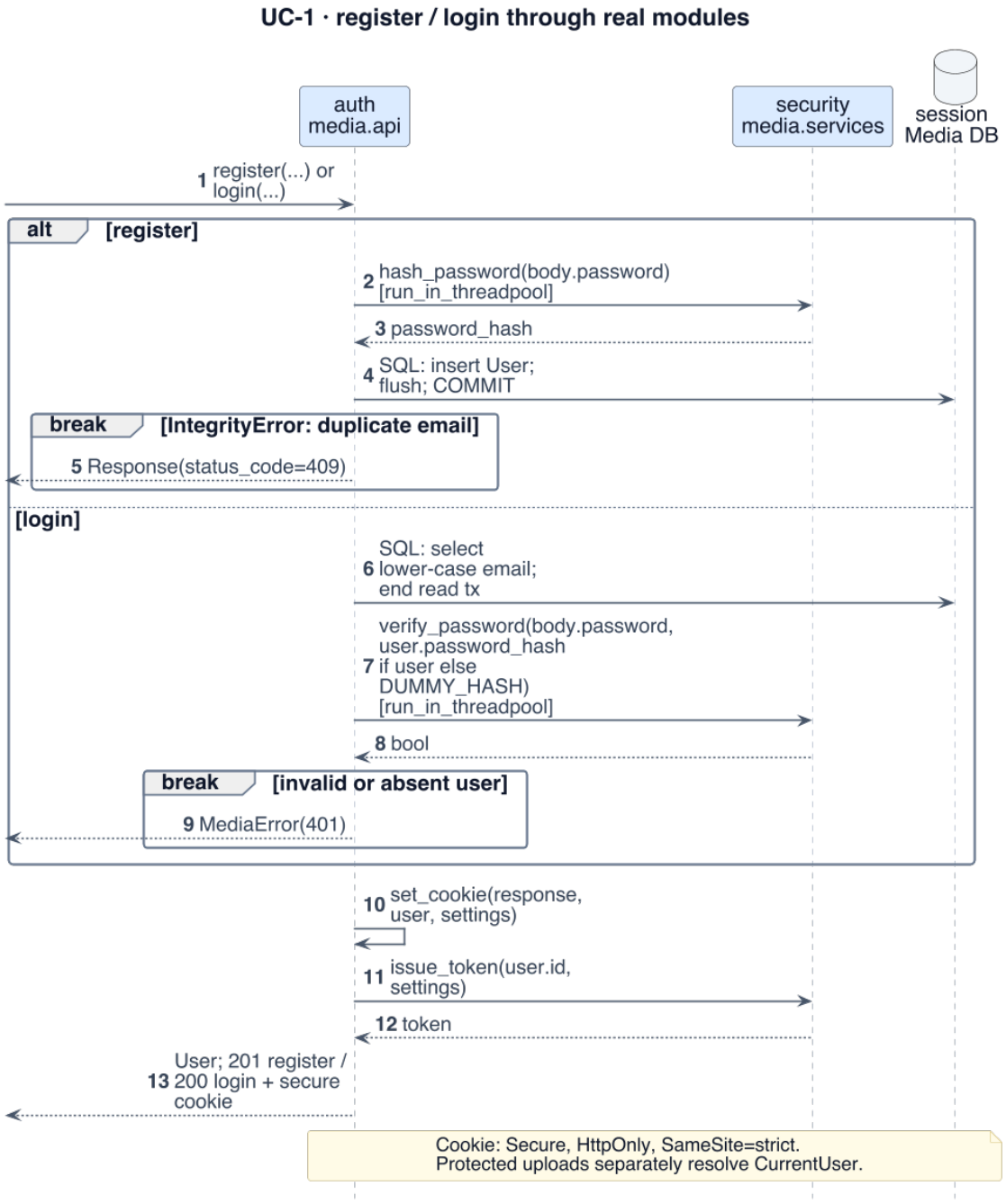


Figure 34. UC-1 real account functions and secure-cookie response.



Figure 35. UC-1 reservation, direct S3 transport and atomic finalization.

The assertions in `tests/media/test_direct_intake.py::test_finalize_atomic_adoption_and_replay` check adoption and replay; `test_finalize_rechecks_expiry_after_slow_storage` checks the second expiry gate. SQL/S3 integration coverage in `tests/v1/test_direct_intake_s3.py` includes `test_finalize_pins_the_exact_version_it_verified`, wrong-owner refusal and `test_missing_object_is_retryable_not_adopted`.

Analysis to design

`Owner` becomes `User` plus trusted request identity; `UploadRequest` becomes `UploadReservation`, and `OwnedAsset` becomes `Asset`. Intake control becomes `direct_intake` functions and the `Storage` adapter. Atomic acceptance becomes pin + Asset + `MediaOutbox` in one SQL transaction, so accepted work survives a queue outage. `contracts.v1.Source` defines the source descriptor carried to Processing; no media bytes cross the API integration.

6.4.2 UC-2 Process the video into playable outputs

Description

Actors and conditions. Processing's command consumer and encoder execute this asynchronous use case; Media receives results. UC-1 has committed a valid source and command intent. For an unedited upload, success records one primary MP4 and, when available, an HLS manifest under the current job/attempt namespace; conversion edits record an MKV or MP3 primary instead. Failure becomes visible through the result projection.

Normal flow. Processing commits inbox acceptance, a guarded Job/Attempt and result/work outboxes together. The relay sends work to the selected lane. A finite executor claims the current attempt before fetching the source, binds its task identity and receives a leased `Claim`. It verifies the pinned source, probes and encodes it, uploads acknowledged output descriptors, then commits terminal state and result intent under live authority. Media consumes results into its local projection (`processing/messaging.py`, `processing/lifecycle.py`, `media/projections.py`).

Exception flows. Identical messages replay harmlessly; conflicting identity or body rolls back acceptance. A short executor that rejects probe admission creates a fenced standard-lane successor. An ordinary encode error instead records failure when authority remains valid. Lease/deadline loss prevents finish; recovery abandons the expired attempt and fails the Job. Ordinary optional-HLS failure preserves the primary output. Missing queued delivery is repaired for the same Attempt; malformed records remain eligible for redrive and DLQ handling. None of these cases permits a stale worker to replace current outputs.

Analysis class diagram

In Figure 36 the Job keeps one identity across attempts, outputs belong to successful execution, and the processing controller owns scheduling and outcome decisions.

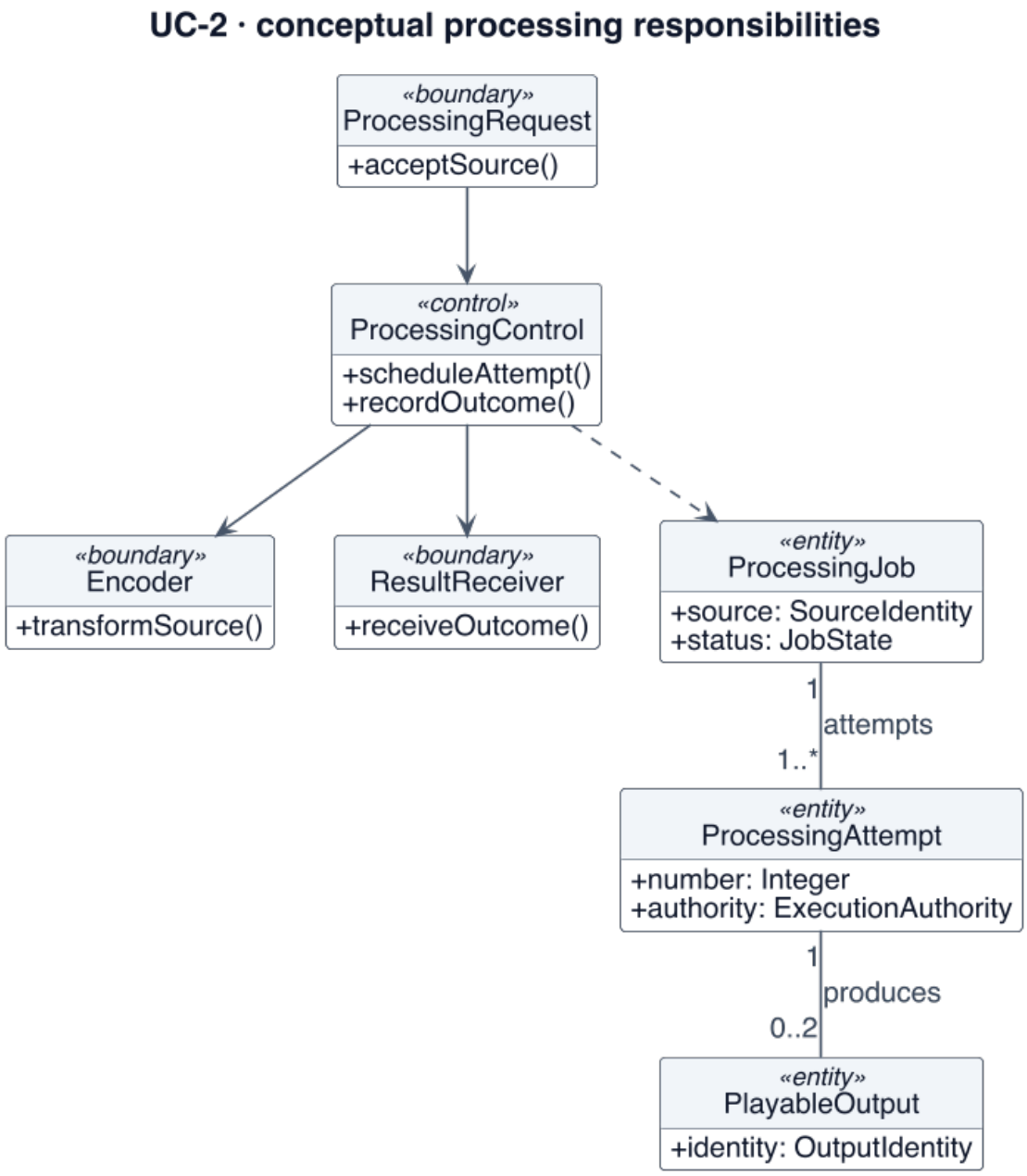


Figure 36. UC-2 conceptual job, execution authority and playable outputs.

Analysis sequence diagrams

Figure 37 combines the shared processing steps with the principal failure and admission alternatives. Durable acceptance, execution and result application stay separate responsibilities.

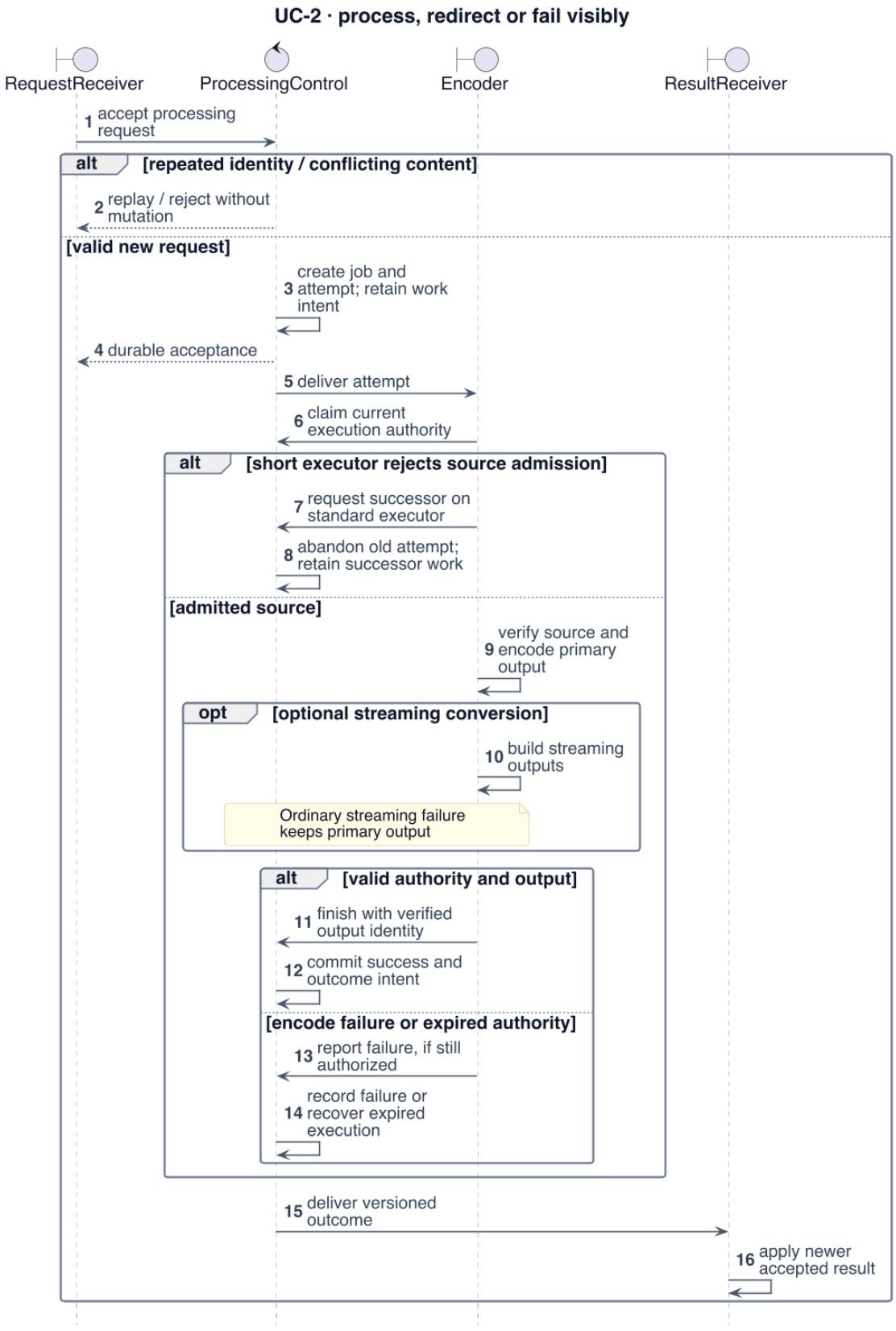


Figure 37. UC-2 normal processing, admission redirect and visible failure.

Design class diagram

Figure 38 shows `Job`, `Attempt`, the unique `AttemptTask` binding and the captured `Claim`. A Claim freezes its field bindings; its nested source and operation containers stay ordinary Python values. Lifecycle functions enforce current-attempt, token, processing-state and database-clock expiry checks.

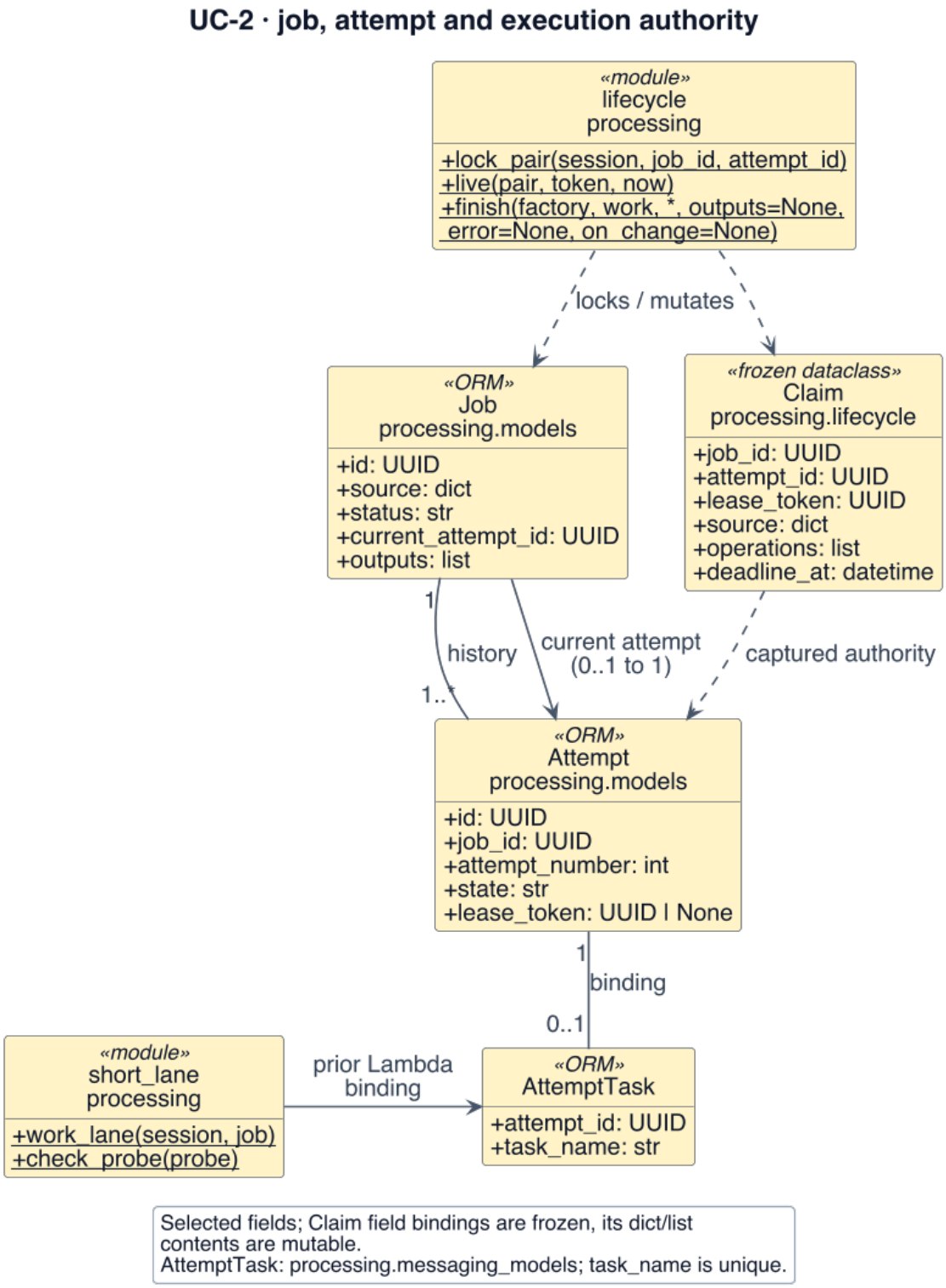


Figure 38. UC-2 implemented execution records and lifecycle functions.

Design sequence diagrams

Figure 39 distinguishes `handle_command` from `handlers.invoke`: the callback returns a boolean after its transaction, while batch consumption builds the partial-failure response. The handler then attempts publication, and a failed publication leaves committed intent for repair (`processing/handlers.py`, `delivery/sqs.py`).

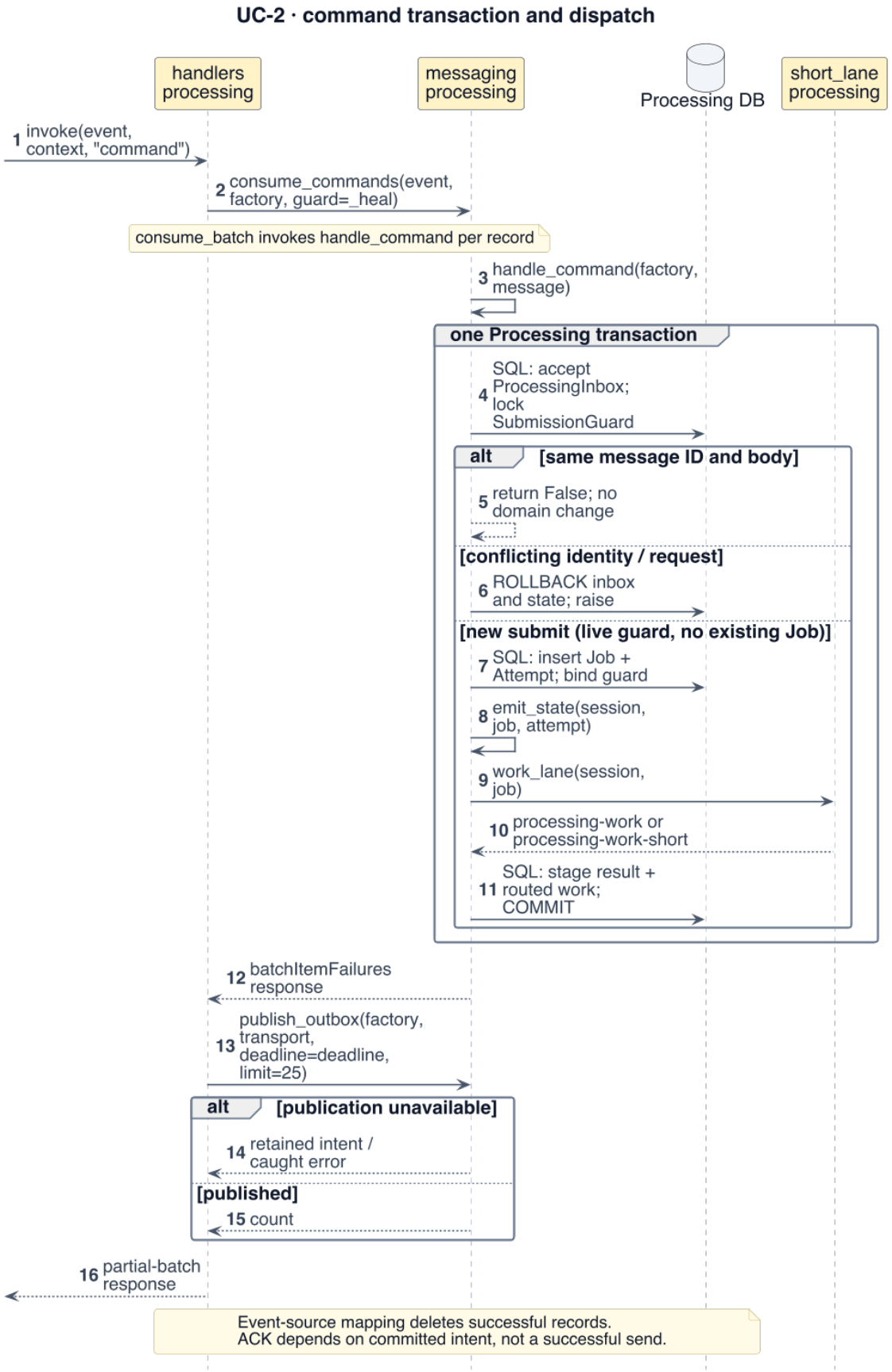


Figure 39. UC-2 atomic command acceptance and subsequent publication.

Lane choice is made by `short_lane.work_lane` and re-checked by `check_probe` after download; a refusal calls `redirect_to_long_lane` and the new Attempt runs on the standard lane. The thresholds are in the lane-decision table of section 3.2.1 and Figures 17 and 18, and they are independent of the standard launcher's capacity decision (`processing/short_lane.py`, `processing/short_encoder.py`).

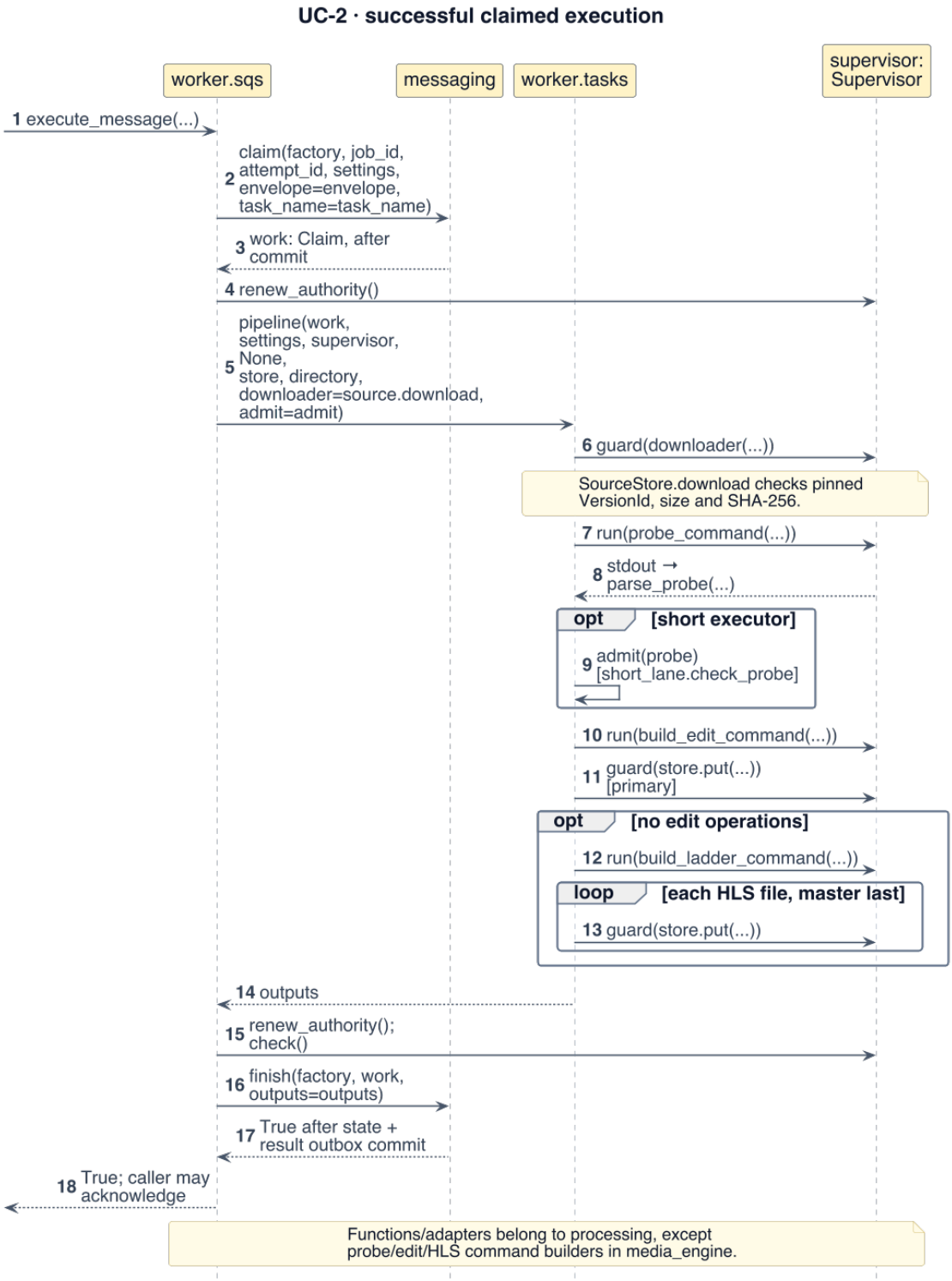


Figure 40. UC-2 claimed execution and optional HLS under live authority.

Figure 40 follows `worker.sqs.execute_message` and `worker.tasks.pipeline`. `media_engine.probe`, `operations` and `hls` build and parse commands, and `Supervisor.run`

executes the subprocesses. `SourceStore.download` checks pinned version, size and hash; `OutputStore.put` returns acknowledged descriptors, and HLS uploads its master last. The standard worker deletes a receipt after a durable outcome or SQL-proved stale work and keeps it when execution is uncertain; on Lambda the event-source mapping deletes records.

Figure 41 expands the main exception paths. Media's result consumer calls `consume_result(session, envelope, apply)`, so inbox acceptance and `apply_result` commit together; `validate_transition` protects the local projection. A stale message can commit its inbox row without changing the projection, and conflicting content rolls back both. The result handoff follows the same pattern as Figure 81.

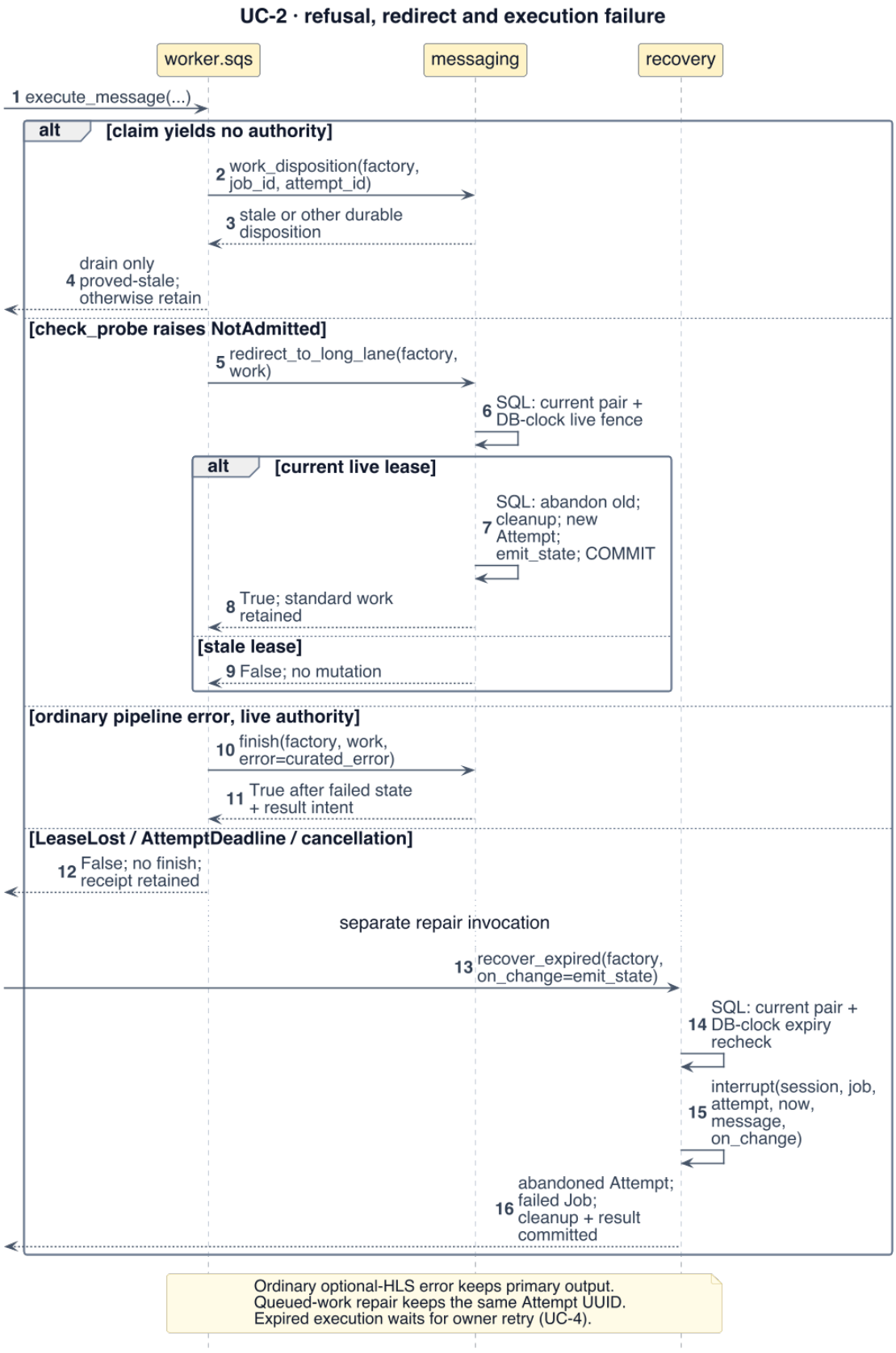


Figure 41. UC-2 refused claims, short-lane redirect and interrupted execution.

`tests/v1/test_messaging_sql.py` checks duplicate/conflict rollback, source-substitution refusal and atomic terminal results, including output namespace validation.

`tests/v1/test_short_encoder_sql.py::test_a_source_the_probe_rejects_becomes_a_new_attempt_on_the_standard_lane` covers redirect. `tests/processing/test_worker.py::test_hls_failure_retains_primary` covers preserving the MP4 after a ladder failure. These local tests verify redirect, terminal commit and stale-authority rejection; measured cloud timings are in Quality Attributes.

Analysis to design

`ProcessingJob` and `ProcessingAttempt` become `Job` and `Attempt`; execution authority becomes `Claim`, lease columns and `AttemptTask`. `PlayableOutput` becomes validated `Output` descriptors stored as JSON rather than an output table. Processing control becomes `messaging`, `lifecycle` and worker modules. Result reception becomes Media's inbox plus `ProcessingProjection`, separating Processing's write authority from Media's owner-facing reads.

6.4.3 UC-4 Retry from the stored source

Description

Actors and conditions. The owner retries through the web application or CLI. The owned Asset has a failed projection and a retained accepted source. Success retains Asset, Job, source and operations, while queuing a new numbered Attempt.

Normal flow. A bodyless request optionally supplies `Idempotency-Key`. Media locks the owned Asset, resolves replay before checking status, verifies the failed projection and reads the pinned descriptor. It commits request identity and stable retry intent. Processing accepts the command, locks guard then Job, validates durable identity and creates one successor plus work/result intent (`media/services/v1_lifecycle.py`, `processing/messaging.py`).

Exception flows. Foreign/deleted Assets return 404; a non-failed projection returns 409. A replay returns the prior public acceptance; Processing's request ledger prevents another successor and returns the current Attempt. Retry-before-submit rolls back the inbox so later redelivery can succeed. Deleted guards cannot resurrect work. Old worker claims and stale result versions cannot overwrite a newer attempt. No source upload is repeated.

Analysis class diagram

Figure 42 adds retry identity to the Job/Attempt relationship: a retry changes execution authority, not source identity.

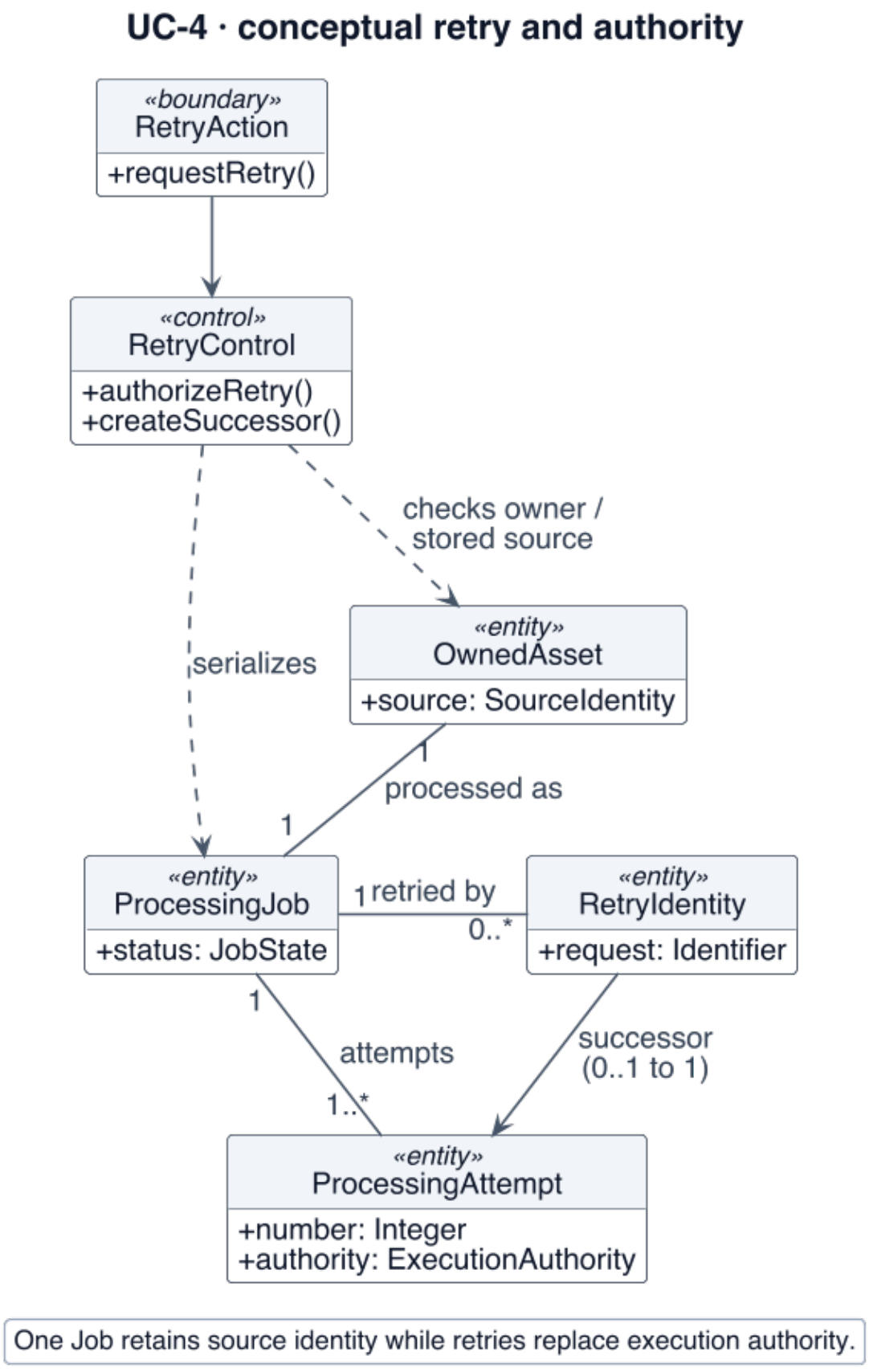


Figure 42. UC-4 conceptual retry identity and successor attempt.

Analysis sequence diagrams

Figure 43 combines eligibility, replay and the late-worker exception around the shared retry flow.

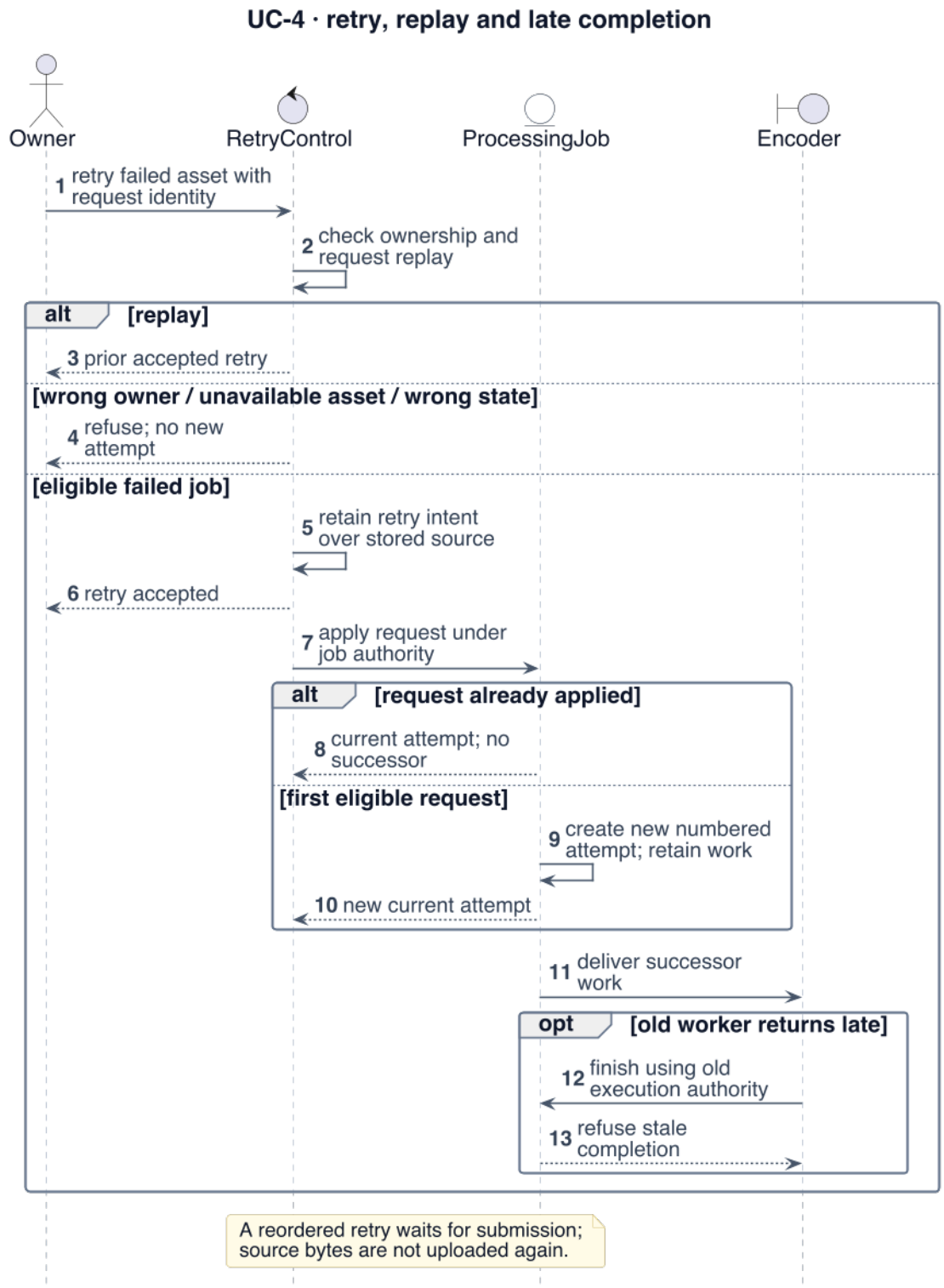


Figure 43. UC-4 owner retry, replay and refused stale completion.

Design class diagram

`RequestKey` deduplicates the public Media operation; `RetryRequest` deduplicates Processing's successor creation. Figure 44 keeps these separate local records explicit. Without a public key, Media derives message identity from Asset and failed state version.

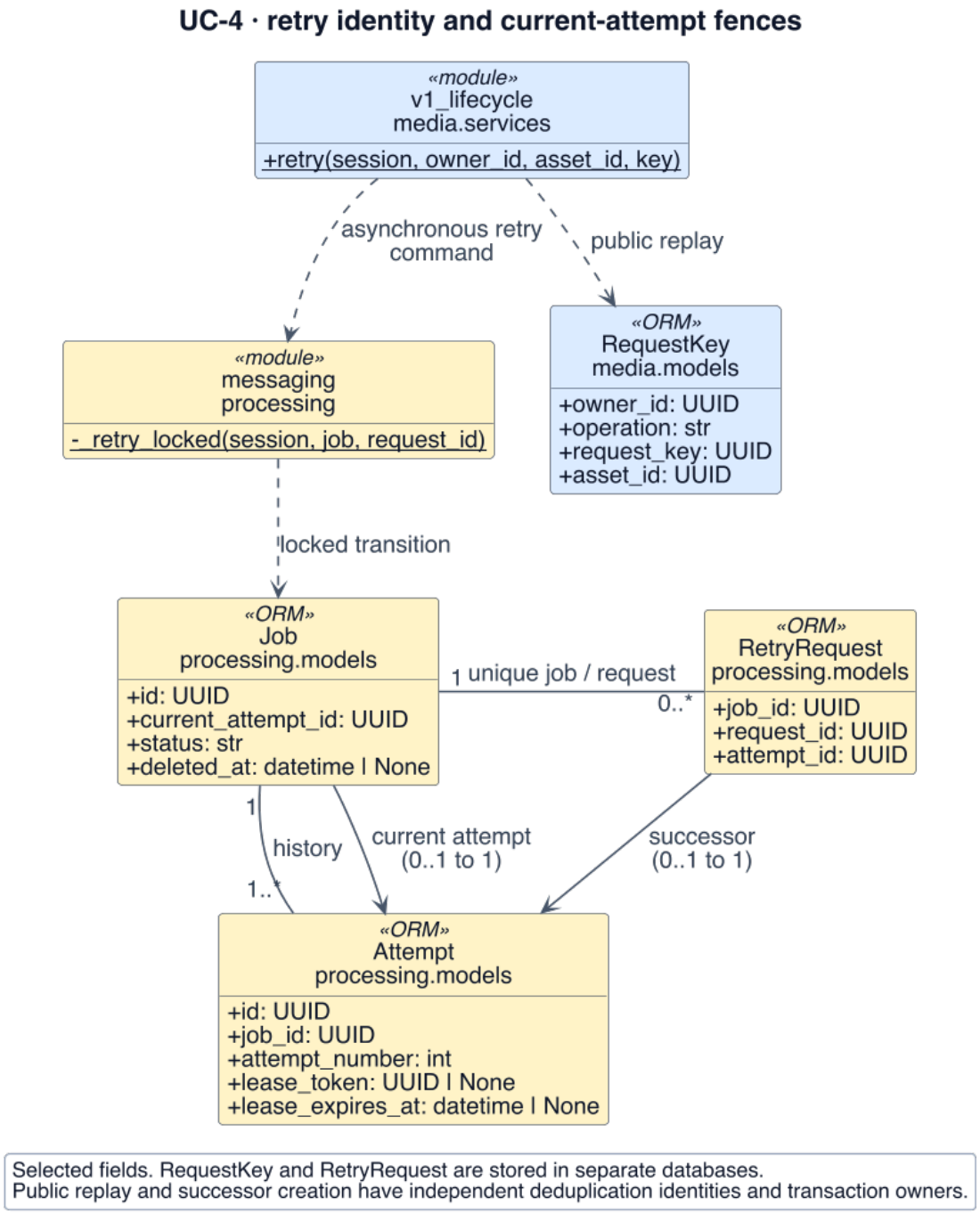


Figure 44. UC-4 implemented public and Processing retry ledgers.

Design sequence diagrams

Figures 45 and 46 show local transactions separated by asynchronous delivery. `_retry_locked` retains source/operations, schedules old-attempt cleanup, increments the attempt number, clears prior error/outputs and calls `emit_state`. Direct Processing retry calls `messaging.retry_job` in its own transaction (`processing/v1_api.py`).

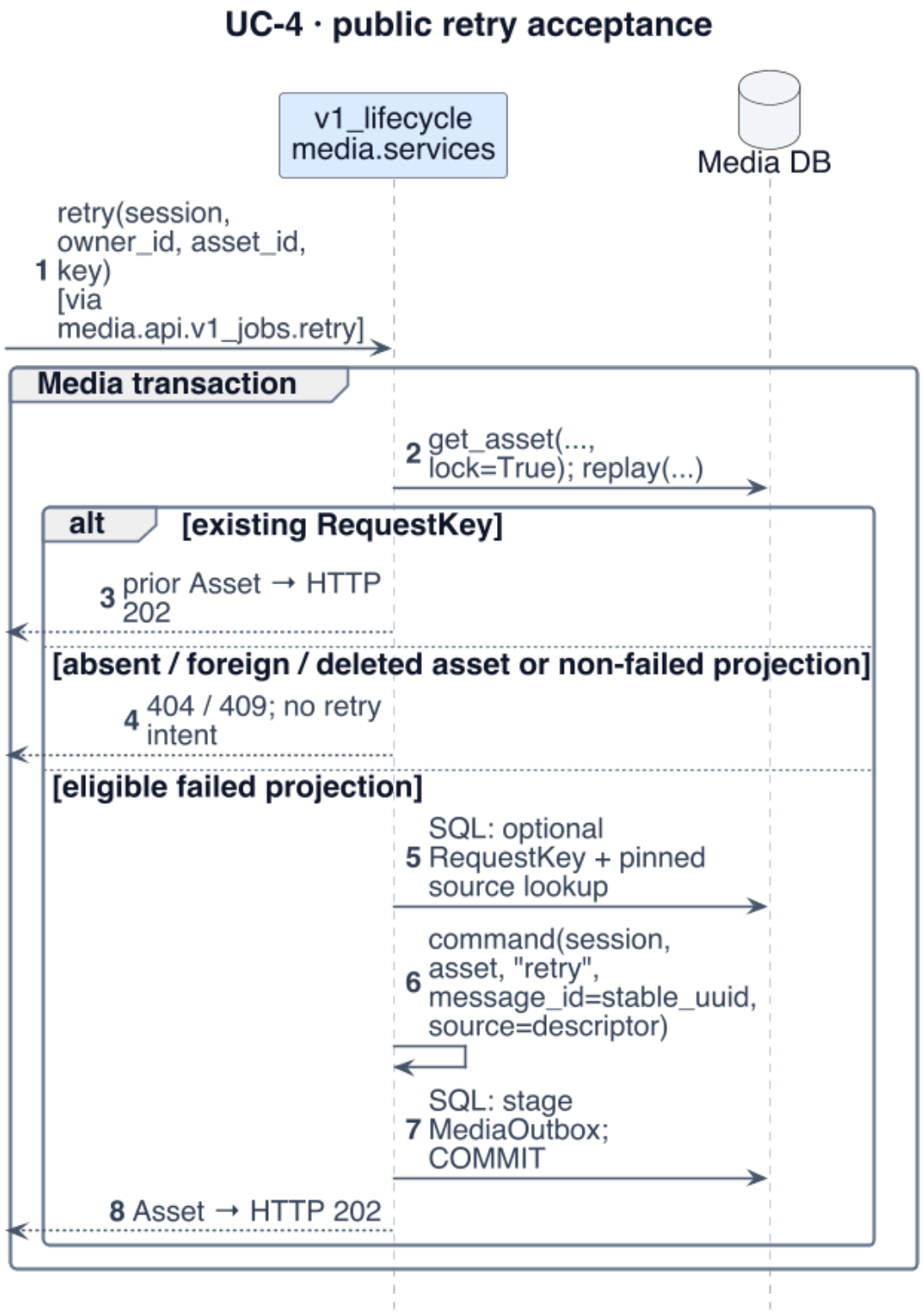


Figure 45. UC-4 public retry acceptance.

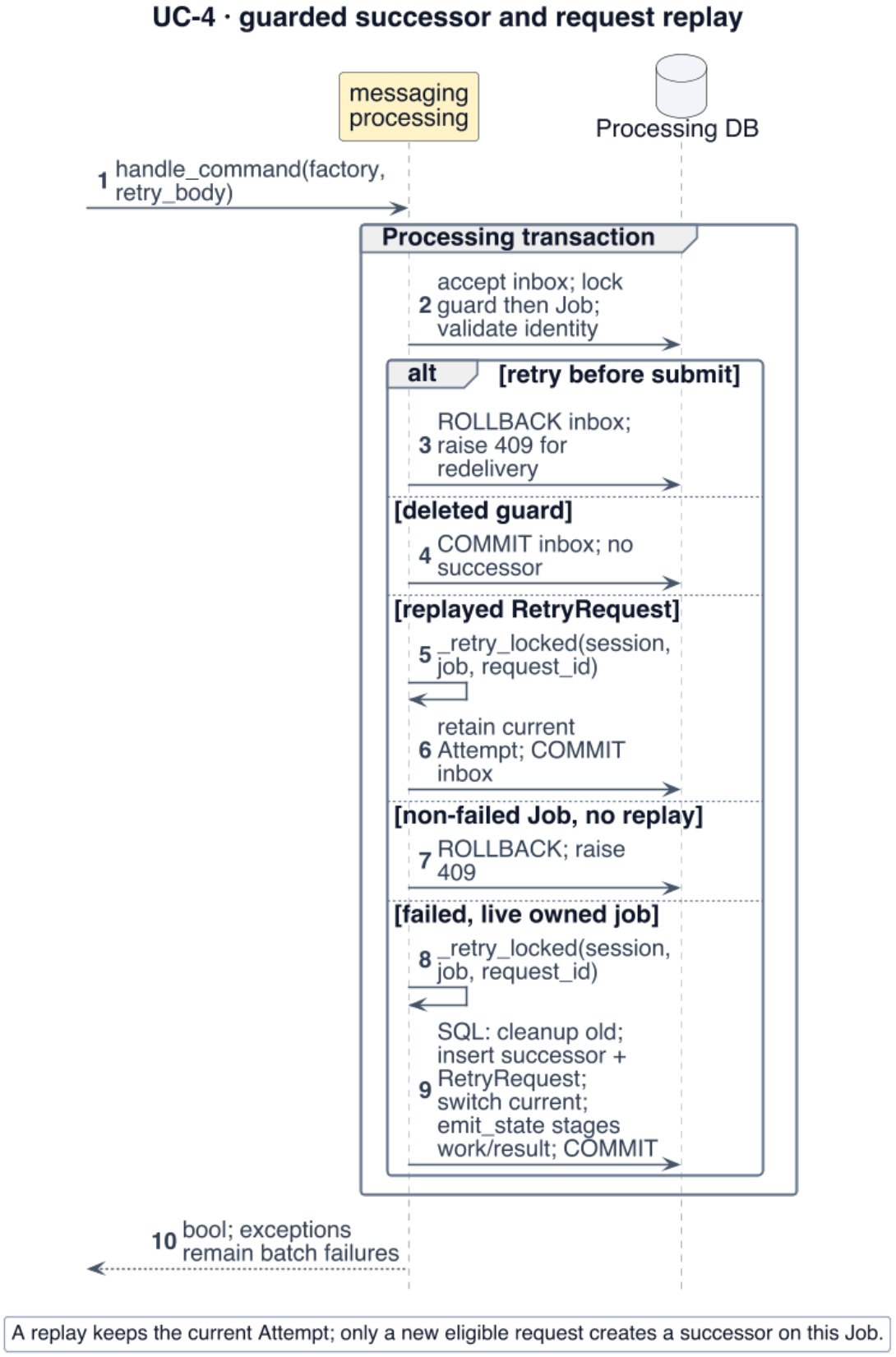


Figure 46. UC-4 guarded successor creation and retry replay.

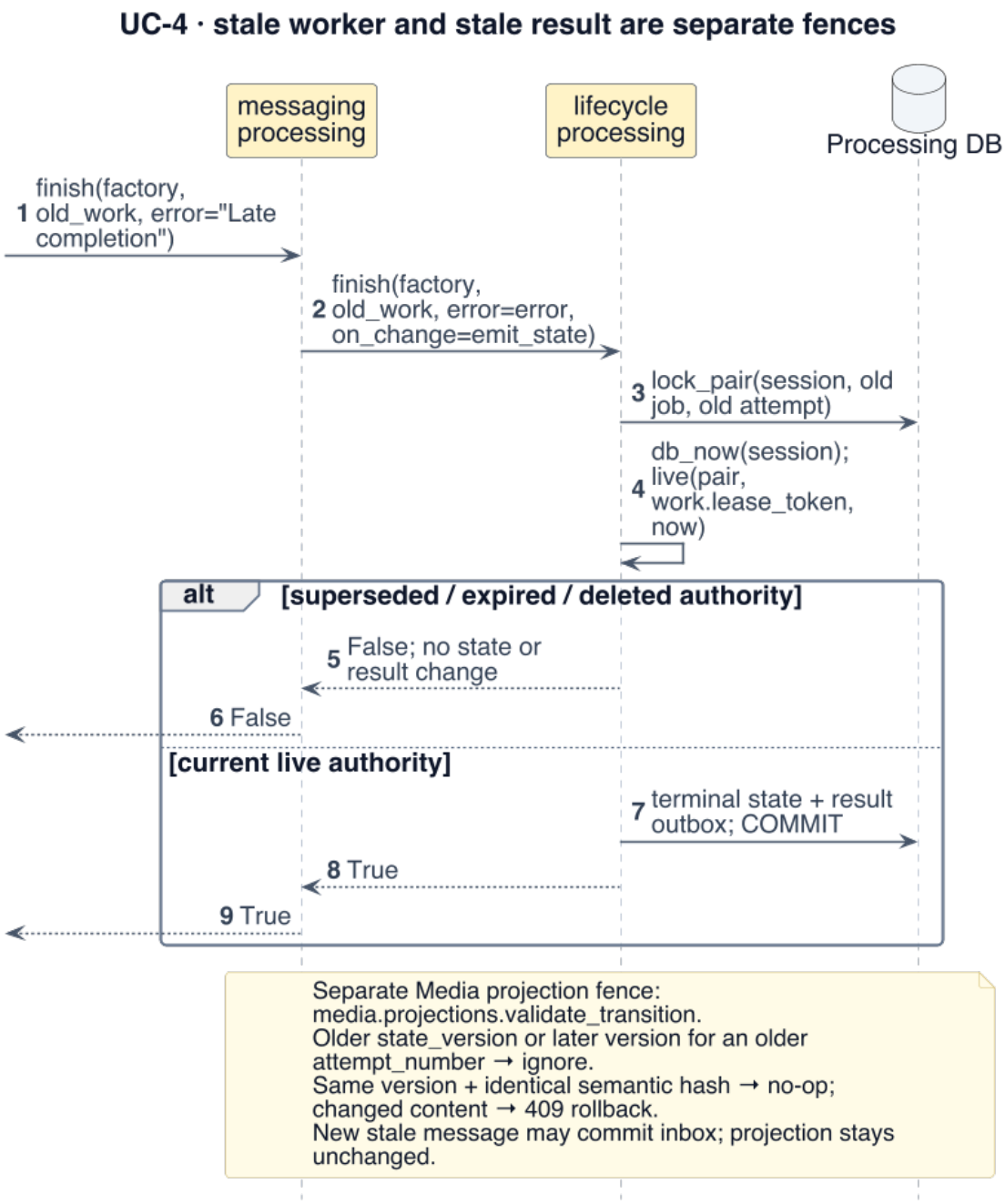


Figure 47. UC-4 independent worker-authority and result-projection fences.

Figure 47 separates `lifecycle.live` from `projections.validate_transition`. The former rejects stale execution tokens; the latter ignores old versions/attempts and rejects conflicting same-version content. `tests/v1/test_messaging_sql.py::test_retry_idempotency_and_cancel_fence_old_and_current_attempts` asserts successor uniqueness and rejected old finishes. `tests/media/test_v1_projection.py::test_retry_and_cancel_are_commands` checks Media intent; its transition tests cover version and attempt fencing.

Analysis to design

Retry control becomes Media's `v1_lifecycle.retry` and Processing's `_retry_locked`, connected by a versioned `Envelope`. Retry identity becomes `RequestKey`, stable message identity and `RetryRequest`, each addressing a different replay boundary. The conceptual

authority check becomes current-attempt and lease validation plus a separate projection fence. An expired Attempt remains abandoned; owner retry creates a new UUID rather than returning that terminal Attempt to queued.

6.4.4 UC-3 Track status and play the result

Description

Analysis and design: Nguyễn Kim Long. The primary actor is an authenticated owner (Maya or Arun) using the browser or Publisher CLI. The precondition is an owned Asset accepted through UC-1; encoding need not have finished. The postcondition is an owner-visible status and, after successful processing, expiring playback access. Reading status never mutates domain state.

In the normal flow the owner requests the Asset by its public ID, Media reads the local processing projection and displays the status, and a `done` Asset returns a signed primary MP4 URL whose bytes (including Range requests) come directly from object storage. Where an HLS manifest exists, the player reads playlists through Media and follows segment redirects to storage. In Media's `/jobs/{job_id}` routes, `job_id` identifies the **Asset**, while `processing_job_id` identifies Processing's Job (`media/api/v1_jobs.py`, `media/schemas/public.py`).

Exception flows preserve ownership and ordering. Unknown, foreign and deleted Assets return 404. Pending intake displays `accepted`; a projected job can be `queued`, `processing`, `done`, `failed` or `cancelled`, and only `done` has a playback URL. Missing HLS leaves MP4 available; unsafe HLS paths or unavailable playlists return 404. An expired capability needs another authorized status read. Duplicate or stale results cannot move the projection backwards, and conflicting results are rejected (`media/projections.py`, `media/api/v1_hls.py`).

Analysis class diagram

Figure 48 separates the viewing boundary, status control and owned result entities.

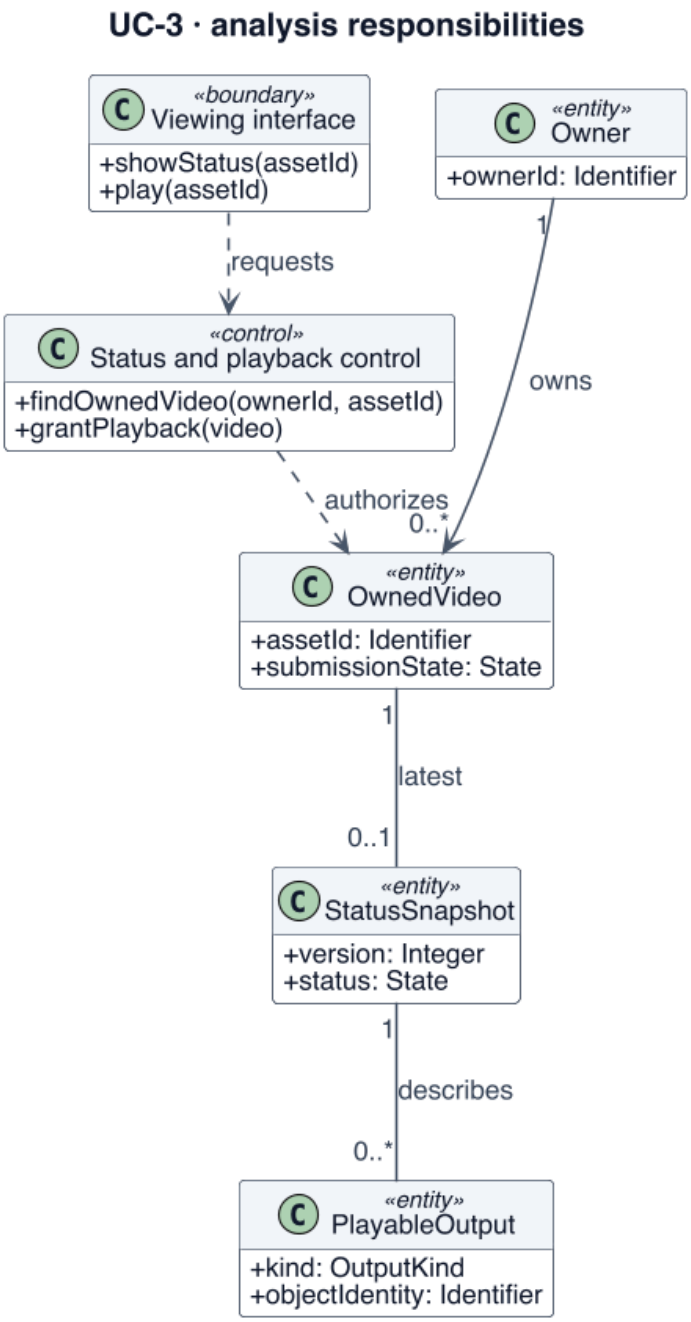


Figure 48. UC-3 analysis objects and ownership.

Analysis sequence diagrams

Figures 49 and 50 show status with optional playback, then denied access and result ordering.

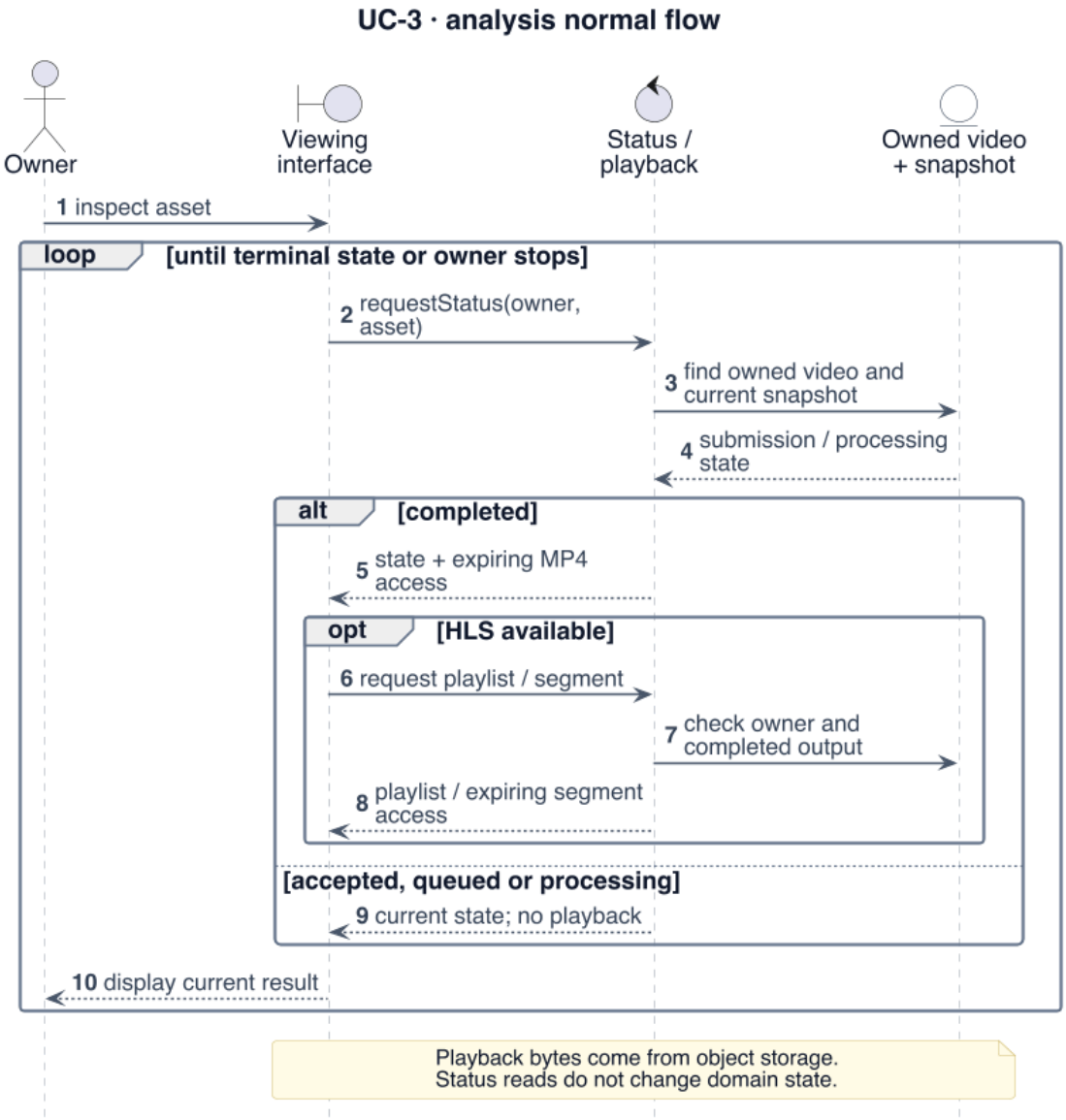


Figure 49. UC-3 normal status and playback flow.

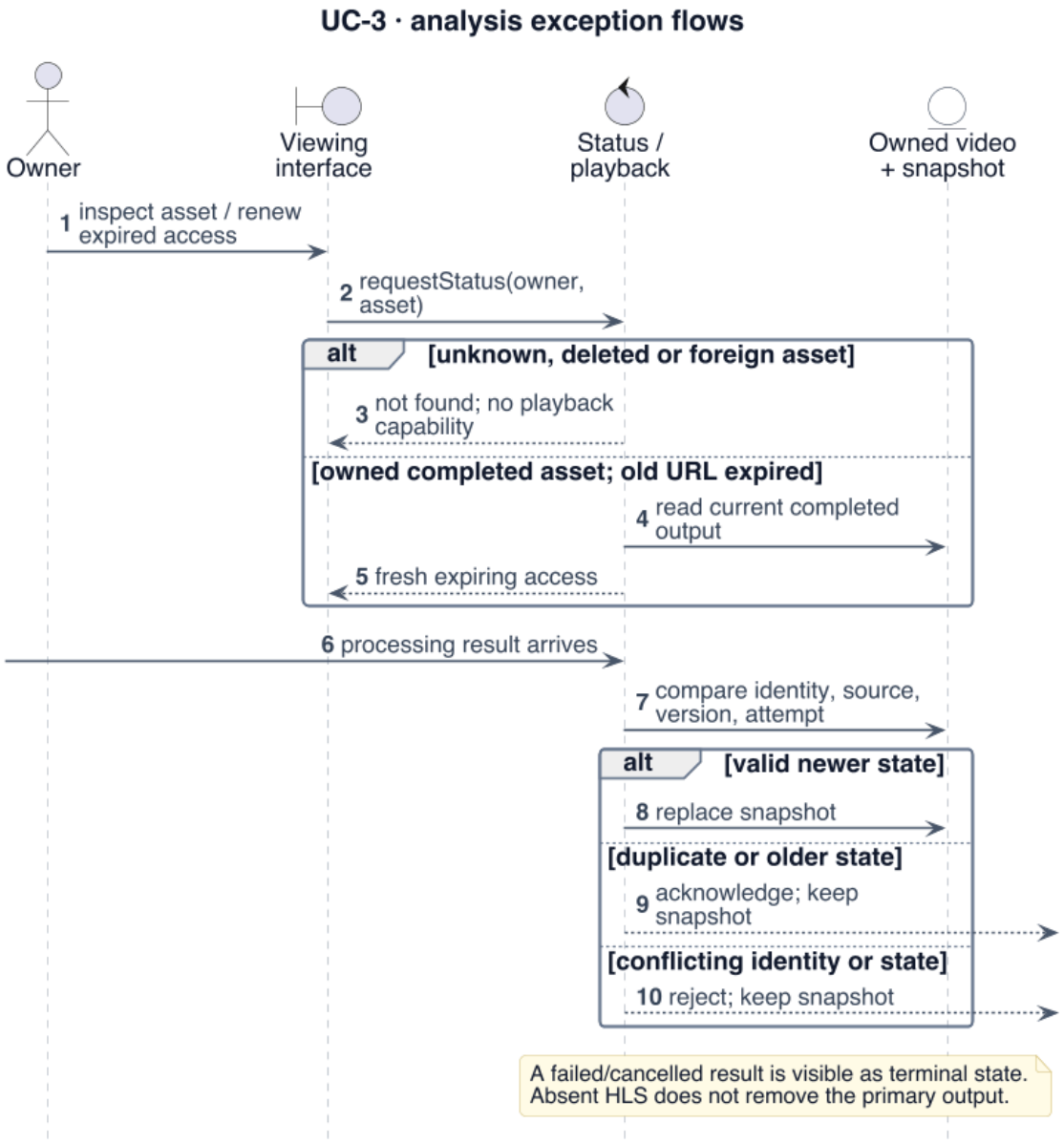


Figure 50. UC-3 ownership, replay and conflicting-result flows.

Design class diagram

Figure 51 refines the status record into `Asset` and `ProcessingProjection`. The `projections` module maps them into `PublicJob` and validates transitions; `Storage` supplies playback capabilities. The projection belongs to Media and has an Asset foreign key; its Processing identifiers are plain values (`media/models.py`, `media/projection_models.py`).

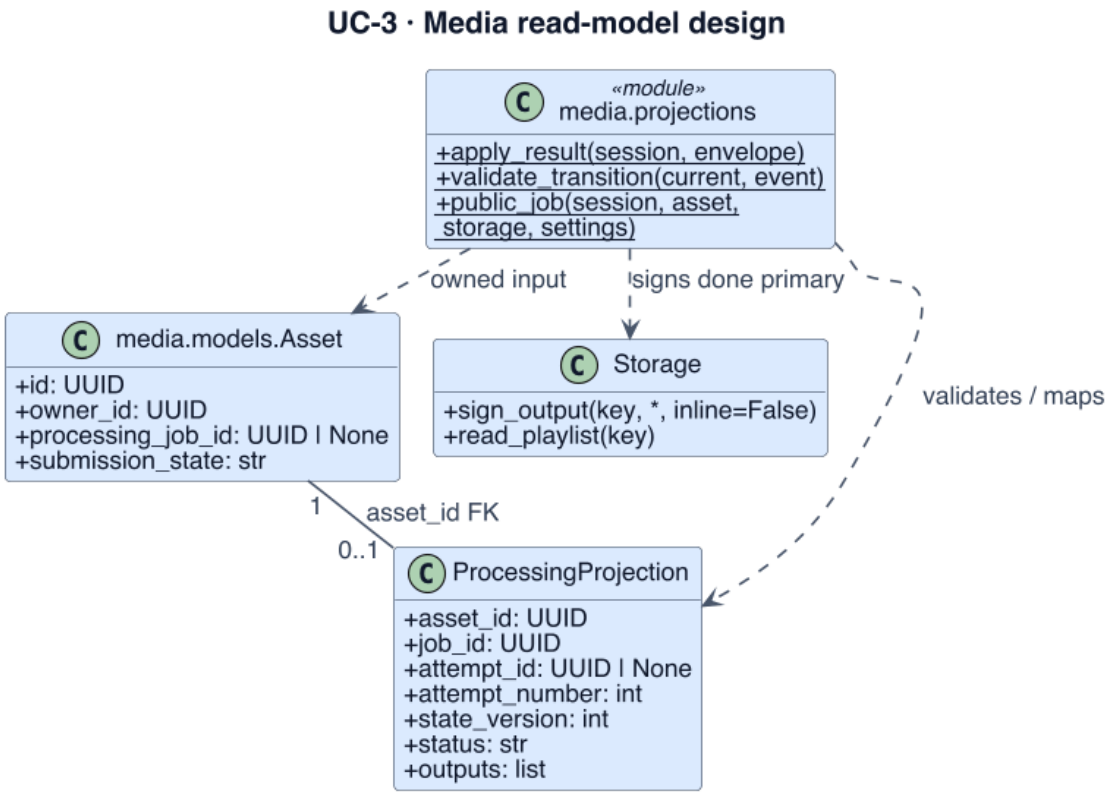


Figure 51. UC-3 implemented projection and playback collaborators, with selected members.

Design sequence diagrams

Figure 52 shows the authorized read and HLS branches. `get_asset` applies the owner predicate before mapping; `public_job_from_projection` signs only a completed primary output. `part` repeats ownership and readiness checks on every playlist or segment request. Playlists are bounded to 1 MiB, segments return HTTP 307, and authenticated responses carry `private, no-store` (`media/repositories/assets.py`, `media/services/storage.py`, `contracts/gateway.py`).

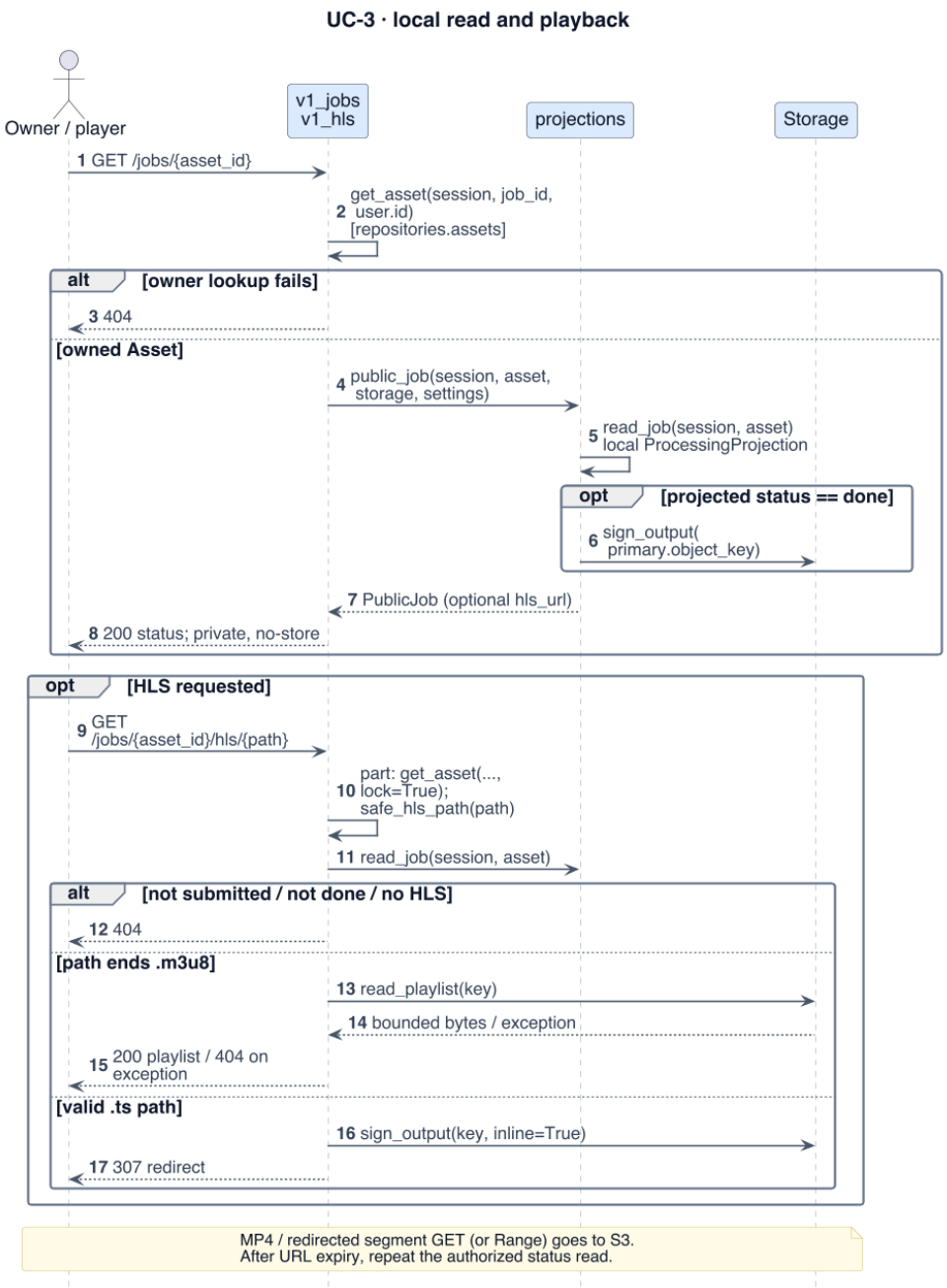


Figure 52. UC-3 local read, MP4 capability and optional HLS access.

Figure 53 shows the result transaction. Inbox deduplication precedes `apply_result`, which validates owner, accepted source, Job and output namespace before transition fences. Same-version replay excludes `message_id` from its hash. A new stale result commits its inbox entry without changing the projection; a conflict rolls back both, and the batch handler reports it as a failed record (`media/messaging.py`, `media/handlers.py`, `delivery/sql.py`).

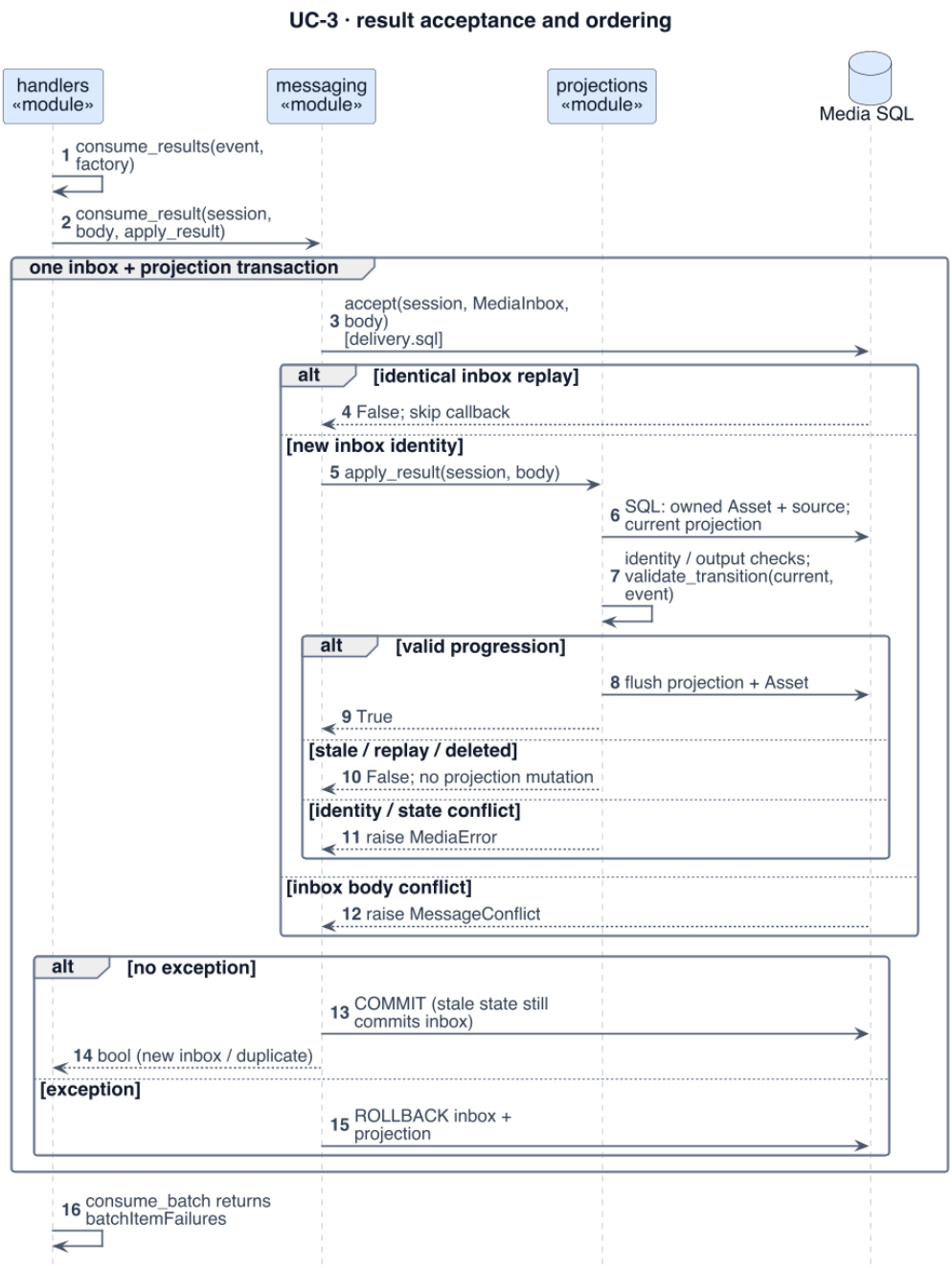


Figure 53. UC-3 atomic inbox/projection update and replay alternatives.

`tests/media/test_v1_projection.py` covers replay, attempt and terminal fences, owner/source/output rejection, status mapping and optional HLS. `test_local_status_hls_and_signed_range_capability` checks routes with mocked storage; the dated playback receipts in §4 cover S3 Range access.

Analysis to design

The owned-video entity becomes `Asset`; its status snapshot becomes `ProcessingProjection`; the view response becomes `PublicJob`. The status control becomes functions in `media/projections.py`, and playback access becomes `Storage.sign_output` and `v1_hls.part`. Separating projection writes from local reads avoids a Processing request during playback and gives result ordering one validation point.

6.4.5 UC-5 Bulk-publish a folder and report results

Description

Analysis and design: Nguyễn Kim Long. Maya is the content owner; Jules integrates the Publisher CLI. Preconditions are a valid session, an HTTPS API base and a folder of owner-held files. Publishing visits immediate regular files in sorted order, skips directories and symlinks, and records each attempted file's outcome. Acceptance means the API returned an Asset ID; `--wait` additionally requires processing to reach `done`.

The normal flow obtains the current owner, reserves an intent key, creates an upload reservation, sends bytes directly to S3, finalizes, and records the accepted ID. Optional bounded polling follows acceptance. A mixed batch continues after individual failures and exits 1 while retaining successful references. Authentication failure or uncertain transport keeps the pending key for replay; definitive rejection completes the intent. Empty batches and exhausted waits also exit unsuccessfully. The postcondition is a per-file report plus durable pending replay keys, not an all-or-nothing batch transaction (`publisher/cli.py`, `publisher/client.py`, `publisher/state.py`).

Analysis class diagram

Figure 54 gives the batch one outcome per attempted file and separates publication intent from accepted reference.

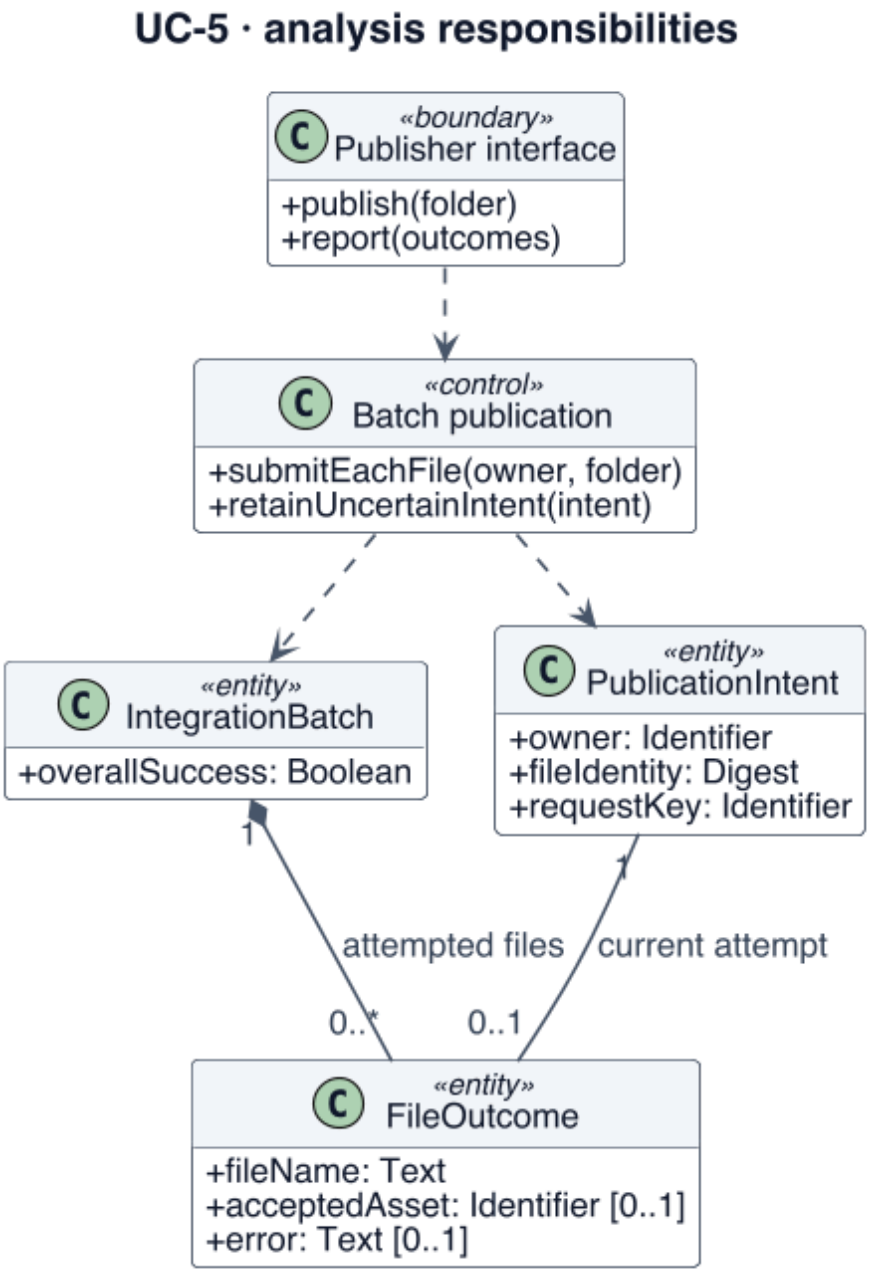


Figure 54. UC-5 conceptual batch, file intent and outcome objects.

Analysis sequence diagrams

Figures 55 and 56 show the normal loop and the partial-failure and replay alternatives; uncertain delivery is unresolved, not rejected.

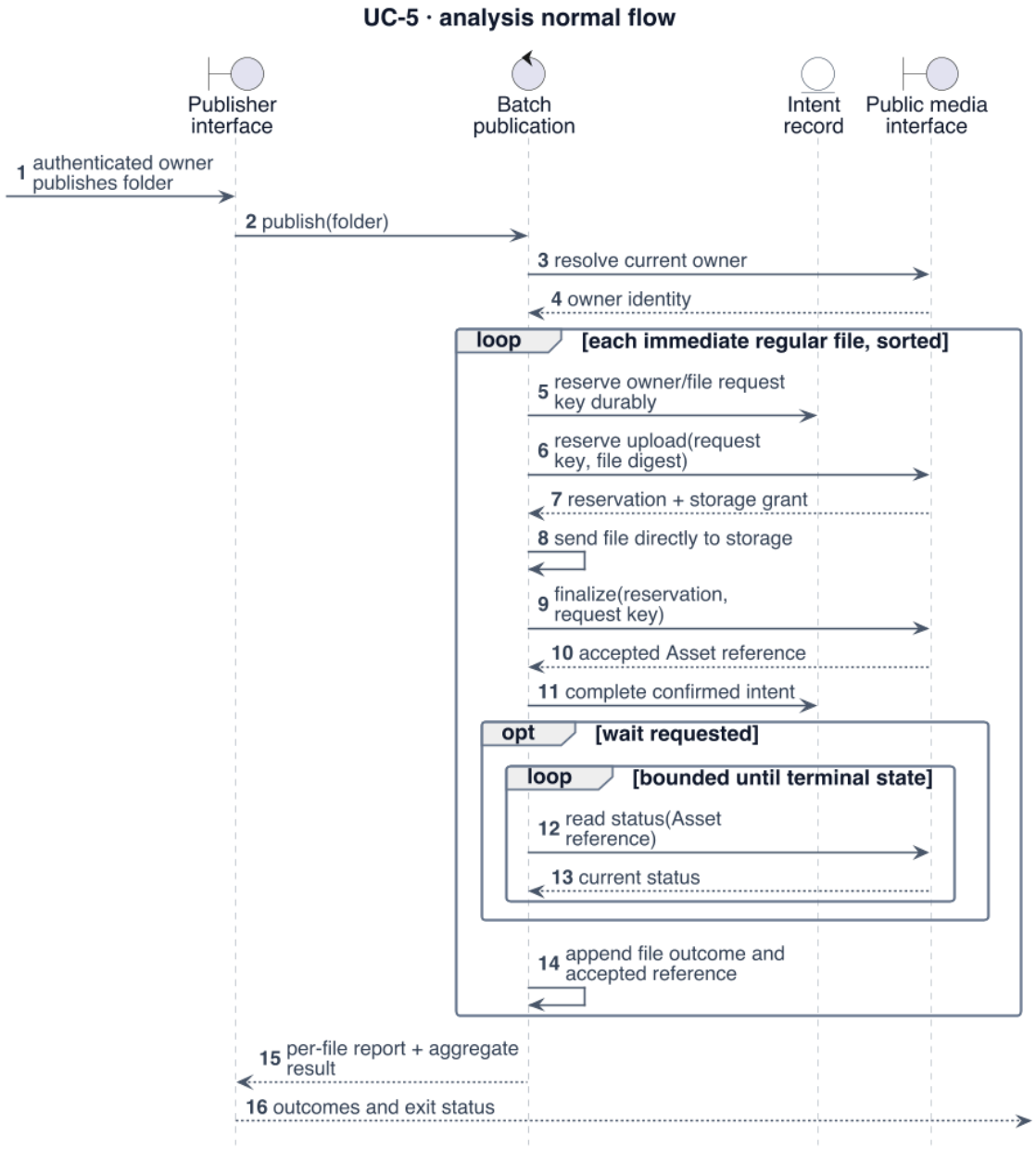


Figure 55. UC-5 per-file publication and optional completion wait.

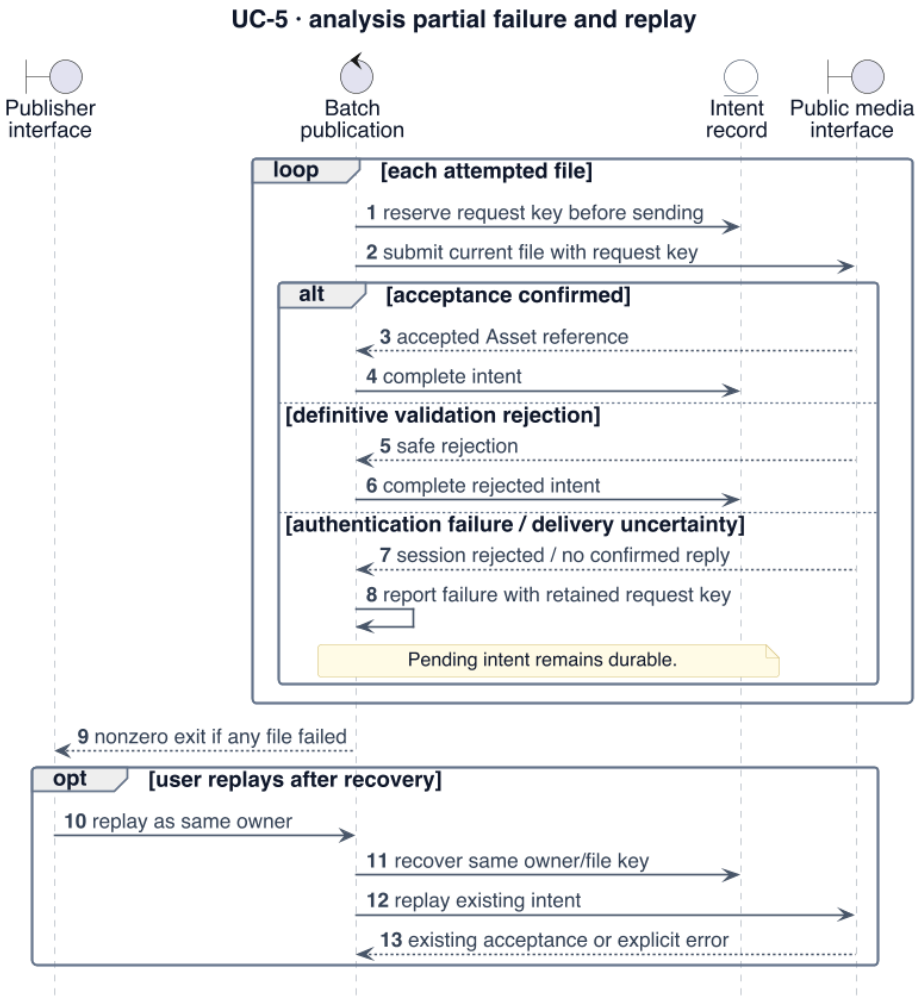


Figure 56. UC-5 mixed outcomes and replay after uncertainty.

Design class diagram

Figure 57 maps the batch control to the `cli` module, API access to `PublicClient`, and pending-key persistence to `Intents`. Scope is API base plus owner; file identity is name, size and SHA-256 of every byte. The CLI locks shared session state across processes, and private writes atomically replace files with mode 0600 (`publisher/state.py`, `publisher/client.py`).

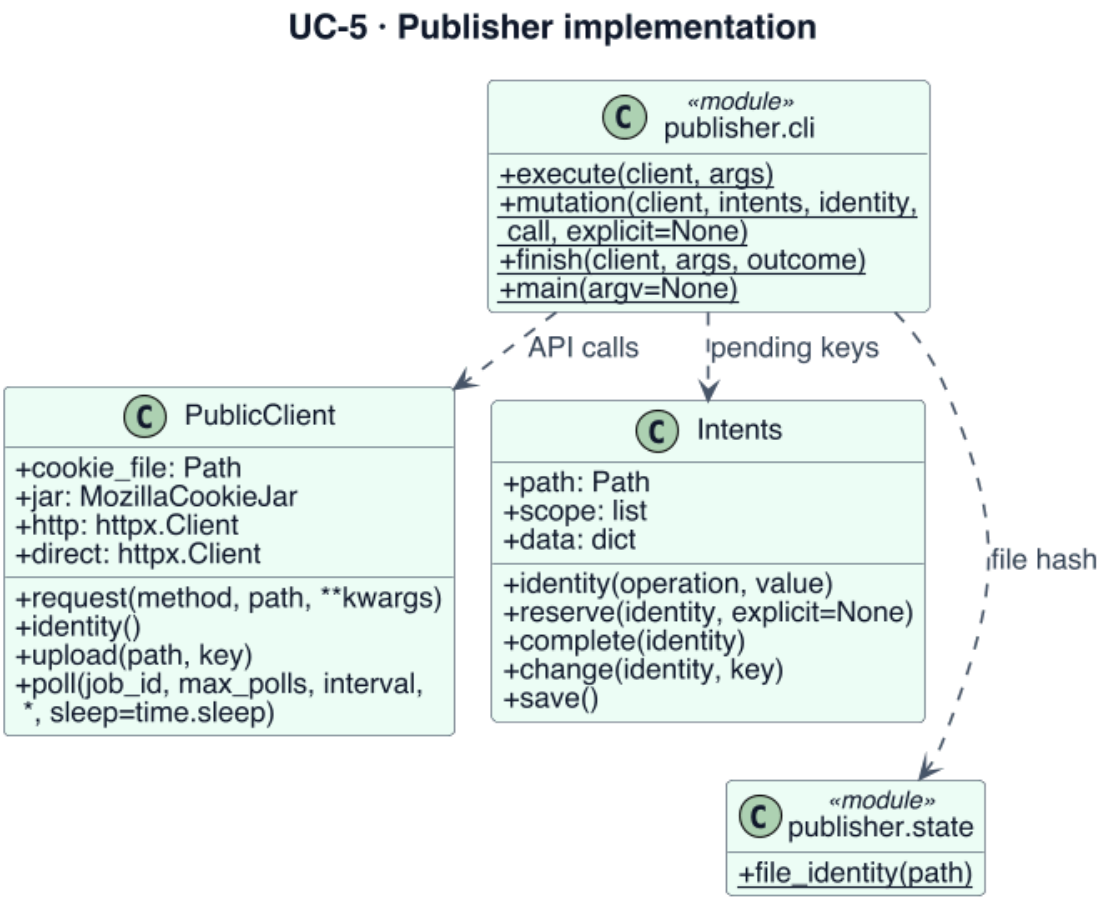


Figure 57. UC-5 implemented client, orchestration and private intent state.

Design sequence diagrams

Figure 58 shows `execute`, `mutation`, `upload` and `finish`. Upload uses a separate HTTP client without cookies; reservation and finalize share one request UUID, and no video bytes traverse API Gateway. A finalized replay may omit the upload grant.

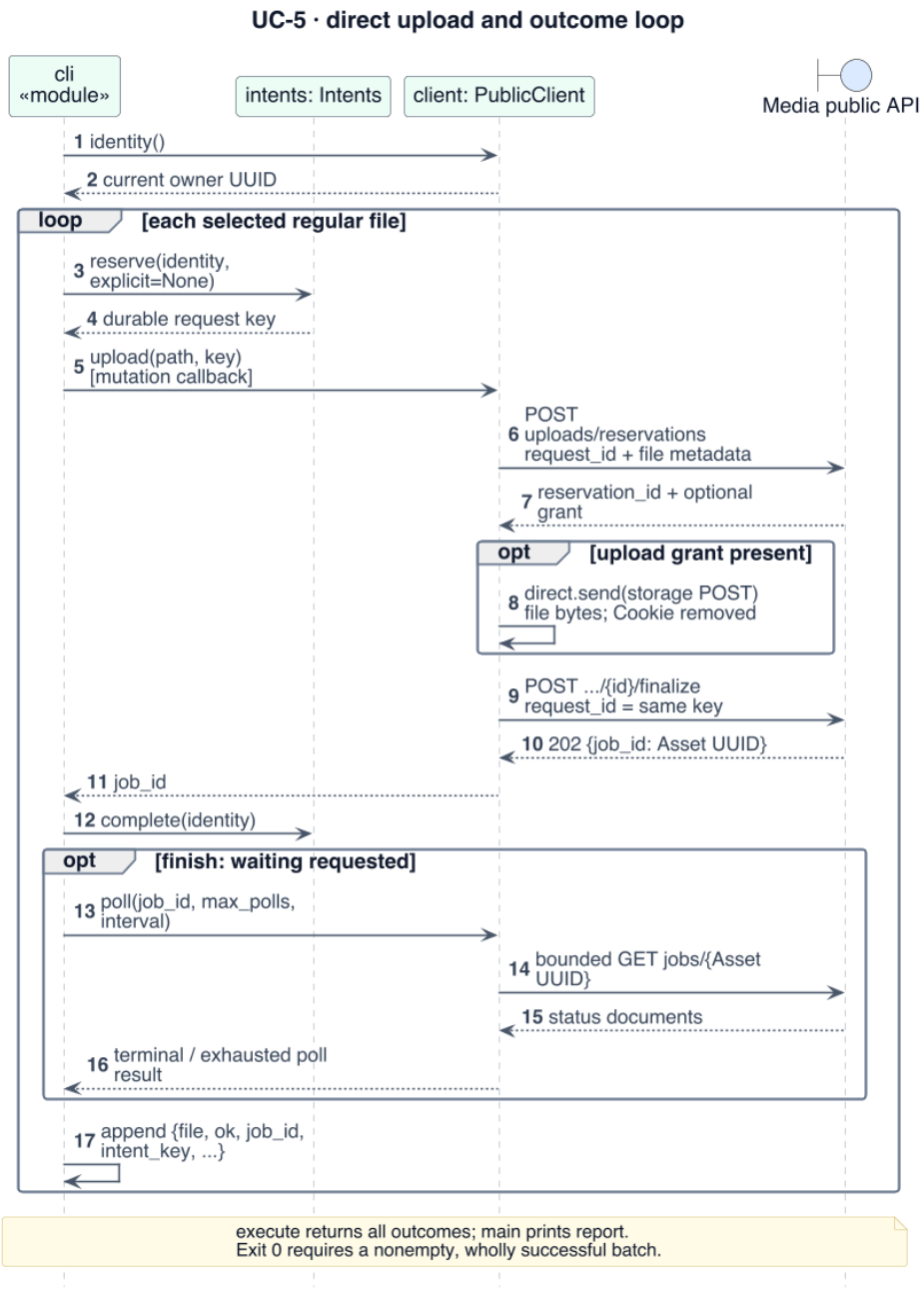


Figure 58. UC-5 direct upload protocol and per-file outcome accumulation.

Figure 59 shows why key reservation precedes network activity. `PublisherError.uncertain` or HTTP 401 preserves the key. Acceptance retains `job_id` in the outcome even if state persistence or polling fails; the report returns accepted IDs and the state file stores pending keys. Diagnostics omit server response bodies (`publisher/cli.py`, `publisher/client.py`).

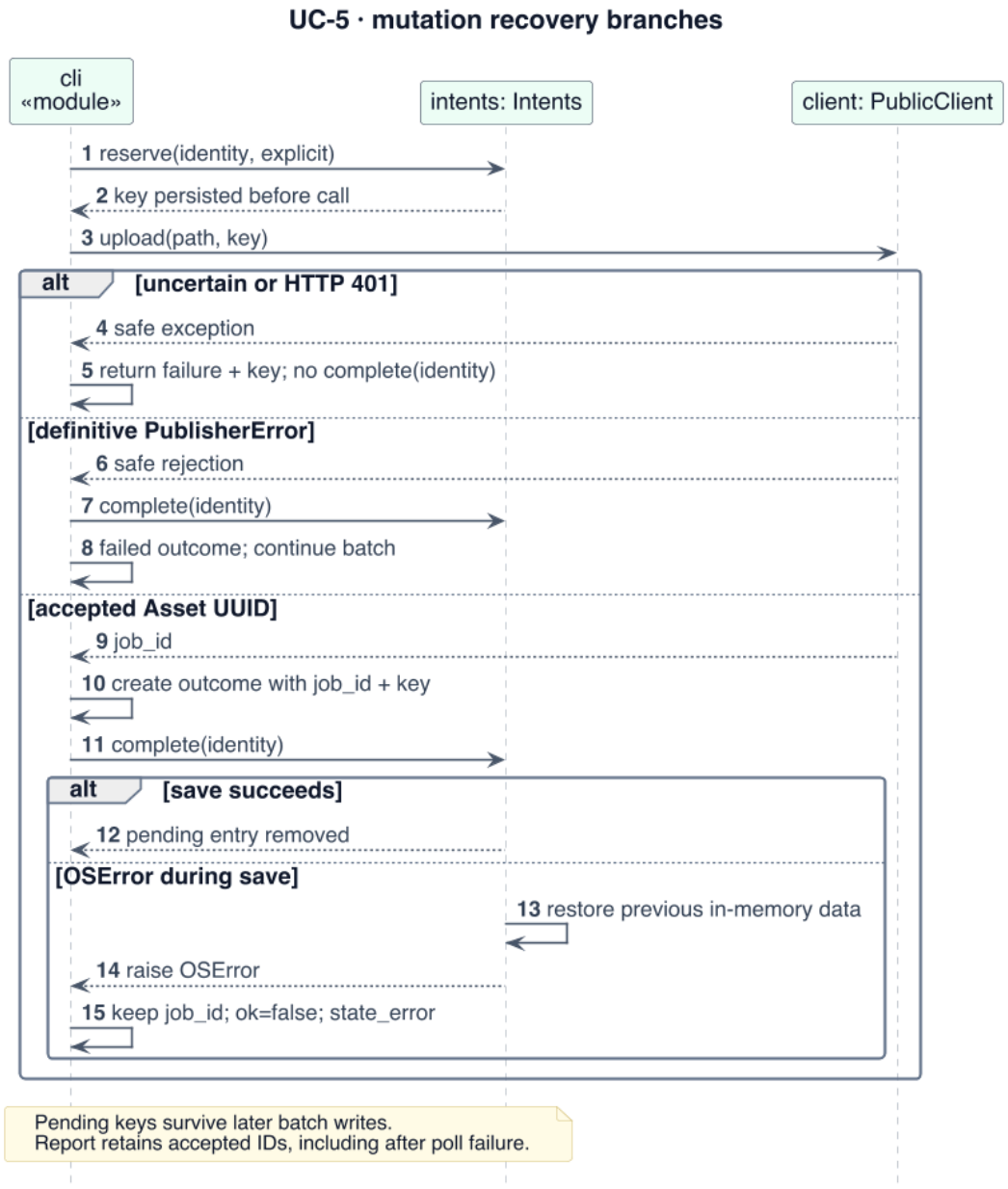


Figure 59. UC-5 uncertain delivery, definitive rejection and retained acceptance.

`tests/publisher/test_publisher.py` checks mixed batches, replay after transport or authentication failure, bounded polling and state-write failure. `test_direct_upload.py` checks cookie isolation and finalize replay; `test_file_identity.py` checks complete-file hashing.

Analysis to design

`IntegrationBatch` becomes the `execute` loop and returned `files` list, with no server-side batch entity. `PublicationIntent` becomes `Intents` with durable UUID reservation. The integration boundary becomes `PublicClient` over the published HTTP schemas. Publisher imports no service or database code, so it follows the same public behavior as the web client.

6.4.6 UC-6 Scale processing capacity independently

Description

Analysis and design: Guruprasath Gopal. Sam, the operator, enables and observes capacity through operator tools; work publication and the fallback timer start automatic decisions.

Preconditions are an enabled launcher, a durable Processing backlog and a configured standard worker definition.

In the normal flow, a successful standard-work send wakes the launcher, and a one-minute schedule recovers a lost wake. The launcher reads queued current Attempts, visible standard-queue messages and worker inventory, reconciles earlier reservations, reserves available slots in one transaction, then starts finite workers. Each worker claims work separately and exits after its bounded execution. The postcondition is capacity between zero and three standard workers; unresolved starts still consume slots and excess work stays queued (`processing/launcher_trigger.py`, `processing/launcher.py`).

Exception flows cover zero visible demand, disabled control, unreadable inventory and uncertain RunTask replies; they prevent new reservations or preserve existing ones. A later observation can identify an uncertain task by its persisted launch identity; otherwise a permitted replay uses the same token and payload. Workers that repeatedly exit without claiming trigger a cooldown and then an idle breaker. Disabling waits for in-flight sends. Short-lane dispatch is a separate decision (section 3.2.1, UC-2).

Analysis class diagram

Figure 60 separates the operator boundary, capacity decision and durable capacity records. A slot reservation has its own identity and lifecycle and does not claim a queued Attempt; its association to an executor is an observed outcome that may start unknown.

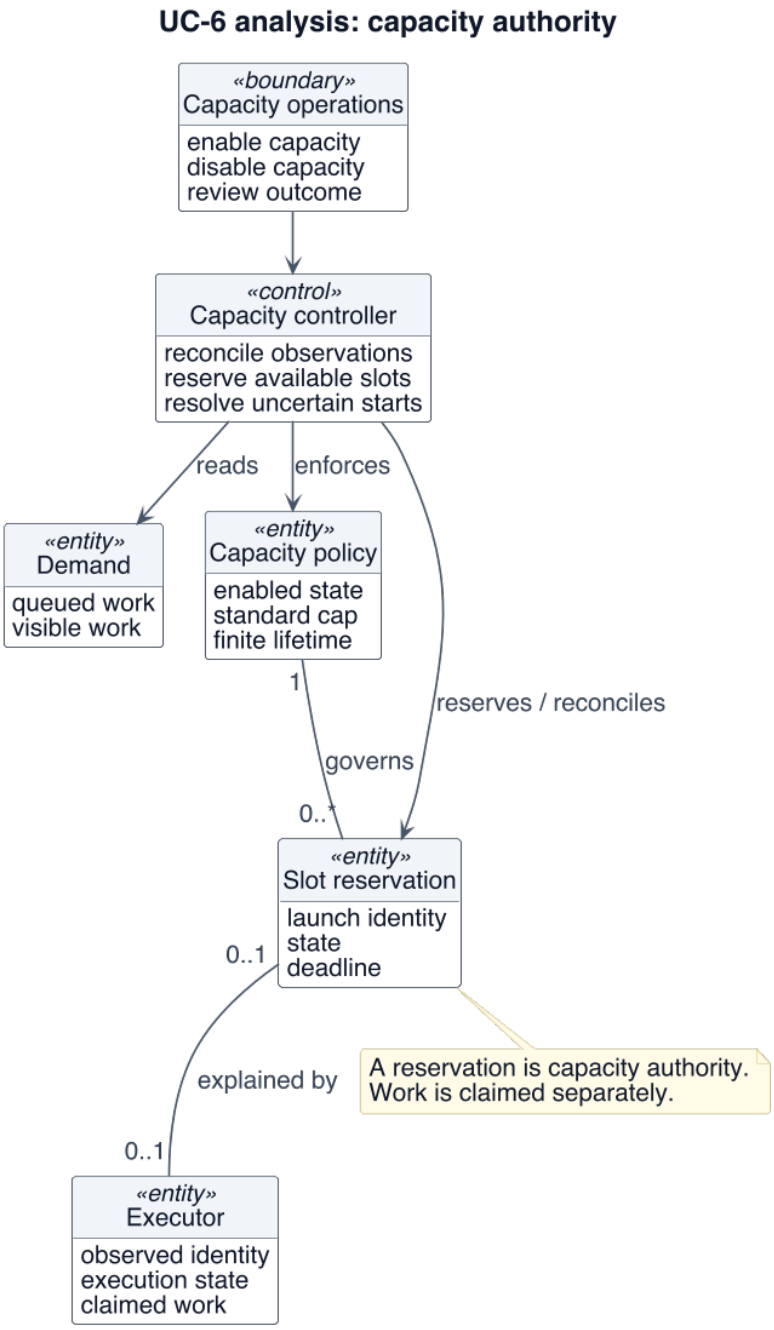


Figure 60. UC-6 conceptual capacity objects and responsibilities.

Analysis sequence diagrams

Figure 61 follows demand through reservation to finite execution; Figure 62 shows the two alternatives, a lost start reply and disabling capacity.

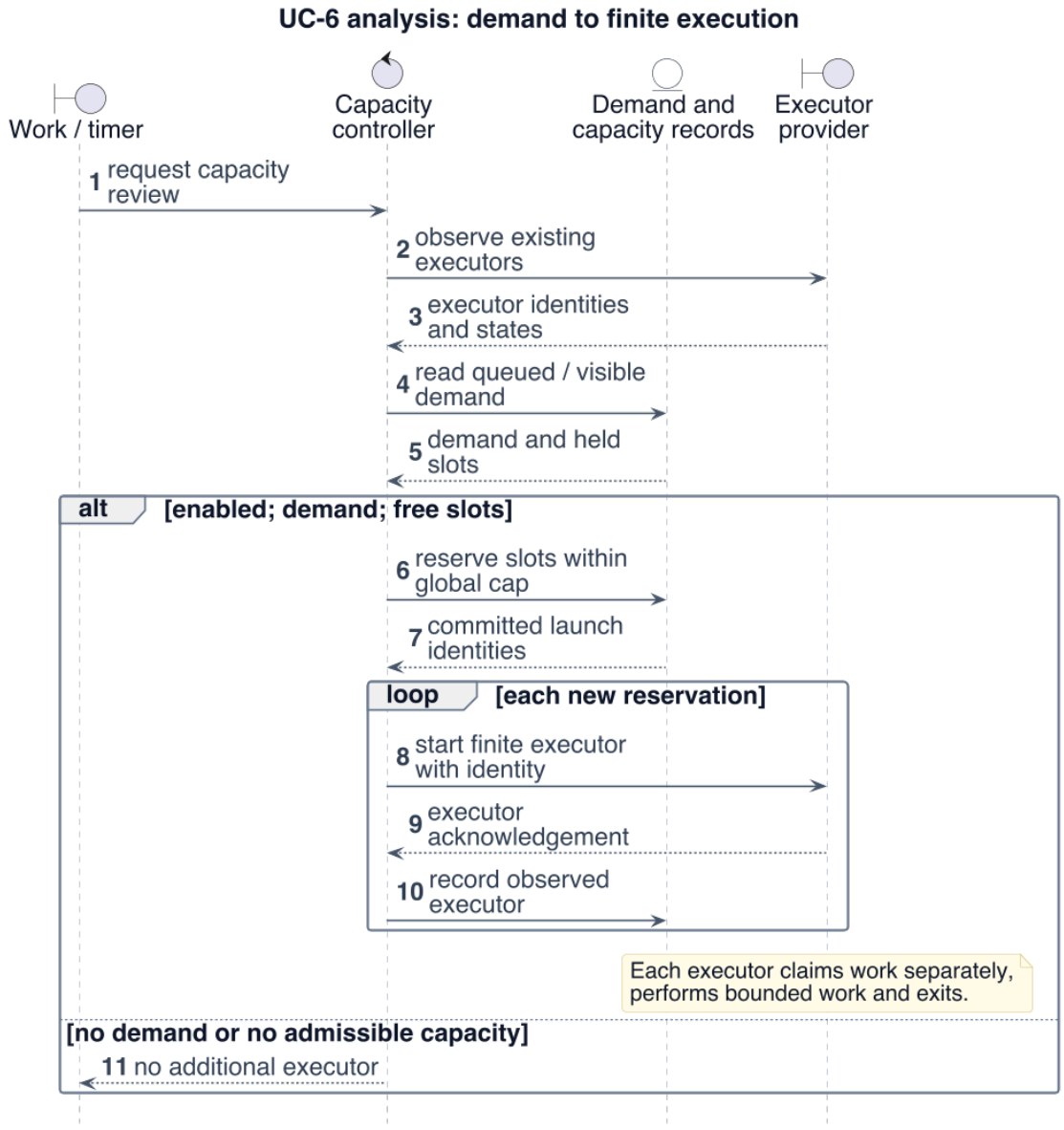


Figure 61. UC-6 normal flow: reserve before requesting an executor.

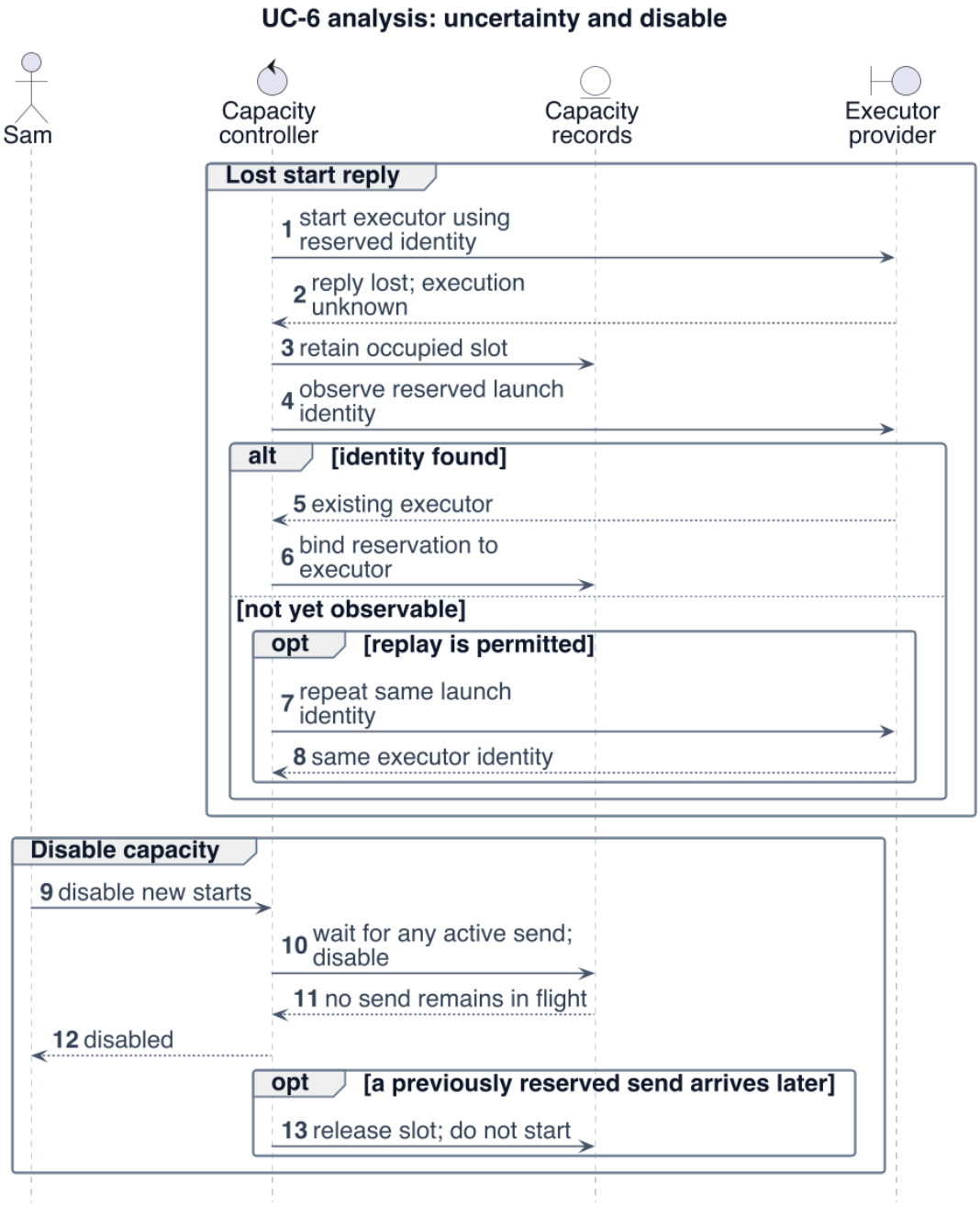


Figure 62. UC-6 alternative flows: preserve uncertainty and interlock disable.

Design class diagram

Figure 63 maps those responsibilities to actual modules and records. `LauncherConfig.slots` clamps the configured cap to `HARD_CAP = 3`. `LauncherControl` is a singleton; `LaunchReservation` has a token primary key and unique `started_by`, linked to the control row by application logic rather than a foreign key. `EcsPort` and `QueueDepth` isolate network operations from the reservation policy (`processing/launcher_models.py`, `processing/launcher_aws.py`).

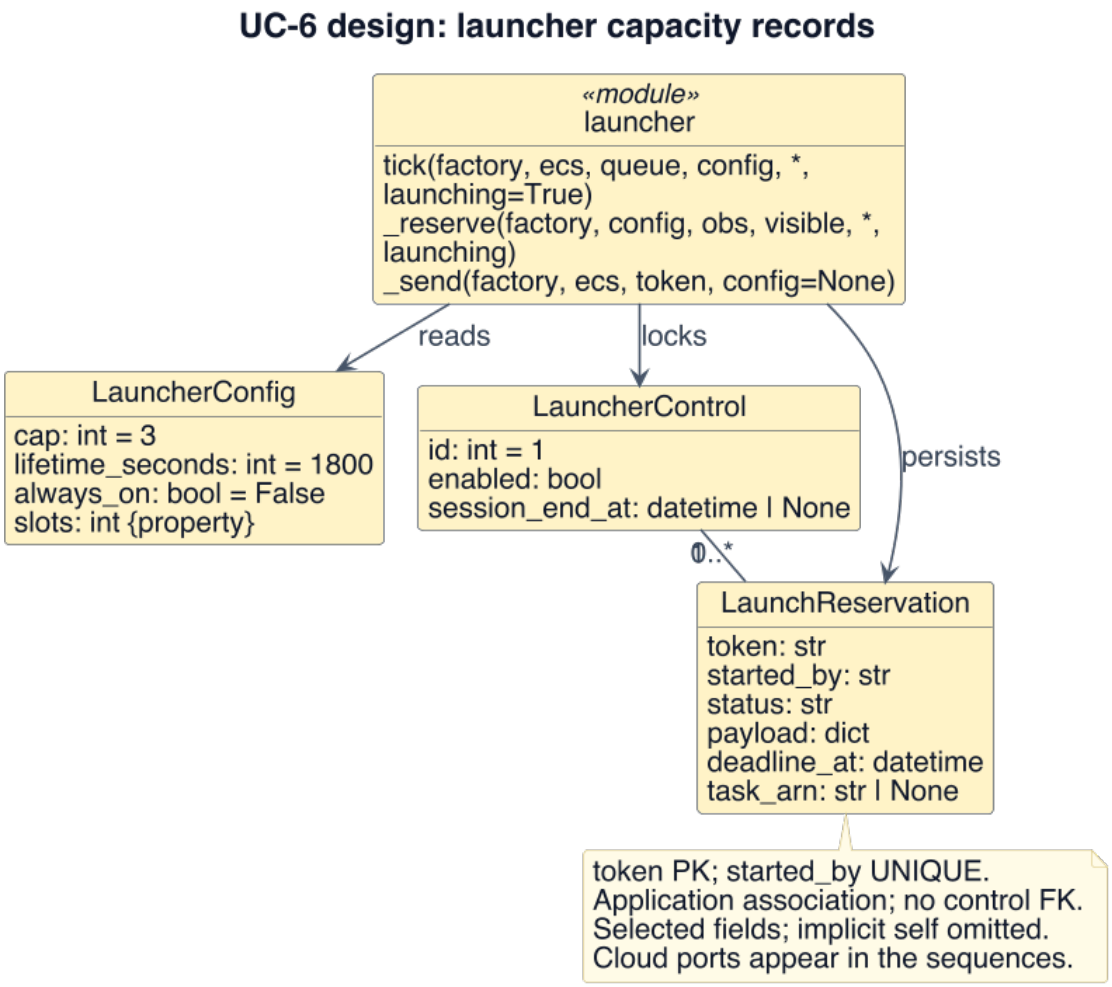


Figure 63. UC-6 selected implementation fields, operations and capacity records.

Design sequence diagrams

Figure 64 expands the normal tick. `tick` calls the module function `observe`, which lists and describes tasks through `EcsPort`. `_reserve` takes the control row `FOR UPDATE`, reconciles observations and commits reservations before `_send` calls RunTask. Demand is the smaller of SQL backlog (no lane filter) and visible standard-queue messages. Live workers plus unexplained reservations consume capacity, and workers not yet bound to an Attempt offset demand.

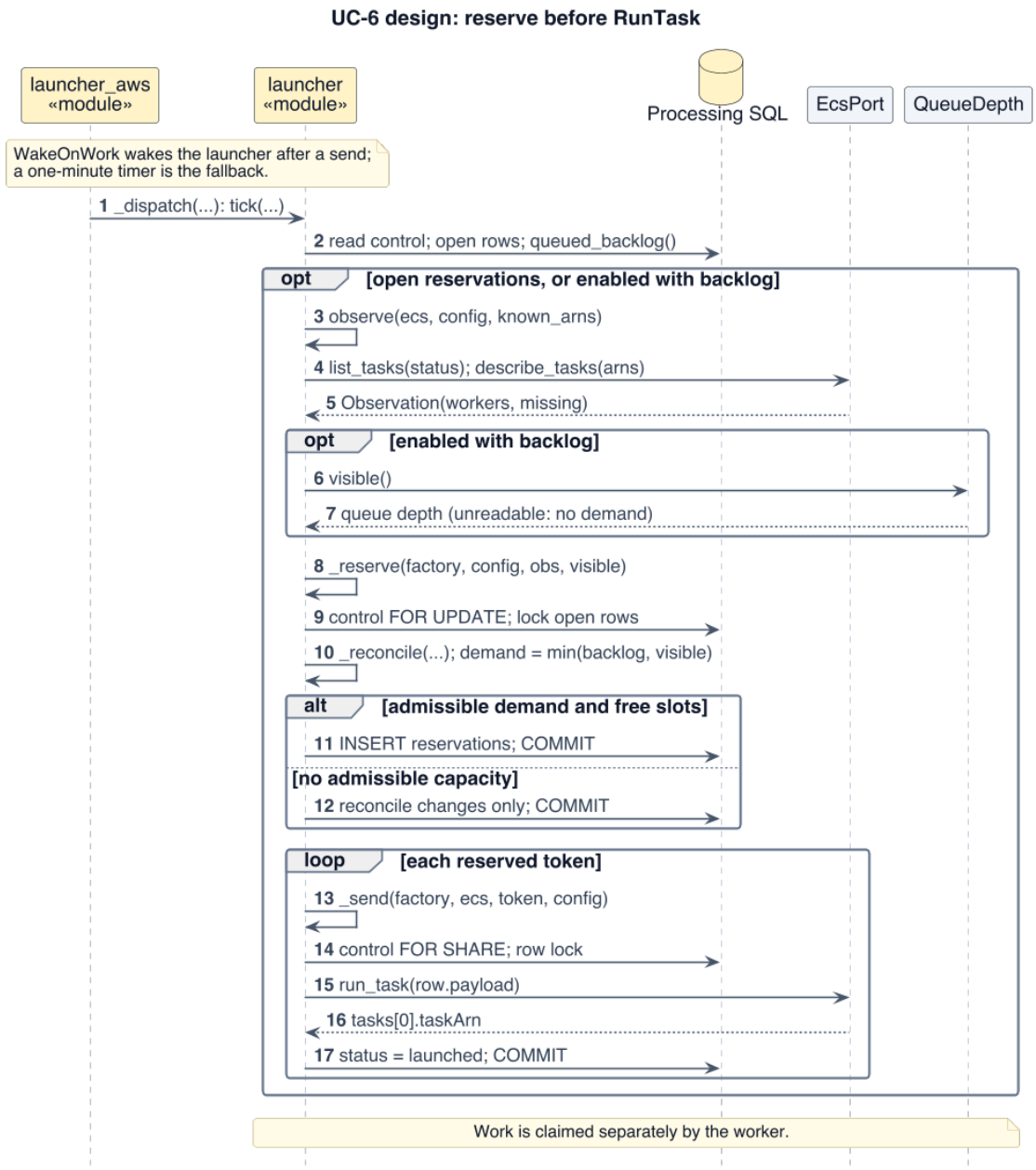


Figure 64. UC-6 implementation: wake, observe, reserve, commit and send.

Figure 65 details response loss and replay. An inventory failure aborts the tick, and an unreadable queue produces no new demand. A refused start releases its slot; an uncertain start keeps it for reconciliation, bounded replay or release at its deadline. Every replay goes back through `_send`, which repeats the shared control lock, the reservation lock and the state, enabled and session checks before `run_task` with the stored payload and the same `clientToken`.

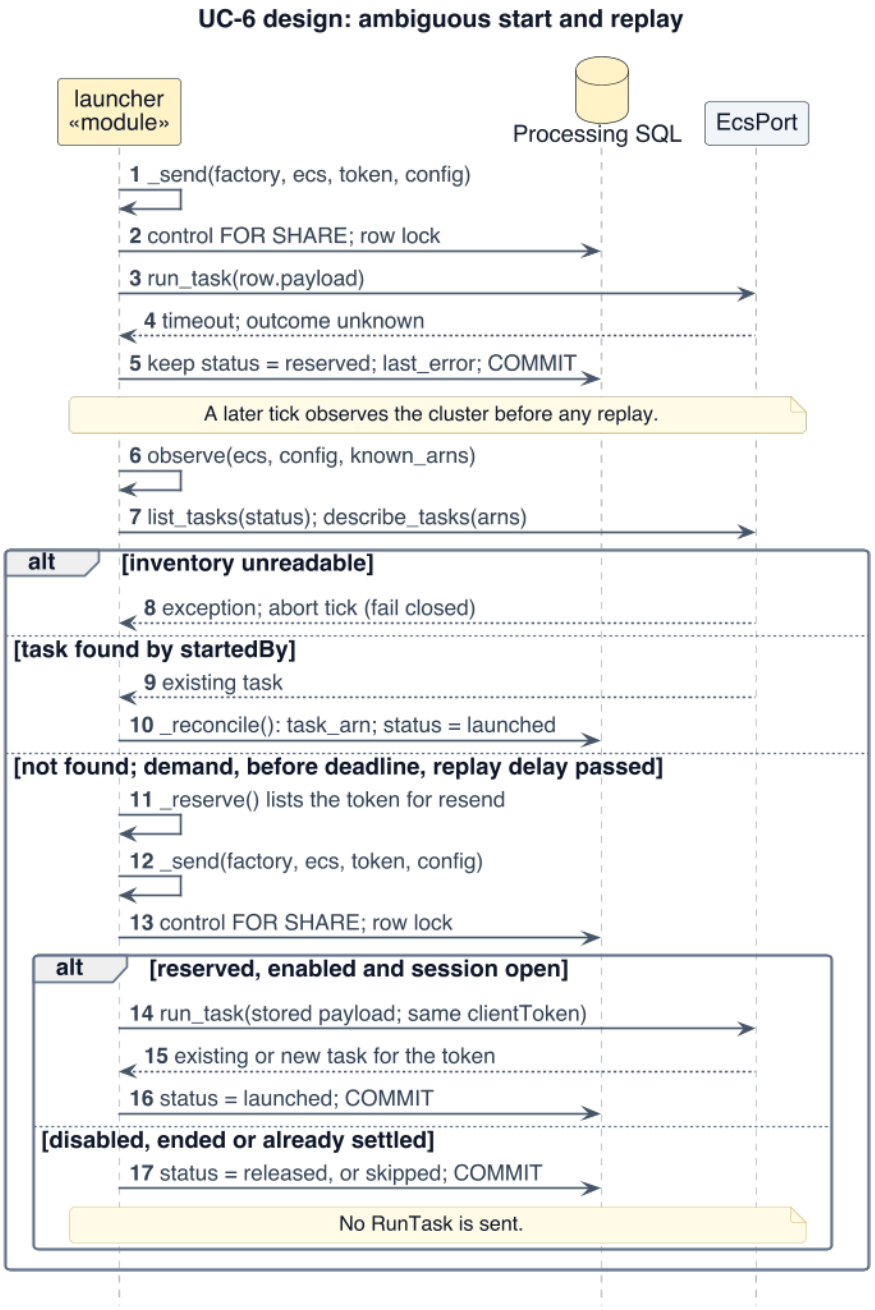


Figure 65. UC-6 implementation: ambiguous start, reconciliation and checked replay.

Figure 66 shows the disable interlock. `_send` holds the control row `FOR SHARE` during RunTask and `set_enabled` needs `FOR UPDATE`, so disabling returns only when no send is in flight. A later `_send` sees the flag off and releases its slot without calling ECS (`processing/launcher.py`).

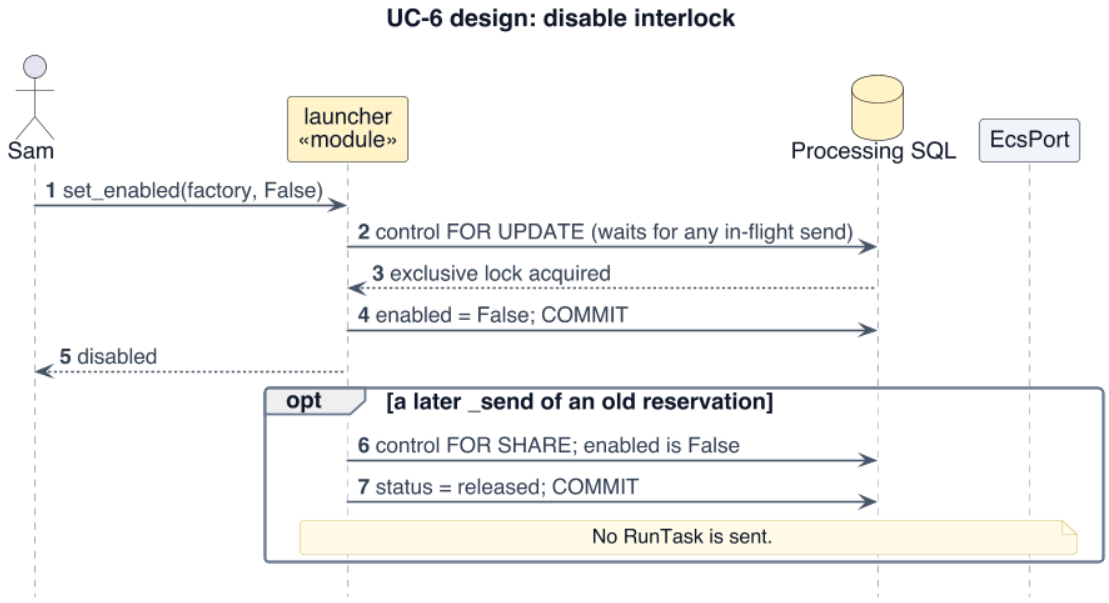


Figure 66. UC-6 implementation: disable waits for in-flight sends.

Analysis to design

Analysis responsibility	Design change and reason
Capacity controller	`launcher.tick`, `_reserve` and `_send` split observation, serialized decisions and uncertain network effects.
Capacity policy and reservation	`LauncherControl`, `LauncherConfig` and `LaunchReservation` add typed control, finite deadlines and stable replay identities.
Executor observation	`EcsPort`, `QueueDepth` and `Observation` make inventory and demand explicit; `AttemptTask` supplies claim bindings.
Demand notification	`WakeOnWork.send` adds one best-effort wake after a standard send; the schedule supplies recovery.

`tests/v1/test_launcher_sql.py` covers concurrent always-on wakes, backlog larger than the cap, lost replies, identical replay, idle exits and disable races. The SQL suite uses PostgreSQL with fake ECS and SQS ports; dated cloud execution results are in §4. `tests/aws/v1/test_unattended_launch.py` checks configuration, IAM scope and operator control.

6.4.7 UC-7 Release Processing on its own and demonstrate rollback

Description

Analysis and design: Guruprasath Gopal. Sam acts through CD and operator tools. Preconditions are an established environment, successful full CI for the exact source, verified immutable image evidence, additive migration evidence and a retained prior version; the release hooks also check the owner's intake and consumer conditions.

Normally, CD admits the source, verifies the selected images and identifies changed owners; a Processing-only change preserves Media. Promotion records private pending intent, runs the owning migration, publishes immutable versions, moves Processing aliases with revision checks and runs the owner public smoke. Adoption then records the images as the deployment baseline and updates the stack, including the launcher's worker definition (`scripts/aws/v1/release.py`, `deploy.py`, `scripts/ci/cd.sh`).

Failed admission moves no alias. A candidate failure prompts restore, which must itself pass smoke; unresolved recovery retains intent, and a provably unchanged candidate may be abandoned. Closing the environment defers release, and changed migrations need explicit additive compatibility attestation. Postconditions distinguish promoted, restored, abandoned and restore-failed outcomes; schema migrations are never reversed automatically.

Analysis class diagram

Figure 67 models release coordination, candidate evidence, prior state and recovery intent. The peer service is observed independently, and its preservation must be evidenced before an owner release is certified.

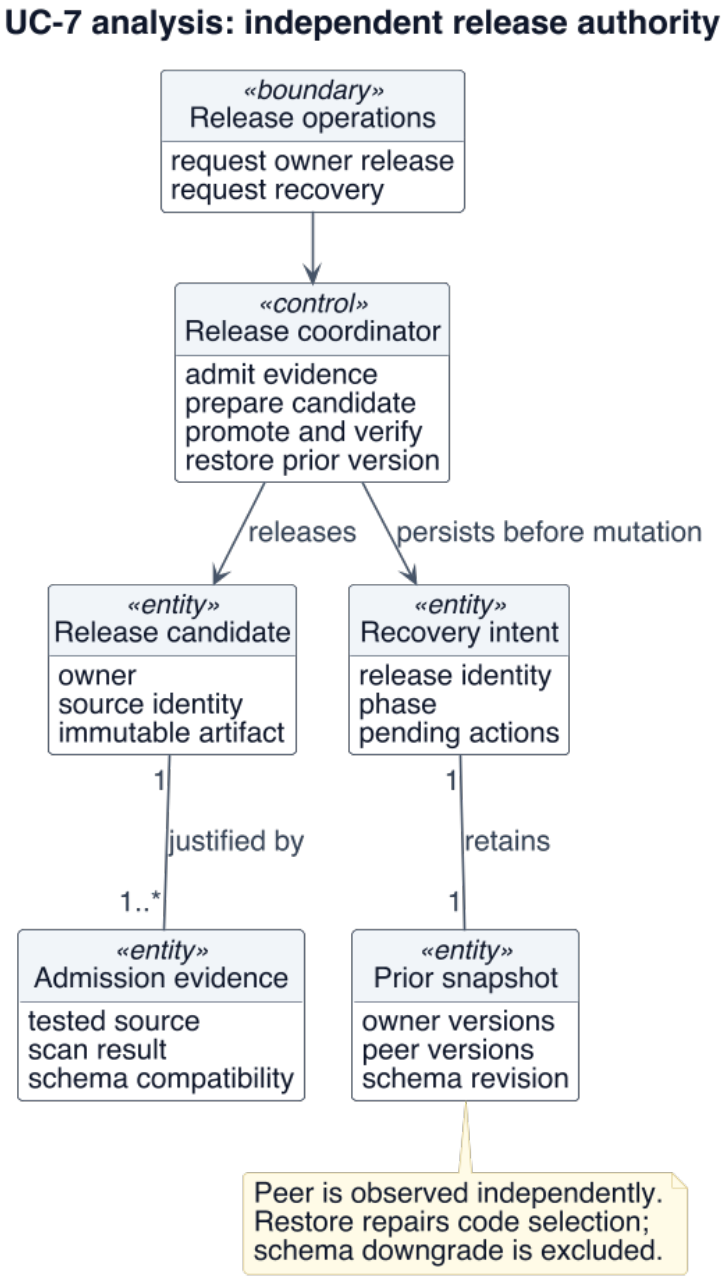


Figure 67. UC-7 conceptual release, evidence and recovery responsibilities.

Analysis sequence diagrams

Figure 68 covers admission rejection and promotion; Figure 69 follows candidate failure through restore or retained recovery intent, distinguishing successful recovery from merely requesting it.

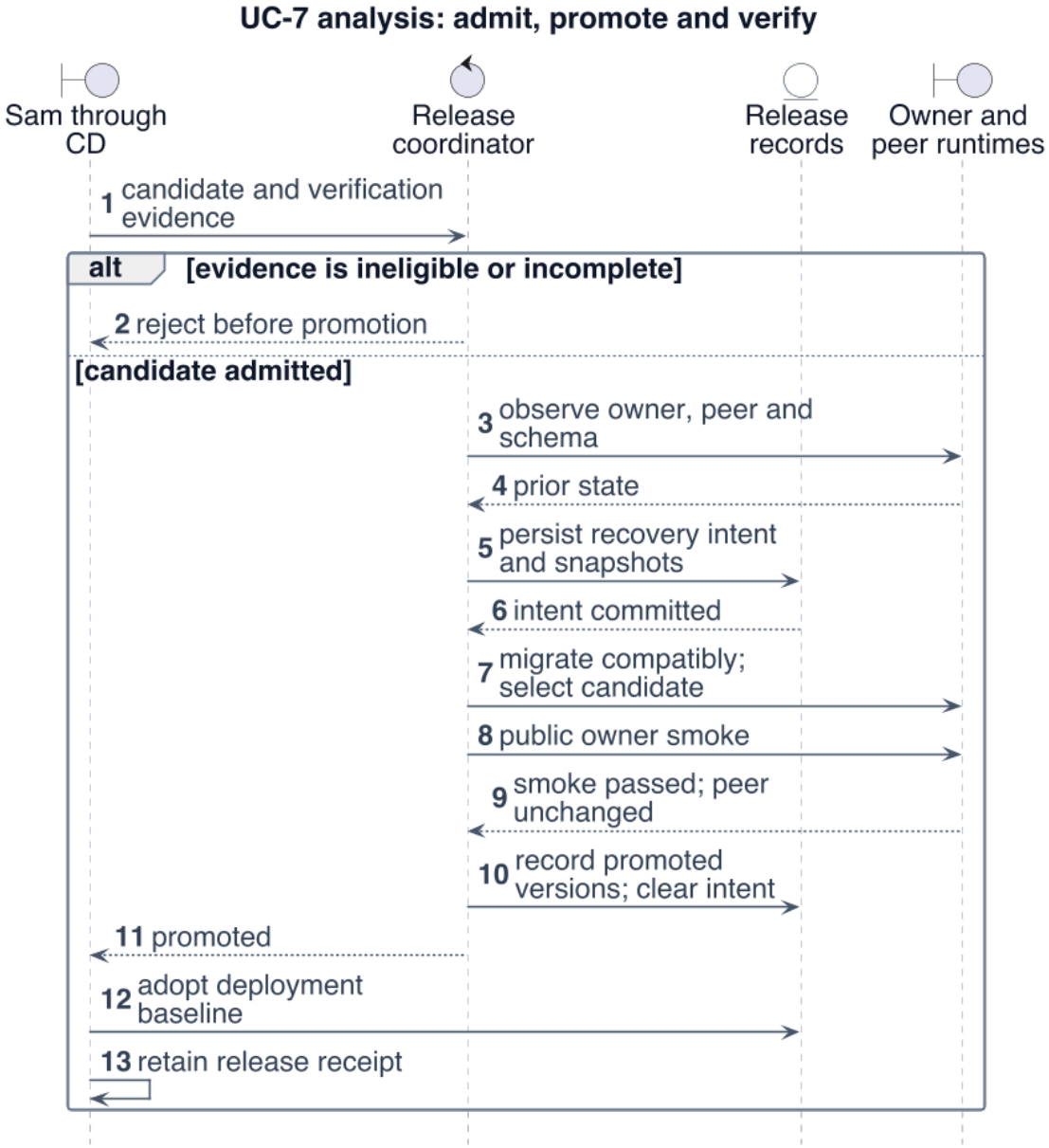


Figure 68. UC-7 admission and normal independent owner release.

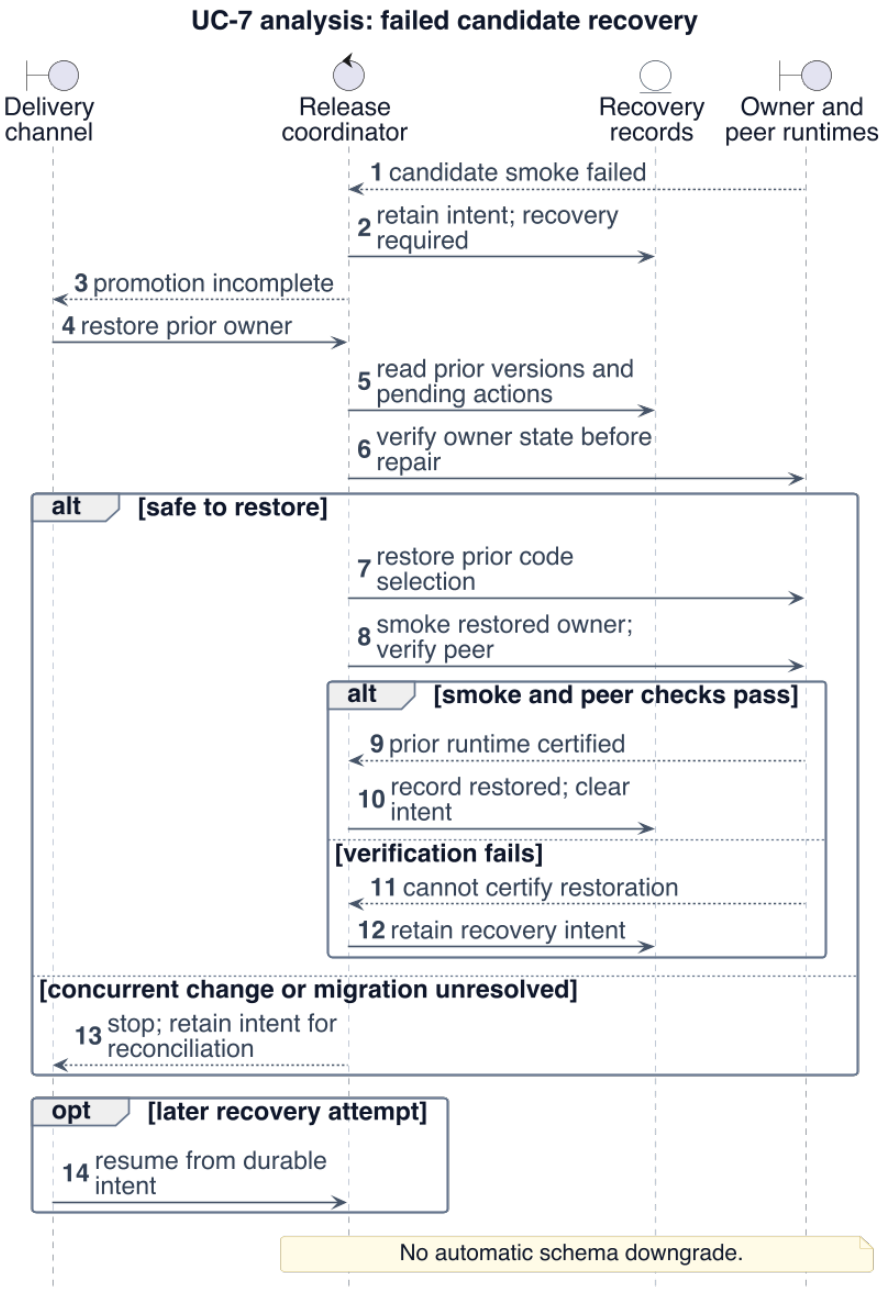


Figure 69. UC-7 failed candidate and resumable recovery.

Design class diagram

Figure 70 uses the implemented release functions, station journal and adoption function. `DeployJournal` extends `Journal`; `Journal.save` atomically replaces private state and its station lock serializes local operation, which is designated-station coordination rather than a distributed lock (`scripts/aws/journal.py`). `OperatorTransport.call` is the AWS execution port; the older `CLITransport` is restricted to planning and emulator execution (`scripts/aws/v1/transport.py`, `scripts/aws/transport.py`).

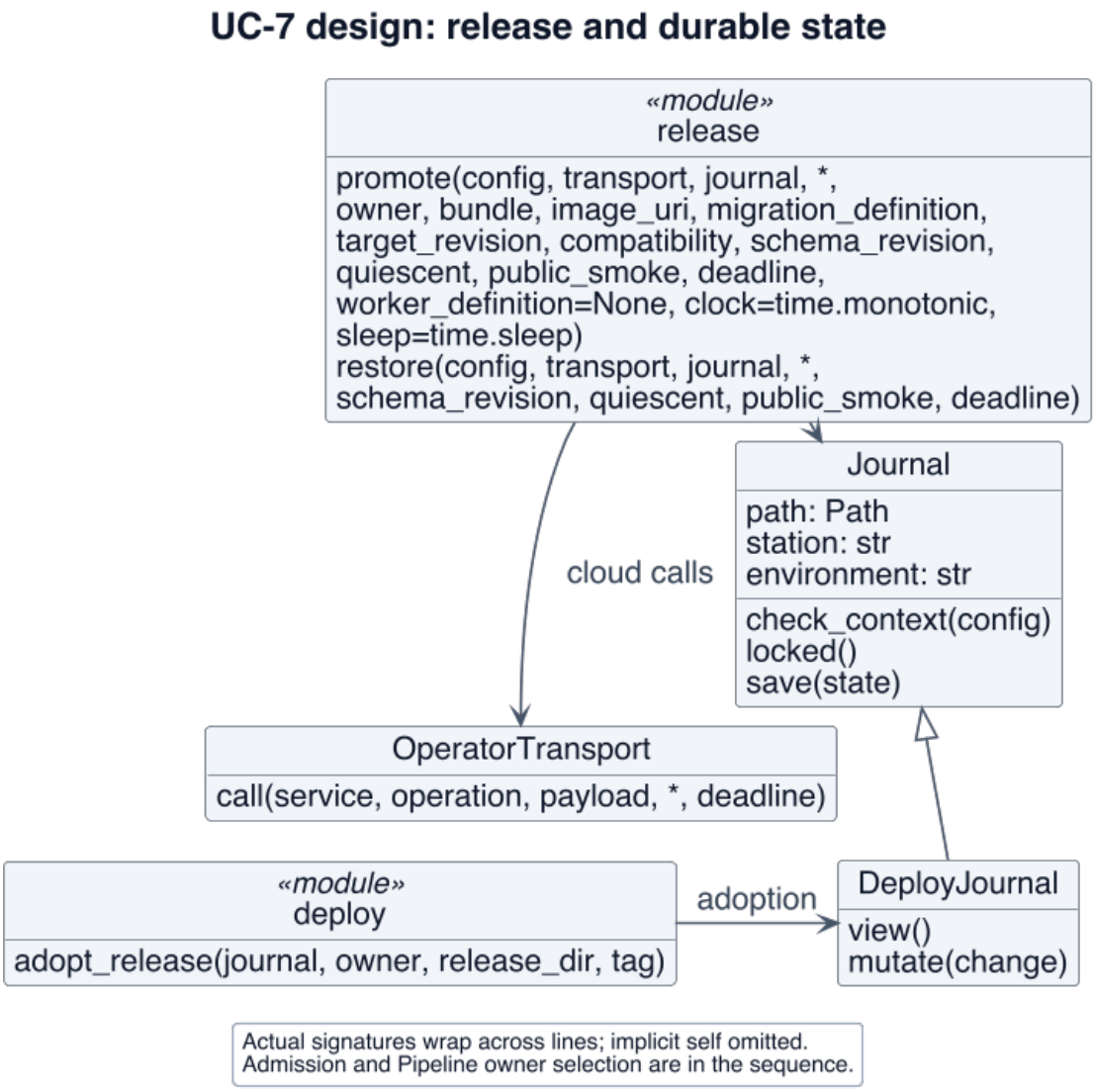


Figure 70. UC-7 actual release, journal, transport and adoption operations.

Design sequence diagrams

Figure 71 shows the principal functions and durable boundaries. The helpers `_version` and `_move` are self-calls of the `release` module; only the AWS operations they issue reach `OperatorTransport`. The Journal records `publication_started` before `publish-version`, then the version receipt and the alias receipt. Automatic admission is tied to the triggering Main push CI run and attempt; manual admission also permits an exact-source full CI dispatch, and PR and scheduled runs are ineligible. CD checks the images it promotes against source and configuration evidence and registry scans (`scripts/ci/cd_gate.py`, `.github/workflows/cd.yml`).

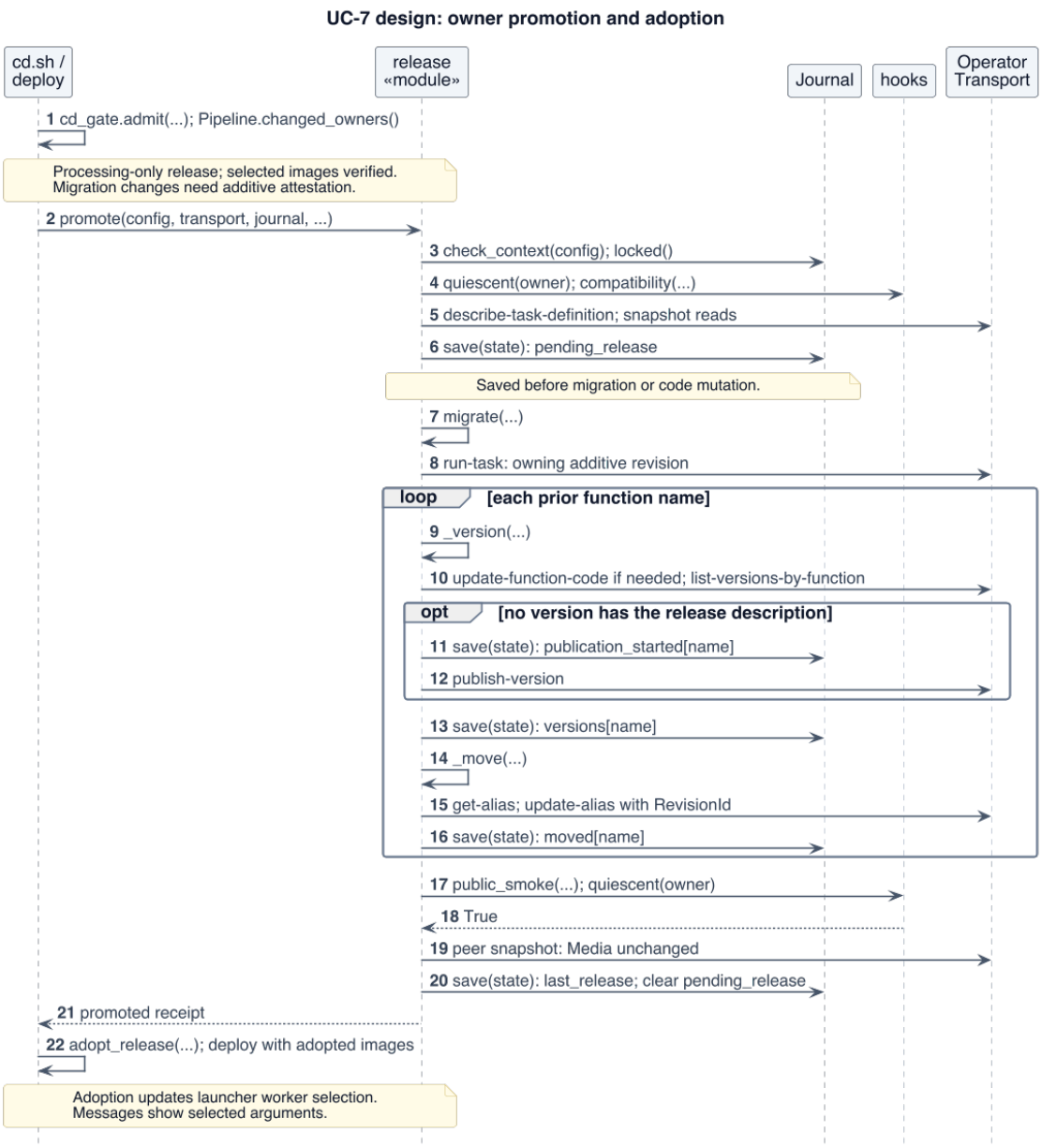


Figure 71. UC-7 implementation: exact-source admission, owner promotion and adoption.

Figure 72 covers restore: `_move` checks alias version and revision, restore refuses concurrent drift and checks the prior function receipt, and intent is retained if restored smoke fails. Figure 73 covers publication uncertainty: `_version` looks for a version carrying the unique release description and never publishes blindly. The later upload-to-playback smoke fails CD on error but has no automatic alias restore.

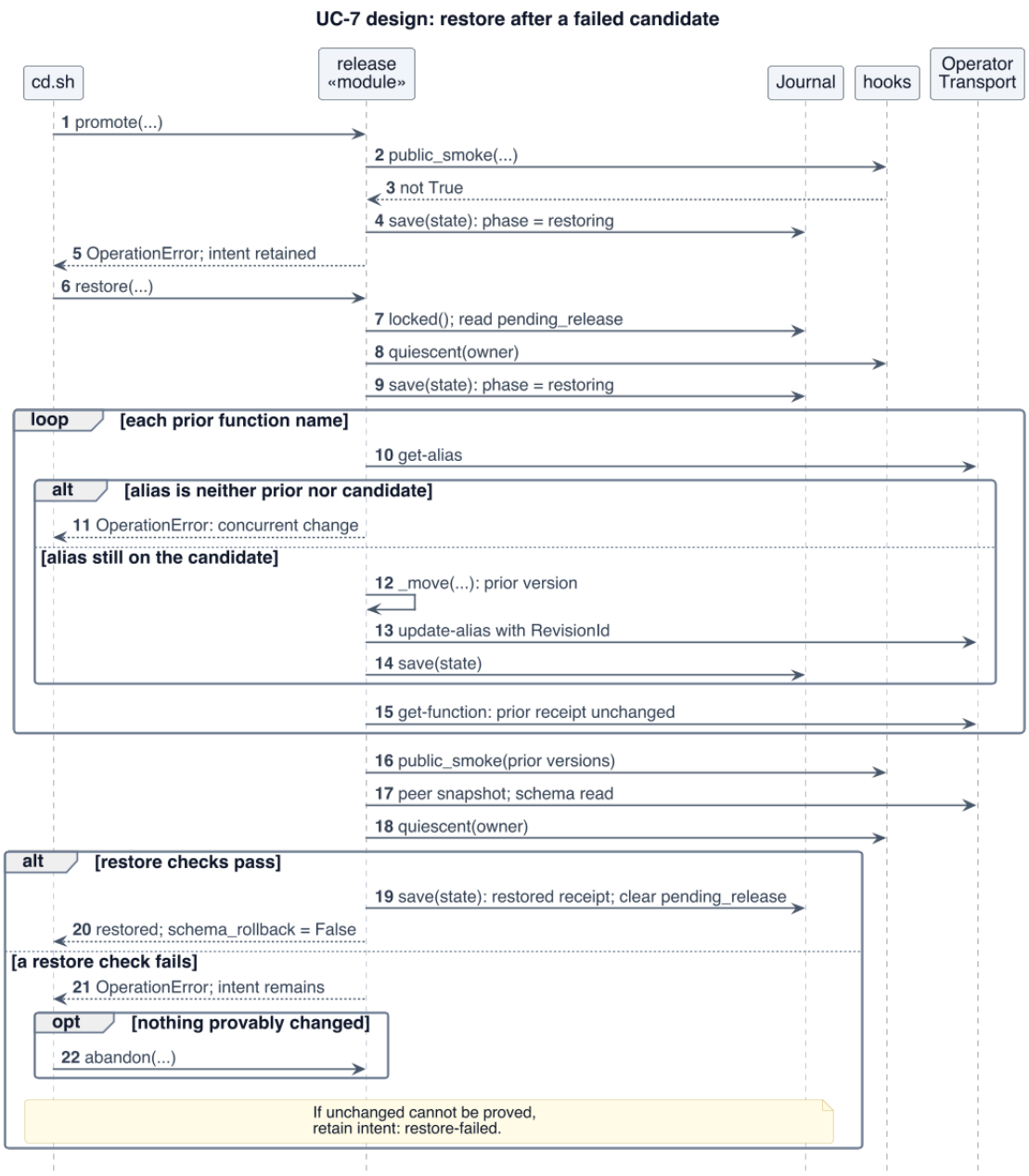


Figure 72. UC-7 implementation: restore checks and recovery outcomes.

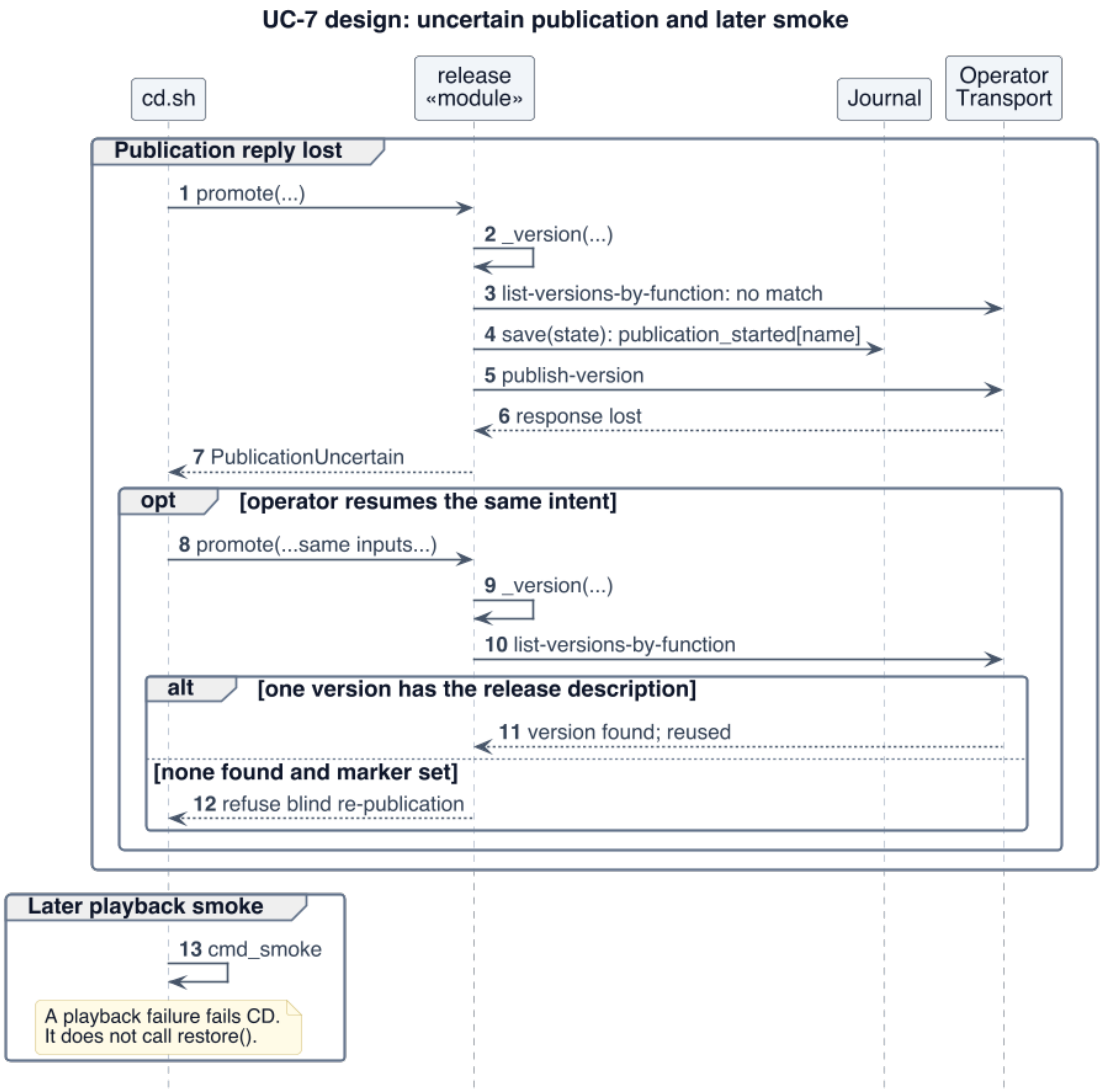


Figure 73. UC-7 implementation: lost publication reply and post-release smoke.

Analysis to design

Analysis responsibility	Design change and reason
Candidate evidence	`cd_gate.validate`, `validate_inputs` and the release plan bind source, image, scans and migration definitions before promotion.
Release coordinator	`release.promote`, `_version` and `_move` introduce durable preparation and revision-checked alias mutation.
Prior state and recovery intent	`snapshot` and `Journal` preserve owner/peer receipts and unresolved operations across interrupted runs.
Recovery and adoption	`restore` checks the prior runtime without schema downgrade; `deploy.adopt_release` updates the persistent baseline after success.

`tests/aws/v1/test_v1_release.py` exercises peer preservation, lost responses, failed-candidate restore, concurrent alias changes and retained recovery intent with deterministic AWS protocol fakes. `test_owners_are_released_only_when_their_images_build_inputs_changed` (`test_persistent_environment.py`) checks owner selection and `test_release_plan.py` checks image evidence and scans. Dated cloud demonstrations are listed in §5.3.

6.5 Design Problems and Patterns

DP-1 to DP-3 follow the ownership allocation in the Project Plan. Each problem has five parts: the problem with class and sequence models, candidate patterns, the rationale for the chosen pattern, solution class and sequence models, and implementation decisions with verification evidence.

6.5.1 DP-1 Bounded cloud capacity and safe per-owner release (Guruprasath Gopal)

The problem

Processing must absorb queued video work without idle encoders and without changing Media capacity. Several wake sources can see the same backlog, so a naive observe-then-launch decision lets each caller spend the same apparent free slots. A lost launch response is worse: retrying as a fresh request can start another billable executor. Figure 74 shows the missing shared capacity authority; Figure 75 illustrates the race. The problem models illustrate the unprotected observe-then-launch alternative.

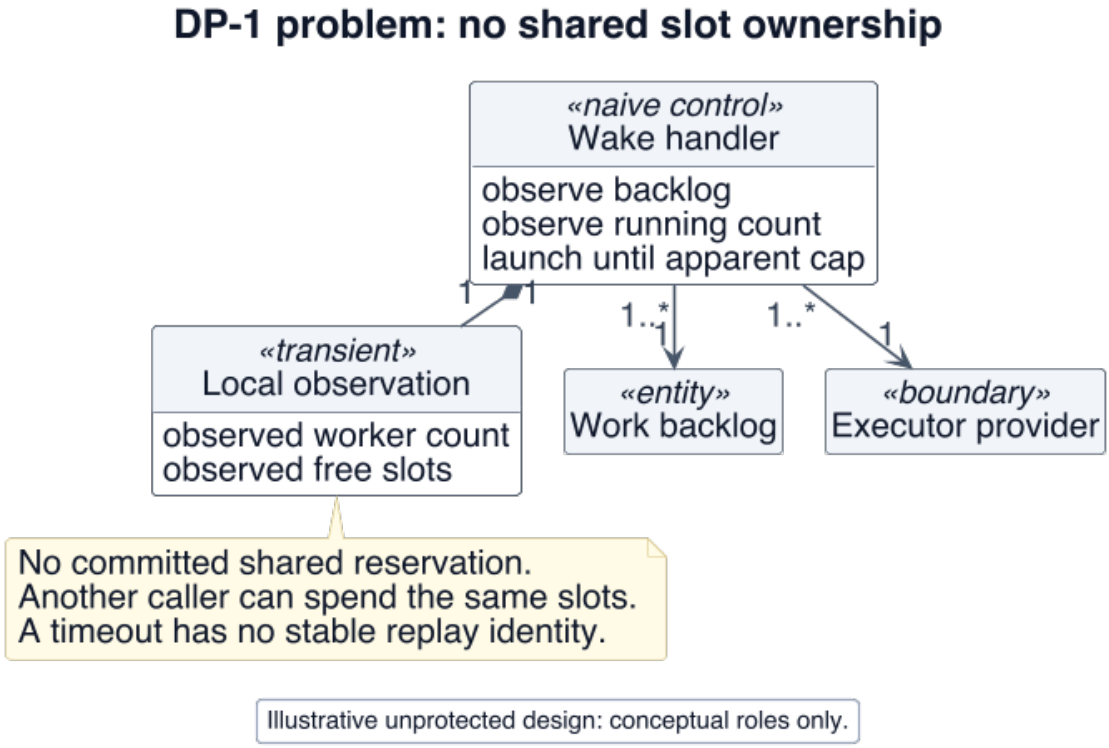


Figure 74. DP-1 problem structure: independent launch decisions without durable reservations.

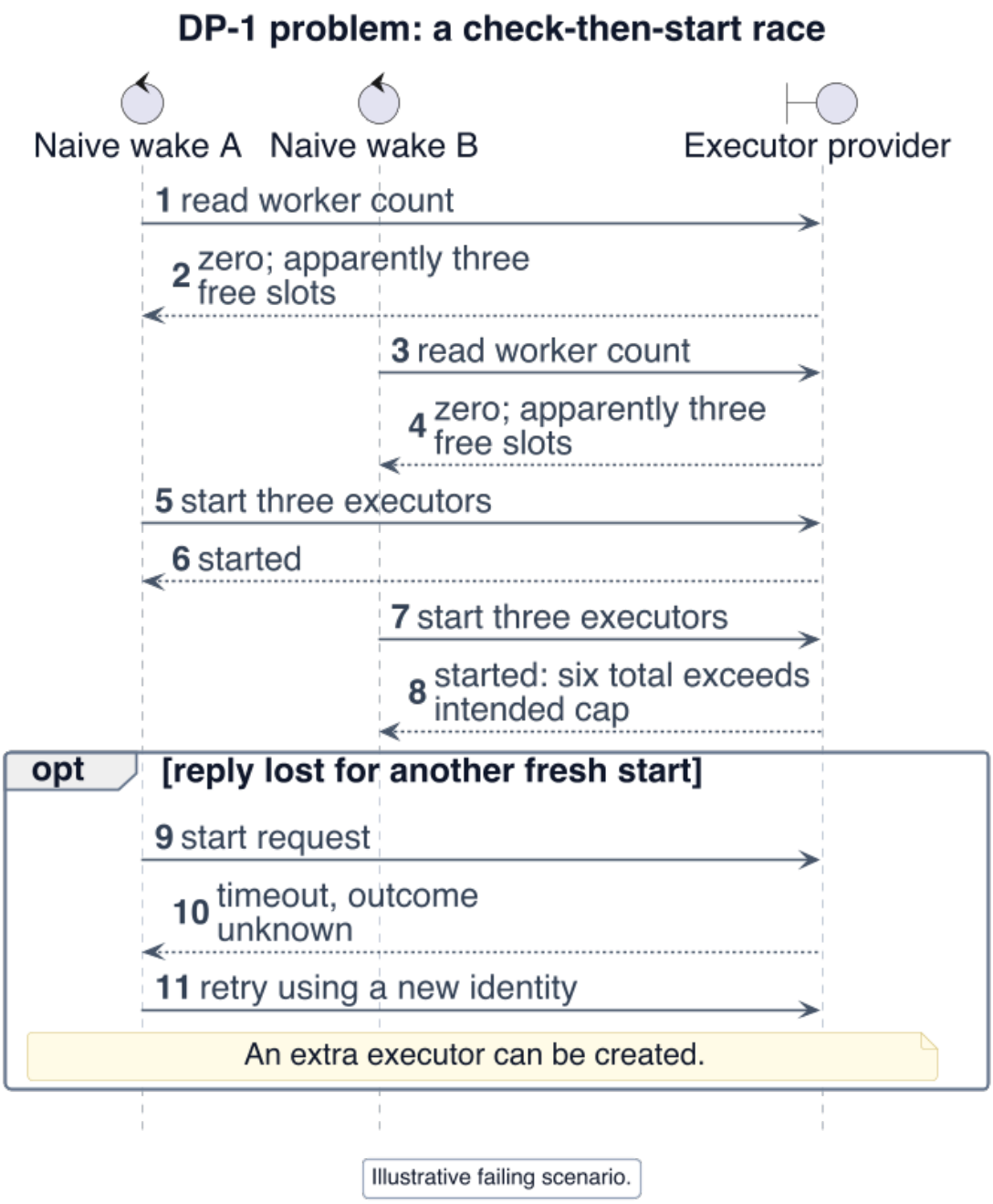


Figure 75. DP-1 failing scenario: concurrent callers reuse the same free capacity.

The forces are bounded spend, asynchronous demand, eventual inventory visibility and recovery after interruption. Release is a related sub-scenario: a Processing change must preserve Media and keep enough prior state to recover a failed candidate (UC-7 holds its models).

Candidate patterns

Capacity candidate	Benefit	Cost or limitation for FlickPond
ECS service autoscaling on queue depth	Managed desired-count adjustment for a continuing consumer service.	Requires a service and scale-in policy in place of the current one-Attempt finite task lifecycle; queue depth alone does not explain uncertain starts.
Fixed always-on worker pool	Predictable warm consumers and simple capacity accounting.	Pays for idle workers and weakens the zero-idle-encoder goal.
Reservation-based launcher with a global cap	Records slot ownership before launch; supports finite tasks and stable replay.	Requires SQL locking, inventory reconciliation and explicit uncertainty handling. **Selected.**

Release candidate	Benefit	Cost or limitation
All-at-once stack update	One deployment operation.	Couples code changes to a wider update and lacks the owner-specific smoke/recovery protocol.
Blue/green	Keeps separate candidate and prior environments for cutover.	Duplicates capacity and requires additional traffic and database coordination.
Per-owner journaled release	Exact-source admission, owner smoke, retained prior receipts and checked restore.	Requires resumable intent and additive schema compatibility. **Selected.**

The comparison evaluates each alternative against finite-task operation, bounded spend and recoverability.

Why the chosen pattern

The selected capacity design combines queue buffering and competing consumers with durable reservations and an idempotent launch request. The reservation is the local authority to spend a slot; inventory explains what happened externally. Serializing reservation transactions closes the race between callers, and persisting the token before the external call makes response loss recoverable without extra capacity. Finite workers return to zero by exiting; cooldown and idle-breaker rules stop endless empty replacements.

Per-owner release applies the same principle to a different uncertainty: save intent before mutation, identify the resulting version and check current state before repair. It is a journaled recovery protocol, not an atomic transaction across cloud resources; separate owner evidence and peer snapshots support independent deployment (`processing/launcher.py`, `scripts/aws/v1/release.py`).

Solution models

Figure 76 labels the implemented capacity-gate, reservation and adapter roles. The singleton/reservation association is logical and has no foreign key. Figure 77 labels the same roles at run time: `LauncherControl` is the serialization root, `LaunchReservation` the durable reservation and `EcsPort` the adapter. The figure shows serialized admission, then reconciliation by token, with every replay passing through the checked `_send`. The corresponding release solution appears in Figures 70, 71 and 72.

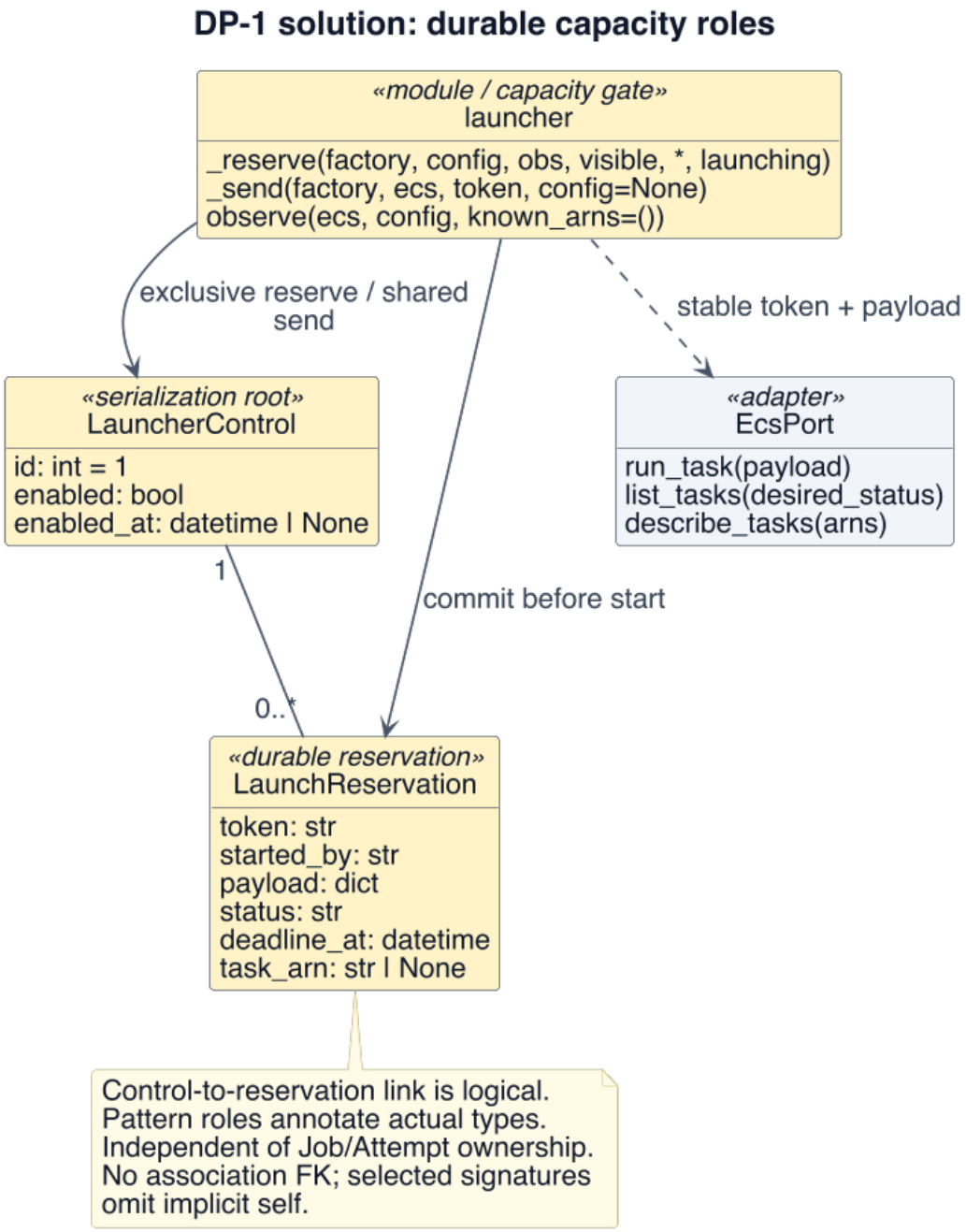


Figure 76. DP-1 solution roles implemented by the launcher and its SQL records.

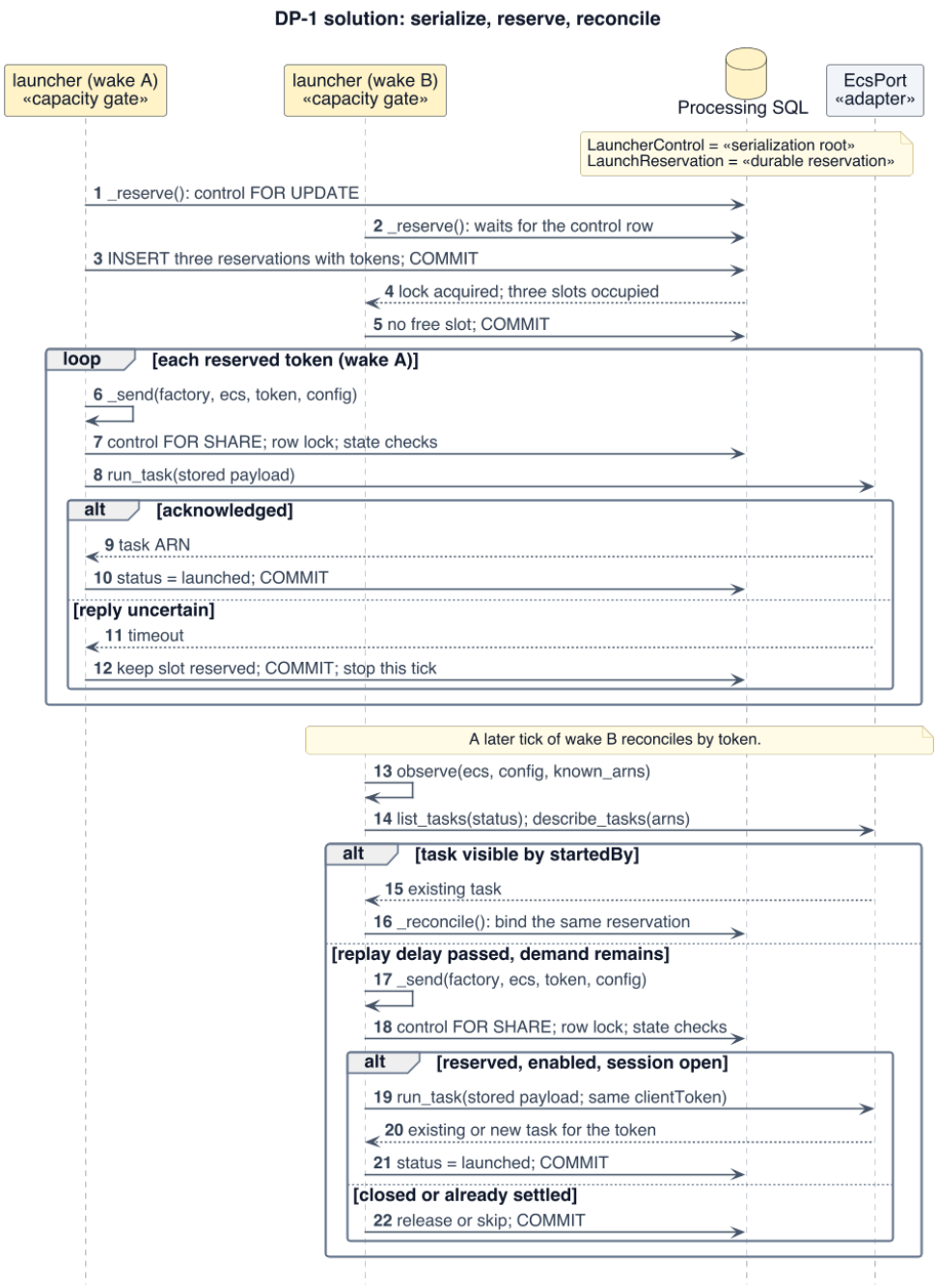


Figure 77. DP-1 solution: reserve once, retain uncertainty and reconcile the same identity.

Implementation decisions

`_reserve` calculates `occupied = live workers + unexplained reservations`, then bounds new reservations by remaining slots and unmet demand; `LauncherConfig.slots` cannot exceed three. Unclaimed live workers and pending starts offset demand, so no executor starts for work that existing capacity can still claim. A shared control lock spans each send, while disable takes an exclusive lock. Always-on configuration removes the session-end gate; deadlines, reservations, the enable flag and idle protection remain (`processing/launcher.py`, `processing/launcher_models.py`).

The launcher-specific SQL tests cover ten queued jobs against cap three, concurrent always-on wakes, lost RunTask replies, identical replay, restarts and disabling during a send

(tests/v1/test_launcher_sql.py). The suite uses real PostgreSQL with fake cloud ports. Infrastructure tests check bounded permissions and operator controls (tests/aws/v1/test_unattended_launch.py).

For release, Pipeline.changed_owners compares image build inputs and cd.sh preserves unchanged owners, and the same principle applies: intent is saved before mutation, and restore never downgrades schema. Candidates and kept-running images require strict scans; the detailed protocol is in UC-7 and §5.3. Release protocol, scan and adoption tests reside in tests/aws/v1/test_v1_release.py, test_release_plan.py and test_persistent_environment.py. The tests verify release selection, image policy and recovery behavior; the recorded cloud demonstrations are listed in §5.3.

6.5.2 DP-2 Durable asynchronous Processing handoff (Ibrahim Mammadov)

The problem

Ibrahim Mammadov is responsible for DP-2's analysis and design. An accepted video must survive temporary unavailability of Processing or the queue without losing its processing request. Media and Processing own separate databases; SQL commit and queue publication cannot share one local atomic transaction. The design must also tolerate duplicate delivery, reordered commands and lost replies without accepting stale execution or displaying older results.

Figure 78 models the direct dual-write candidate. Figure 79 shows its failure window: the Asset commits, then the process crashes before publishing. Sending first only moves the window, because Processing could receive work for an acceptance that later rolls back. The proposal's synchronous peer SubmitJob/Redis-RQ arrangement is the recorded earlier alternative; the current code uses SQL ledgers and SQS (docs/architecture/decisions.md, AD-11).

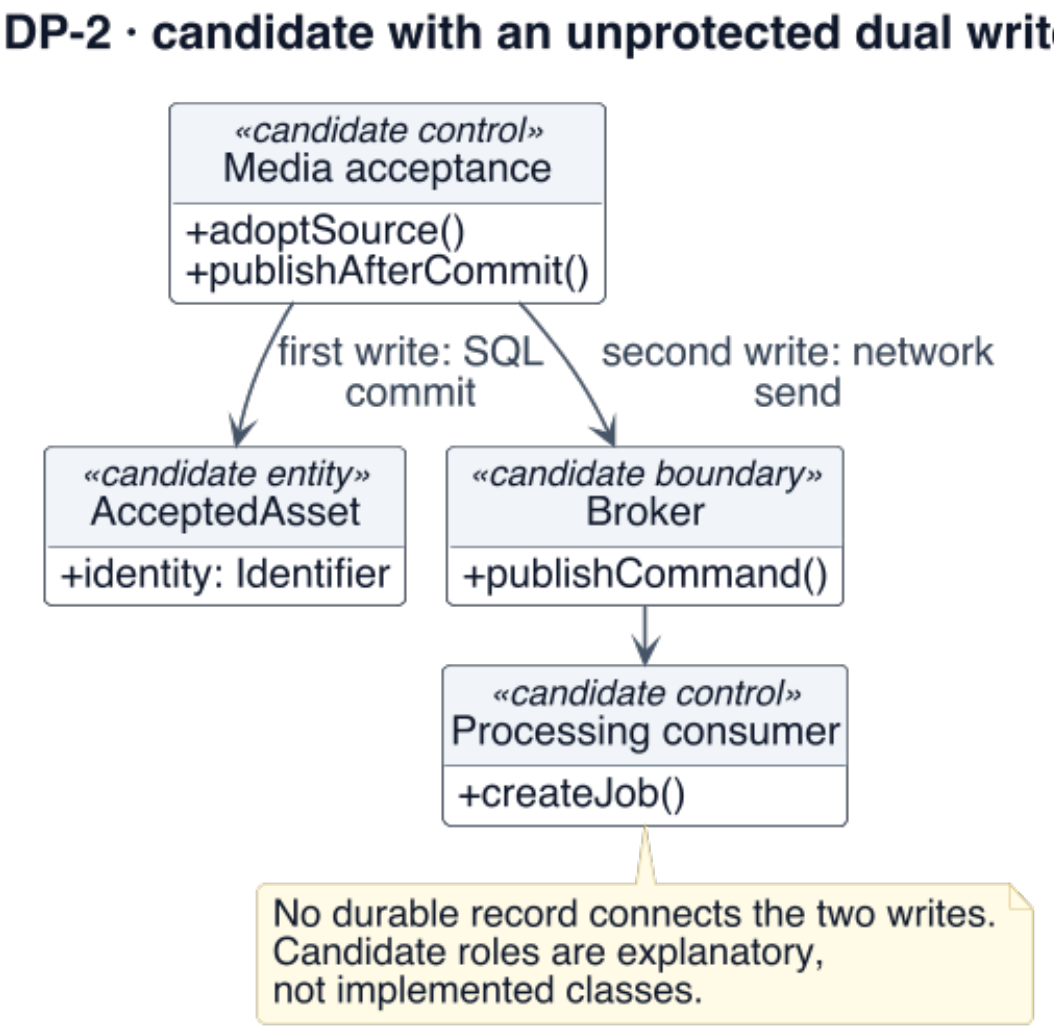


Figure 78. DP-2 candidate structure with an unprotected SQL/broker dual write.

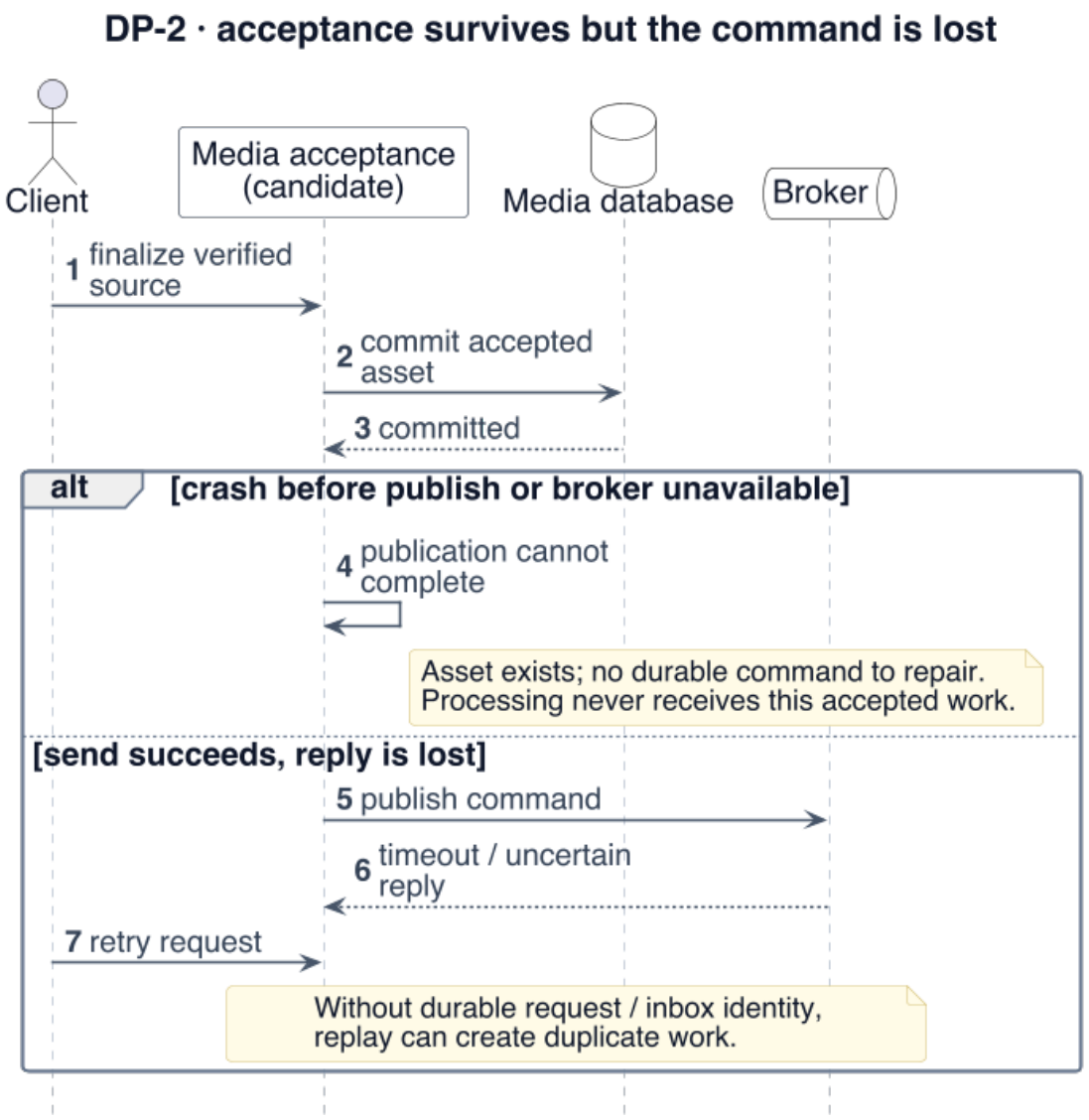


Figure 79. DP-2 lost publication and ambiguous-send failure windows.

Candidate patterns

Candidate	Benefit	Main consequence
Transactional outbox + idempotent inbox	Owned state and message intent commit locally; consumers deduplicate in their mutation transaction.	Requires a relay, repair, retained ledgers and explicit ordering fences; transport remains at least once.
Direct dual write: publish after commit	Small API path and immediate publication when dependencies are healthy.	Crash or broker outage after commit loses intent; uncertain sends require durable reconciliation and deduplication.
Synchronous peer HTTP with request identity	Immediate peer response and familiar request handling.	Couples acceptance to peer availability; a timeout still leaves an ambiguous outcome and requires durable retry/reconciliation.

Why the chosen pattern

The outbox/inbox pair keeps each commit local and reuses the SQS transport, so Media accepts work whether or not Processing is available. An inbox alone cannot recover an unsent command, and an outbox alone cannot stop repeated consumer mutations; together they

cover both failure boundaries, while guard, version and lease checks handle domain ordering. The trade-off is eventual status propagation and explicit repair instead of a synchronous result.

Solution models

Figure 80 labels the implemented pattern roles. `delivery.sql` supplies the shared algorithms («relay»), and each context owns independent «outbox» and «idempotent inbox» ledgers that commit with its local state. `Envelope` and `Source` in `contracts/v1.py` carry message identities and source values; `Actor` represents trusted HTTP request identity and is not part of the queue envelope. IAM and queue policies establish message-producer authority.

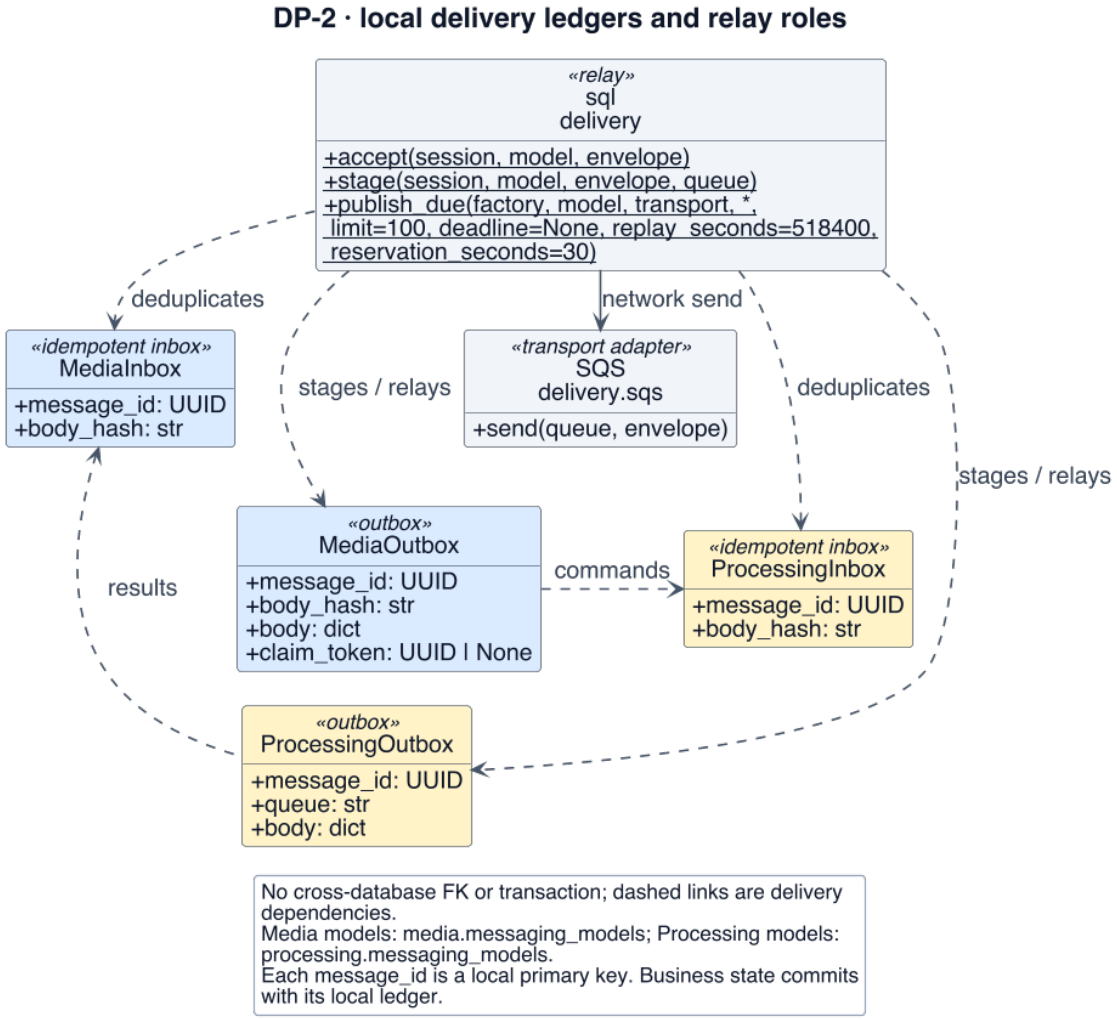


Figure 80. DP-2 owner-local outboxes, idempotent inboxes and relay adapter.

Figure 81 follows one accepted command through a lost send reply, with SQS as an explicit participant. The relay reserves a due row under `SKIP LOCKED`, commits its claim, and sends to the queue outside SQL locks. A second transaction records success or retry only if the claim token still matches, so a lost reply causes another send of the same identity. On delivery, Processing's inbox, guarded Job/Attempt and follow-on outboxes commit together; the result path repeats the pattern for Media's projection.

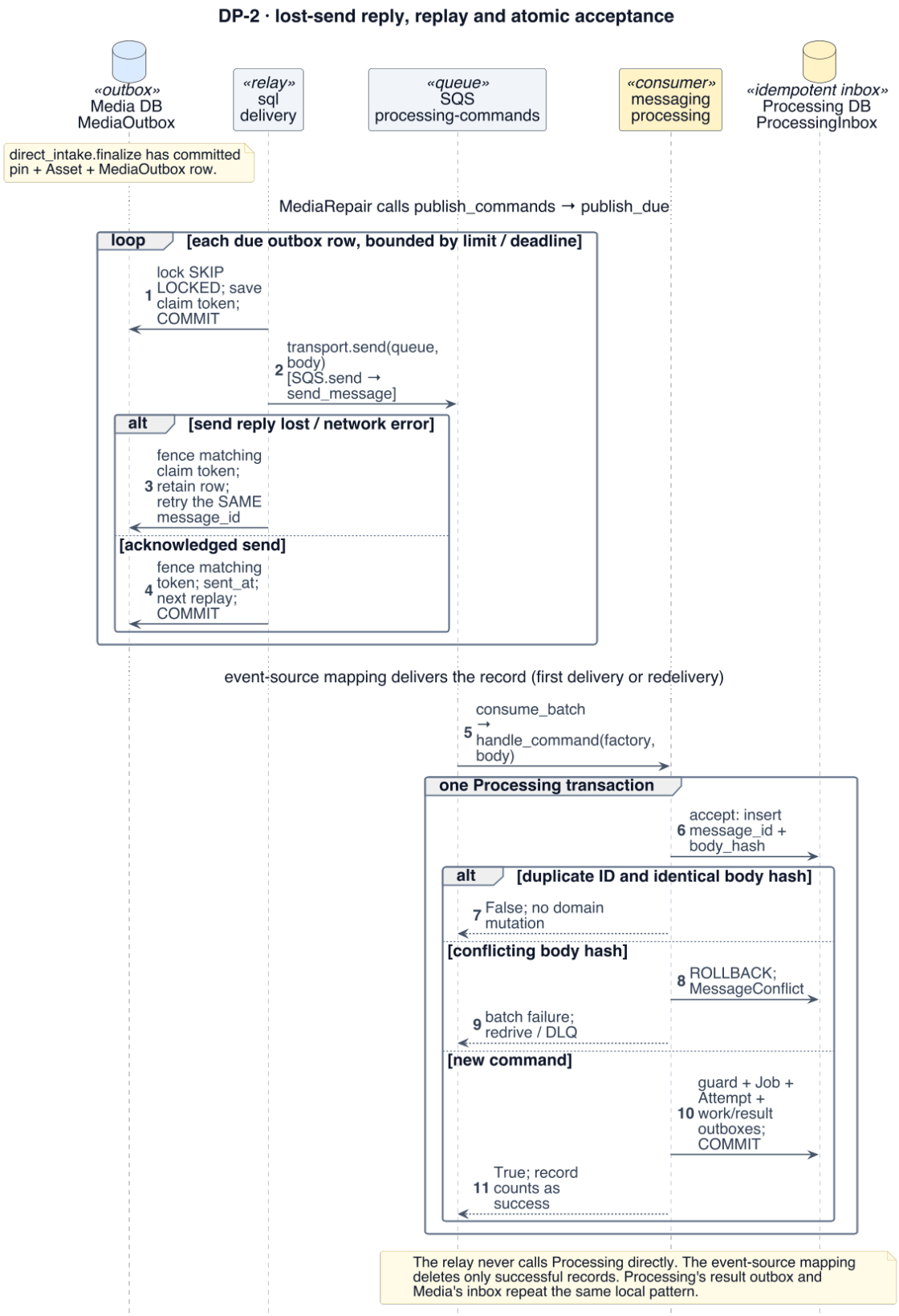


Figure 81. DP-2 committed intent, fenced relay and duplicate-safe consumption.

Implementation decisions

`delivery.sql.stage` never commits implicitly. Direct finalize calls `media.messaging.enqueue_submit` inside adoption, and edit calls `stage_submit` inside its source-adoption transaction. `MediaRepair` calls `publish_commands` later, so the invariant is commit-before-send, even when repair runs before the HTTP response reaches the client (`media/handlers.py`, `media/services/direct_intake.py`, `media/services/v1_lifecycle.py`).

`delivery.sql.accept` keys deduplication by message UUID and canonical body hash: an identical replay returns false and changed content raises `MessageConflict`. `handle_command` commits inbox and owned changes before returning. `delivery.sqs.consume_batch` records callback exceptions as partial failures and the Lambda mapping deletes successful records. `handlers.invoke` attempts best-effort publication after consumption, while repair covers unavailable sends. Encoders stage result intent in SQL, and the Processing relay publishes it.

`processing.recovery_v1.repair_work` rearms transport intent for queued current attempts. Execution expiry uses `recover_expired`/`interrupt`, which abandons the Attempt, fails the Job and stages cleanup and result intent; a user retry creates a new Attempt. These mechanisms complement inbox deduplication, and transport remains at least once.

The SQL assertions in `tests/v1/test_messaging_sql.py::test_lost_send_response_replays_after_commit_outside_locks` acquire the rows with `NOWAIT` during a simulated send and verify stable replay identities. `test_submit_duplicate_conflict_and_rollback` checks atomic acceptance and conflict rollback; `test_success_output_namespace_validation_and_atomic_terminal_result` checks terminal intent. `tests/media/test_v1_result_delivery.py::test_projection_commits_before_successful_batch_ack` checks projection/ACK ordering. These PostgreSQL tests verify atomic acceptance, replay identity and acknowledgement ordering.

6.5.3 DP-3 Trusted owner identity and reusable public contract (Nguyễn Kim Long)

The problem

Browser and Publisher requests must share an owner decision across separately packaged services. A service credential authenticates a forwarding service but cannot establish where a caller-controlled `X-Owner-Id` came from. Distributing the JWT signing secret lets every verifier mint sessions and couples each service to account rules.

Figures 82 and 83 model the forwarded-header candidate and its failure when the forwarder copies an unverified identity; it is a rejected design, not the current runtime. The forces are owner isolation, current account decisions, independent service releases and a public interface usable without domain imports.

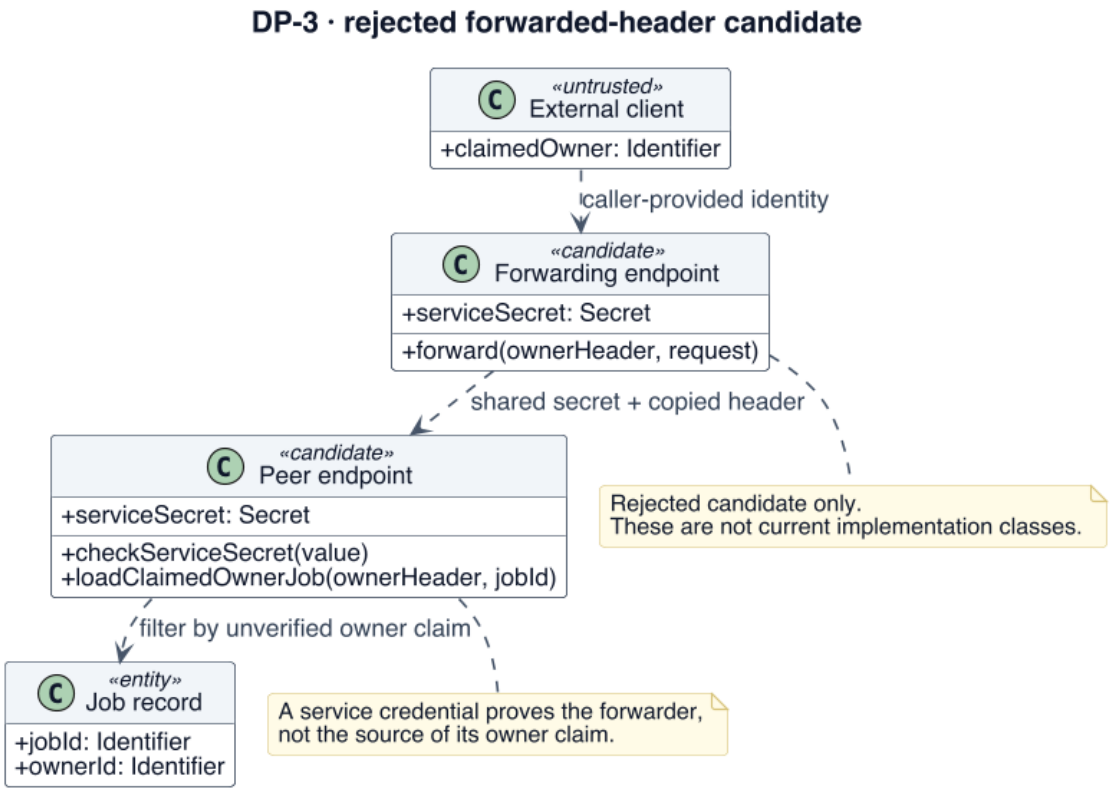


Figure 82. DP-3 rejected service-secret/forwarded-identity structure.

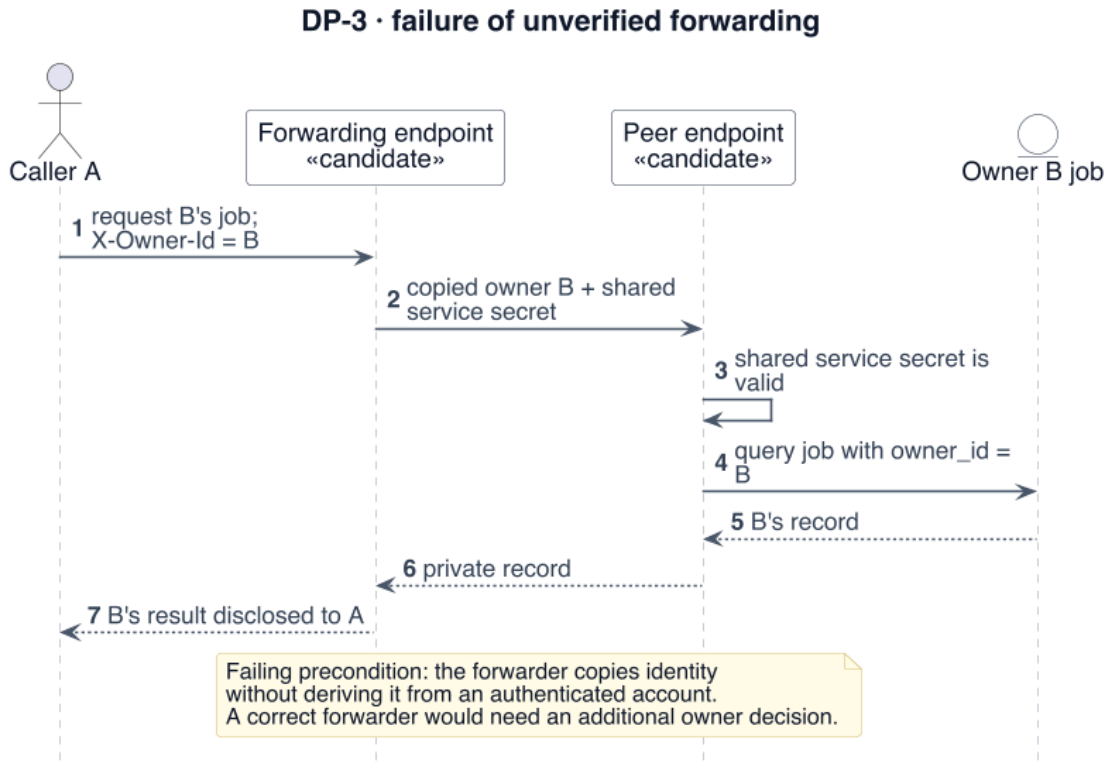


Figure 83. DP-3 failure when a trusted forwarder copies untrusted owner identity.

Candidate patterns

Candidate	Benefit	Trade-off
Forwarded identity header plus shared service secret	Simple private HTTP integration	Requires the forwarder to derive identity correctly; a shared secret alone cannot validate a copied owner claim.
Shared JWT secret, verified by every service	Each service verifies sessions locally	Distributes signing authority; current user/role checks duplicate account access or need another dependency.
Gateway Lambda REQUEST authorizer, trusted Actor and per-service owner filtering	Centralizes session validation while each service enforces resource ownership	Adds a synchronous Media identity dependency and requires restricted backend invocation. **Chosen.**

Why the chosen pattern

The chosen boundary combines an authorization gateway, an event-to-application adapter and a small shared contract. Media keeps session and account authority: API Gateway invokes the **Media-owned Lambda REQUEST authorizer**, then invokes the selected API separately with its trusted Actor context, and each API filters its own resources by owner. The trade-off is one synchronous Media dependency for protected routes, with no extra business service (`media/authorizer.py`, `scripts/aws/v1/definitions.py`; the sequence is in Figure 85).

Solution class and sequence diagrams

Figure 84 labels the implementation roles. Each API instantiates `GatewayAdapter`, which strips public owner and role headers, rebuilds cookies from the Gateway event, validates `Actor` and injects request state. The integration marker is a schema fence; Lambda resource policies that restrict invocation to the configured Gateway establish the trust boundary.

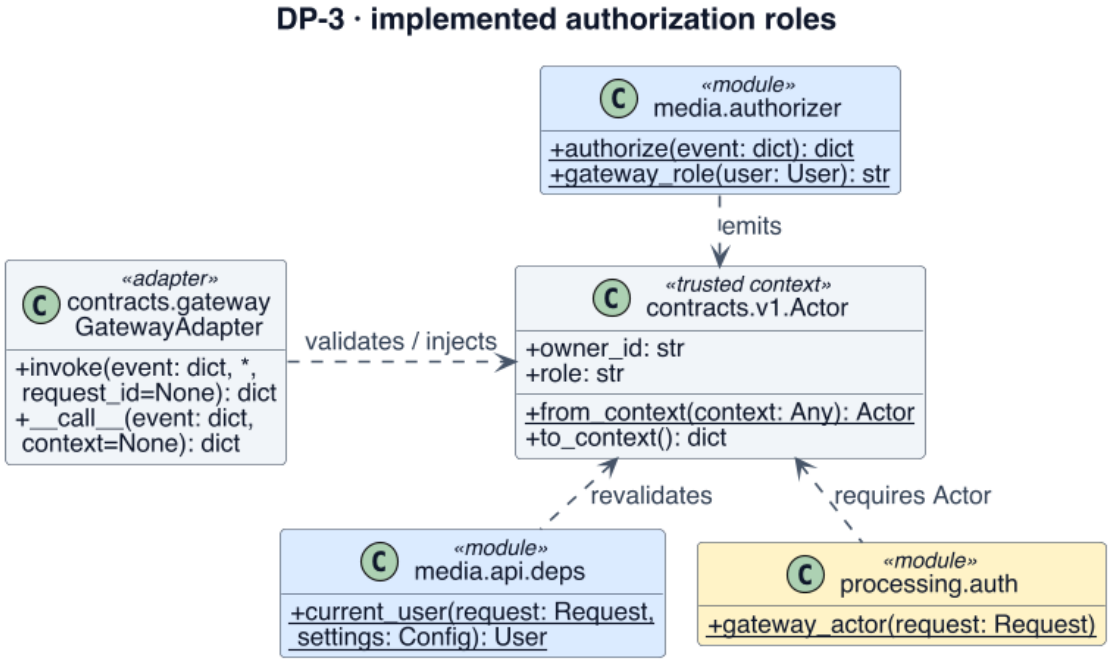


Figure 84. DP-3 actual authorization, adapter and local enforcement roles.

Figure 85 shows the separate invocations. The authorizer verifies HS256 claims and current `User`, returning role `publisher` for current user/operator accounts. Media revalidates its cookie and current User and compares them to Actor. Processing accepts adapter-injected Actor and scopes queries to `owner_id` (`media/api/deps.py`, `processing/auth.py`, `processing/v1_api.py`).

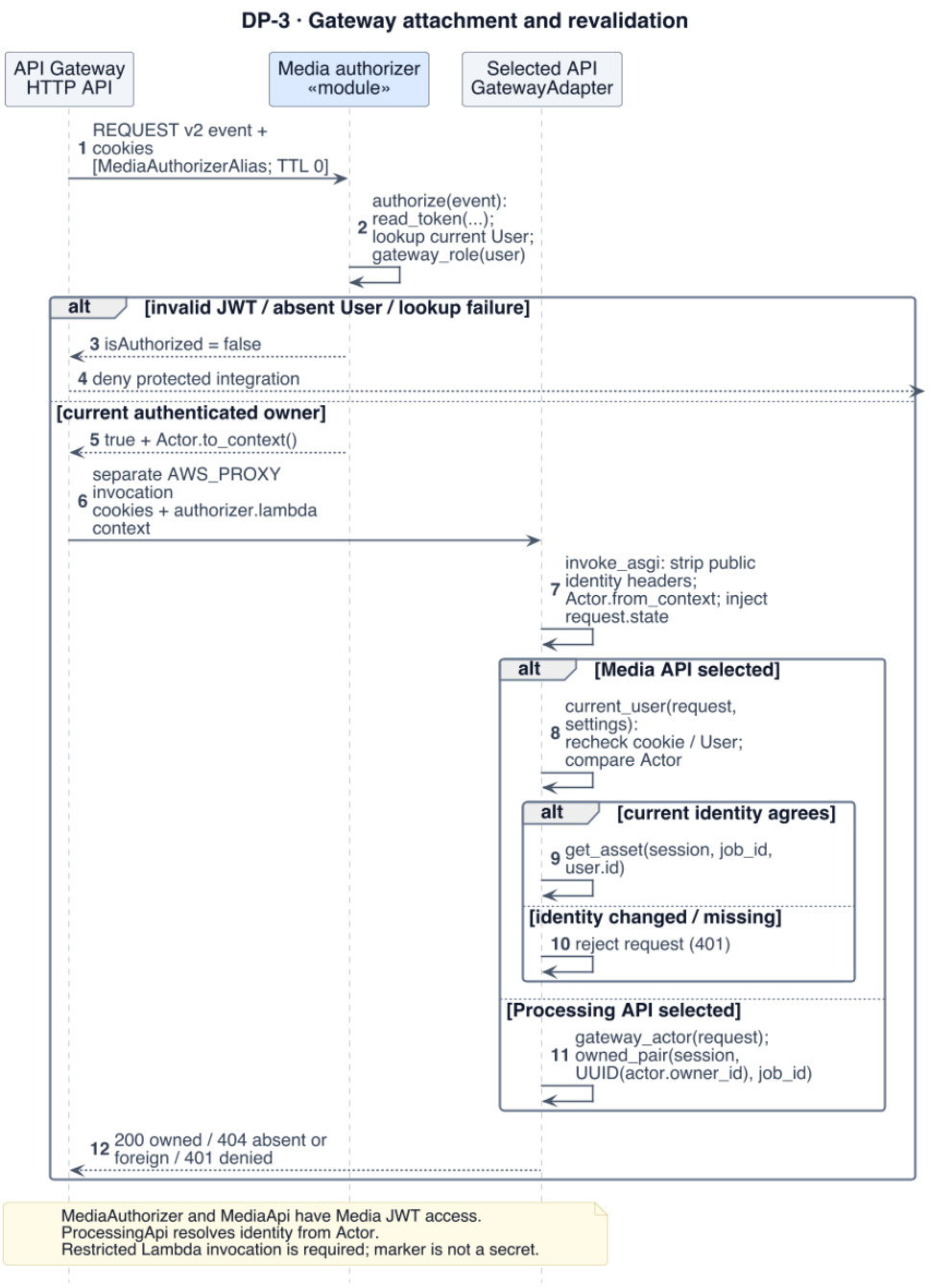


Figure 85. DP-3 Gateway attachment, current-user validation and service-local ownership checks.

Implementation decisions and verification

`GatewayAuthorizer` uses payload v2, a Cookie identity source and cache TTL zero, and targets `MediaAuthorizerAlias`; public health and Media register/login/logout routes bypass it. Processing's handler needs neither Media's JWT secret nor its User table.

The reusable contract comprises `contracts/` and the OpenAPI HTTP document (Figure 86). `contracts/v1.py` defines `Actor`, `Source` and `Envelope`; `contracts/gateway.py` provides the common adapter. Publisher and `frontend/app.js` consume the HTTP interface described by `docs/api/media.openapi.json`, including reservation/finalize, status and optional playback links. They do not import the Python contract package. `contracts/README.md` specifies additive

changes and staged consumer-before-producer version upgrades; only version 1 is currently implemented.

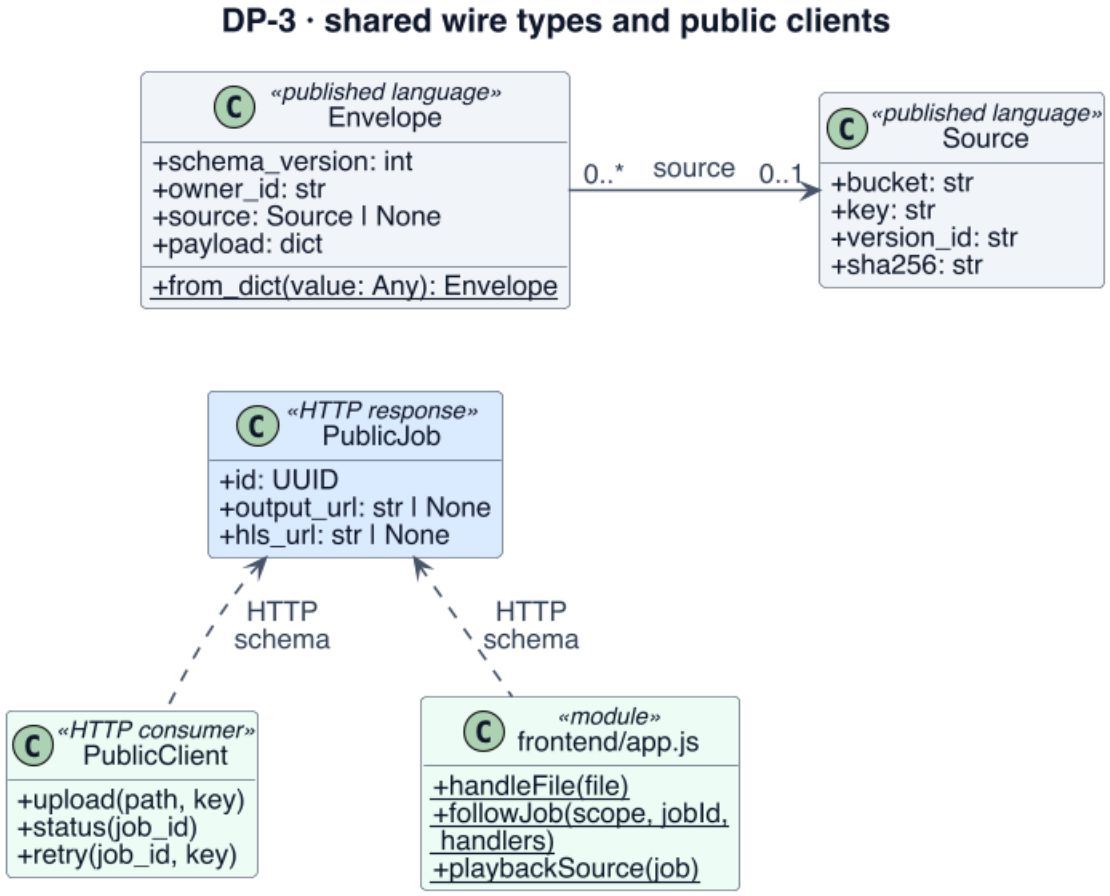


Figure 86. DP-3 shared wire types and independent consumers of the OpenAPI HTTP contract.

`tests/media/test_lambda_boundary.py` checks current roles, removed users and repeated decisions. `tests/v1/test_contract_fixtures.py` checks Media revalidation, owner-scoped Processing access, wire shapes and negative contexts. `tests/v1/test_local_gateway.py` checks forged-header stripping and anonymous denial. Publisher's import-boundary, direct-upload and schema tests support independent consumption; `test_public_schema_conformance_and_explicit_ca` specifically validates the proposal contract in `project-roadmap/design/05-contracts/public.openapi.json`. These local tests verify current-user checks, trusted context, owner filtering and independent client consumption.

6.6 Member Contributions and Lessons Learned

6.6.1 What Team 17 Contributed beyond the Inherited Baseline

Team 17 has three members and was formed with Prof Boon Kui Heng's permission. FFmpeg processing, HLS output, worker replicas and basic CI came from Project 1; the comparison table under Background in the Introduction separates them from this project's work. Team 17 built the platform around those capabilities: two bounded contexts with owned data, durable asynchronous integration, an independent Publisher client, bounded cloud execution on Lambda and finite Fargate tasks, per-owner delivery and restore, and retained tests and receipts that tie each design claim to a dated result.

6.6.2 Per-Member Contribution Summary

Each member led one specialism, is responsible for the analysis and design of the use cases and design problem named below, and reviewed the other two members' work. Work packages follow the proposal and the team-agreed specialisms in `project-documentation/effort-ledger.md`; effort per member is reported in Project Metrics.

Guruprasath Gopal

Guruprasath led cloud and delivery (UC-6, UC-7, DP-1). This work took the reproducible baseline through AWS deployment, CI quality gates, per-owner releases and release freeze, connecting the proposal's independent-capacity and repeatable-release goals to the deployed runtime and its operator tools.

- Led work packages 1 and 7: baseline reproducibility, the inherited scan repair, CloudFormation, Lambda, Fargate, Aurora, S3, CloudFront, image scanning and the deploy/release/restore workflow.
- Led delivery and Aurora-pause repairs during integration, coordinated the release freeze and its recorded image/runtime identities, and approves promotion to Main.
- Developed the capacity and release analysis and design for UC-6, UC-7 and DP-1.
- Contributed to the presentation and final report, with cloud/delivery review of the other specialisms.

Ibrahim Mammadov

Ibrahim led backend services (UC-1, UC-2, UC-4, DP-2): extracting Processing's execution model and persistence, adapting Media's owned-asset workflow and enforcing security at the service and storage boundaries. This turned the proposal's service split into durable acceptance, execution and retry behavior.

- Led work packages 3 and 4: Processing's database, migrations, worker and recovery; Media's asset records, integration, web routes and retry flow.
- Led the boundary-security implementation recorded in the effort ledger: owner checks, database-role separation, presigned storage access and negative tests.
- Developed the analysis and design of UC-1, UC-2, UC-4 and DP-2.
- Contributed backend integration, design explanations and verification evidence to the presentation and final report.

Nguyễn Kim Long

Long led domain analysis, contracts and verification (UC-3, UC-5, DP-3): defining the service vocabulary and public interfaces, building the independent Publisher client and organizing the experiments that connect the proposal's goals to observed outcomes.

- Led work packages 2 and 6: DDD, service contracts, OpenAPI documents, Publisher and its contract tests.
- Led work package 8: scaling, load, recovery and playback experiments and their raw results; ran the queue-drain re-test during integration.
- Developed the analysis and design of UC-3, UC-5 and DP-3.
- Maintained the evidence index and release-freeze documentation, and contributed to the presentation and final report.

6.6.3 Lessons Learned

1. **Commit intent before external effects.** Adopting the outbox/inbox early made the SQL-to-SQS boundary explicit. Treat response loss, duplicate delivery and repair as normal protocol paths.
2. **Measure startup separately from execution.** Scale-to-zero reduces idle compute but adds database resume, image initialization and task-start latency. Separate these stages before choosing an encoding lane or changing capacity.
3. **Verify idle behavior.** Activity markers and cleanup backoff allow Aurora to pause without routine repair waking it unnecessarily. Returning to zero is an observable requirement.
4. **Test delivery as application logic.** Durable intent, immutable identities, additive migrations and focused smoke checks make interrupted releases recoverable. A separate upload-to-playback smoke verifies the completed workflow after owner readiness.
5. **Retain evidence through change.** Rescan candidates and rollback targets, bind reports to their source and scope, and keep original acceptance targets so later fixes remain comparable.

6.6.4 Evidence and How to Reproduce It

The repository retains the inputs and receipts behind the report. Local runs exercise code and emulated cloud ports; dated AWS receipts record the cloud scenarios. The deployment endpoints below identify the demonstration environment; its planned teardown is after 16 November 2026.

Evidence	Location or reproduction command
Architecture decisions, domain model, views and use-case catalogue	`docs/architecture/decisions.md`, `docs/architecture/ddd.md`, `docs/architecture/views.md`, `docs/architecture/use-case-design.md`
Proposal and work-package allocation	`project-documentation/Team17-Proposal.docx`; `project-documentation/effort-ledger.md`
Results against original targets	`docs/results/actual-vs-target.md`; regenerate the summary with `python scripts/dev/results_summary.py`
Scenario-by-scenario evidence index	`docs/evidence-index.md`
Sanitized cloud receipts	`docs/execution/aws/v1/evidence/`, `docs/results/engineering-20261020/`, `docs/results/engineering-20261021/`
Release identity and engineering handoff	`docs/release/demo-release-manifest.md`, `docs/release/engineering-handoff.md`
Deployment and DLQ procedures	`docs/execution/aws/v1/deploy-runbook.md`, `docs/execution/aws/v1/dlq-runbook.md`
Public study guide	https://collaboration95.github.io/dc2/
Local demonstration	`make setup-v1`, then `make v1-up V1_RUN_ID=demo1`, then `make v1-demo V1_RUN_ID=demo1 OPEN=1`
Static checks and fast tests	`make lint`, `make format-check`, `make test`; frontend: `cd frontend && npm ci && npm test`
Real-dependency V1 flows	With the local namespace running: `make verify-v1 V1_RUN_ID=demo1 FLOW=all`
Unit-test inventory and security scanners	Extensibility and Maintainability (test inventory); Continuous Integration (scanner inventory)
Demonstration deployment addresses	Frontend: `https://d1e1dpefd6hp9x.cloudfront.net`; API: `https://nqxf2hclhi.execute-api.ap-southeast-1.amazonaws.com`
Report sources and build instructions	`project-documentation/final-report/src/`; `project-documentation/final-report/README.md`

6.6.5 AI Tool Declaration

The team used AI coding and writing assistants: Anthropic Claude models through Claude Code, and OpenAI GPT models for independent review. They supported code and test generation, refactoring, pull-request review, documentation drafting and proofreading, diagram production, and execution and recording of verification steps under the team's direction.

Engineering results in this report come from retained tests, CI artifacts and cloud receipts. The team reviewed the outputs and is responsible for the content and quality of the submitted work.

7. Appendices

7.1 Glossary

Term	Meaning
ACU	Aurora capacity unit; each Serverless v2 instance is configured for 0–2 ACU here.
ADR / AD-nn	Architecture decision record and identifier. `docs/architecture/decisions.md` retains AD-01–AD-29; Physical Architecture & Design adds the report analyses AD-30 and AD-31.
AISS-nn	Architectural issue identifier in Architectural Limitation.
Asset	Media-owned accepted video, identified by the public Media `job_id`; distinct from Processing's internal Job UUID.
Attempt	One numbered execution of a Job; a retry creates a new Attempt identity.
Bounded context	A boundary within which a domain vocabulary, model and rules apply: Media Management or Processing.
CD / CI	Continuous delivery / continuous integration.
`dc2-live`	The persistent V2-live demonstration environment's AWS stack name.
DLQ	Dead-letter queue; configured redrive follows five receives.
Inbox / outbox	Owner-local tables that deduplicate received messages / stage outgoing intent in the same transaction as domain changes.
Lease	A token and time-limited execution authority for the current Attempt, checked against the database clock.
OIDC	OpenID Connect; GitHub Actions uses it to assume an AWS role without a stored cloud key.
Projection	Media's local Processing read model, checked against source, identity, attempt and `state_version`.
Release (per owner)	Promotion of one service's immutable image through checked alias changes, owner smoke and restore; deployment adoption records the resulting worker definition.
SAST / DAST	Static application security testing / dynamic application security testing.
SCA	Software composition analysis: dependency vulnerability checks.
Seed / producer / consumer	The platform's owned video and Job / the submitting client or owner / the owner using the processed result.
V1 / V2-live	Historical per-run, booked-session proof stacks / the persistent environment introduced on 18 October (AD-27).
VersionId	The S3 object-version identifier pinned as part of an accepted source descriptor.

7.2 Use-Case Traceability Matrix

Section 6.2 defines the seven goals, and section 6.4 holds their descriptions, flows and models. The matrix gives the route from each owner to code and verification; test names are selected assertions, and the named suites hold the remaining cases.

UC	Analysis/design owner	Model section	Key code modules	Key tests
UC-1	Ibrahim Mammadov	§6.4 UC-1	`media/services/direct_intake.py`, `media/services/storage.py`, `media/messaging.py`	`tests/media/test_direct_intake.py::test_finalize_atomic_adoption_and_replay`; `tests/v1/test_direct_intake_s3.py::test_finalize_pins_the_exact_version_it_verified`
UC-2	Ibrahim Mammadov	§6.4 UC-2	`processing/messaging.py`, `processing/lifecycle.py`, `processing/worker/tasks.py`, `media_engine/`	`tests/v1/test_messaging_sql.py::test_success_output_namespace_validation_and_atomic_terminal_result`; `tests/processing/test_worker.py::test_hls_failure_retains_primary`
UC-3	Nguyễn Kim Long	§6.4 UC-3	`media/projections.py`, `media/repositories/assets.py`, `media/api/v1_hls.py`	`tests/media/test_v1_projection.py::test_wrong_owner_source_job_and_output_rejected`; `test_local_status_hls_and_signed_range_capability` in the same suite
UC-4	Ibrahim Mammadov	§6.4 UC-4	`media/services/v1_lifecycle.py`, `processing/messaging.py`, `processing/lifecycle.py`	`tests/media/test_v1_projection.py::test_retry_and_cancel_are_commands`; `tests/v1/test_messaging_sql.py::test_retry_idempotency_and_cancel_fence_old_and_current_attempts`
UC-5	Nguyễn Kim Long	§6.4 UC-5	`publisher/cli.py`, `publisher/client.py`, `publisher/state.py`, `docs/api/media.openapi.json`	`tests/publisher/test_publisher.py::test_cli_mixed_batch_keeps_every_file_outcome`; `test_module_entry_and_import_boundary` in the same suite
UC-6	Guruprasath Gopal	§6.4 UC-6	`processing/launcher.py`, `processing/launcher_models.py`, `processing/launcher_trigger.py`	`tests/v1/test_launcher_sql.py::test_always_on_concurrent_wakes_never_exceed_the_cap_without_a_session_end`; `test_a_lost_runtask_response_is_resolved_by_token_without_a_second_launch` in the same suite
UC-7	Guruprasath Gopal	§6.4 UC-7	`scripts/aws/v1/release.py`, `scripts/aws/v1/deploy.py`, `scripts/aws/journal.py`, `scripts/ci/cd.sh`	`tests/aws/v1/test_v1_release.py::test_failed_candidate_restores_prior_aliases_without_schema_downgrade`; `tests/aws/v1/test_persistent_environment.py::test_owners_are_released_only_when_their_images_build_inputs_changed`

7.3 Supporting Assets

The UI frames come from retained presentation recordings captured on 23 October 2026. The asset register records each source clip, extraction timecode and image size. The persona illustration shows the fictional planning personas.

Asset	Purpose
Upload screen	Owner upload area and supported-file guidance.
Accepted upload	Accepted file waiting for Processing.
Library and playback	Completed owner library and inline playback.
Publisher batch	Public-API batch status for three files.
Foreign access refused	Cross-owner and anonymous access checks.
Bounded capacity	Finite Fargate encoder activity and queue counts.
Planning personas	Maya, Arun, Jules and Sam, from left to right.